Digital Logic Design

Number Systems, Operations and Codes

Part 1

Decimal Numbers

The position of each digit in a weighted number system is assigned a weight based on the **base** or **radix** of the system. The radix of decimal numbers is ten, because only ten symbols (0 through 9) are used to represent any number.

The column weights of decimal numbers are powers of ten that increase from right to left beginning with $10^0 = 1$:

$\ldots 10^5\ 10^4\ 10^3\ 10^2\ 10^1\ 10^0$.

For fractional decimal numbers, the column weights are negative powers of ten that decrease from left to right:

$10^2\ 10^1\ 10^0.\ 10^{-1}\ 10^{-2}\ 10^{-3}\ 10^{-4}\ \ldots$

Decimal Numbers

Decimal numbers can be expressed as the sum of the products of each digit times the column value for that digit. Thus, the number 9240 can be expressed as

$$(9 \times 10^3) + (2 \times 10^2) + (4 \times 10^1) + (0 \times 10^0)$$

or

$$9 \times 1{,}000 + 2 \times 100 + 4 \times 10 + 0 \times 1$$

Example

Express the number 480.52 as the sum of values of each digit.

Solution

$$480.52 = (4 \times 10^2) + (8 \times 10^1) + (0 \times 10^0) + (5 \times 10^{-1}) + (2 \times 10^{-2})$$

Binary Numbers

For digital systems, the binary number system is used. Binary has a radix of two and uses the digits 0 and 1 to represent quantities.

The column weights of binary numbers are powers of two that increase from right to left beginning with $2^0 = 1$:

$\ldots 2^5 \; 2^4 \; 2^3 \; 2^2 \; 2^1 \; 2^0$ weights

32 16 8 4 2 1 weights

For fractional binary numbers, the column weights are negative powers of two that decrease from left to right:

$2^2 \; 2^1 \; 2^0 . \; 2^{-1} \; 2^{-2} \; 2^{-3} \; 2^{-4} \ldots$ weights

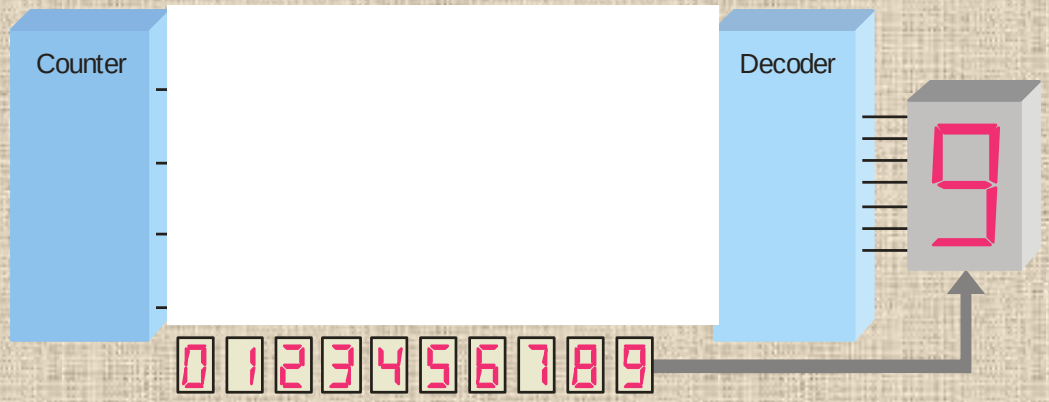
4 2 1 . ½ ¼ 1/8 1/16 weights

Binary Numbers

A binary counting sequence for numbers from zero to fifteen is shown.

Notice the pattern of zeros and ones in each column.

Digital counters frequently have this same pattern of digits:



Decimal Number	Binary Number
0	0 0 0 0
1	0 0 0 1
2	0 0 1 0
3	0 0 1 1
4	0 1 0 0
5	0 1 0 1
6	0 1 1 0
7	0 1 1 1
8	1 0 0 0
9	1 0 0 1
10	1 0 1 0
11	1 0 1 1
12	1 1 0 0
13	1 1 0 1
14	1 1 1 0
15	1 1 1 1

Binary Conversions

The decimal equivalent of a binary number can be determined by adding the column values (weights) of all of the bits that are 1 and discarding all of the bits that are 0.

Example Convert the binary number 100101.01 to decimal.

Solution Start by writing the column weights; then add the weights that correspond to each 1 in the number.

2^5	2^4	2^3	2^2	2^1	2^0 .	2^{-1}	2^{-2}
32	16	8	4	2	1 .	½	¼
1	0	0	1	0	1.	0	1

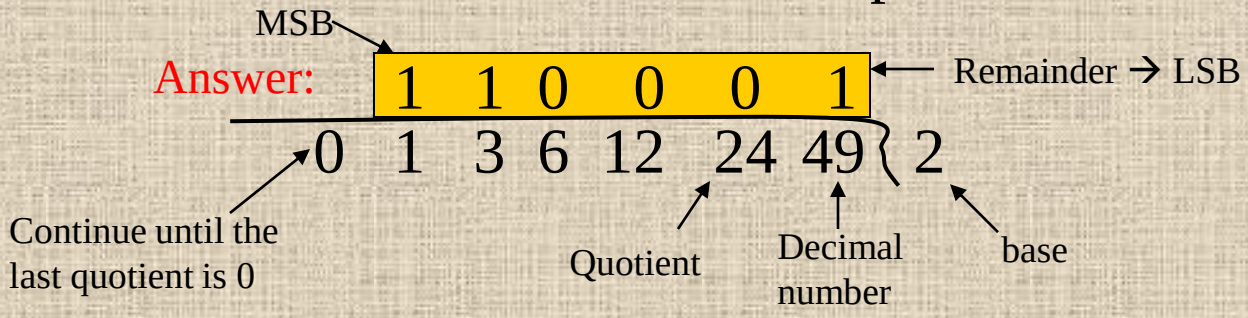
32 +4 +1 +¼ = 37¼ = 37.25

Binary Conversions

To convert **decimal** to any other **base**, repeatedly divide by the **base**.
For conversion to binary, repeatedly divide by 2:

Example Convert the decimal number 49 to binary by repeatedly dividing by 2.

Solution You can do this by "reverse division" and the answer will read from left to right. Put quotients to the left and remainders on top.



Binary Addition

The rules for binary addition are

0 + 0 = 0	Sum = 0, carry = 0
0 + 1 = 0	Sum = 1, carry = 0
1 + 0 = 0	Sum = 1, carry = 0
1 + 1 = 10	Sum = 0, carry = 1

When an input carry = 1 due to a previous result, the rules are

1 + 0 + 0 = 01	Sum = 1, carry = 0
1 + 0 + 1 = 10	Sum = 0, carry = 1
1 + 1 + 0 = 10	Sum = 0, carry = 1
1 + 1 + 1 = 11	Sum = 1, carry = 1

Binary Addition

Example Add the binary numbers 00111 and 10101 and show the equivalent decimal addition.

Solution

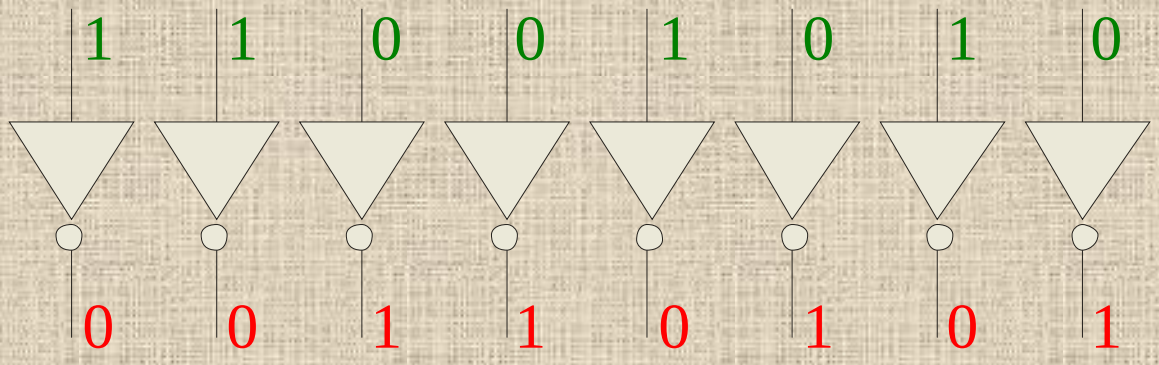
$$\begin{array}{rr} {\scriptstyle 0\ 1\ 1\ 1} & \\ 00111 & 7 \\ 10101 & 21 \\ \hline 11100 & = 28 \end{array}$$

1’s Complement

The 1’s complement of a binary number is just the inverse of the digits. To form the 1’s complement, change all 0’s to 1’s and all 1’s to 0’s.

For example, the 1’s complement of 11001010 is 00110101

In digital circuits, the 1’s complement is formed by using inverters:



2’s Complement

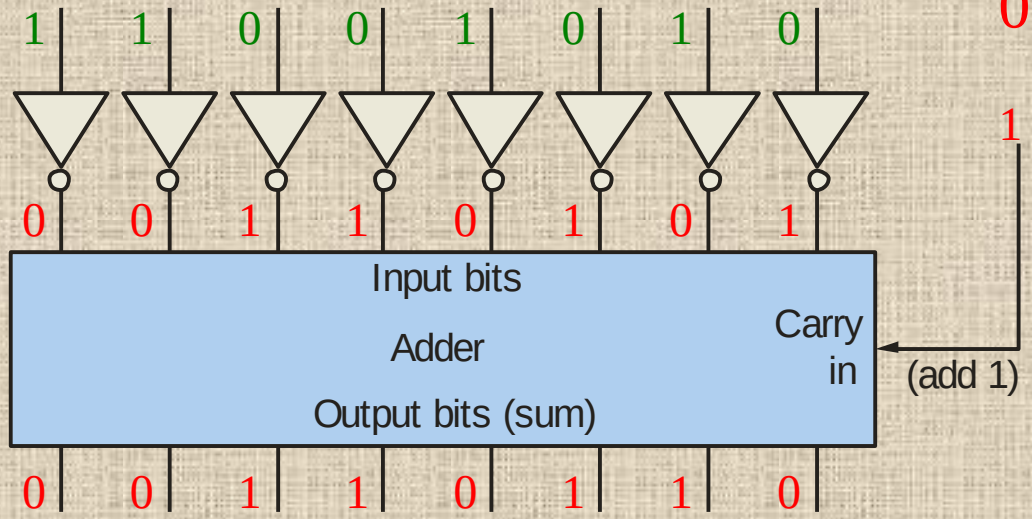
The 2’s complement of a binary number is found by adding 1 to the LSB of the 1’s complement.

Recall that the 1’s complement of 11001010 is 00110101 (1’s complement)

To form the 2’s complement, add 1:

$$\begin{array}{r} 00110101 \\ +\quad\;\; 1 \\ \hline 00110110 \end{array}$$

(2’s complement)



Signed Binary Numbers

There are several ways to represent signed binary numbers. In all cases, the MSB in a signed number is the sign bit, that tells you if the number is positive or negative.

For example, the positive number 25 is written using 8-bits as 00011001 (true form).



The negative number –25 is written as:

–25 = 10011001



In the **sign magnitude form**, a negative number has the same magnitude bits as the corresponding positive number but the sign bit is 1 rather than a zero. The sign bit has no weight value.

1’s Complement form

Negative numbers can be the 1’s complements of the corresponding positive numbers.
For example, the decimal number -25 is expressed as the 1’s complement of 00011001 (+25) which is: 11100110 (-25).

The 1’s complements of negative numbers are the corresponding positive numbers.
For example the 1’s complement of 11100110 (-25) is 00011001 (+25).

2’s Complement

Negative numbers can be also the 2’s complements of the corresponding positive numbers. For example, the decimal number -25 is expressed as the 2’s complement of 00011001 (+25) by inverting each bit and adding 1, which is: 11100111 (-25).

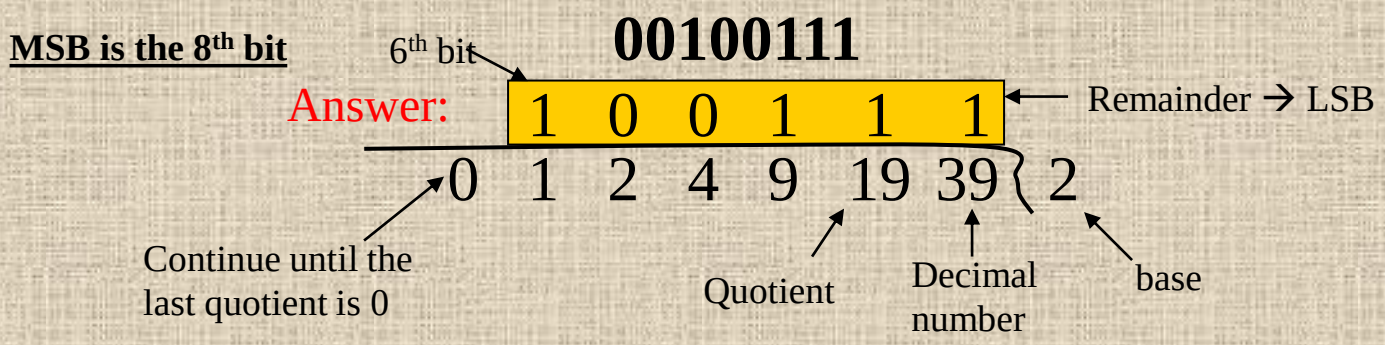
The 2’s complements of negative numbers are the corresponding positive numbers.
For example, the 2’s complement of 11100111 (-25) is 00011001 (+25).

Example

Express the decimal number -39 as an 8-bit binary number in the sign magnitude, 1's complement, and 2's complement.

Solution:

8-bit binary number for + 39 by repeatedly dividing by 2:



The sign magnitude form, -39: **10100111**

In the 1's complement form, -39 is produced by taking 1's complement of +39: **11011000**

In the 2's complement form, -39 is produced by taking the 2's complement of +39: **11011001**

-39 in 1’s complement: 11011000

\+ 1

-39 in 2’s complement: 11011001

2’s complement of -39 is +39 because:

1’s complement of 11011001: 00100110

\+ 1

+39 in 2’s complement: 00100111

Decimal	Sign-Magnitude	1’s comp.	2’s comp.
+39	**00100111**	**00100111**	**00100111**
-39	**10100111**	**11011000**	**11011001**

Problem:

Express +19 and -19 as 8-bit signed binary numbers in sign magnitude, 1’s complement, and 2’s complement.

Digital Logic Design

Number Systems, Operations and Codes

Part 2

Hexadecimal Numbers

Hexadecimal uses sixteen characters to represent numbers: the numbers 0 through 9 and the alphabetic characters A through F.

Large binary number can easily be converted to hexadecimal by grouping **4 bits** at a time and writing the equivalent hexadecimal character.

Example

Express $(1001\ 0110\ 0000\ 1110)_2$ in hexadecimal:

Solution

Group the binary number by 4-bits starting from the right.

Thus, the answer is: $(960E)_{16}$

Decimal	Hexadecimal	Binary
0	0	0000
1	1	0001
2	2	0010
3	3	0011
4	4	0100
5	5	0101
6	6	0110
7	7	0111
8	8	1000
9	9	1001
10	A	1010
11	B	1011
12	C	1100
13	D	1101
14	E	1110
15	F	1111

Hexadecimal Numbers

Hexadecimal is a weighted number system. The column weights are powers of 16, which increase from right to left.

Column weights { 16^3 16^2 16^1 16^0 .
4096 256 16 1 .

Example

Express $(1A2F)_{16}$ in decimal.

Solution

Start by writing the column weights:

4096 256 16 1
1 A 2 F_{16}

$1(4096) + 10(256) + 2(16) + 15(1) = 6703_{10}$

Decimal	Hexadecimal	Binary
0	0	0000
1	1	0001
2	2	0010
3	3	0011
4	4	0100
5	5	0101
6	6	0110
7	7	0111
8	8	1000
9	9	1001
10	A	1010
11	B	1011
12	C	1100
13	D	1101
14	E	1110
15	F	1111

Octal Numbers

Octal uses eight characters the numbers 0 through 7 to represent numbers. There is no 8 or 9 character in octal.

Binary number can easily be converted to octal by grouping **3 bits** at a time and writing the equivalent octal character for each group.

Example

Express $(1\ 001\ 011\ 000\ 001\ 110)_2$ in octal:

Solution

Group the binary number by 3-bits starting from the right.

Thus, the result is: $(113016)_8$

Decimal	Octal	Binary
0	0	0000
1	1	0001
2	2	0010
3	3	0011
4	4	0100
5	5	0101
6	6	0110
7	7	0111
8	10	1000
9	11	1001
10	12	1010
11	13	1011
12	14	1100
13	15	1101
14	16	1110
15	17	1111

Octal Numbers

Octal is also a weighted number system. The column weights are powers of 8, which increase from right to left.

Column weights { 8^3 8^2 8^1 8^0 . .
512 64 8 1

Example

Express $(3702)_8$ in decimal.

Solution

Start by writing the column weights:

512 64 8 1

$(3\ 7\ 0\ 2)_8$

$3(512) + 7(64) + 0(8) + 2(1) = 1986_{10}$

Decimal	Octal	Binary
0	0	0000
1	1	0001
2	2	0010
3	3	0011
4	4	0100
5	5	0101
6	6	0110
7	7	0111
8	10	1000
9	11	1001
10	12	1010
11	13	1011
12	14	1100
13	15	1101
14	16	1110
15	17	1111

BCD

Binary coded decimal (BCD) is a weighted code that is commonly used in digital systems when it is necessary to show decimal numbers such as in clock displays.

The table illustrates the difference between straight binary and BCD. BCD represents each decimal digit with a 4-bit code. Notice that the codes 1010 through 1111 are not used in BCD.

Decimal	Binary	BCD
0	0000	0000
1	0001	0001
2	0010	0010
3	0011	0011
4	0100	0100
5	0101	0101
6	0110	0110
7	0111	0111
8	1000	1000
9	1001	1001
10	1010	00010000
11	1011	00010001
12	1100	00010010
13	1101	00010011
14	1110	00010100
15	1111	00010101

BCD

You can think of BCD in terms of column weights in groups of four bits. For an 8-bit BCD number, the column weights are: (80 40 20 10) (8 4 2 1).

Question: What are the column weights for the BCD number 1000 0011 0101 1001?

Answer:

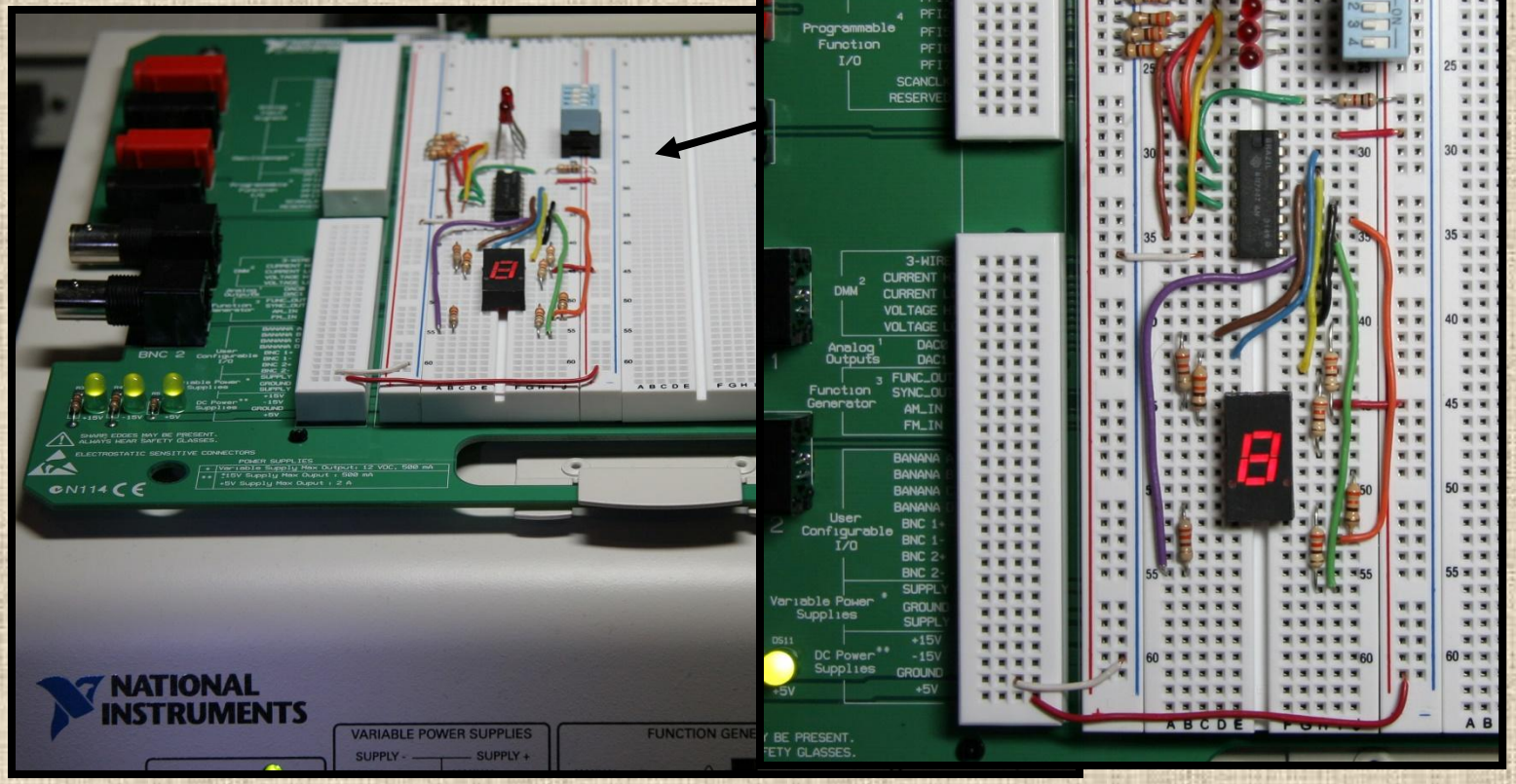
8000 4000 2000 1000 800 400 200 100 80 40 20 10 8 4 2 1

Note that you could add the column weights where there is a 1 to obtain the decimal number. For this case:

$$(8000) + (200 + 100) + (40 + 10) + (8 + 1) = (8359)_{10}$$

BCD

A lab experiment in which BCD is converted to decimal is shown.



Selected Key Terms

Byte	A group of eight bits
Hexadecimal	A number system with a base of 16.
Octal	A number system with a base of 8.
BCD	Binary coded decimal; a digital code in which each of the decimal digits, 0 through 9, is represented by a group of four bits.

Digital Logic Design

Number Systems, Operations and Codes

Part 3

Arithmetic Operations with Signed Numbers

Using the signed number notation with negative numbers in 2's complement form simplifies addition and subtraction of signed numbers.

Rules for **addition**: Add the two signed numbers. Discard any final carries. The result is in signed form.

Examples with 8-bit signed binary numbers:

00011110	= +30	00001110	= +14	11111111	= –1
+ 00001111	= +15	+ 11101111	= –17	+ 11111000	= –8
00101101	= +45	11111101	= –3	111110111	= –9

Discard carry

Verification:

$2^5 + 2^3 + 2^2 + 2^0 =$

$32 + 8 + 4 + 1 = +45$

Verification:

$-2^7 + 2^6 + 2^5 + 2^4 + 2^3 + 2^2 + 2^0 =$

$-128+64+32+16+8+4+1 = -3$

Arithmetic Operations with Signed Numbers

Note that if the number of bits required for the answer is exceeded, **overflow** will occur. The overflow may only occur if both numbers have the same sign. The overflow will be indicated by an incorrect sign bit.

Two examples are:



Wrong! The answer is incorrect (out of range) and the sign bit has changed.

Repair/Correction! Add a bit to the MSB of each signed binary number

Arithmetic Operations with Signed Numbers

Two examples are:

001000000 = +64
001000001 = +65
010000001 = +129

110000011 = –125
111110111 = –9
Discard carry → 1101111010 = –134

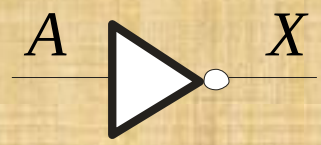


Correct! The answer is correct (in range) and the sign bit has not changed.

Repair/Correction occurs with 9 bits!

Digital Logic Design
Logic Gates

The Inverter

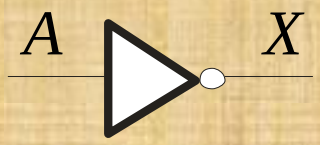


The inverter performs the Boolean **NOT** operation. When the input is LOW, the output is HIGH; when the input is HIGH, the output is LOW.

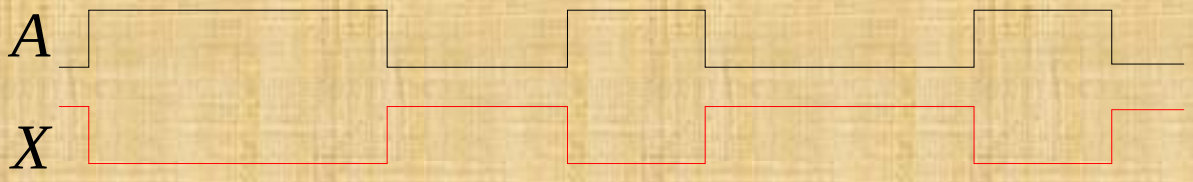
Input	Output
A	*X*
LOW (0)	HIGH (1)
HIGH (1)	LOW(0)

The **NOT** operation (complement) is shown with an overbar. Thus, the Boolean expression for an inverter is $X = \overline{A}$.

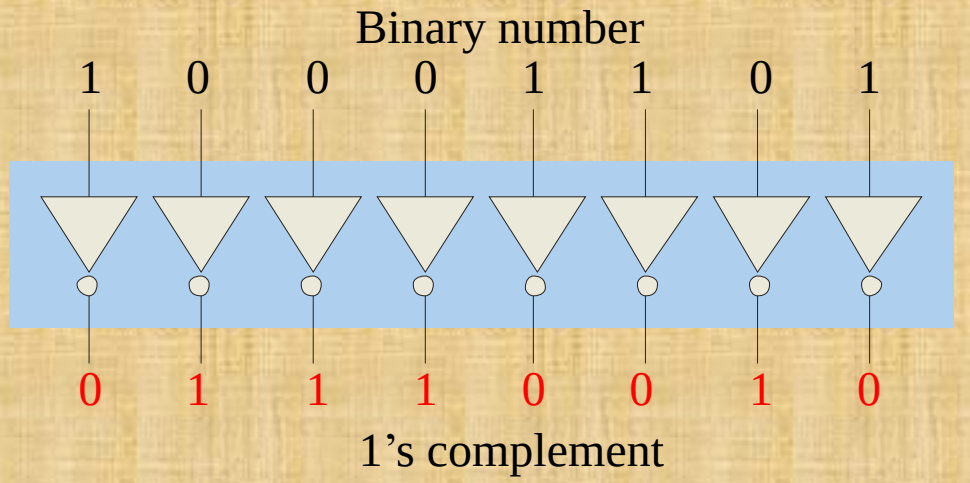
The Inverter



Example waveforms:



A group of inverters can be used to form the 1's complement of a binary number:



The AND Gate

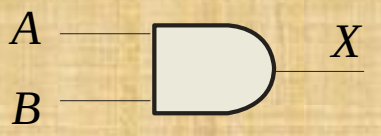


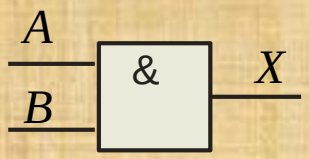
The **AND gate** produces a HIGH output when all inputs are HIGH; otherwise, the output is LOW. For a 2-input gate, the **truth table** is

Inputs		Output
A	*B*	*X*
0	0	0
0	1	0
1	0	0
1	1	1

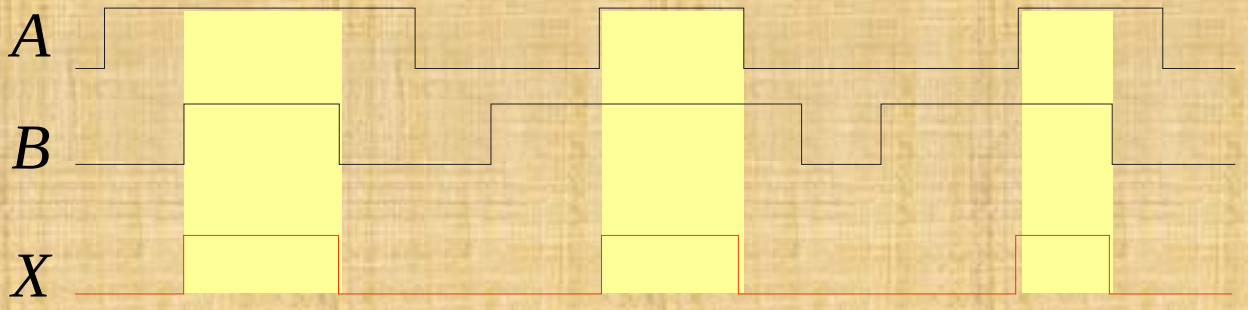
The **AND** operation is usually shown with a dot between the variables but it may be implied (no dot). Thus, the AND operation is written as $X = A \cdot B$ or $X = AB$.

The AND Gate





Example waveforms:



The AND operation is used in computer programming as a selective mask. If you want to retain certain bits of a binary number but reset the other bits to 0, you could set a mask with 1's in the position of the retained bits.

Example If the binary number 10100011 is ANDed with the mask 00001111, what is the result? 00000011

The OR Gate



The **OR gate** produces a HIGH output if any input is HIGH; if all inputs are LOW, the output is LOW. For a 2-input gate, the truth table is

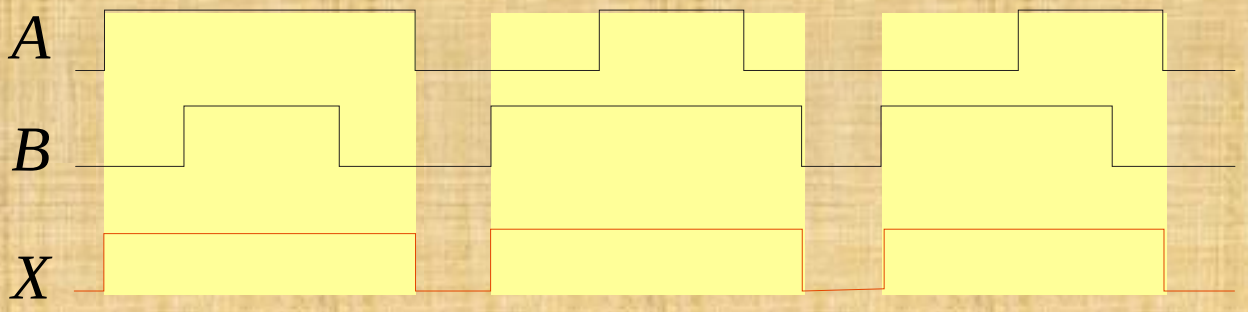
Inputs		Output
A	*B*	*X*
0	0	0
0	1	1
1	0	1
1	1	1

The **OR** operation is shown with a plus sign (+) between the variables. Thus, the OR operation is written as $X = A + B$.

The OR Gate



Example waveforms:



The OR operation can be used in computer programming to set certain bits of a binary number to 1.

Example ASCII letters have a 1 in the bit 5 position for lower case letters and a 0 in this position for capitals. (Bit positions are numbered from right to left starting with 0.) What will be the result if you OR an ASCII letter with the 8-bit mask 00100000?

Solution The resulting letter will be lower case.

For example: A: 01000001; a: 01100001

If A is ORed bit-by-bit with 00100000, the result will be: 01100001

The NAND Gate

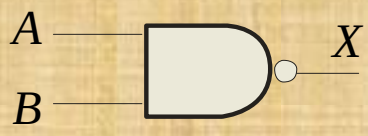


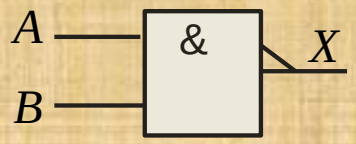
The **NAND gate** produces a LOW output when all inputs are HIGH; otherwise, the output is HIGH. For a 2-input gate, the truth table is

Inputs		Output
A	*B*	*X*
0	0	1
0	1	1
1	0	1
1	1	0

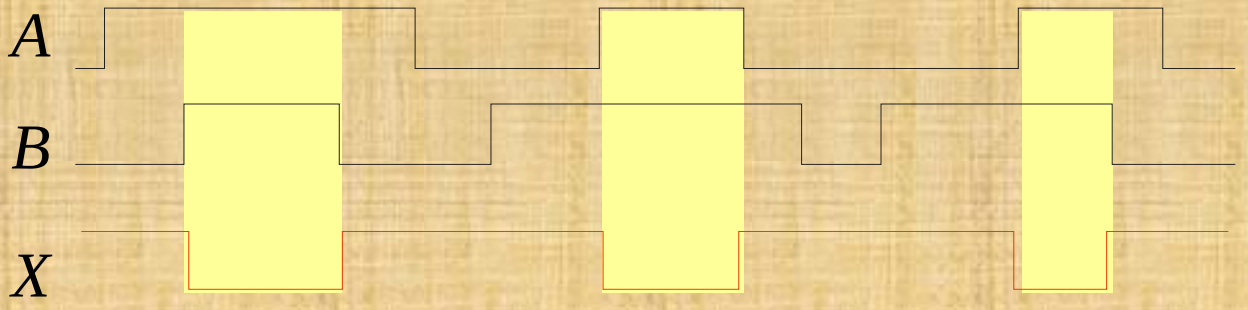
The **NAND** operation is shown with a dot between the variables and an overbar covering them. Thus, the NAND operation is written as $X = \overline{A \cdot B}$ (Alternatively, $X = \overline{AB}$.)

The NAND Gate



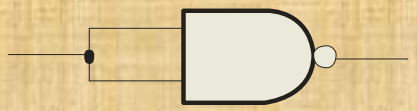


Example waveforms:



The NAND gate is particularly useful because it is a "universal" gate – all other basic gates can be constructed from NAND gates.

Question How would you connect a 2-input NAND gate to form a basic inverter?



The NOR Gate

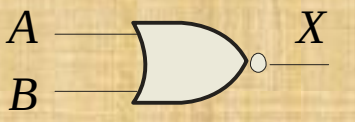


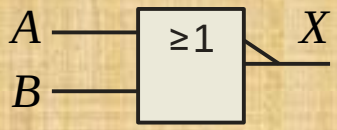
The **NOR gate** produces a LOW output if any input is HIGH; if all inputs are HIGH, the output is LOW. For a 2-input gate, the truth table is

Inputs		Output
A	*B*	*X*
0	0	1
0	1	0
1	0	0
1	1	0

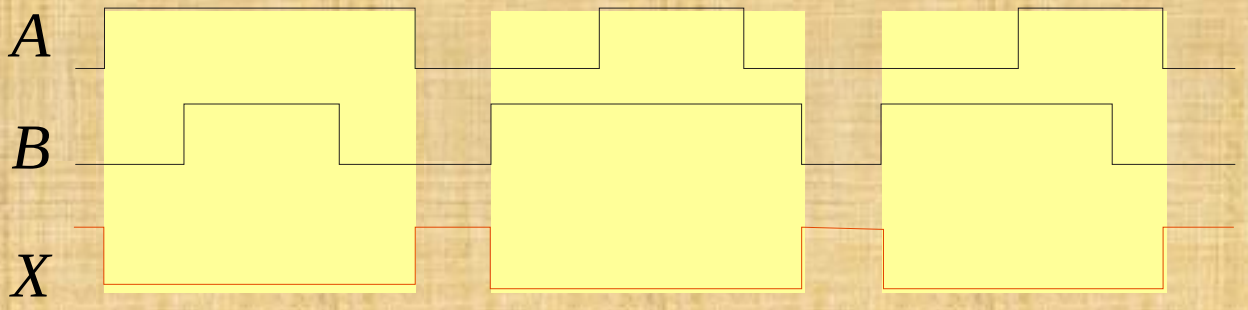
The **NOR** operation is shown with a plus sign (+) between the variables and an overbar covering them. Thus, the NOR operation is written as $X = \overline{A + B}$.

The NOR Gate



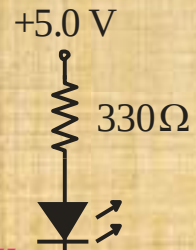


Example waveforms:

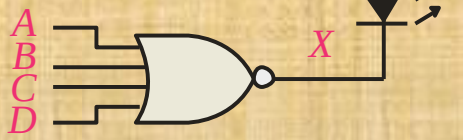


The NOR operation will produce a LOW if any input is HIGH.

Example When is the LED is ON for the circuit shown?



Solution The LED will be on when any of the four inputs is HIGH.



The XOR Gate

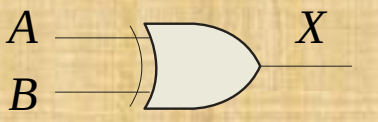


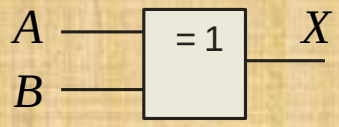
The **XOR gate** produces a HIGH output only when both inputs are at opposite logic levels. The truth table is

Inputs		Output
A	*B*	*X*
0	0	0
0	1	1
1	0	1
1	1	0

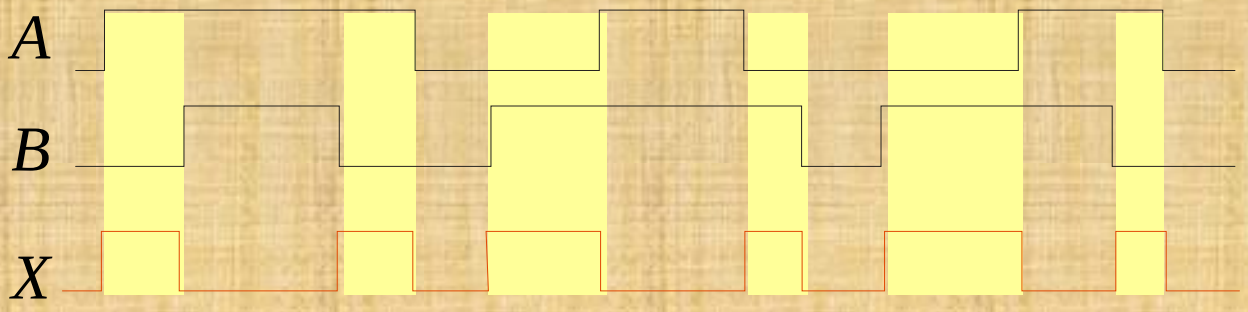
The **XOR** operation is written as $X = \overline{A}B + A\overline{B}$. Alternatively, it can be written with a circled plus sign between the variables as $X = A \oplus B$.

The XOR Gate





Example waveforms:



Notice that the XOR gate will produce a HIGH only when exactly one input is HIGH.

Question If the *A* and *B* waveforms are both inverted for the above waveforms, how is the output affected?

There is no change in the output.

The XNOR Gate

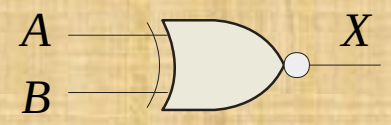


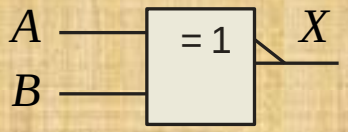
The **XNOR gate** produces a HIGH output only when both inputs are at the same logic level. The truth table is

Inputs		Output
A	*B*	*X*
0	0	1
0	1	0
1	0	0
1	1	1

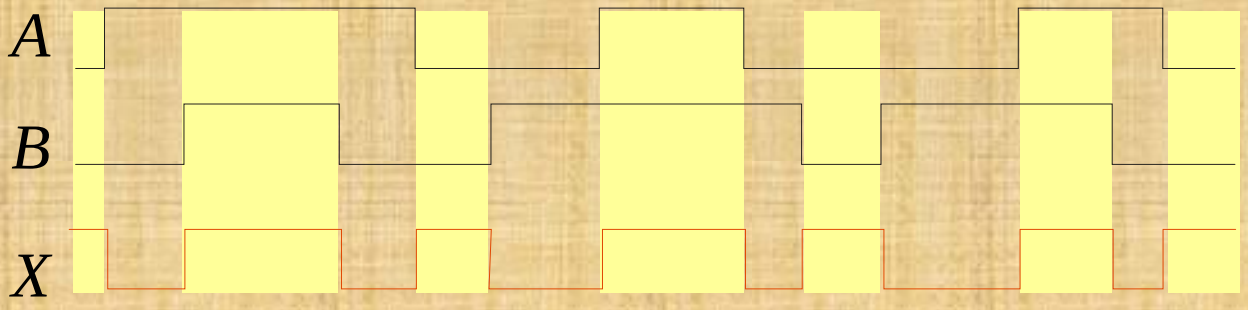
The **XNOR** operation shown as $X = \overline{A}\,\overline{B} + AB$. Alternatively, the XNOR operation can be shown with a circled dot between the variables. Thus, it can be shown as $X = A \odot B$.

The XNOR Gate





Example waveforms:



Notice that the XNOR gate will produce a HIGH when both inputs are the same. This makes it useful for comparison functions.

Question If the *A* waveform is inverted but *B* remains the same, how is the output affected?

The output will be inverted.

Selected Key Terms

Inverter	A logic circuit that inverts or complements its inputs.
Truth table	A table showing the inputs and corresponding output(s) of a logic circuit.
Timing diagram	A diagram of waveforms showing the proper time relationship of all of the waveforms.
Boolean algebra	The mathematics of logic circuits.
AND gate	A logic gate that produces a HIGH output only when all of its inputs are HIGH.

Selected Key Terms

OR gate	A logic gate that produces a HIGH output when one or more inputs are HIGH.
NAND gate	A logic gate that produces a LOW output only when all of its inputs are HIGH.
NOR gate	A logic gate that produces a LOW output when one or more inputs are HIGH.
Exclusive-OR gate	A logic gate that produces a HIGH output only when its two inputs are at opposite levels.
Exclusive-NOR gate	A logic gate that produces a LOW output only when its two inputs are at opposite levels.

Boolean Algebra and Logic Gates

Binary Logic

- Binary logic deals with

1 - **Variables** that can take on **two** discrete **values**

→*Values can be called **True**, **False**, **yes**, **no**, etc.*

2 - **Operations** that assume **LOGICAL** Meaning

→ *Binary logic is equivalent to **Boolean algebra***

Boolean Algebra

•Basic mathematics required for the description of digital circuits

• used to describe the different interconnections of digital circuits

• the variable used in the Boolean algebra are called Boolean variables

→ We will study two-valued Boolean algebra and functions with simplifications using basic Boolean Identities

Two-valued Boolean Algebra

- It consists of

1- Boolean Variables

- Designated by letters of the alphabet such as A, B, C, x, y, z etc.

- Each variable can have two and only two distinct values: 1 and 0 (True, False)

- Can be a Function of some other Boolean variables (F=ABC)

2- Boolean Operations

-There are three Basic logical operations:

AND, OR, and NOT

Basic Boolean Operations- *AND* *operation*

- Represented by a dot or by the absence of an operator

Example: *x.y = or* *xy=z*

read: *x AND y is equal to z*

Interpretation: Z = 1 if and only if x= 1 AND y= 1

Otherwise z = 0

Truth table:

x	y	xy
0	0	0
0	1	0
1	0	0
1	1	1

Don't confuse this with binary multiplication operation

Truth table gives the value of z for all possible values of x and y

Basic Boolean Operations- *OR* *operation*

- *Represented by a plus sign (+)*

Example: $x + y = z$

read: *x OR y is equal to z*

Interpretation: z = 1 if x= 1 or if y= 1 or if both x =1 and y =1. z = 0 if x = 0 and y=1

Don't confuse this with binary addition operation

Truth table:

x	y	x+y
0	0	0
0	1	1
1	0	1
1	1	1

Truth table gives the value of z for all possible values for x and y

Basic Boolean Operations- *NOT* operation

- *Represented by a prime or an overbar (also called complement)*

Example: $x' = z$ *(or* $\bar{x} = z$*)*

read: *Not x is equal to z*

Interpretation: *z = "what x is not"*

x= 1 then z=0; x= 0 then z=1

Truth table:

x	x'
0	1
1	0

Truth table gives the value of z for all possible values for x

Binary Logic and Binary Signals

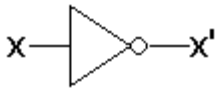
• For simplicity, we often still write digits instead:

  – 1 is true
  – 0 is false

• We will use this interpretation along with special operations to *design functions* and *logic circuits* for doing arbitrary computations.

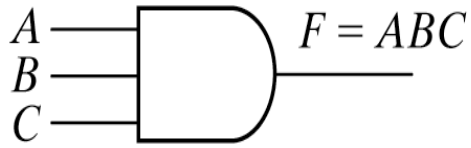
Logic Gates

- **Logic gates are electronic circuits that operate on one or more input signal to produce an output signal**
- Basic operations can be implemented in hardware using a Basic logic gate.
  - Symbols for each of the logic gates are shown below.
  - These gates output the product, sum or complement of their inputs

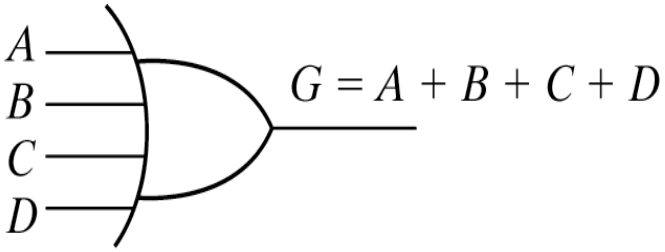
Logic Operation:	AND (product) of two inputs	OR (sum) of two inputs	NOT (complement) With one input
Representation:	x.y, or xy	x + y	x’
Logic gate:			

Gates with Multiple Inputs

- AND and OR Gates may have more than 2 input signals



(a) Three-input AND gate



(b) Four-input OR gate

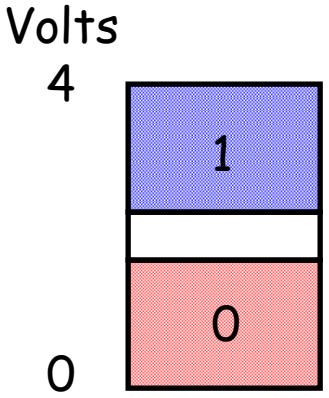
Binary Signals

- Computers use voltages to represent information.
- Two voltage levels are used to represent a binary value

  “1” and “0”

- Some digital systems for example may define that:
  - Binary ‘0” is equal to 0 Volt
  - Binary “1” is equal to 4 Volt

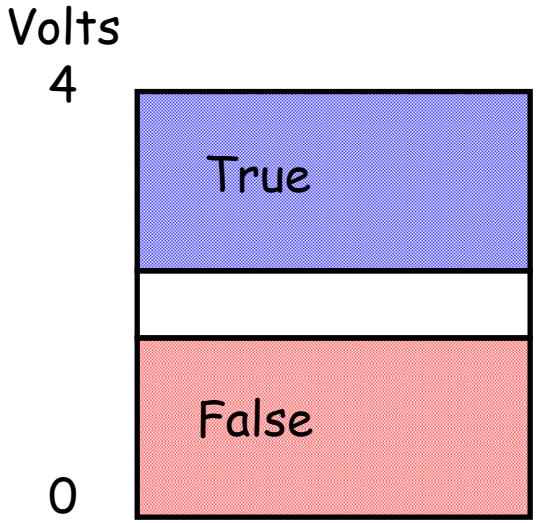
→ *It’s convenient for us to translate these voltages into values 1 and 0.*



Binary Logic and Binary Signals

•It's also possible to think of voltages as representing two *logical* values, ***true*** and ***false***.

→ These logical values are called Boolean values



Logic Gates - Signals

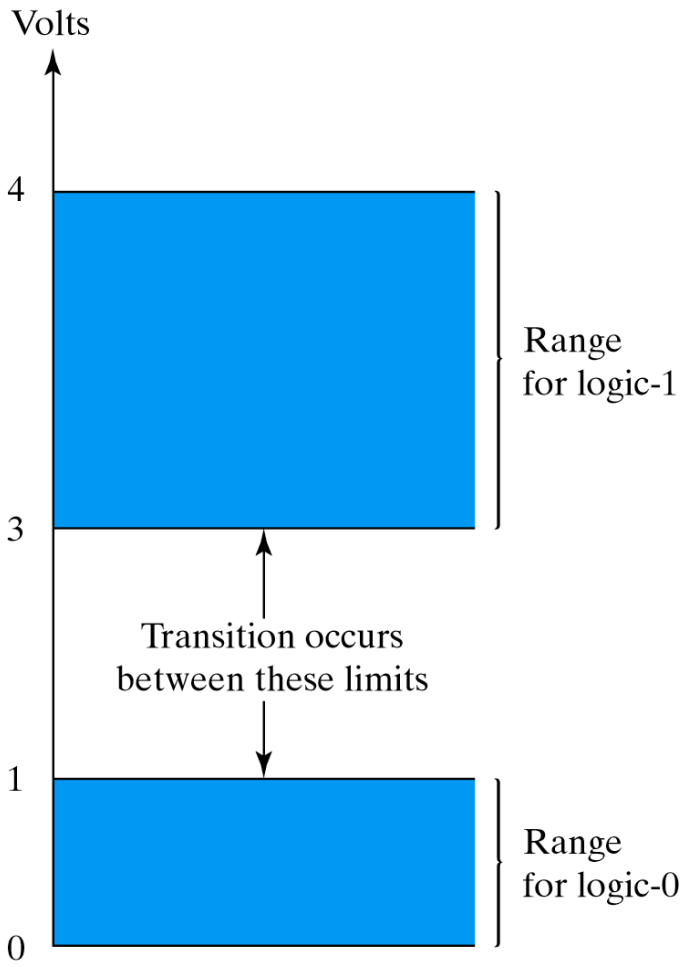
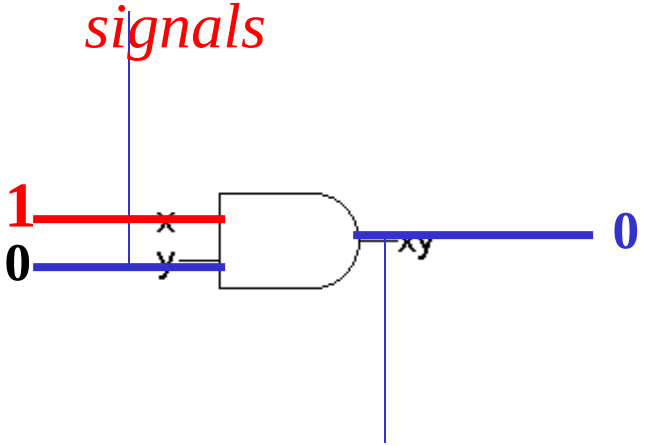


Fig. 1-3 Example of binary signals

Example

two input signals



one output signal

Logic Gates - Signals

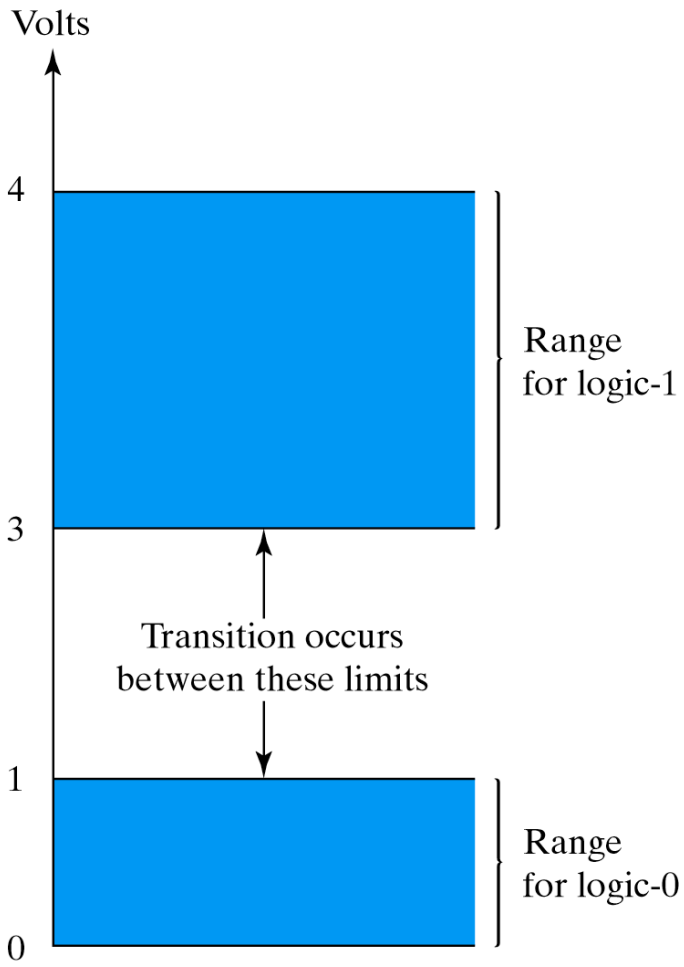
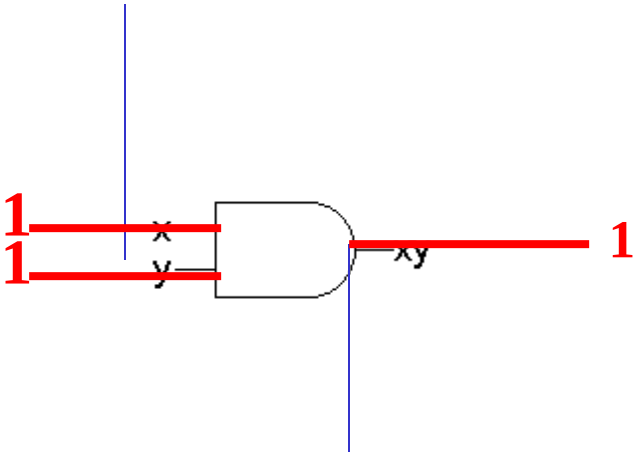


Fig. 1-3 Example of binary signals

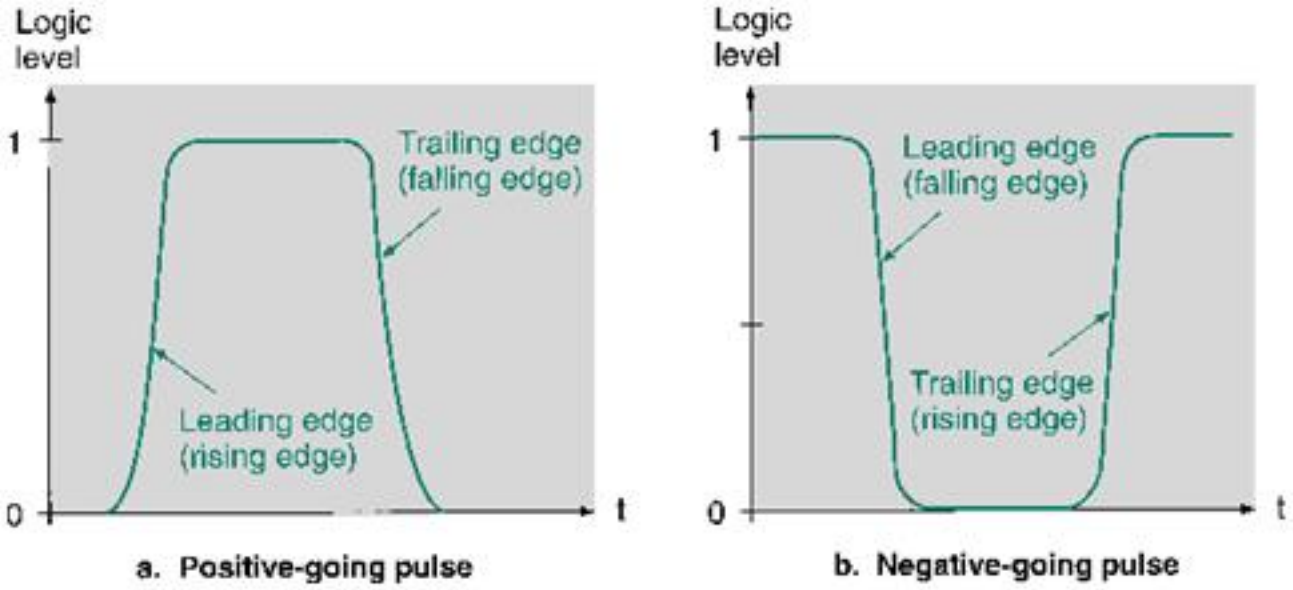
Example

2 input signals



1 output signal

Timing Diagram –Input and output signals



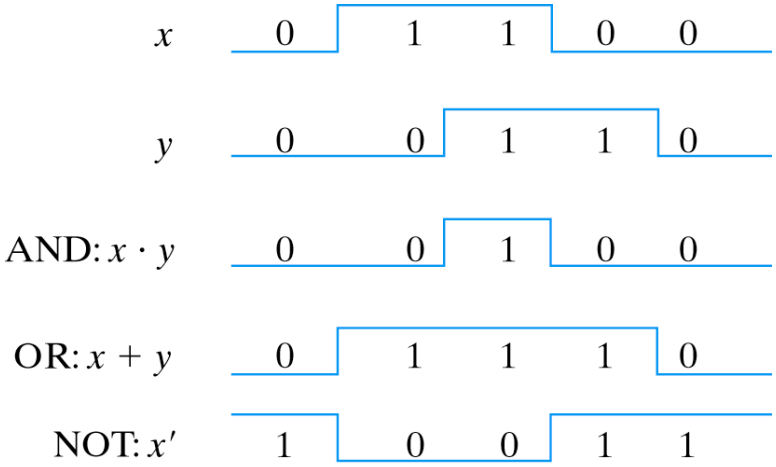


Fig. 1-5 Input-output signals for gates

Boolean expressions (functions)

- **We can use the basic operations to form more complex expressions:**

$$f(x,y,z) = x\,y' + z\,x'$$

- **Some terminology and notation:**
  - **f** is the name of the function.
  - ***Term*** is an implementation with a gate: in this example f has two terms $x\,y'$ and $z\,x$
  - ***(x,y,z)*** are the ***input variables***, each representing 1 or 0.
  - *A **literal*** is any occurrence of an input variable or its complement. The function above has four literals: x, y', z, and x'.

Precedence for Evaluation of Boolean Expression

- Precedence are important.

  – Parentheses first (if any) then

  NOT has the highest precedence, followed by AND, and then OR.

  $\rightarrow f(x,y,z) = (x + y')z + x'$

  –Fully parenthesized, the function above would be kind of messy:

  $$f(x,y,z) = (((x +(y'))z) + x')$$

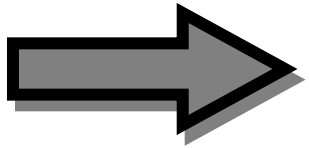
Truth Table

- A truth table shows all possible inputs and outputs of a function. Each input variable represents either 1 or 0.
  - A function with n variables has 2 power n possible combinations of inputs.
- Inputs are listed in binary order-example, from 000 to 111.

f(x,y,z) = (x + y')z + x'



f(0,0,0) = (0 + 1)0 + 1 = 1
f(0,0,1) = (0 + 1)1 + 1 = 1
f(0,1,0) = (0 + 0)0 + 1 = 1
f(0,1,1) = (0 + 0)1 + 1 = 1
f(1,0,0) = (1 + 1)0 + 0 = 0
f(1,0,1) = (1 + 1)1 + 0 = 1
f(1,1,0) = (1 + 0)0 + 0 = 0
f(1,1,1) = (1 + 0)1 + 0 = 1



x	y	z	f(x,y,z)
0	0	0	1
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

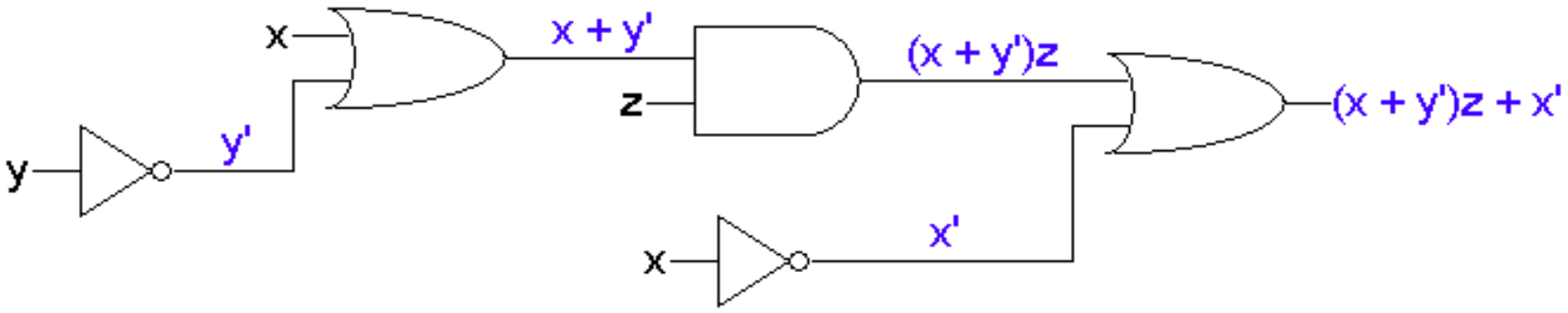
Boolean Expression and Logic Circuits

- *A* Boolean expression (function) can be converted into a circuit by *combining* basic gates.
- Example:

  - The diagram below shows the inputs and outputs of each gate.

  - The precedences are explicit in a circuit.

f(x,y,z) = (x + y')z + x'



Obtaining Boolean Expressions

- **Expressions may be obtained from:**
  - English language description
  - Truth table;
  - Logic circuit.

Obtaining Boolean Expressions

- The Boolean expression (un-simplified) can be obtained from the truth table: Consider the following arbitrary Truth Table

A	B	C	F_1	
0	0	0	0	
0	0	1	0	
0	1	0	1	*A'BC'*
0	1	1	1	*A'BC*
1	0	0	1	*AB'C'*
1	0	1	1	*AB'C*
1	1	0	1	*ABC'*
1	1	1	1	*ABC*

We can also write the function as:

$F_1(A,B,C) =$ A'BC'+ A'BC+ AB'C'+ AB'C+ ABC'+ ABC

0 → prime

1 → real

Obtaining Boolean Expressions

Using the false terms in the truth table

•Sometimes it is easier to work with the terms that describe when the function is false.

•For example, if a function has four variables then there are sixteen possible states.

→If for instance thirteen out of sixteen were true, then only three out of sixteen are false. Fewer terms makes it easier

• In our example, two out of eight are false.

Obtaining Boolean Expressions

- The Boolean expression (un-simplified) can be obtained from the truth table using false terms

A	B	C	F_1	
0	0	0	0	*A'B'C'*
0	0	1	0	*A'B'C*
0	1	0	1	
0	1	1	1	
1	0	0	1	
1	0	1	1	
1	1	0	1	
1	1	1	1	

So we can also write the function **NOT** F_1 as:

$\overline{F_1}(A,B,C) = A'B'C' + A'B'C$

Obtaining The Truth Table - Example

Design a digital circuit that will be used to control an Alarm bell. This Alarm bell is to be installed in a room to protect it from unauthorized entry.

Sensor devices provide the following logic signals

C = 1 The control system is active

D = 1 The room door is closed

M = 1 There is a motion in the room

Q = 1 The room is open to the public

i) Obtain the truth table

ii) derive the Boolean expression using **true** terms

Truth table

C	D	M	Q	Alarm
0	0	0	0	0
0	0	0	1	0
0	0	1	0	0
0	0	1	1	0
0	1	0	0	0
0	1	0	1	0
0	1	1	0	0
0	1	1	1	0
1	0	0	0	1
1	0	0	1	0
1	0	1	0	1
1	0	1	1	0
1	1	0	0	0
1	1	0	1	0
1	1	1	0	1
1	1	1	1	0

Alarm= CD'M'Q' + CD'MQ' + CDMQ'

Door should not be open

room is closed to the public → door open + motion

room is closed to the public + door closed → motion

Boolean Identities

• Boolean algebra is used in digital design to reduce any logical function (expression) to its simplest form

→ the minimization of the number of literals and the number of terms

→ a circuit with less equipment

• It is a hard problem (no specific rules to follow)

Boolean Identities

1.	$x + 0 = x$	
2.	$x \cdot 1 = x$	
3.	$x + x' = 1$	
4.	$x \cdot x' = 0$	
5.	$x + x = x$	Basic to Boolean algebra
6.	$x \cdot x = x$	
7.	$x + 1 = 1$	
8.	$x \cdot 0 = 0$	
9.	$(x')' = x$	
10.	$x + y = y + x$	Commutative
11.	$xy = yx$	Commutative
12.	$x + (y + z) = (x + y) + z$	Associative
13.	$x(yz) = (xy)z$	Associative
14.	$x(y + z) = xy + xz$	Distributive
15.	$x + yz = (x+y)(x+z)$	Associative
16.	$(x + y)' = x'y'$	DeMorgan
17.	$(xy)' = x' + y'$	DeMorgan
18.	$x + xy = x$	Absorption
19.	$x(x + y) = x$	Absorption

Verifying Boolean Identities-Examples

Theorem : $x+x = x$

$x+x = (x+x)\ 1$ — x.1=x

$= (x+x)\ (x+x')$ — x+x’=1

$= x+xx'$ — $x+yz =(x+y)(x+z)$

$= x+0$ — x.x’=0

$= x$ — x+0=x

Theorem : x x = x

$xx = x\ x + 0$

$= xx + xx'$

$= x\ (x + x')$

$= x\ 1$

$= x$

Verifying Boolean Identities-Examples

- DeMorgan's Theorems
  - → $(x+y)' = x'\,y'$
  - → $(x\,y)' = x' + y'$
- By means of truth table

x	y	x+y	(x+y)‘	x‘	y‘	x‘y’
0	0	0	1	1	1	1
0	1	1	0	1	0	0
1	0	1	0	0	1	0
1	1	1	0	0	0	0

Verifying Boolean Identities-Examples

- Theorem $x + xy = x$

By means of truth table

x	y	xy	x + xy
0	0	0	0
0	1	0	0
1	0	0	1
1	1	1	1

Simplifying Boolean Expressions

→ **Use the Rules of Boolean Algebra**

We can simply the function as:

$F_1(A,B,C) = A'BC' + A'BC + AB'C' + AB'C + ABC' + ABC$

$= A'B(C'+C) + AB'(C+C') + AB(C'+C)$

$= A'B + AB' + AB$

$= A'B + A(B'+B)$

$= A + A'B$

Simplifying Boolean Expressions

Function with four variables

- Giving the following function:

$F_{2a}(A,B,C,D) = (AB'(C + BD) + A'B')C$

$= (AB'C + \mathbf{AB'BD} + A'B')C$

$= (AB'C + \mathbf{A0D} + A'B')C$

$= (AB'C + \mathbf{0} + A'B')C$

$= (AB'C + A'B')C$

$= AB'\mathbf{CC} + A'B'C$

$= AB'C + A'B'C$

$= (\mathbf{A+A'})B'C$

$= B'C$

$\boldsymbol{F_{2b}(A,B,C,D)} = B'C$

→ the two expressions are equivalent!

→ **F_{2a} requires more logic gates than F_{2b}**

Basic and Other Logic gates

- **Basic Logic gate**

  - AND
  - OR
  - NOT

  These are called “fundamental logic gates” as all other gates and digital Circuits can be created from these gates.

- **Other Logic gates**

  - NAND
  - NOR

  These are called “Universal logic gates” as any digital circuit can be designed by just using these gates

  - XOR
  - XNOR

The NAND & NOR Gates

• We can use a NAND and NOR gates to implement all three of the *basic operations* (AND,OR,NOT).

→ They are said to be **functionally complete**

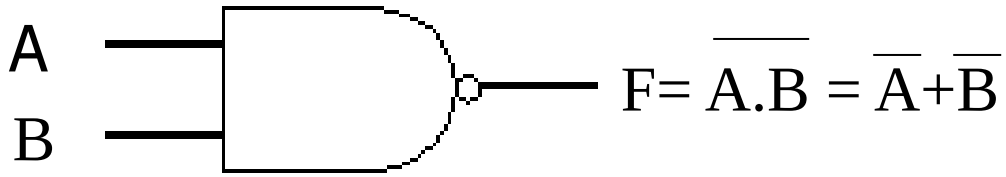
→ Both NAND and NOR gates are very valuable as any design can be realized using either one.

•It is easier to build digital circuits using all NAND or NOR gates than to combine AND,OR, and NOT gates.

•NAND/NOR gates are typically faster and cheaper to produce.

The NAND Gate

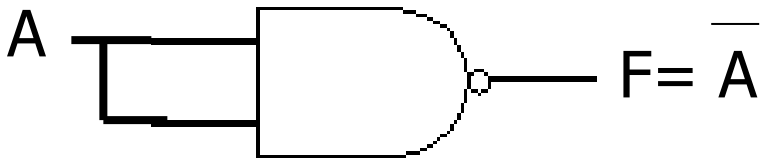
- The NAND gate is a combination of an AND gate followed by an inverter (NOT gate).
- We can use a NAND gate to implement all three of the *basic operations* (AND,OR,NOT).
- Such a gate is said to be **functionally complete**.



A	**B**	**F**
0	0	1
0	1	1
1	0	1
1	1	0

The NAND Gate

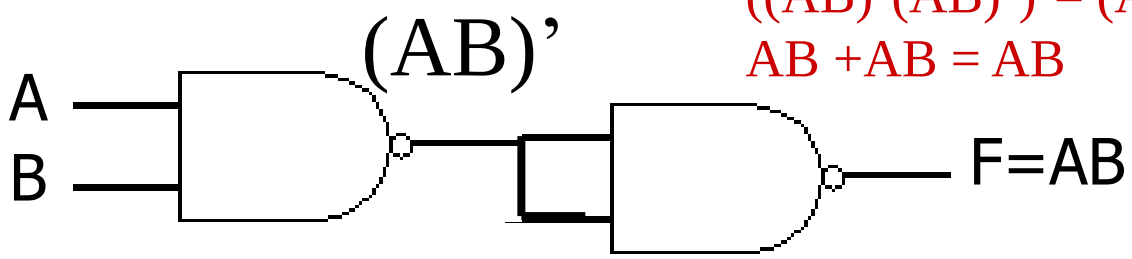
→ a NAND gate with both of its inputs driven by the same signal is equivalent to a NOT gate



NOT Gate

→a NAND gate whose output is complemented is equivalent to an AND gate

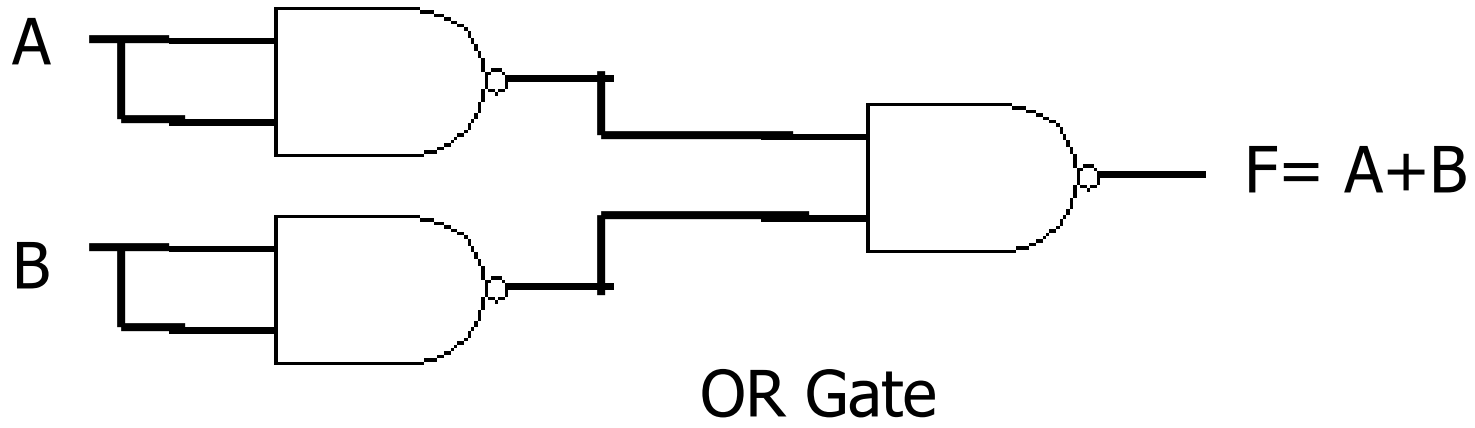
((AB)'(AB)')'= (AB)''+(AB)''= AB +AB = AB



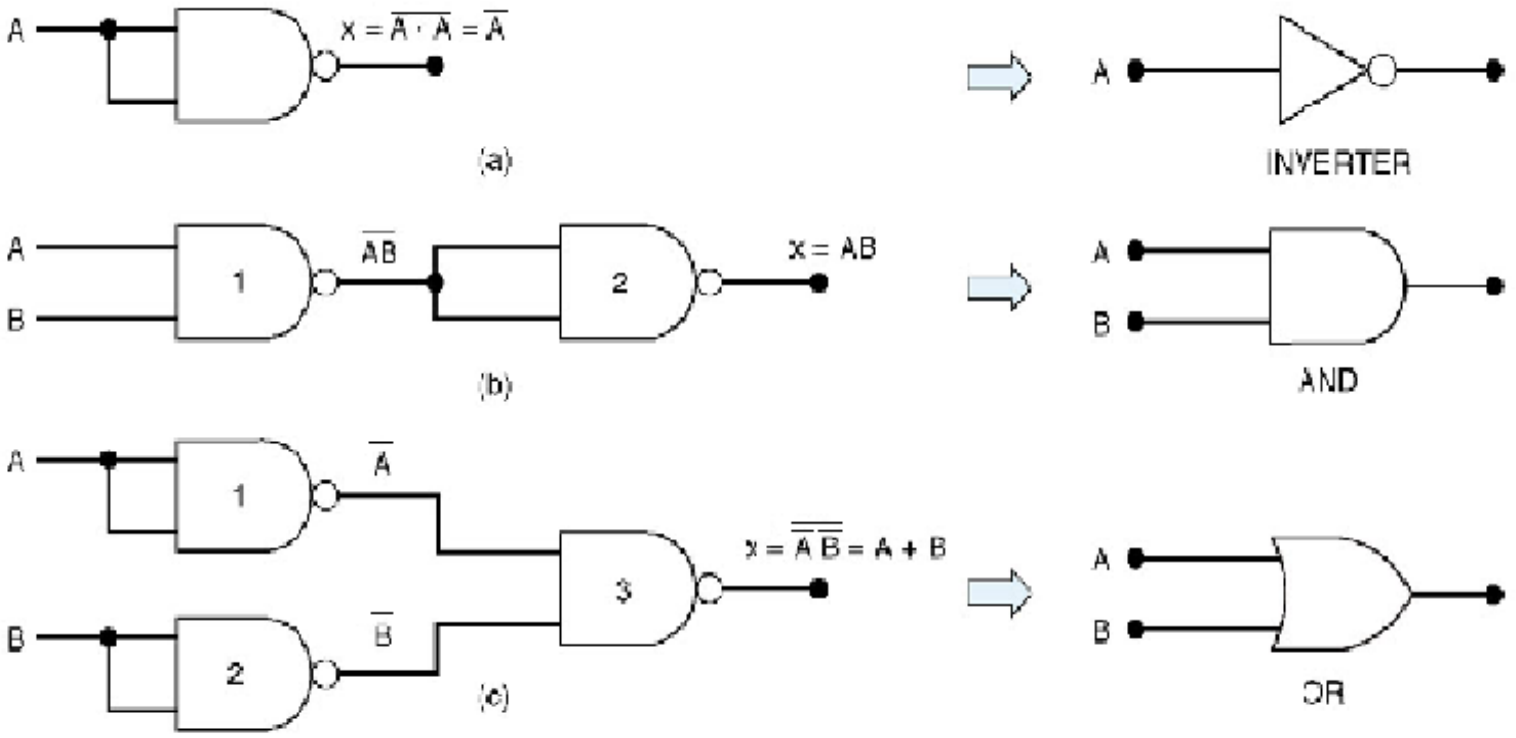
AND Gate

The NAND Gate

•a NAND gate with complemented inputs acts as an OR gate.



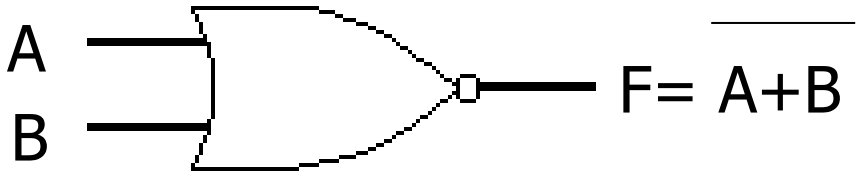
Universality of NAND



The NOR Gate

•This is a NOR gate. It is a combination of an OR gate followed by an inverter.

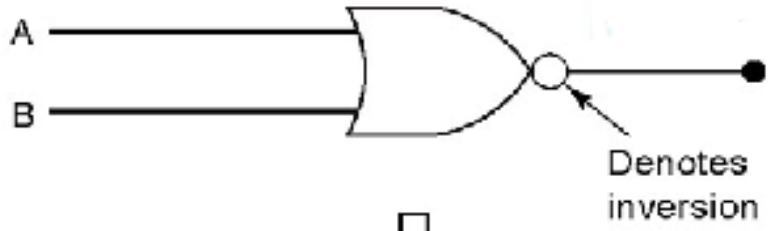
•like the NAND gate, the NOR gate is **functionally complete** →any logic function can be implemented using just NOR gates.



A	**B**	**F**
0	0	1
0	1	0
1	0	0
1	1	0

NOR Gate Equivalence

- NOR Symbol, Equivalent Circuit, Truth Table



(a)

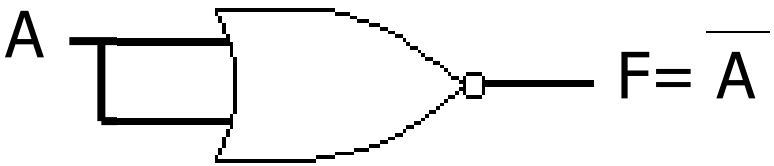


(b)

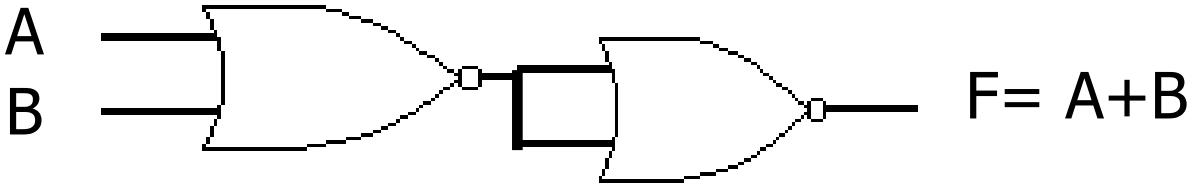
		OR	NOR
A	B	A + B	$\overline{A + B}$
0	0	0	1
0	1	1	0
1	0	1	0
1	1	1	0

(c)

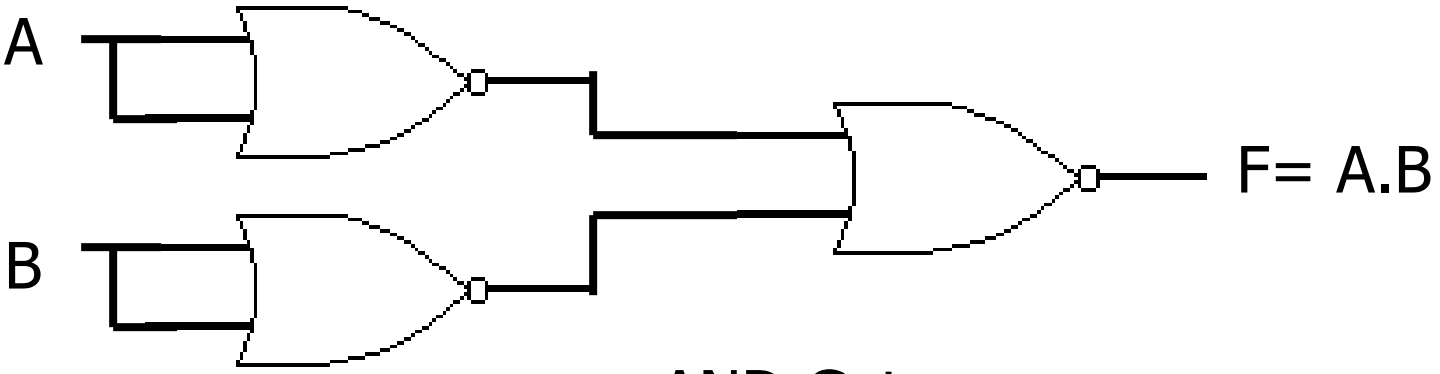
NOR Gates-**functionally complete**



NOT Gate

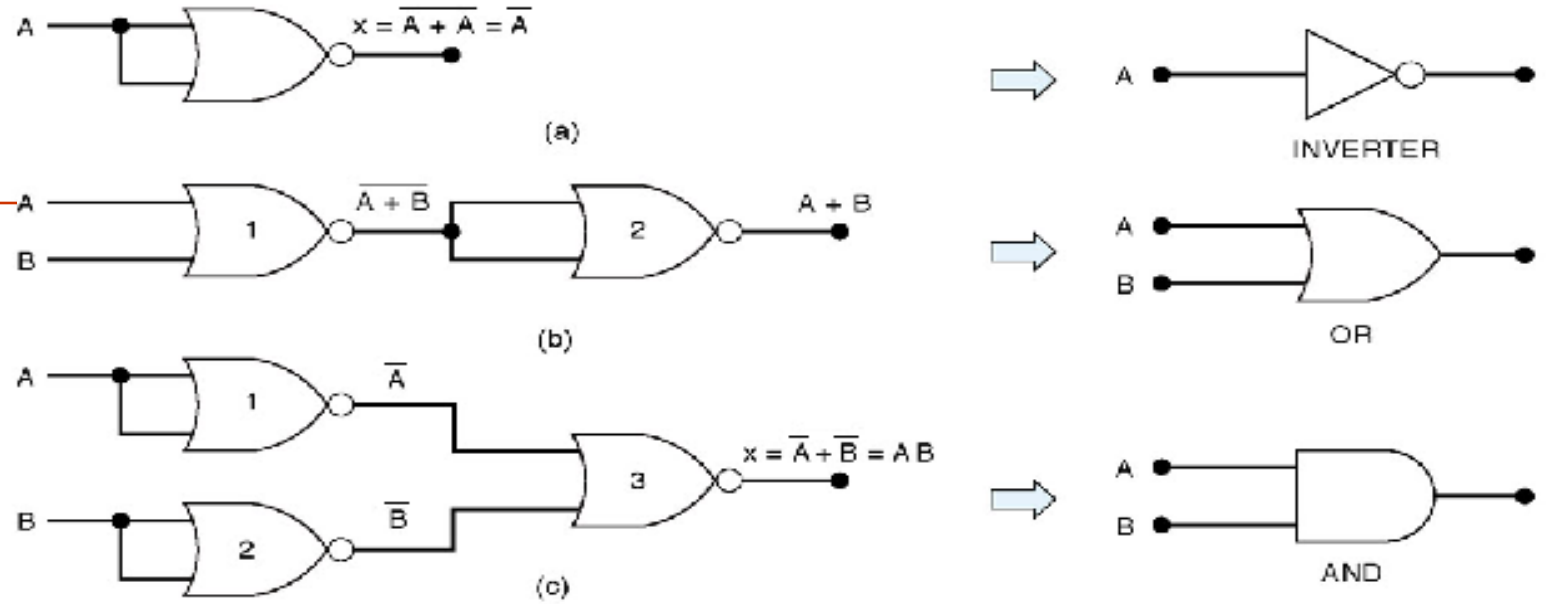


OR Gate



AND Gate

Universality of NOR gate



- Equivalent representations of the AND, OR, and NOT gates

(A+A)' = A'A' = A'

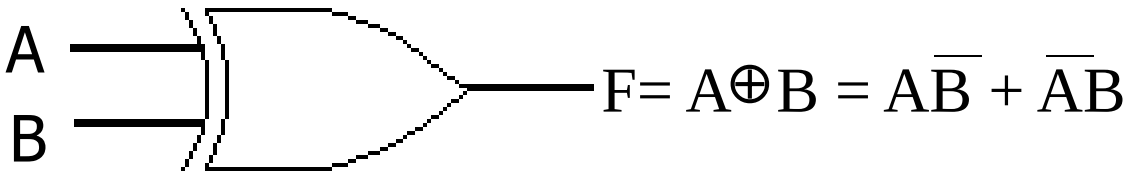
((A+B)' + (A + B)')' = (A+B)"(A+B)" = (A'B')' (A'B')'

= (A"+B") (A"+B") = (A+B) (A+B) = (A+B)

The XOR Gate (Exclusive-OR)

- This is a XOR gate.
- XOR gates assert their output when exactly one of the inputs is asserted, hence the name.
- The operator symbol for this operation is $\oplus$

  $1 \oplus 1 = 0$ and $1 \oplus 0 = 1$.

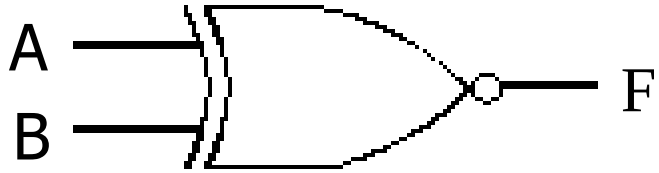


A	B	F
0	0	0
0	1	1
1	0	1
1	1	0

The XNOR Gate

- This functions as an exclusive-NOR gate, or simply the complement of the XOR gate.
- The symbol for this operation is ⊙

  1 ⊙ 1 = 1 and 1 ⊙ 0 = 0.



$$F = \overline{A \oplus B} = (AB) + (\overline{A} \cdot \overline{B}) = AB + A'B'$$

A	B	F
0	0	1
0	1	0
1	0	0
1	1	1

Standard Forms

- We have seen how to interpret truth tables, obtain Boolean expressions (functions) then build logic circuits.

- We have simplified Boolean expressions using Boolean algebra.

→There is a "standard" way of writing Boolean expressions (Functions):

- **The standard Sum of Products (SOP)**
- **The standard Product of Sums (POS)**

The standard Sum of Product-Minterms

- A Minterm is one in which all variables appear (only) once.
- Each Minterm represents exactly one combination (row) in truth table.
- n variables give 2^n Minterms.

Truth Table

Decimalvalue	A	B	C	F	Minterm
0	0	0	0	0	m_0
1	0	0	1	0	m_1
2	0	1	0	1	m_2
3	0	1	1	1	m_3
4	1	0	0	1	m_4
5	1	0	1	1	m_5
6	1	1	0	1	m_6
7	1	1	1	1	m_7

The standard Sum of Products-Function

- SOP are expressions of the form:

F (A,B,C, …) = (…) + (…) + (…) + …

- Brackets can contain single or multiple variables

- Such expressions can be implemented using:

F (A,B,C, …) = (AND's) OR (AND's) OR (AND's) OR ...

The standard Sum of Product-**Function**

• SOP form not unique, and doesn't necessarily contain all variables, for example:

F (A,B,C) = A'B'C' + A'BC + C'A'B + C'AB' + BAC + BAC

and F (A,B,C) = B + B'C'

are both valid SOP expressions.

The standard Sum of Product-**Function**

- We Can obtain SOP from truth table (below)

F(A,B,C) = A'BC' + A'BC + AB'C' + AB'C + ABC' + ABC

A simpler notation is to write as

$$F(A,B,C) = m_2 + m_3 + m_4 + m_5 + m_6 + m_7$$

$$= \sum m_i (2, 3, 4, 5, 6, 7)$$

Decimal value	A	B	C	F	Minterm	
0	0	0	0	0	m_0	A'B'C'
1	0	0	1	0	m_1	A'B'C
2	0	1	0	1	m_2	A'BC'
3	0	1	1	1	m_3	A'BC
4	1	0	0	1	m_4	AB'C'
5	1	0	1	1	m_5	AB'C
6	1	1	0	1	m_6	ABC'
7	1	1	1	1	m_7	ABC

Product of Sums: **Function**

- **From truth table we have** $\bar{F}(A,B,C) = (A'B'C' + A'B'C)$
- **Therefore we obtain F from $\bar{F}$:**

$$F(A,B,C) = \overline{[\bar{F}(A,B,C)]} = \overline{(A'B'C' + A'B'C)}$$

$$= (A' + B'' + C'')(A'' + B'' + C') = (A + B + C)\quad(A + B + C')$$

Form Compact $F = M_0 \cdot M_1 = \prod Mi\ (0, 1)$

Truth Table

Decimal value	A	B	C	F	Minterm	Maxterm	$M_i = \bar{m}_i$
0	0	0	0	0	m_0	M_0	A+B+C
1	0	0	1	0	m_1	M_1	A+B+C'
2	0	1	0	1	m_2	M_2	A+B'+C
3	0	1	1	1	m_3	M_3	A+B'+C'
4	1	0	0	1	m_4	M_4	A'+B+C
5	1	0	1	1	m_5	M_5	A'+B+C'
6	1	1	0	1	m_6	M_6	A'+B'+C
7	1	1	1	1	m_7	M_7	A'+B'+C'

Obtain SOP and POS from a given expression

- Given an arbitrary Boolean expression
- Work out number of terms (2^n) for n inputs.
- Generate truth table and identify terms for which the function is true - the Minterms.
- Write function as:

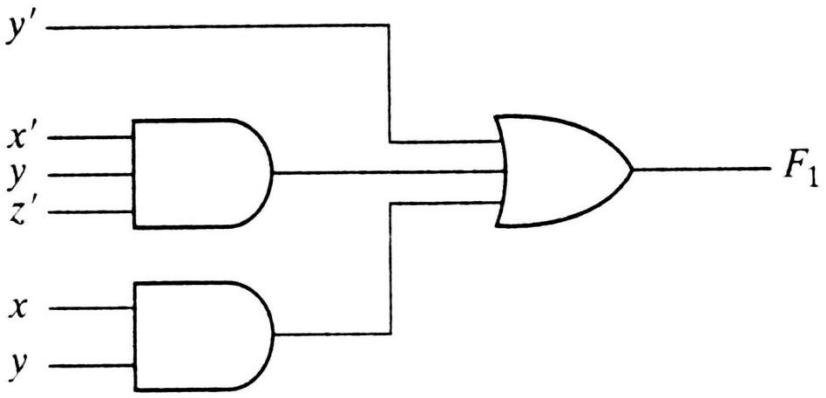
$$F = \sum_{i=0}^{n-1} m_i$$

- Alternatively, identify terms for which the function is false and use a Maxterm description.
- Write function :

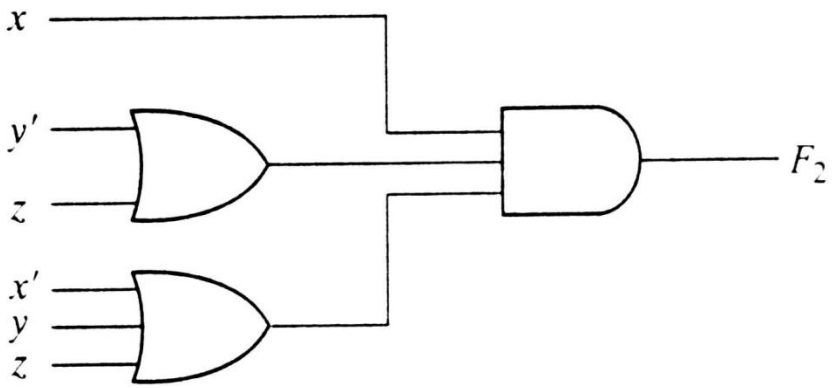
$$F = \prod_{j=0}^{n-1} M_j \quad j \neq i$$

SOP & POS Implementation using AND and OR

- Two-level implementation

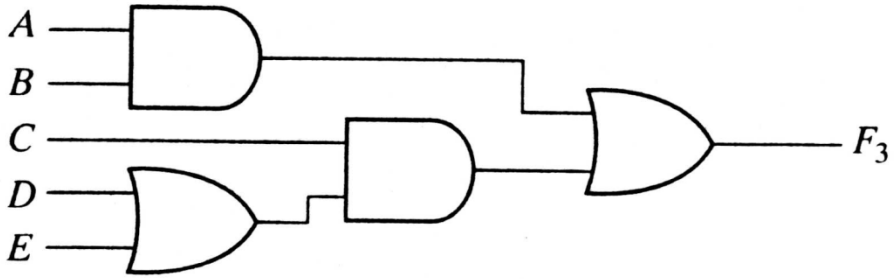


(a) Sum of Products



(b) Product of Sums

- Multi-level implementation



(a) $AB + C(D + E)$

Truth Table

A	B	C	C_o	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

Examples

1- Half Adder

2- Full Adder

3- Deriving SOP and POS from a truth table

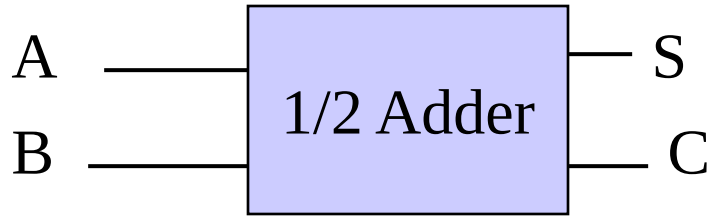
4- Simplifying SOP & POS using Boolean identities

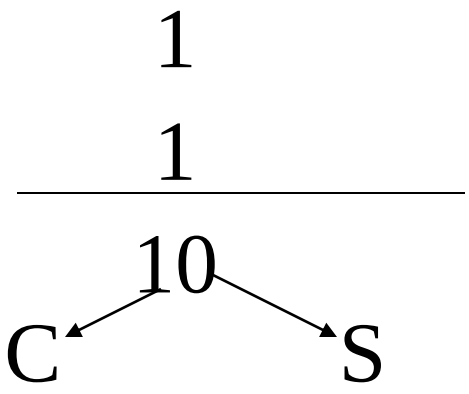
Half Adder

→The half-adder accepts two binary digits on its inputs and produces two binary digits on its outputs: a sum bit and a carry bit.

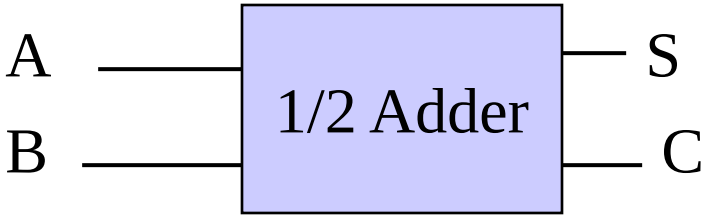
Truth Table

A	B	C	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0





Half-Adder



Truth Table

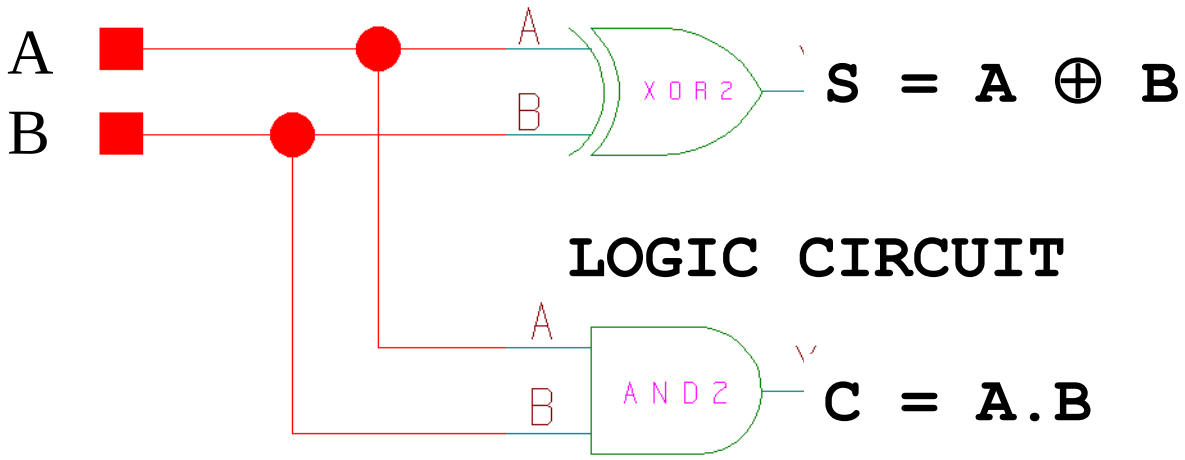
A	B	C	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

Logic Function

$S = A'B + AB'$

$S = A \oplus B$

$C = A.B$



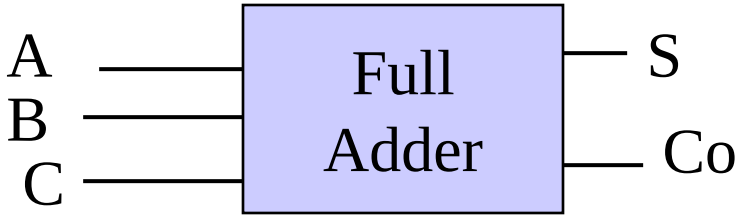
Full Adder

Truth Table

A	B	C	C_o	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

→ The Full-adder accepts two input bits and an input carry and generates a sum output and an output carry

→ Basic difference between a full and a half adder is that the full adder accepts an input carry



Full-Adder

Logic FUNCTION

Truth Table

A	B	C	C_o	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

$$C_o = A'BC + AB'C + ABC' + ABC$$

$$= C[A'B + AB'] + AB[C' + C]$$

$$= C[A \oplus B] + AB.1$$

$$= C(A \oplus B) + AB$$

$$S = A'B'C + A'BC' + AB'C' + ABC$$

$$= A'[B'C + BC'] + A[B'C' + BC]$$

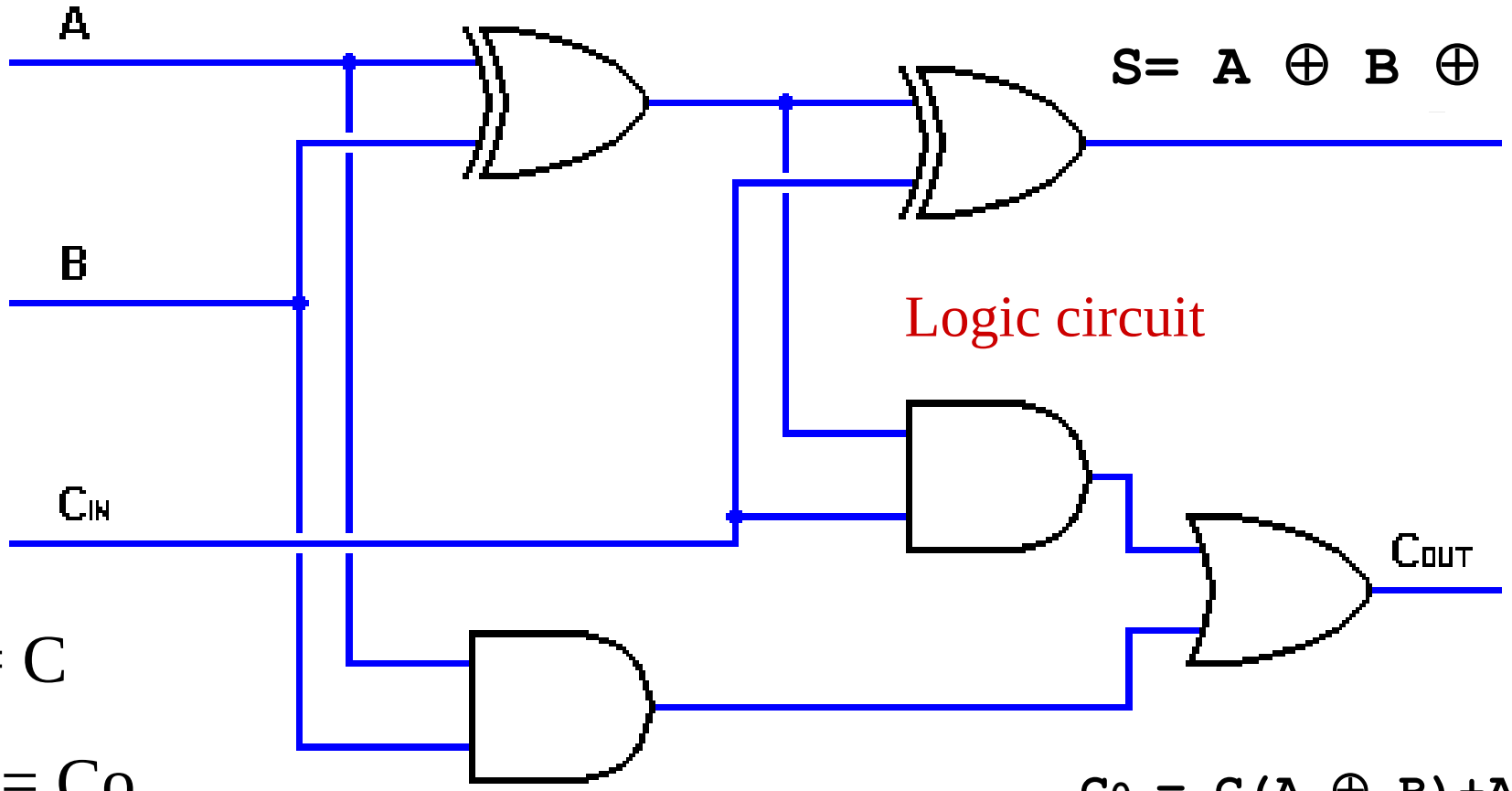
$$= A'[B \oplus C] + A[B \oplus C]'$$

$$= A' \underbrace{[X]}_{X} + A \underbrace{[X]'}_{x'}$$

$$= A \oplus X$$

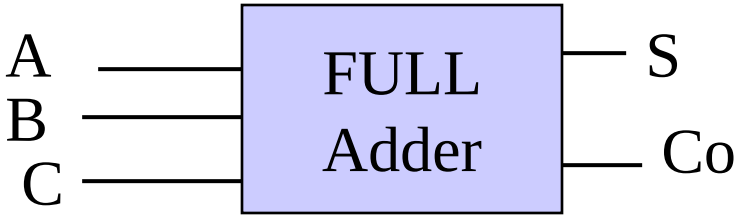
$$= A \oplus B \oplus C$$

Full Adder

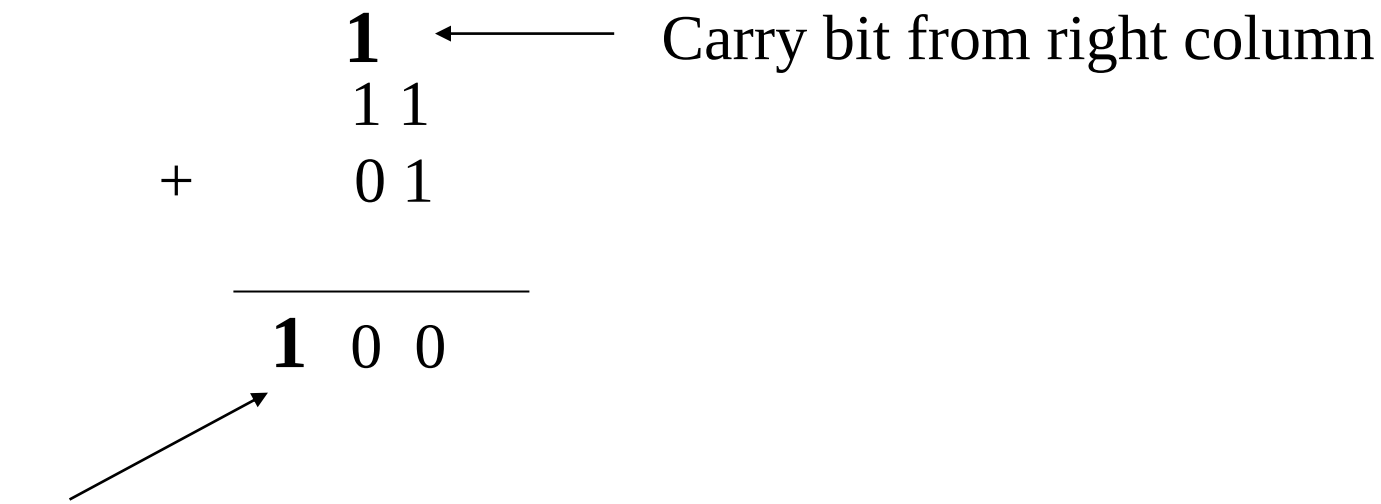


Cin = C

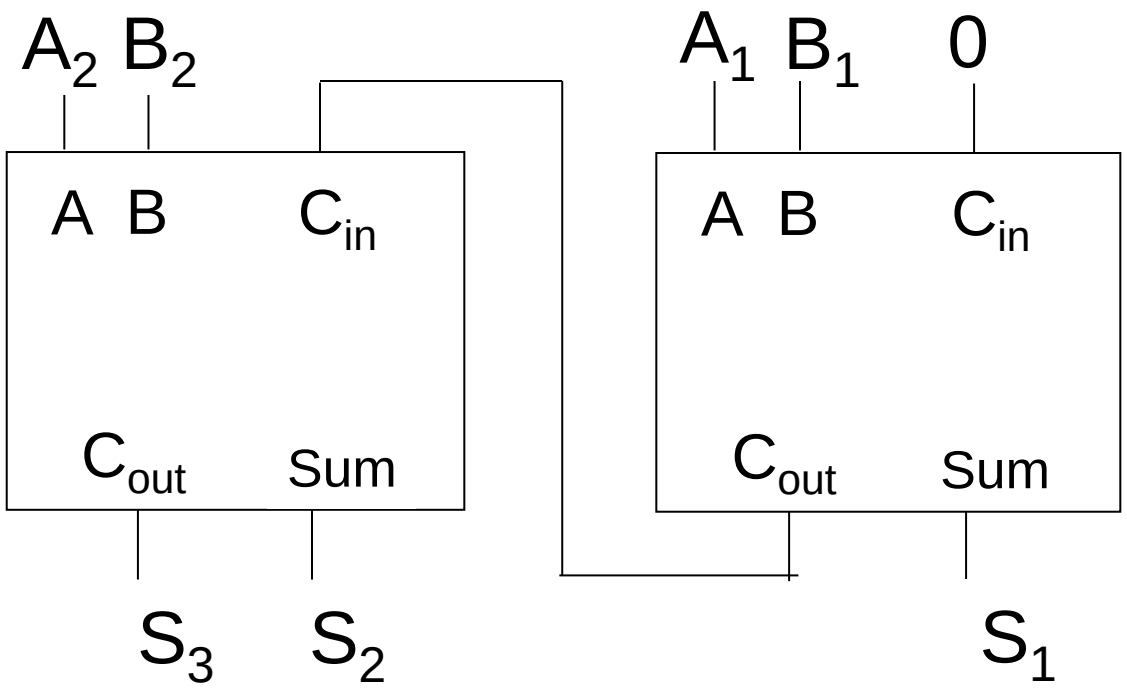
Cout = Co



Two bit parallel adder

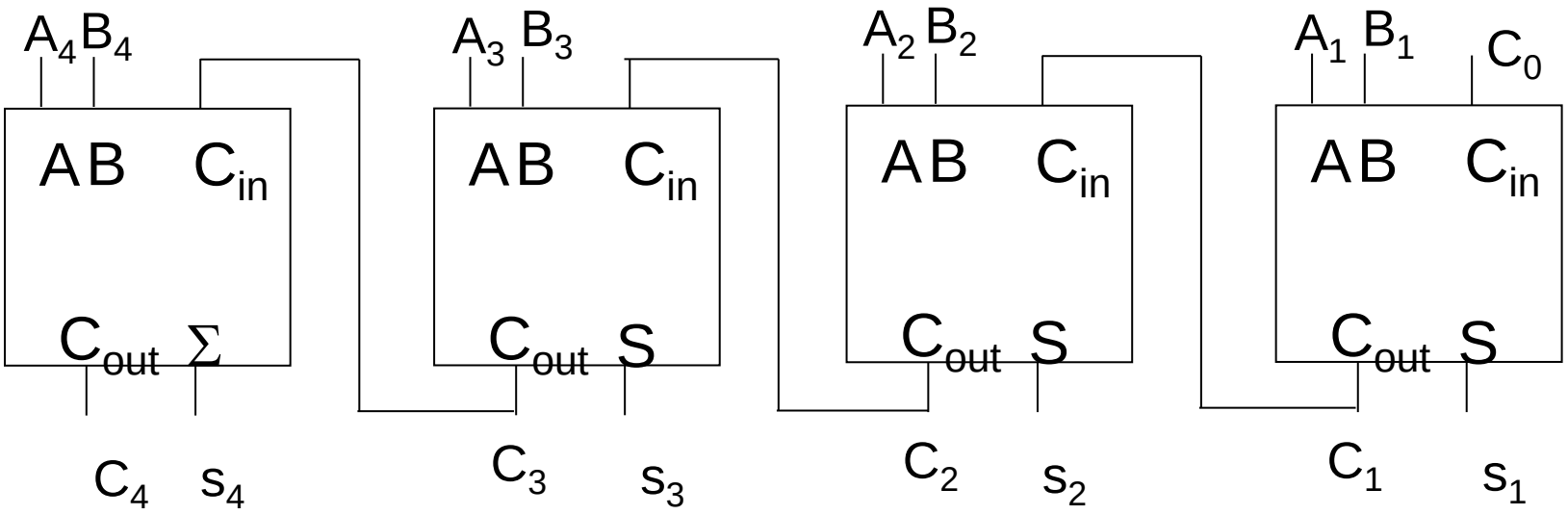


Two bit parallel adder



$$\begin{array}{r} A_2A_1 \\ +\ B_2B_1 \\ \hline S_3\,S_2\,S_1 \end{array}$$

Four bit parallel adder



Overflow Examples (review from chapter 1)

- In a 6-bit register

  + 17 = 010001

  + 16 = +010000

  =100001 → Overflow

- **100001 = 2's : - (11111) = -$(31)_{10}$** instead of **+ $(33)_{10}$**
- Same with a 7-bit register

  + 17 = 0 010001

  + 16 = +0 010000

  = 0 100001

  0100001 = + 33 No Overflow

Four bit parallel adder

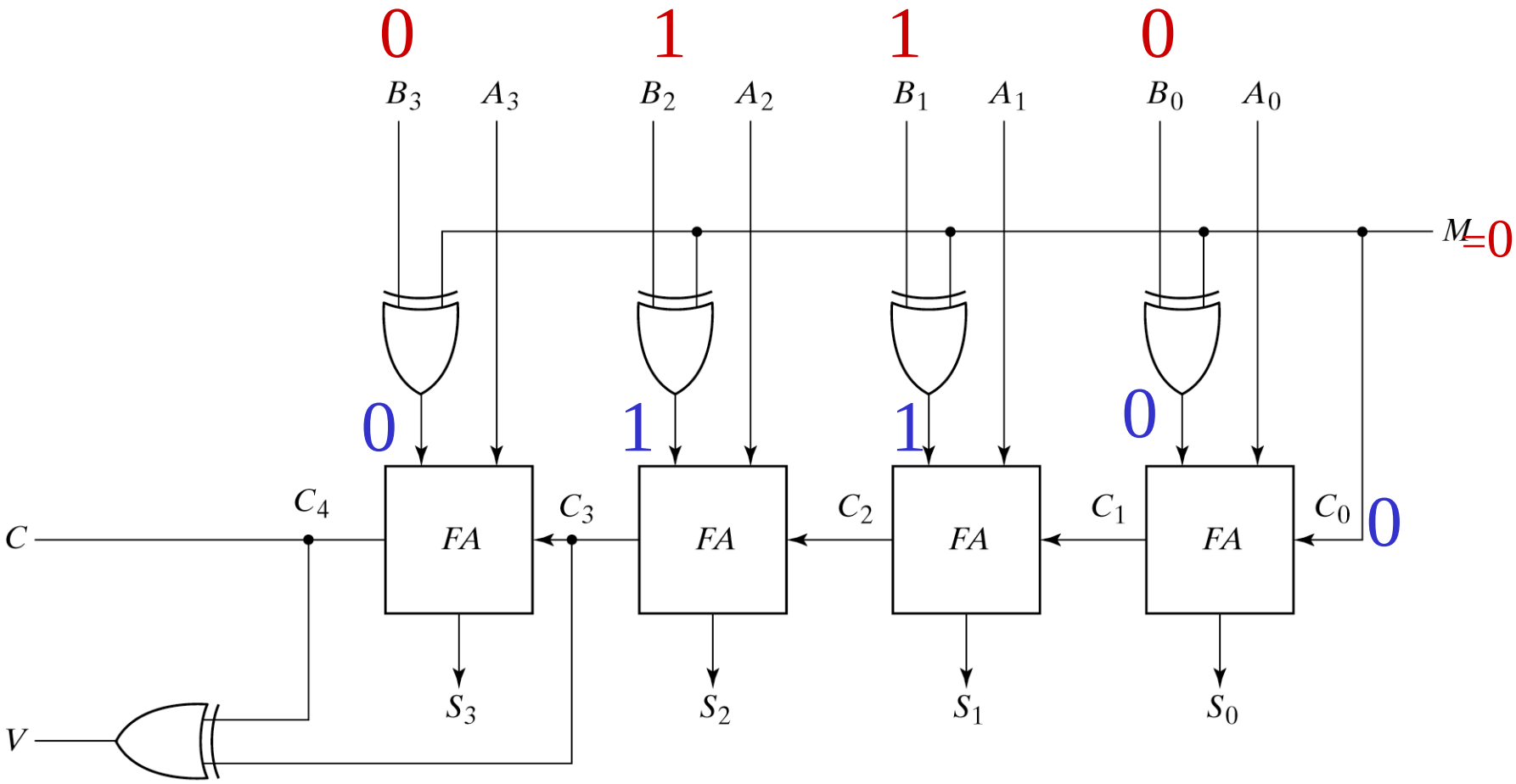


Fig. 4-13 4-Bit Adder Subtractor

Four bit parallel adder

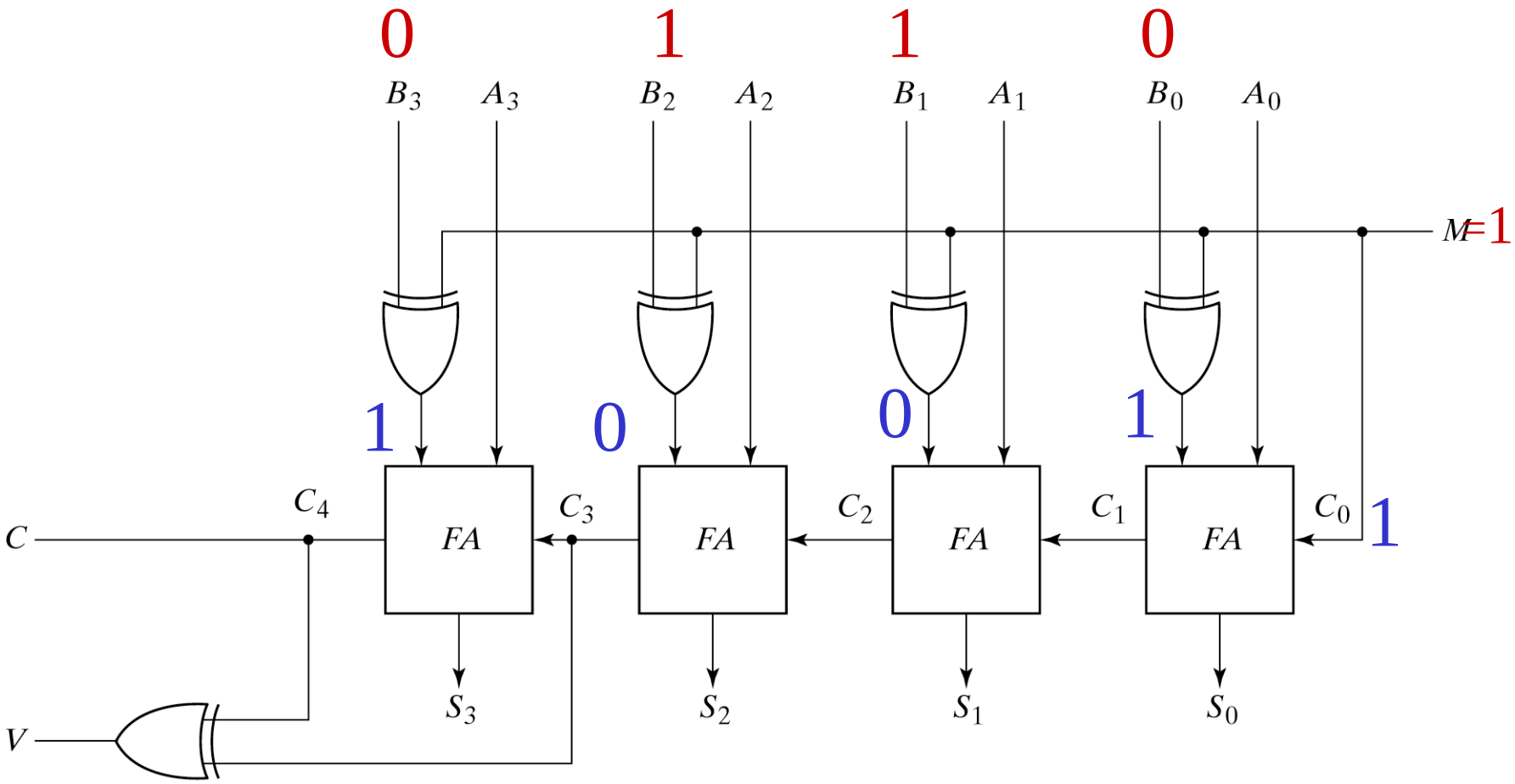
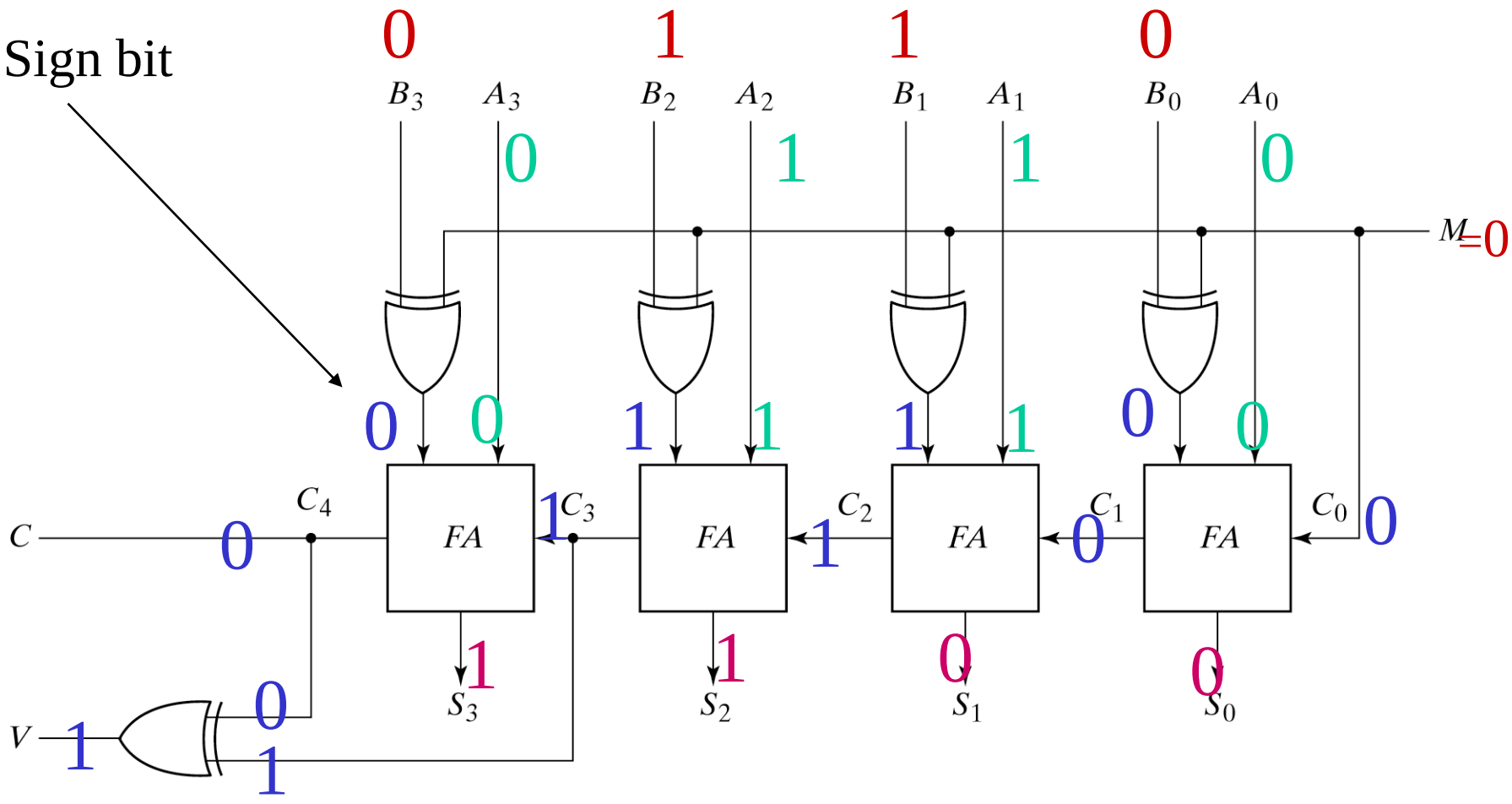


Fig. 4-13 4-Bit Adder Subtractor

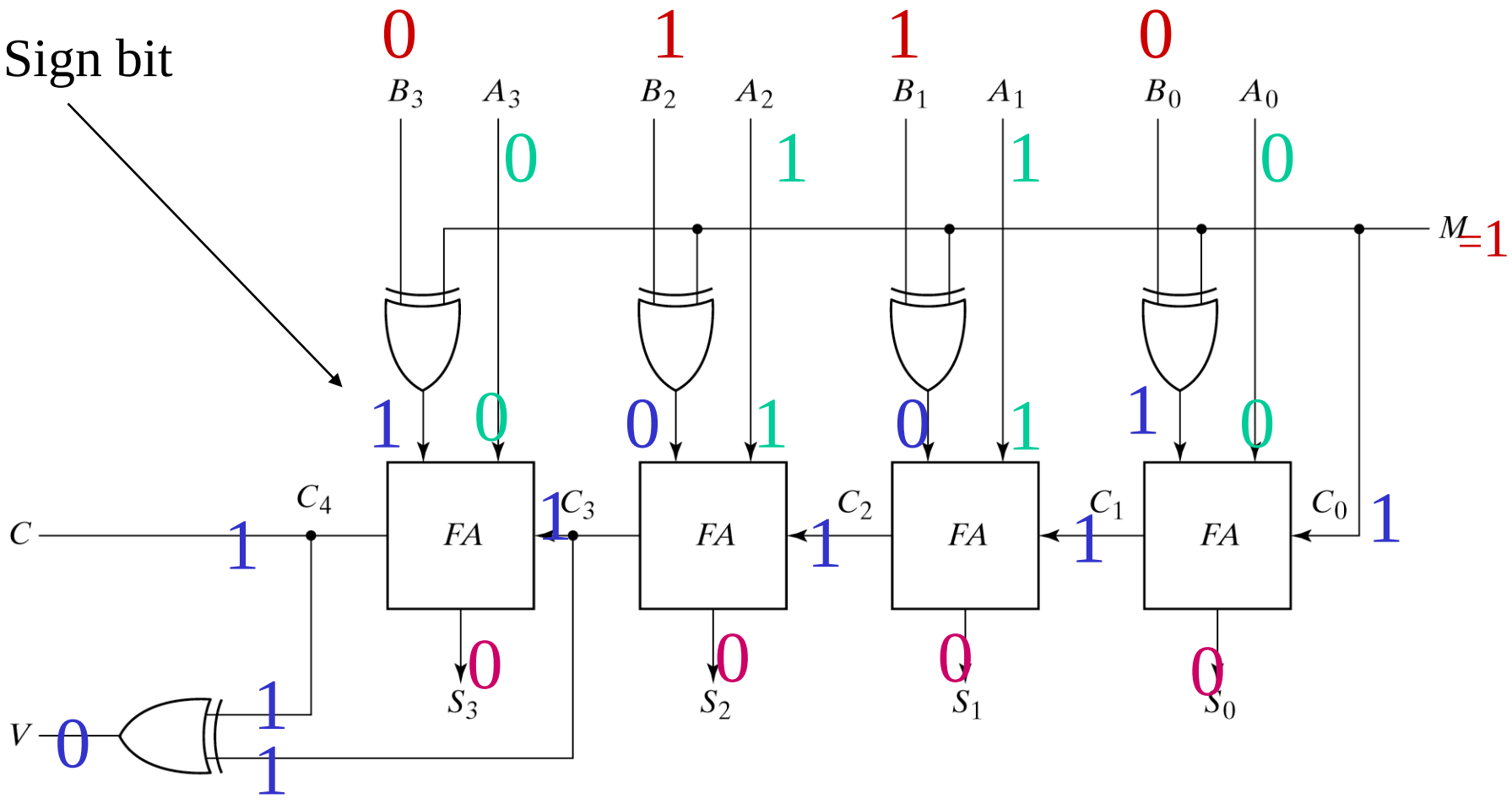
Four bit parallel adder: more examples



Overflow

Fig. 4-13 4-Bit Adder Subtractor

Four bit parallel adder: more examples



NO Overflow

Fig. 4-13 4-Bit Adder Subtractor

SOP & POS Standard Forms- Example

From an arbitrary Truth table (next slide)

- **Part one**

1- obtain SOP representation for F

2- obtain the two level implementation for F without simplification

3- simplify F using Boolean identities

4- obtain two level-implementation for F

5- compare the design obtained in question 4 with the one of question 2

- **Part two**

Repeat part one using POS

Deriving SOP and POS from a truth table

Consider the following arbitrary Truth Table

i) SOP

ii) POS

i	A	B	C	F	Minterms
0	0	0	0	0	
1	0	0	1	0	
2	0	1	0	1	**→m_2 = A'BC'**
3	0	1	1	1	**→m_3 = A'BC**
4	1	0	0	0	
5	1	0	1	1	**→m_5 = AB'C**
6	1	1	0	0	
7	1	1	1	1	**→m_7 = ABC**

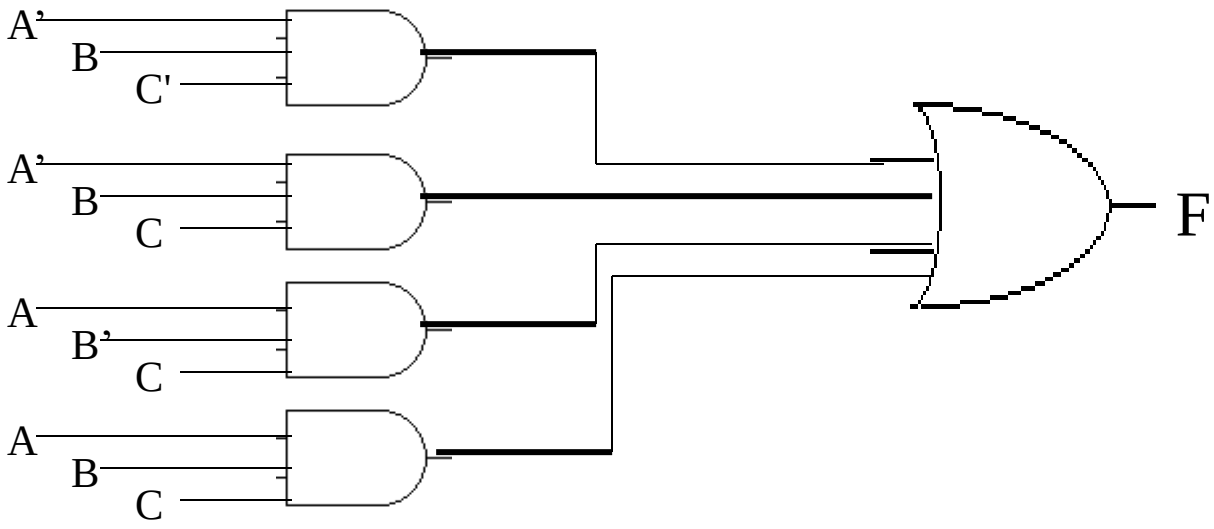
Deriving SOP from a truth table

1- **Expression sum of products**

a) F = m2 + m3 + m5 + m7

= A’BC’ + A’BC + AB’C + ABC

b) Implementation with logic gates (unsimplified)



Two level-Implementation

Simplifying SOP using Boolean identities

c) Simplification

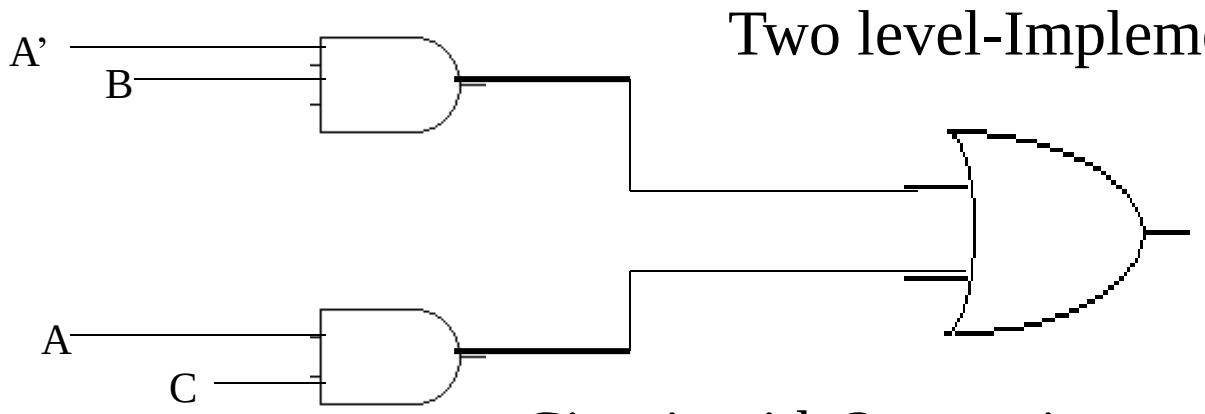
Using the Boolean identity absorption $xy + xy' = x$

We can simplify m_2 with m_3 and m_5 with m_7

$m_2 + m_3 = (A'B)\,C' + (A'B)\,C = A'B$

$m_5 + m_7 = (AC)B' + (AC)B = AC$

Therefore **$F = A'B + AC$**



Two level-Implementation

Circuit with 3 gates instead of 5

Deriving POS from a truth table

2- Expression Product of sums

i	A	B	C	F	Maxterms
0	0	0	0	0	→M_0 = A+B+C
1	0	0	1	0	→M_1 = A+B+C’
2	0	1	0	1	
3	0	1	1	1	
4	1	0	0	0	→M_4 = A’+B+C
5	1	0	1	1	
6	1	1	0	0	→M_6 = A’+B’+C
7	1	1	1	1	

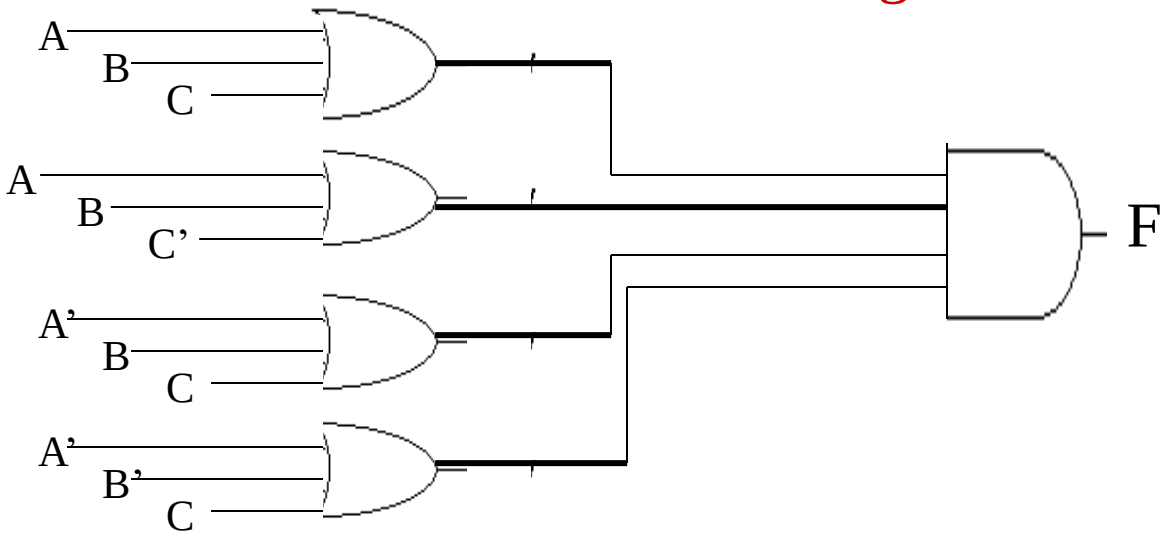
Deriving SOP from a truth table

a) Function

$F = M_0 \; . M_1 \; . M_4 \; . M_6$

$= (A+B+C)\,(A+B+C')(A'+B+C)(A'+B'+C)$

Logic circuit



Two level-Implementation

Simplifying POS using Boolean identities

b) Simplification

Using (X + Y) (X+Y') = X

Verification

X	Y	Y'	X+ Y	X + Y'	(X + Y) (X+Y')
0	0	1	0	1	0
0	1	0	1	0	0
1	0	1	1	1	1
1	1	0	1	1	1

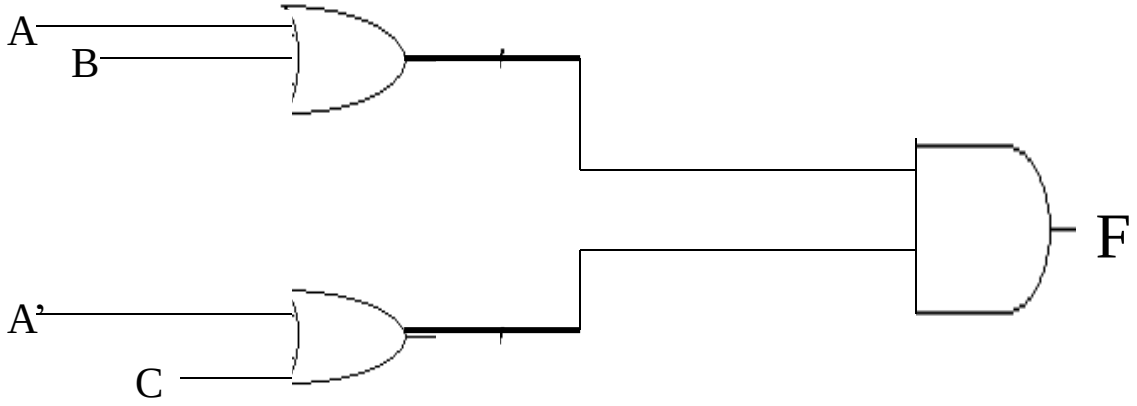
M0.M1 = [(A+B) + C] [(A+B) + C']= **A+B**

M4.M6 = [A' + C) + B] [(A'+C) + B'] = **A'+C**

F = (A+B)(A'+C)

Simplifying POS using Boolean identities

C) Two level Implementation



Gate-Level Minimization

The Karnaugh MAP

- An alternative approach to represent Boolean functions
- used to minimize Boolean functions
- Easy conversion from truth table to K-map
- Easy to obtain minimized SOP function.
- Simple steps used to perform minimization

→ **Much faster and more efficient than previous minimization techniques with Boolean algebra.**

The Karnaugh MAP

- K-MAP is ideally suited for four or less variables, becoming cumbersome for five or more variables.

→Each square represents a Minterm

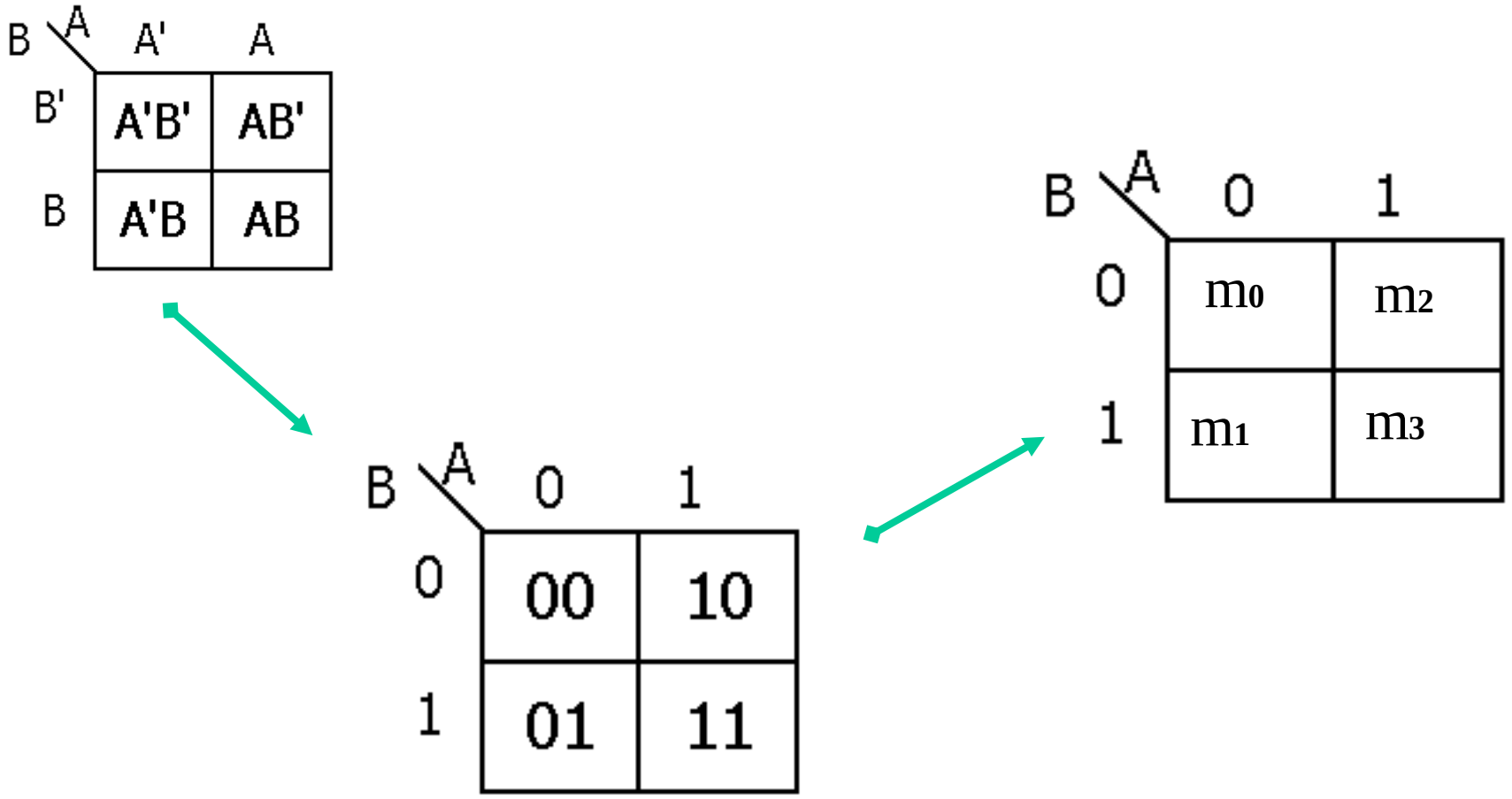
→*Map is arranged such that two neighbors differ in only one variable (e.g. ABC +ABC')*

→ *Two terms must be "adjacent" in the map*

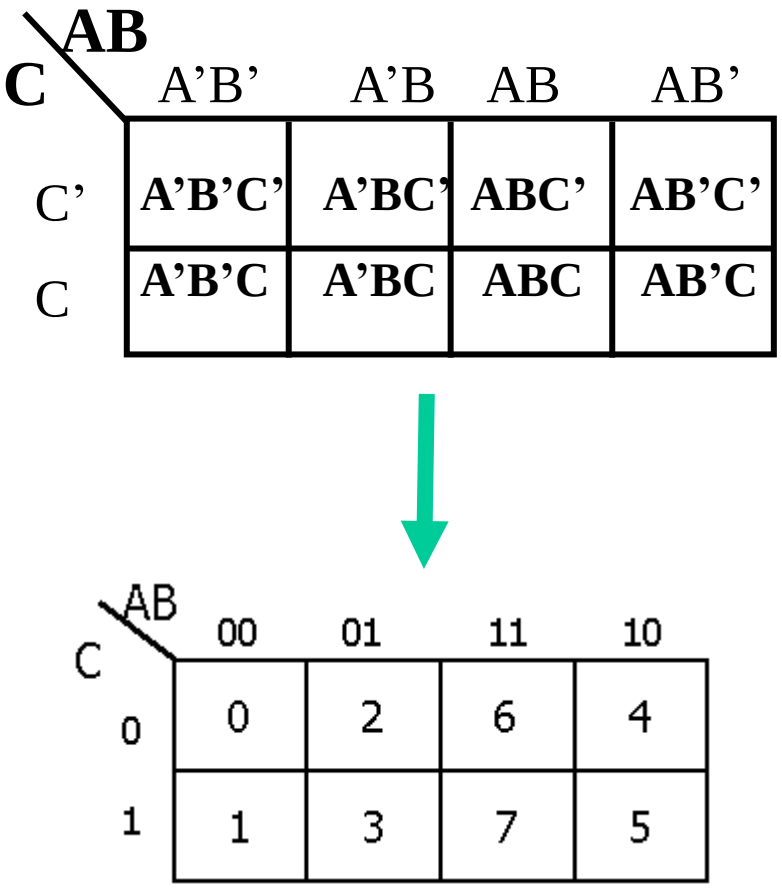
→ A K-map of n variables will have 2^n squares

→ For a Boolean expression, product terms are denoted by 1's, while sum terms are denoted by 0's – or left blank (represented by Minterms in the map)

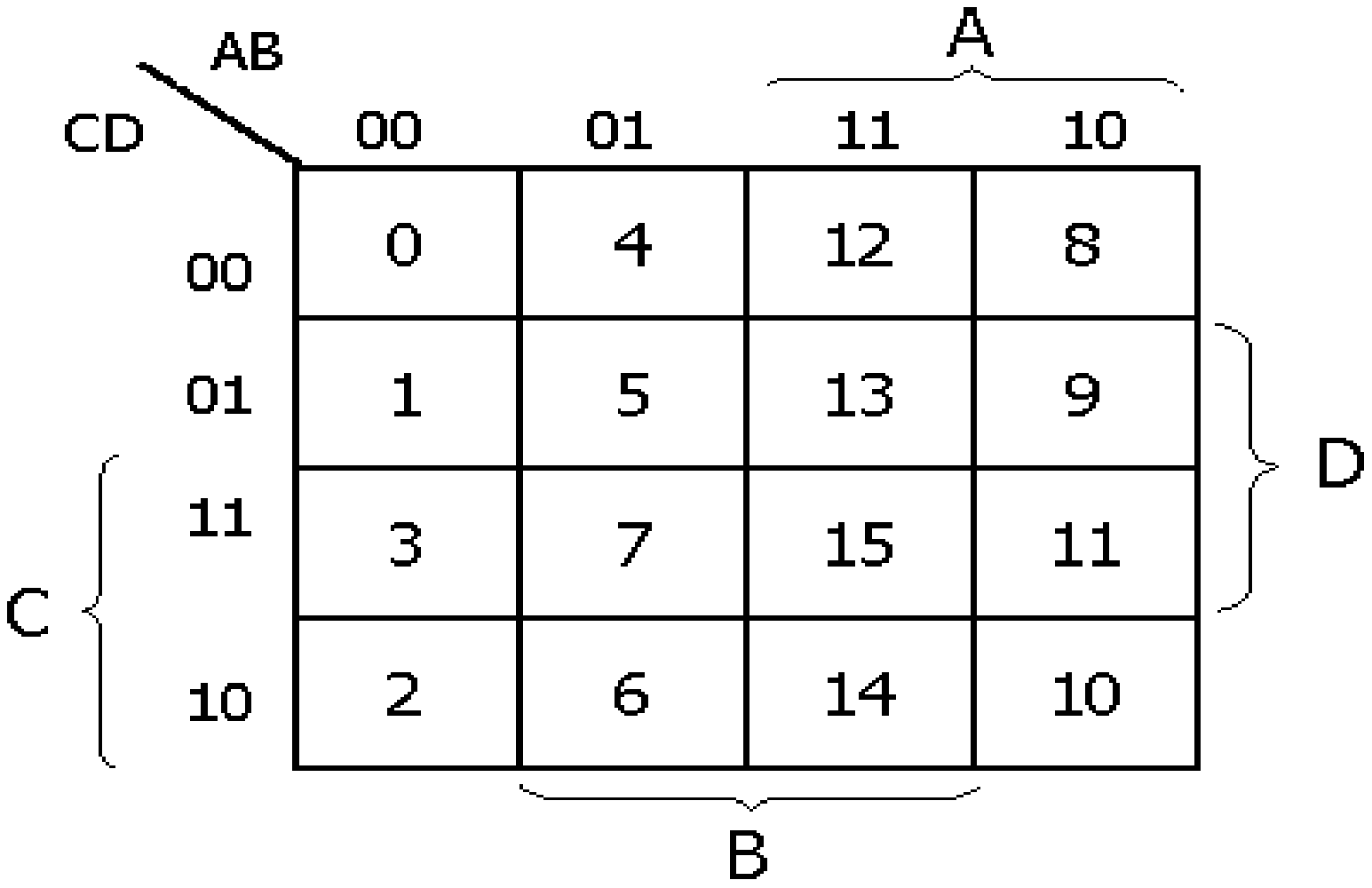
K-Map with Two variables



K-Map with 3 variables



Kmap With 4 variables



Assigning 1's and 0's in Kmap

•Assign the value of the outputs to the corresponding Minterms in the K-map

F (A,B,C,D)= A'B'C'D'+A'BC'D'+ AB'C'D' + A'BC'D+ ABC'D + ABCD'+AB'CD'

CD \ AB	00	01	11	10
00	1	1	0	1
01	0	1	1	0
11	0	0	0	0
10	0	0	1	1

→Consider the squares with 1's to simplify SOP

→ Consider the squares with 0's to simplify POS

Karnaugh Maps - grouping squares

- **Groups of squares are formed in considering the following rules:**

-Every square containing 1 must be considered at least once

- A square containing 1 can be included in as many groups as desired

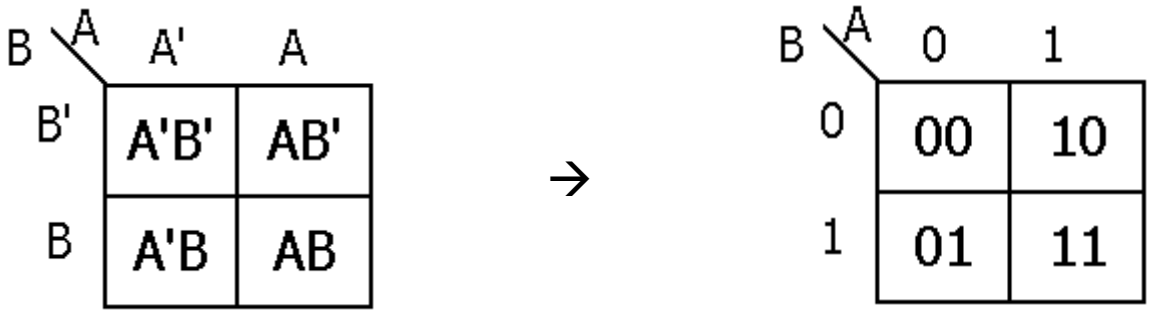
- A group must be as large as possible (i.e. large number of squares)

- The number of squares in a group must be equal to 2^n, i.e. 2,4,8,.

→ the simplified logic expression obtained from a K-map is not always unique. Groupings can be made in different ways.

2 variable Karnaugh Map

$x + x' = 1$



F=AB’+AB

F=A’B’+A’B

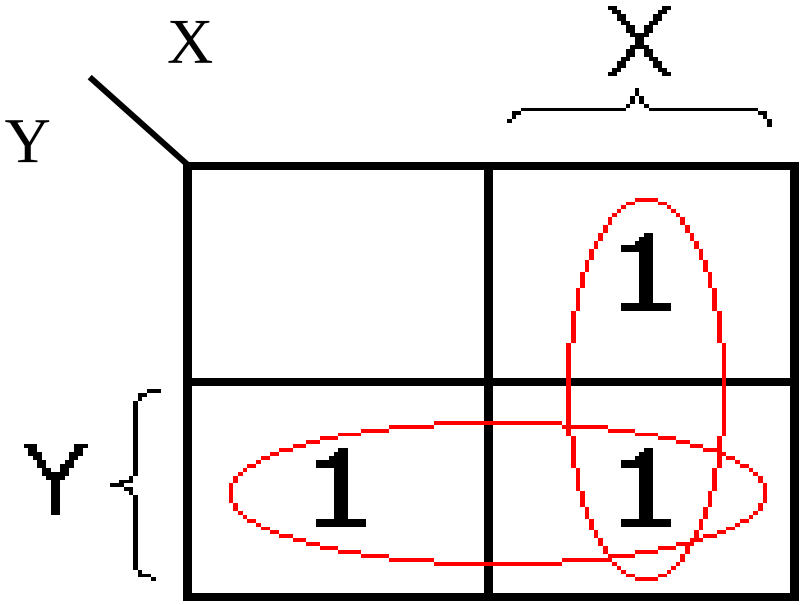


F = A

F = A’

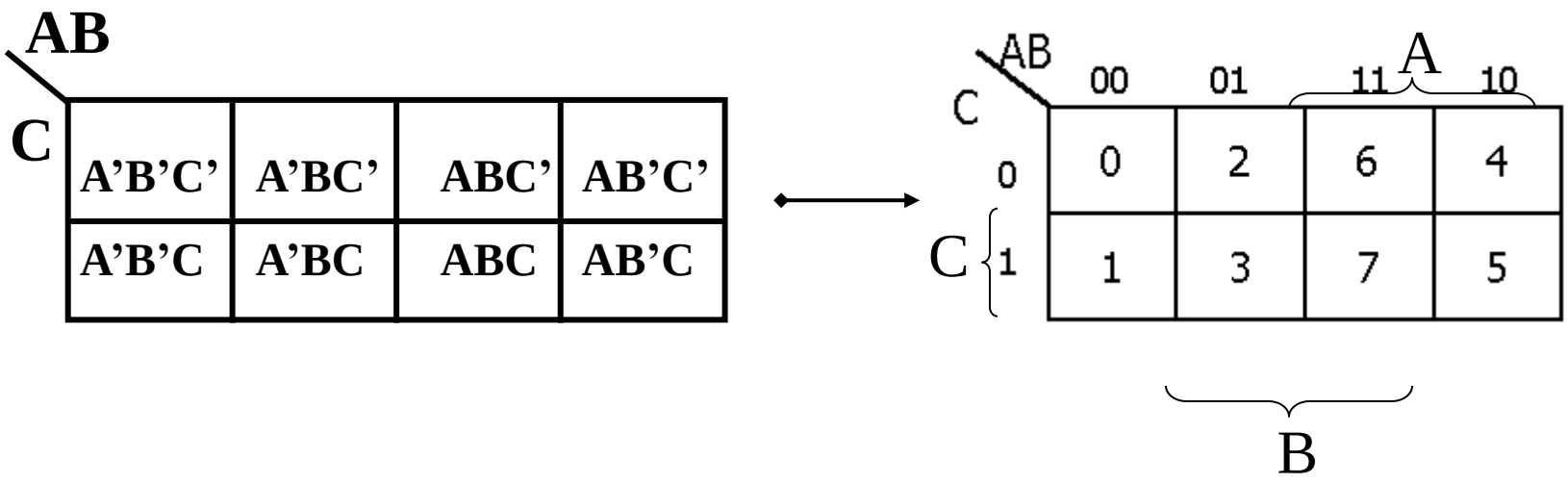
2 variable Karnaugh Map

F =X’Y+XY+XY’

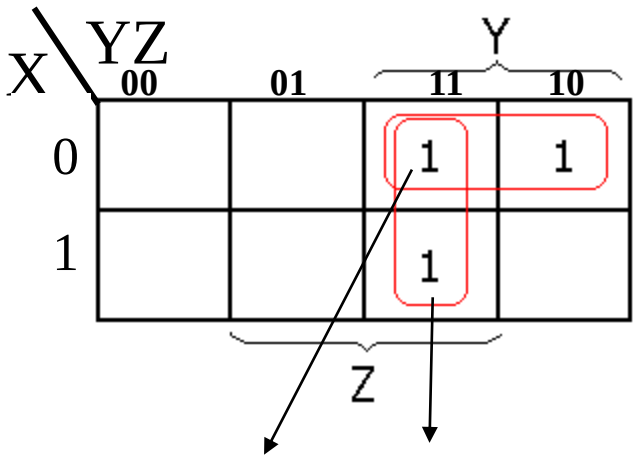


F = X + Y

3 Variable Karnaugh Map

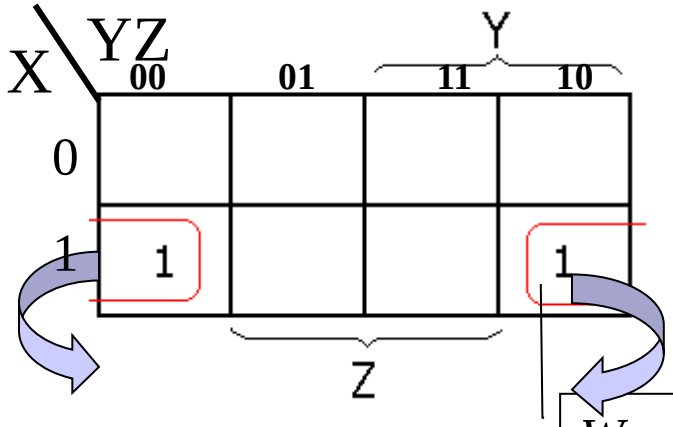


F = X'YZ'+XYZ+X'YZ



F = X'Y+YZ

F = XY'Z'+XYZ'

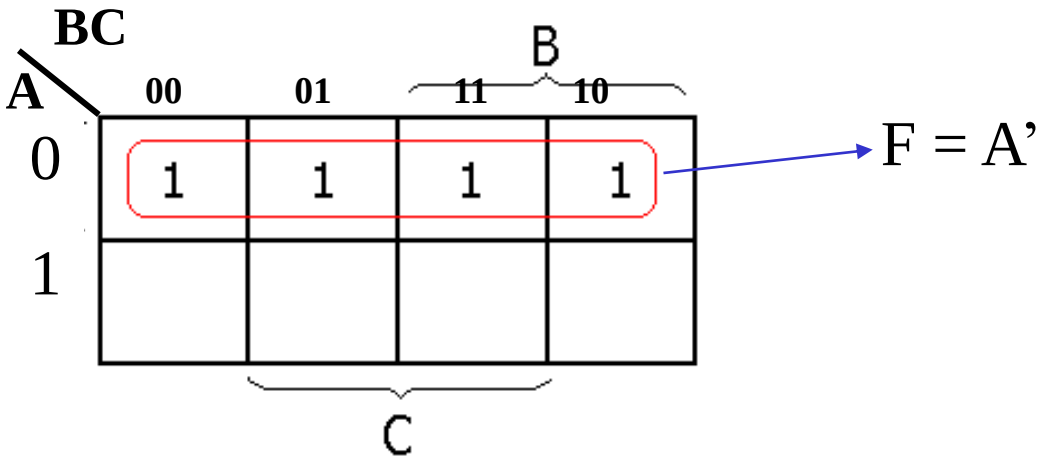


F = XZ'

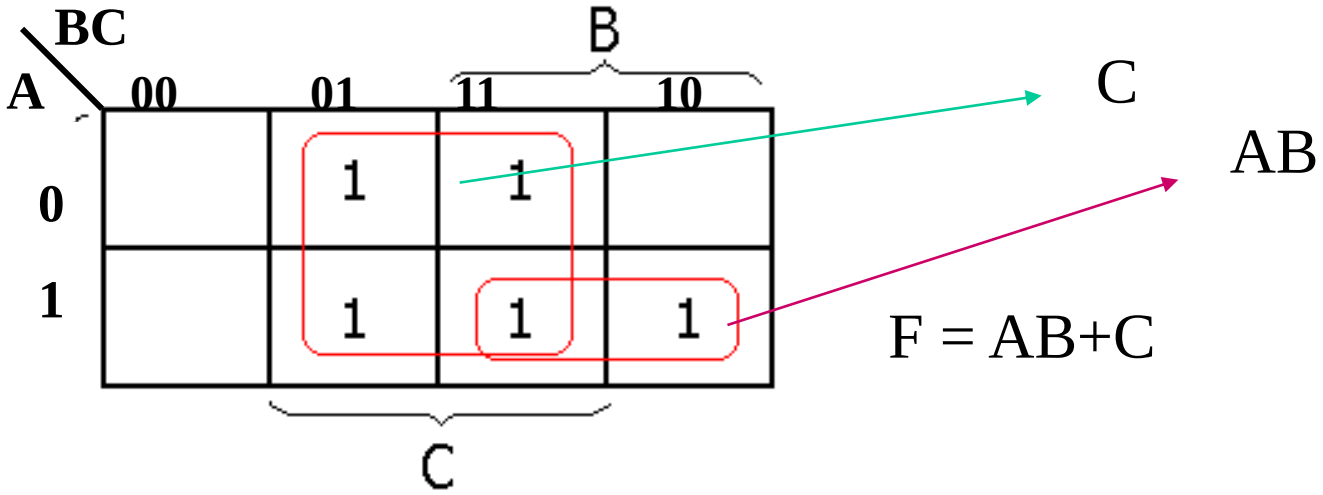
Wrapping around edges

3 variable Karnaugh Map

F(A,B,C) = A'BC'+A'B'C'+A'BC+A'B'C

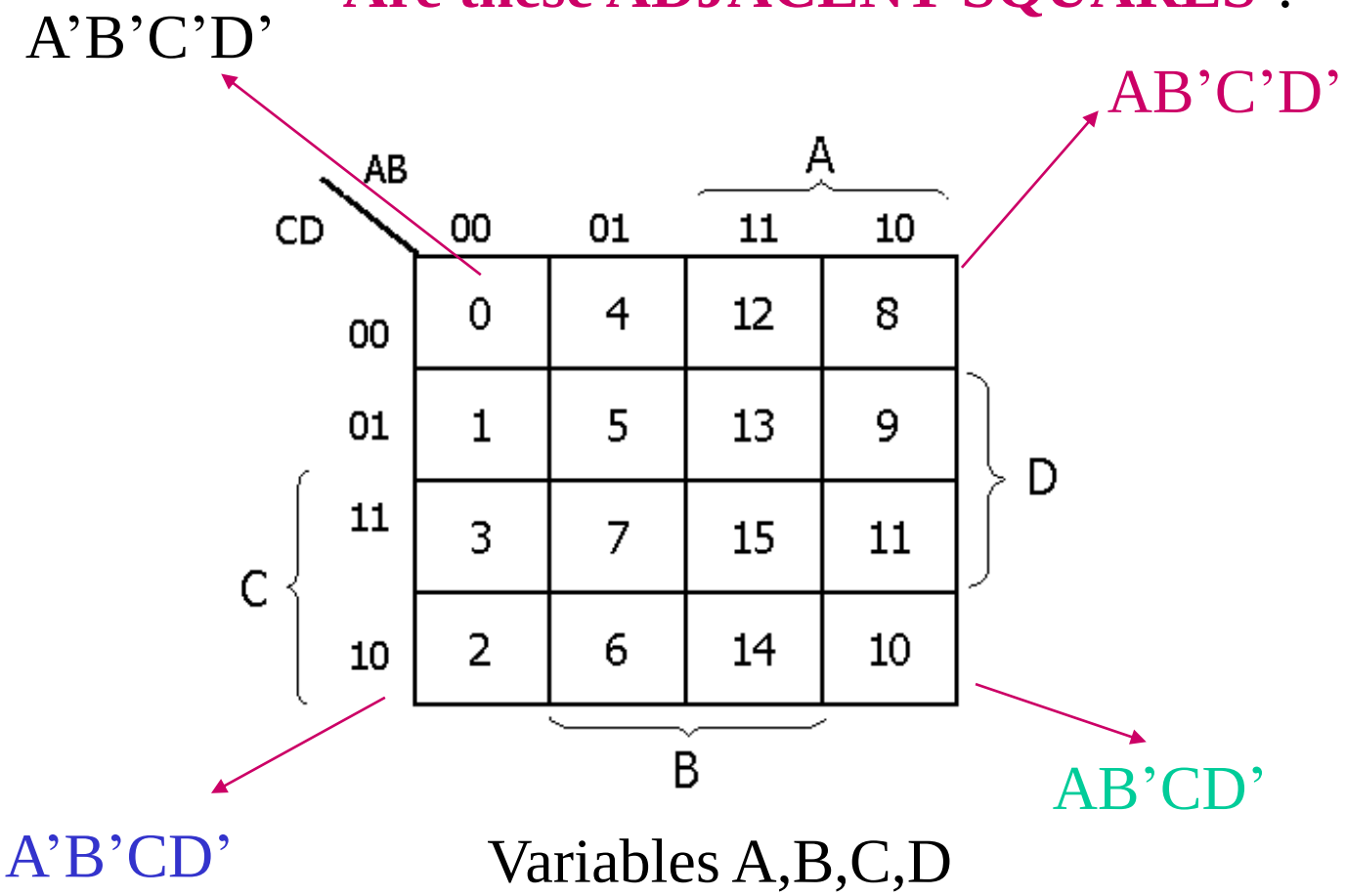


F(A,B,C) = A'BC+A'B'C+AB'C+ABC+ABC'



4 variables K-MAP

Are these ADJACENT SQUARES ?



Function with "don't care" Outputs

- Example

A purely binary number is converted into a 5-4-2-1 BCD number recall that BCD is often used to represent numbers in computers. The truth table is as below.

A	B	C	D	W	X	Y	Z
0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	1
0	0	1	0	0	0	1	0
0	0	1	1	0	0	1	1
0	1	0	0	0	1	0	0
0	1	0	1	1	0	0	0
0	1	1	0	1	0	0	1
0	1	1	1	1	0	1	0
1	0	0	0	1	0	1	1
1	0	0	1	1	1	0	0
1	0	1	0				
1	0	1	1				
1	1	0	0				
1	1	0	1				
1	1	1	0				
1	1	1	1				

Function with "don't care" Outputs

- Example

A purely binary number is converted into a 5-4-2-1 BCD number recall that BCD is often used to represent numbers in computers. The truth table is as below.

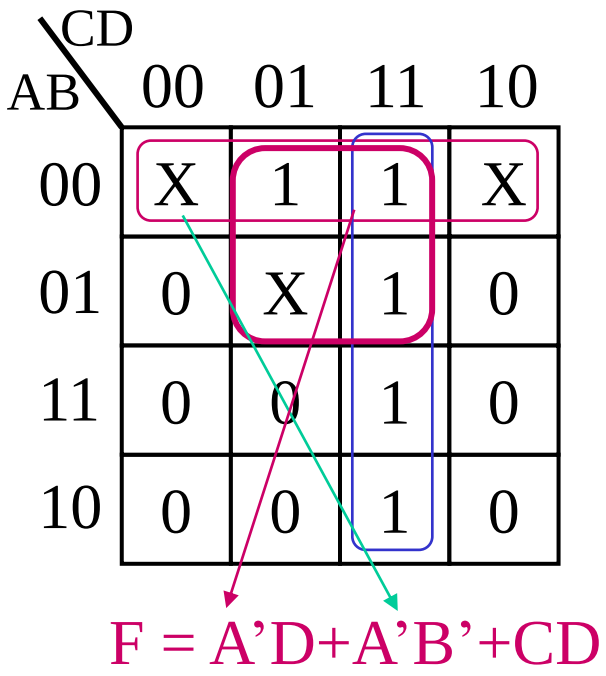
A	B	C	D	W	X	Y	Z
0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	1
0	0	1	0	0	0	1	0
0	0	1	1	0	0	1	1
0	1	0	0	0	1	0	0
0	1	0	1	1	0	0	0
0	1	1	0	1	0	0	1
0	1	1	1	1	0	1	0
1	0	0	0	1	0	1	1
1	0	0	1	1	1	0	0
1	0	1	0				
1	0	1	1				
1	1	0	0				
1	1	0	1				
1	1	1	0				
1	1	1	1				

Don't care terms

Σd(10,11,12,13,14,15) are don't care outputs for W, X, Y,Z

K-map with Don’t Care outputs

- Don’t care outputs can be either 0 or 1.
- This can be used to help simplify logic functions.
- Example: F(A,B,C,D)= Σ m(1,3,7,11,15 with Σ d(0,2,5)



•X denotes a “don’t care” term.

• X are used as 1’s or 0’s to increase the number of squares during the grouping

Solution to the 5-4-2-1 BCD example

A	B	C	D	W	X	Y	Z
0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	1
0	0	1	0	0	0	1	0
0	0	1	1	0	0	1	1
0	1	0	0	0	1	0	0
0	1	0	1	1	0	0	0
0	1	1	0	1	0	0	1
0	1	1	1	1	0	1	0
1	0	0	0	1	0	1	1
1	0	0	1	1	1	0	0
1	0	1	0				
1	0	1	1				
1	1	0	0				
1	1	0	1				
1	1	1	0				
1	1	1	1				

Don’t care terms

Using K-maps for the 4 variable we obtain:

W=A+BD+BC

X=BC’D’+AD

Y=CD+B’C+AD’

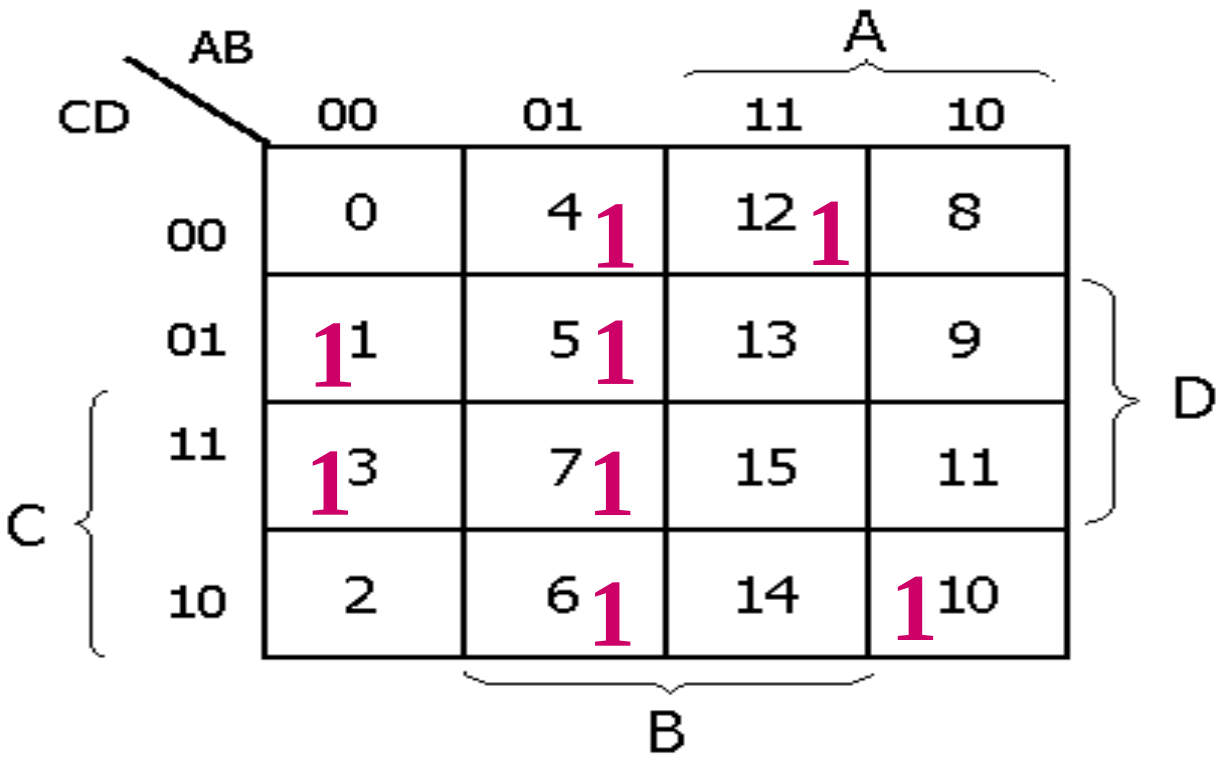
Z=AD’+A’B’D+BCD’

K-Maps- Examples

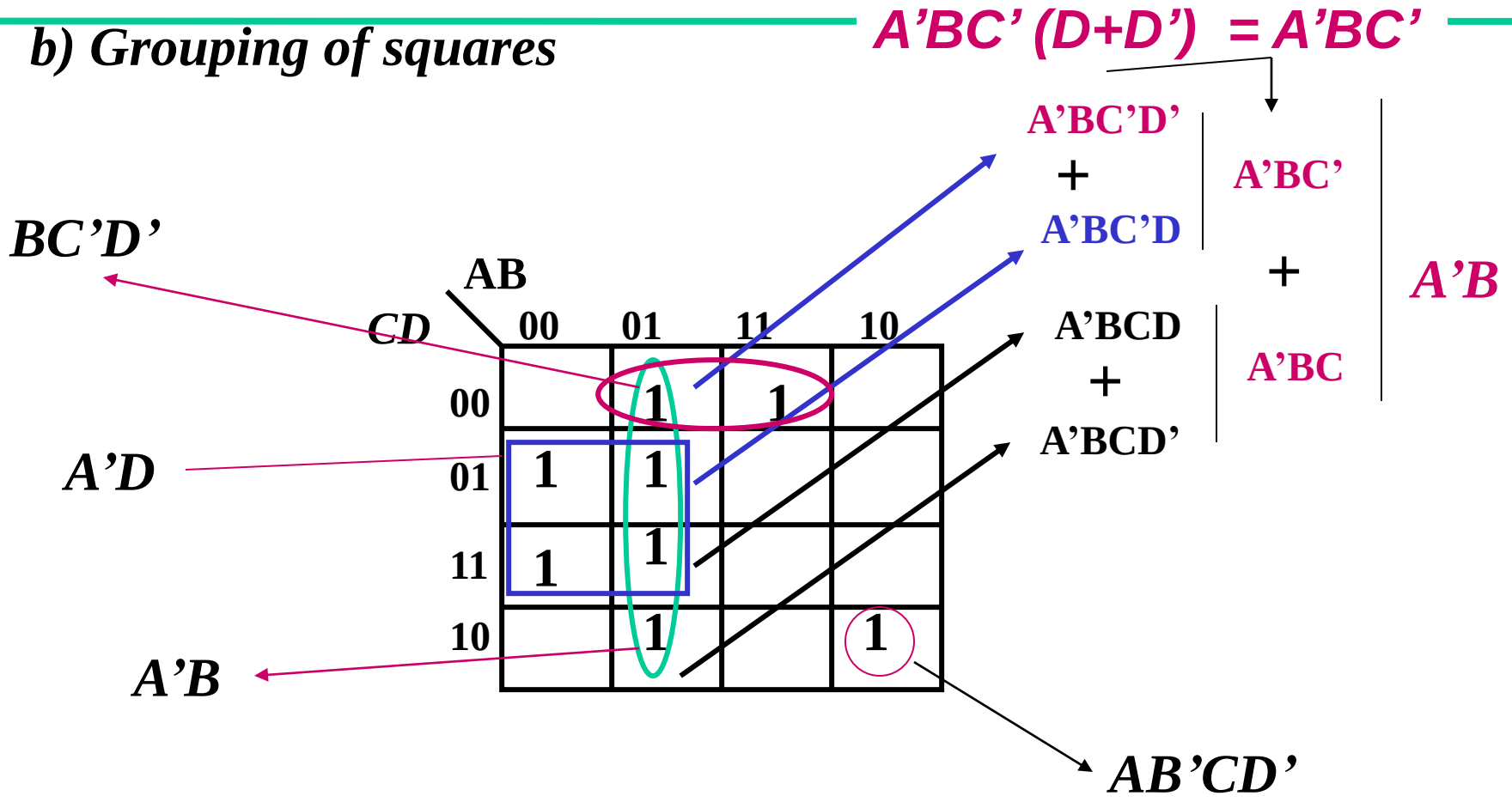
1- simplify the following expression using K-Maps

$$F(A,B,C,D) = \sum m\,(1, 3, 4, 5, 6, 7, 10, 12)$$

a) Building K-Map for F



b) Grouping of squares



c) Write the Simplified Expression

$$F(A,B,C,D) = A'B + A'D + BC'D' + AB'CD'$$

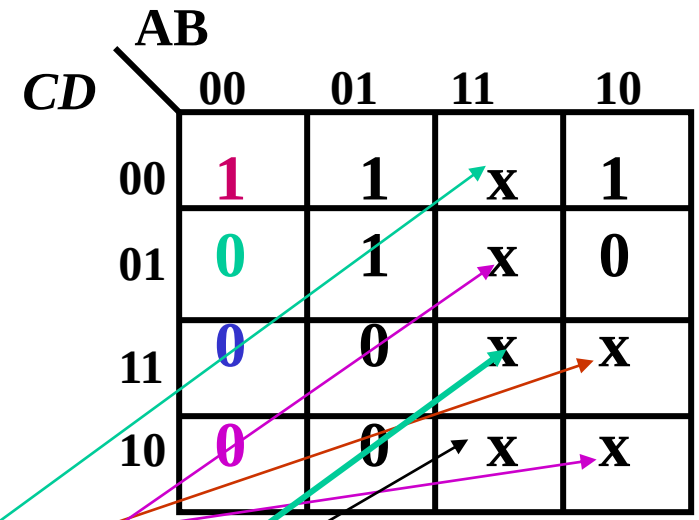
A	*B*	*C*	*D*	*F*
0	0	0	0	1
0	0	0	1	0
0	0	1	0	0
0	0	1	1	0
0	1	0	0	1
0	1	0	1	1
0	1	1	0	0
0	1	1	1	0
1	0	0	0	1
1	0	0	1	0
1	0	1	0	x
1	0	1	1	x
1	1	0	0	x
1	1	0	1	x
1	1	1	0	x
1	1	1	1	x

a) Building K-map from the truth table

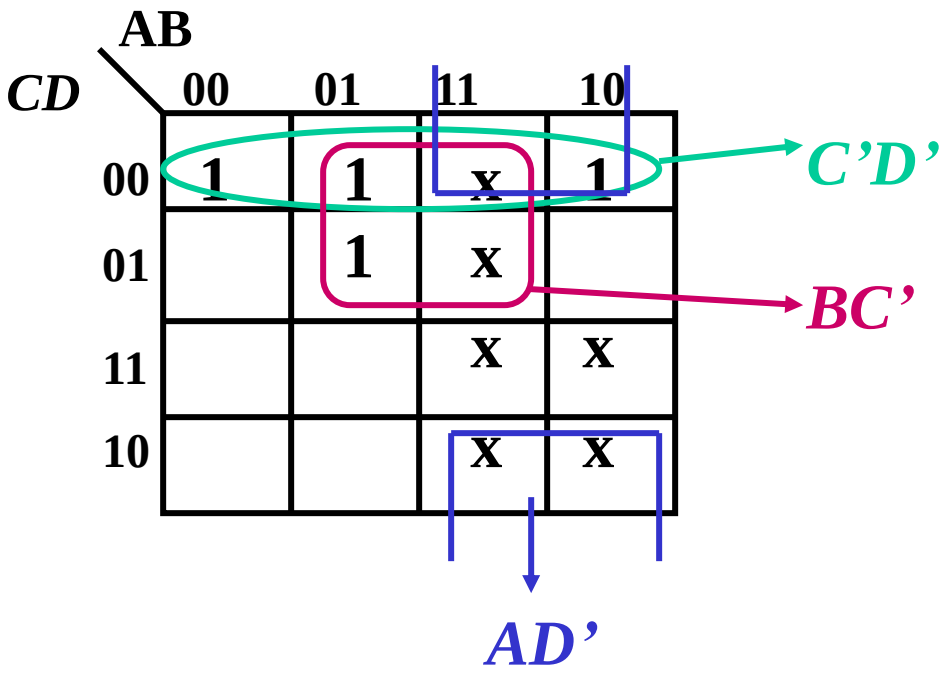
CD \ AB	00	01	11	10
00	1	1	x	1
01	0	1	x	0
11	0	0	x	x
10	0	0	x	x

A	B	C	D	F
0	0	0	0	1
0	0	0	1	0
0	0	1	0	0
0	0	1	1	0
0	1	0	0	1
0	1	0	1	1
0	1	1	0	0
0	1	1	1	0
1	0	0	0	1
1	0	0	1	0
1	0	1	0	x
1	0	1	1	x
1	1	0	0	x
1	1	0	1	x
1	1	1	0	x
1	1	1	1	x

a) Building K-map from the truth table



b) Obtain Sum of Products for F



$$F = BC' + C'D'$$

Prime implicants

When grouping square:

• a group should contain a maximum of adjacent squares

→Known as *PRIME IMPLICANT*

• each group represents one term in the function

• a function should contain a minimum set of terms

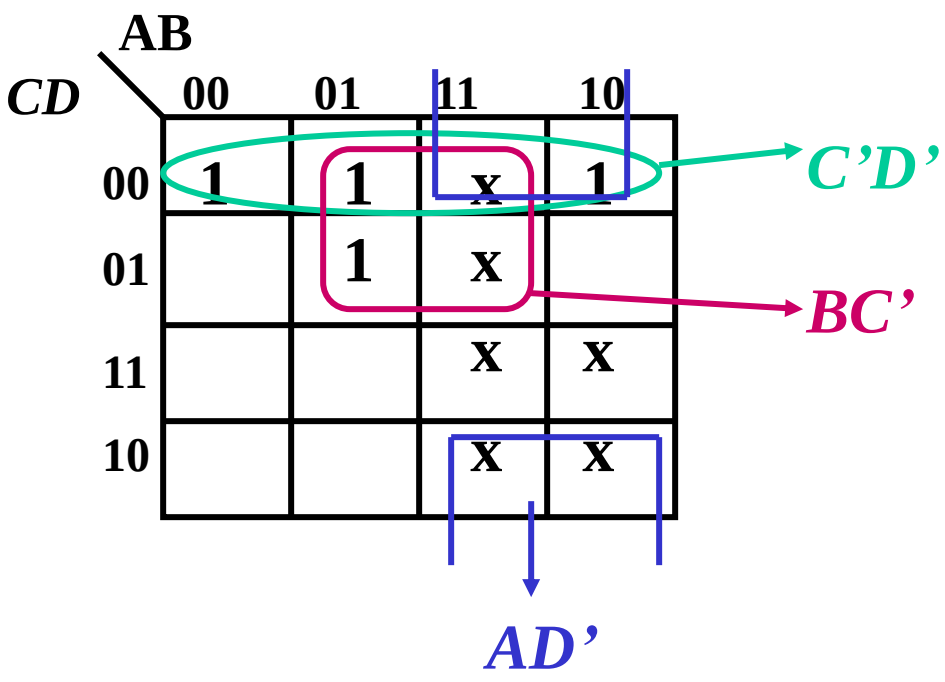
when selecting groups :

- Select only those groups that have at least one square that is not covered by another group:

→ Known as *Essential Prime Implicant*

- Do not select a group that has all its squares covered by other groups: →Known as *Optional prime implicant*

b) Obtain Sum of Products for F



$$F = BC' + C'D'$$

c) Obtain product of Sums : 2 STEPS

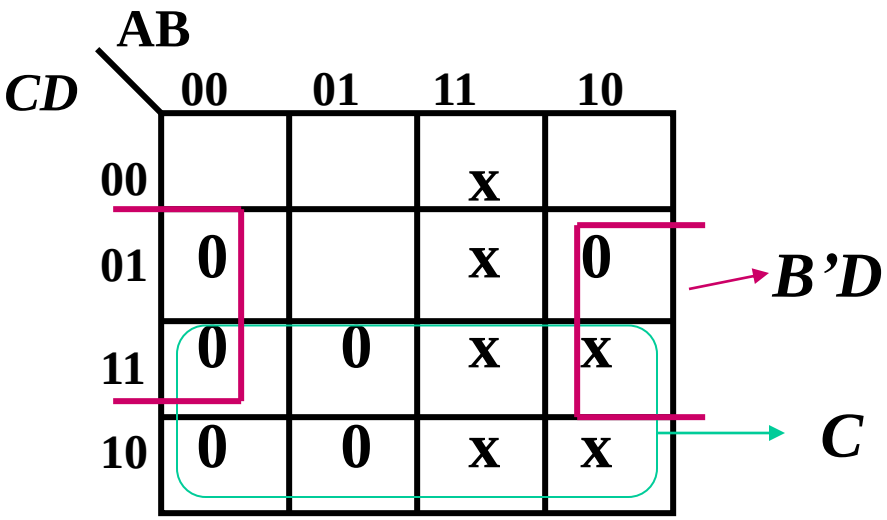
1 - use Minterms to simplify and obtain F'

$$F' = B'D + C$$

2 - complement F' to get the Product of Sum form

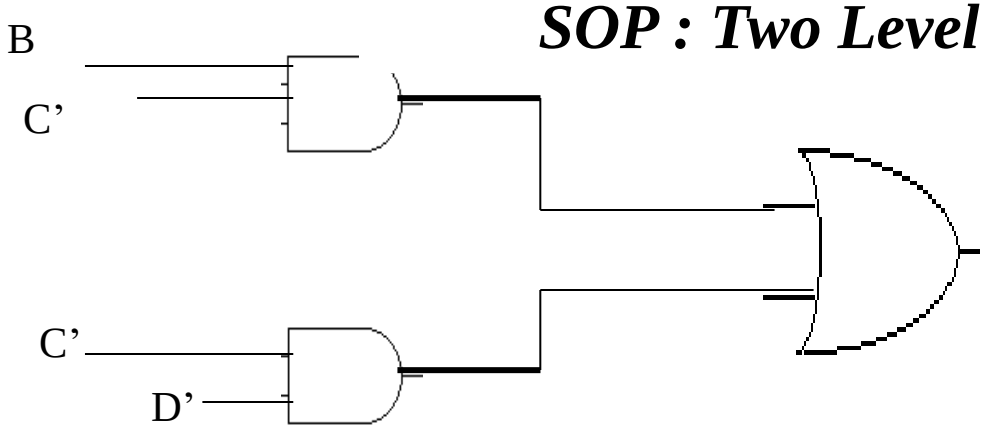
$$F'' = (B'D + C)' = (B''+D').C'$$

$$F = (B +D').C'$$



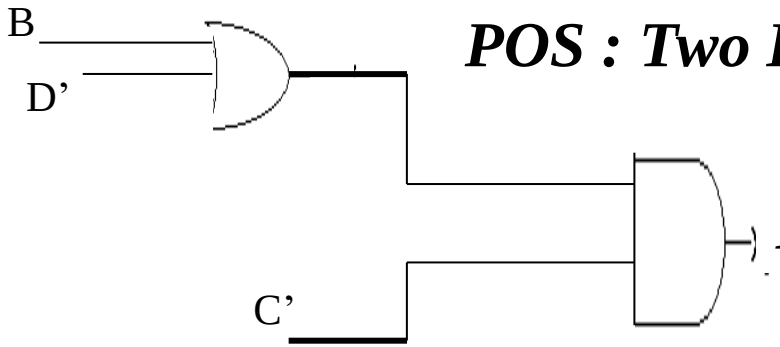
Two Level Implementations

SOP : Two Level Implementation



F = BC’ + C’D’

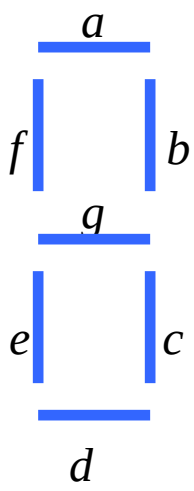
POS : Two Level Implementation



F = (B +D’).C’

Seven Segment Decoder -Example

a BCD to Seven Segment Decoder inputs data in BCD form and converts it to a seven segment output



(a) Segment designation

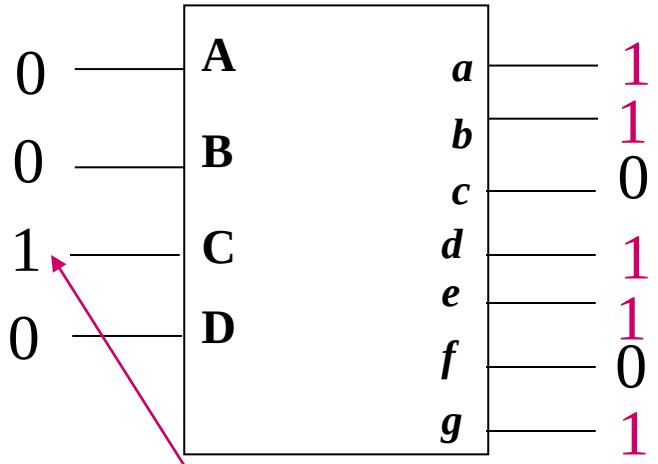


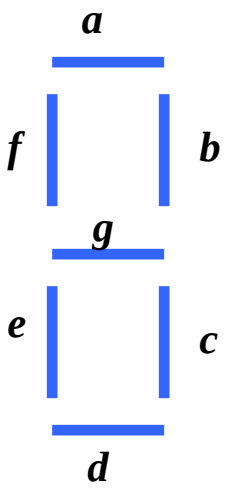
(b) Numerical designation for display

A- BCD to Seven Segment Decoder

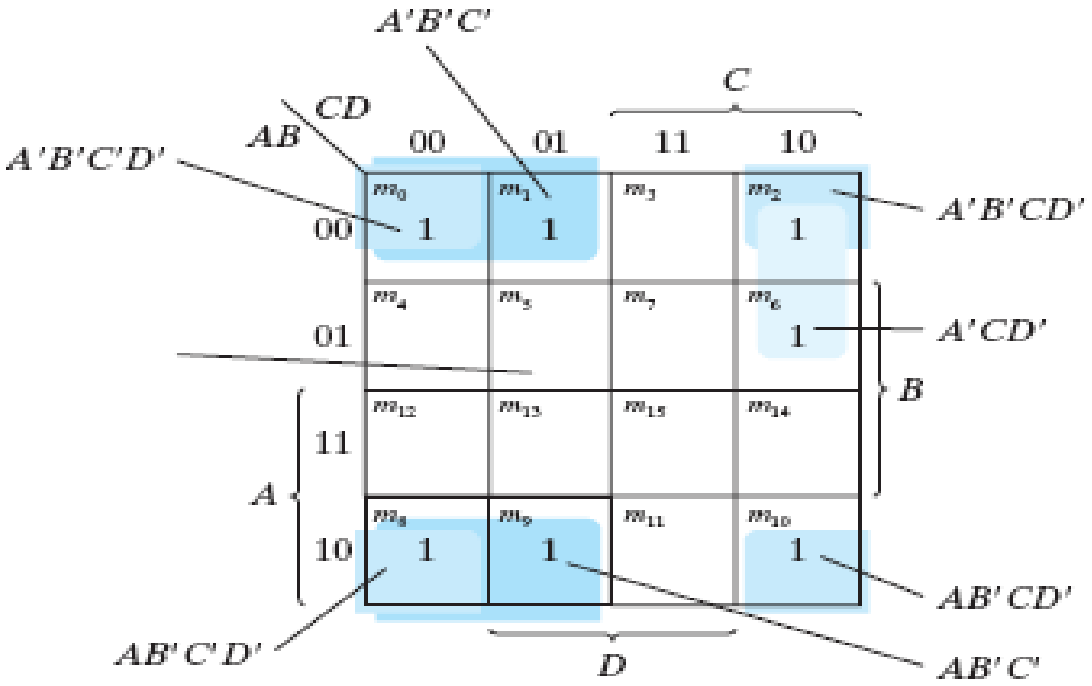
A	B	C	D	a	b	c	d	e	f	g
0	0	0	0	1	1	1	1	1	1	0
0	0	0	1	0	1	1	0	0	0	0
0	0	1	0	1	1	0	1	1	0	1
0	0	1	1	1	1	1	1	0	0	1
0	1	0	0	0	1	1	0	0	1	1
0	1	0	1	1	0	1	1	0	1	1
0	1	1	0	1	0	1	1	1	1	1
0	1	1	1	1	1	1	0	0	0	0
1	0	0	0	1	1	1	1	1	1	1
1	0	0	1	1	1	1	1	0	1	1
1	0	1	0	x	x	x	x	x	x	x
1	0	1	1	x	x	x	x	x	x	x
1	1	0	0	x	x	x	x	x	x	x
1	1	0	1	x	x	x	x	x	x	x
1	1	1	0	x	x	x	x	x	x	x
1	1	1	1	x	x	x	x	x	x	x

Don’t care terms









Note: $A'B'C'D' + A'B'CD' = A'B'D'$

$AB'C'D' + AB'CD' = AB'D'$

$A'B'D' + AB'D' = B'D'$

$A'B'C' + AB'C' = B'C'$

K-MAP

AB \ CD	00	01	11	10
00	1		1	1
01		1	1	1
11				
10	1	1		

$a =$

AB \ CD	00	01	11	10
00	1	1	1	1
01	1		1	
11				
10	1	1		

$b =$

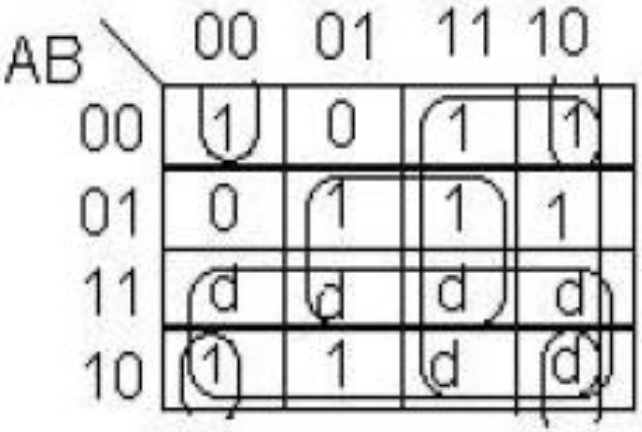
AB \ CD	00	01	11	10
00	1	1	1	
01	1	1	1	1
11				
10	1	1		

$c =$

AB \ CD	00	01	11	10
00	1		1	1
01		1		1
11				
10	1	1		

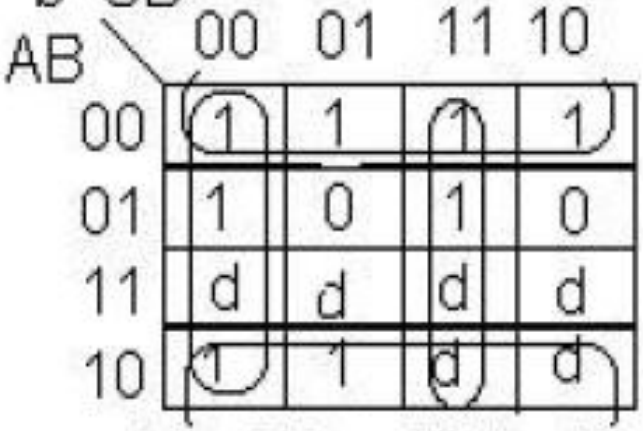
$d =$

a



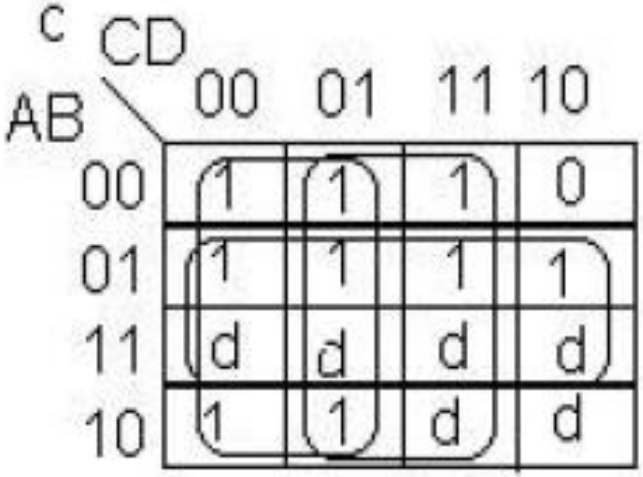
a = A + BD + B'D'+ C

b



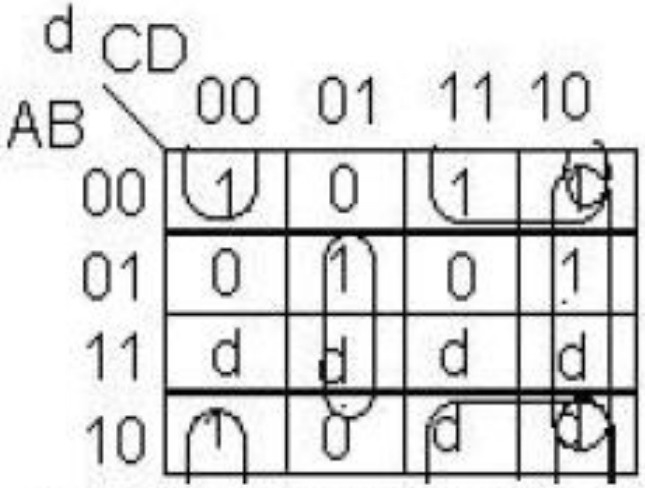
b = CD + C'D' + B'

c

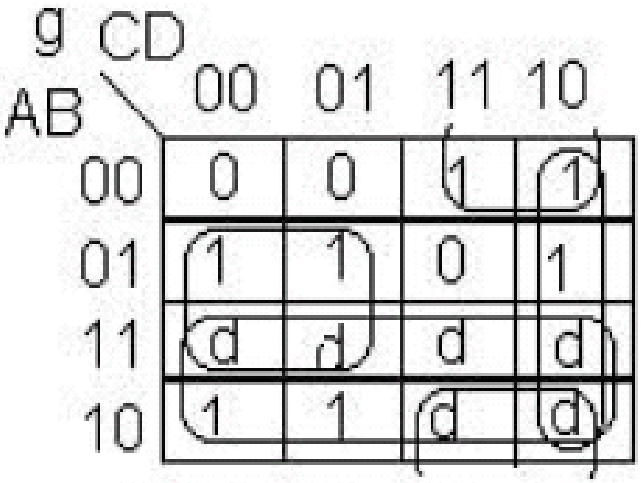


c= B + C'+ D

d



d=BC'D+B'C+B'D'+CD'



g=BC'+B'C+CD'+A

Implementations using NAND & NOR Gates

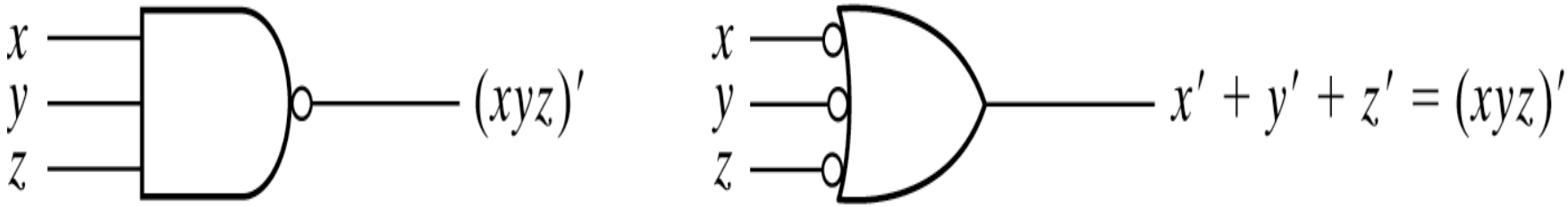
• Digital circuits are frequently constructed with NAND or NOR gates rather with AND and OR gates.

→ Both NAND and NOR gates are very valuable as any design can be realized using either one.

•It is easier to build digital circuits using all NAND or NOR gates than to combine AND,OR, and NOT gates.

•NAND/NOR gates are typically faster and cheaper to produce.

Logic Operations with NAND Gates

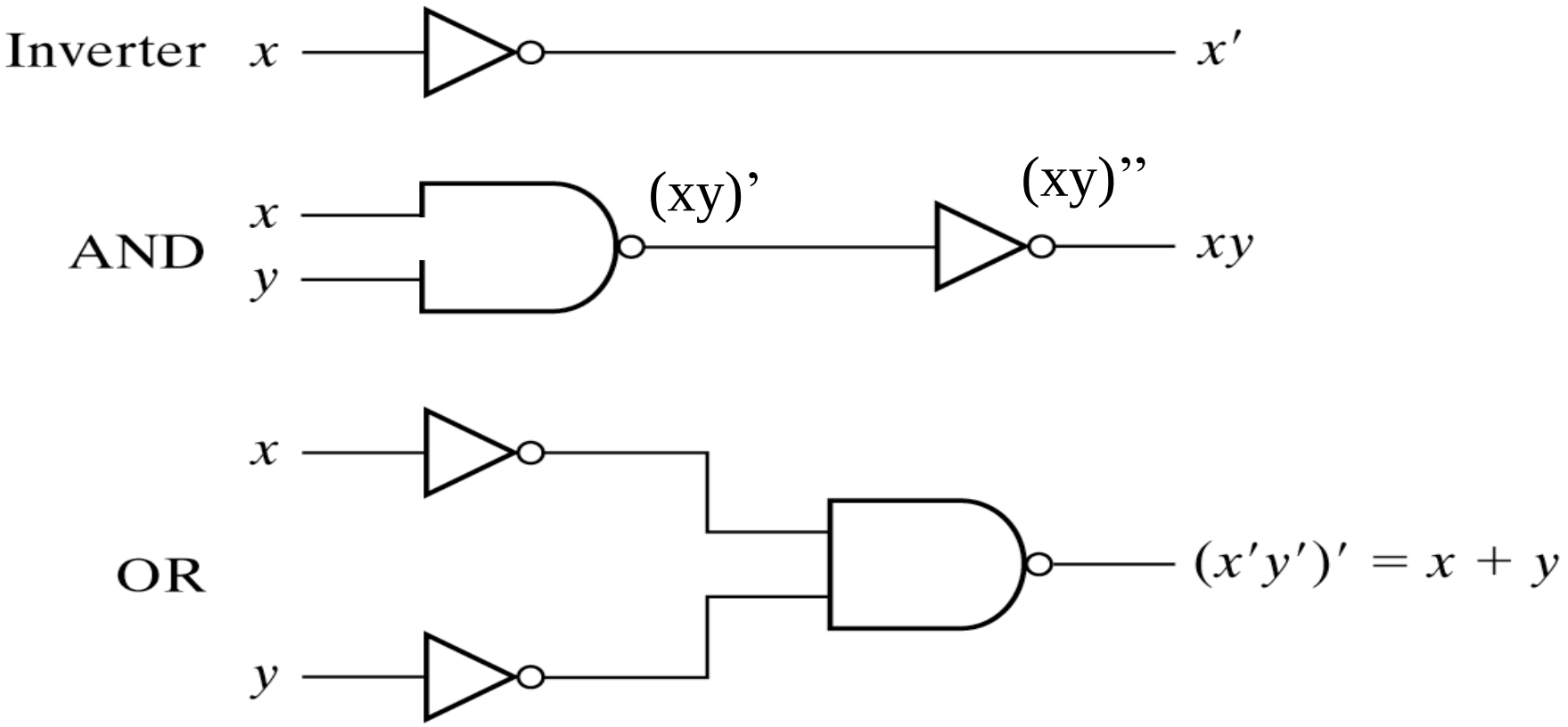


(a) AND–invert

(b) Invert–OR

Two Graphic Symbols for NAND Gate

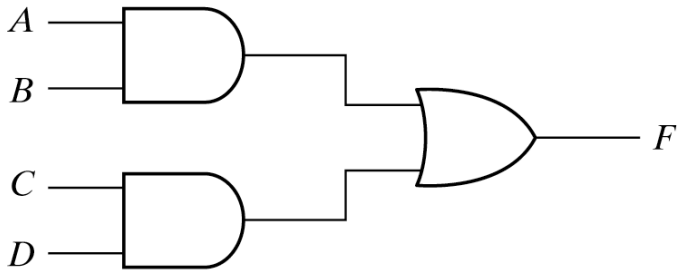
Logic Operations with NAND Gates



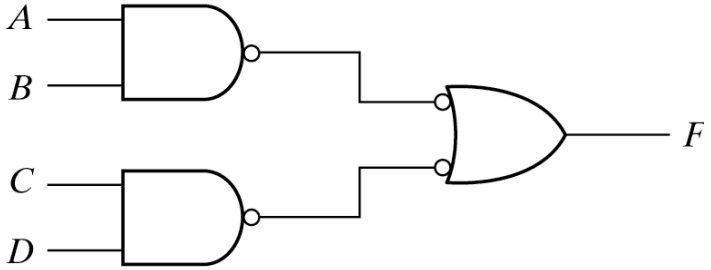
Logic Operations with NAND Gates

NAND gates Implementations

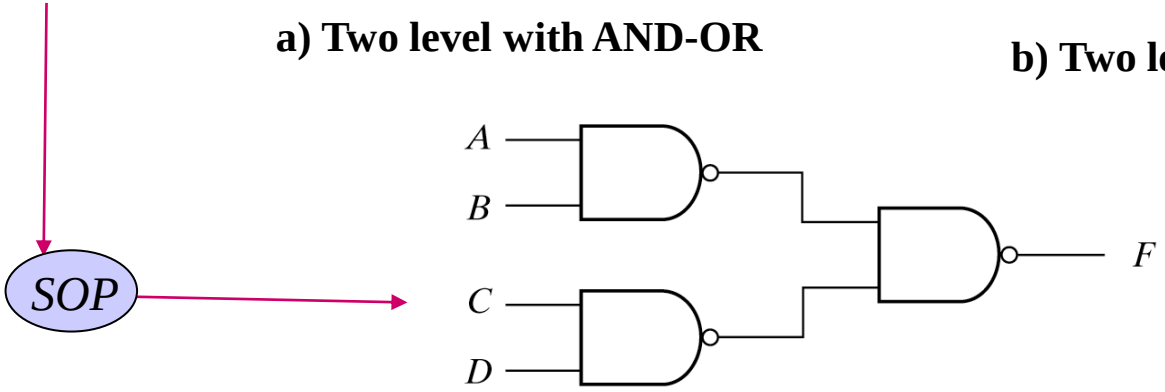
(AB)'' + (CD)''= ((AB)'(CD)')'



a) Two level with AND-OR



b) Two level with NAND& Invert- OR



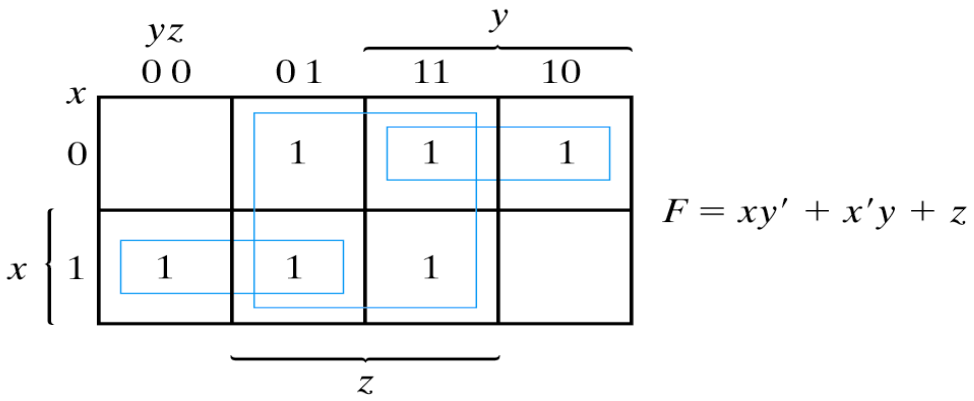
c) Two level with NAND gates
(use this in exam)

Three Ways to Implement F = AB + CD

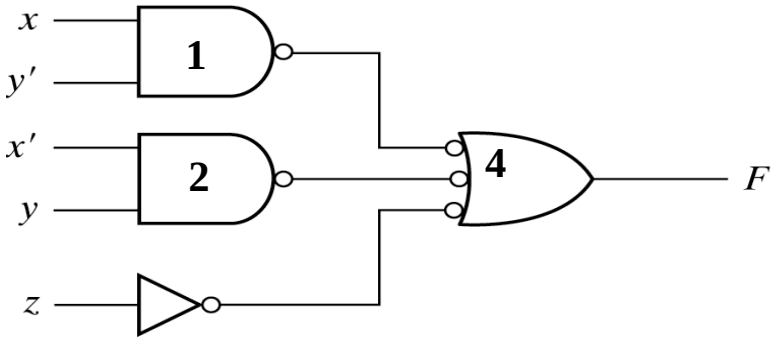
NAND gates implementations -Examples

1: (xy')' 2: (x'y)' 4: ((xy')')' + ((x'y)')' + (z')'= xy' + x'y +z

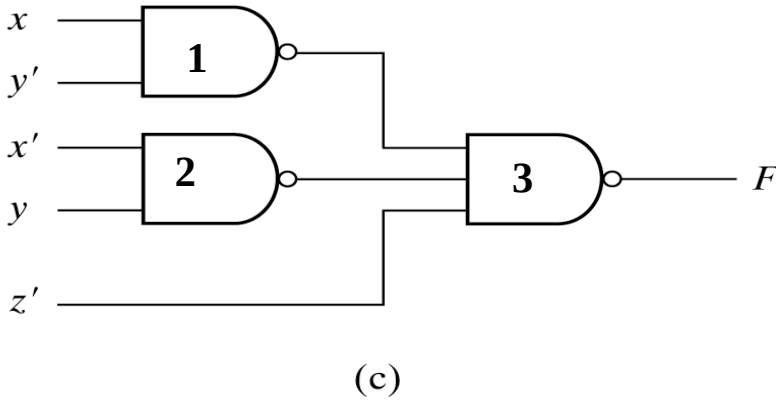
3: ((xy')'(x'y)'(z'))'= (xy')" + (x'y)"+ (z')' = xy' + x'y +z



(a)



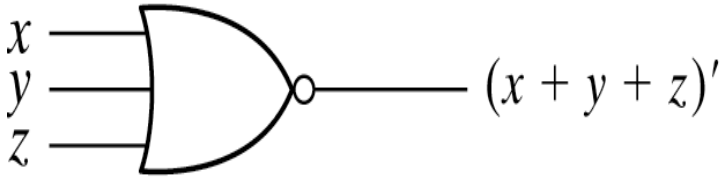
(b)



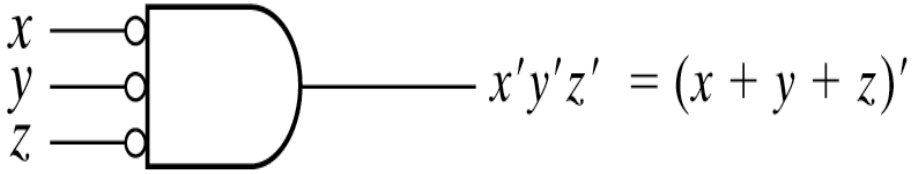
(c)

Using NAND gates to implement SOP

Logic Operations with NOR Gates



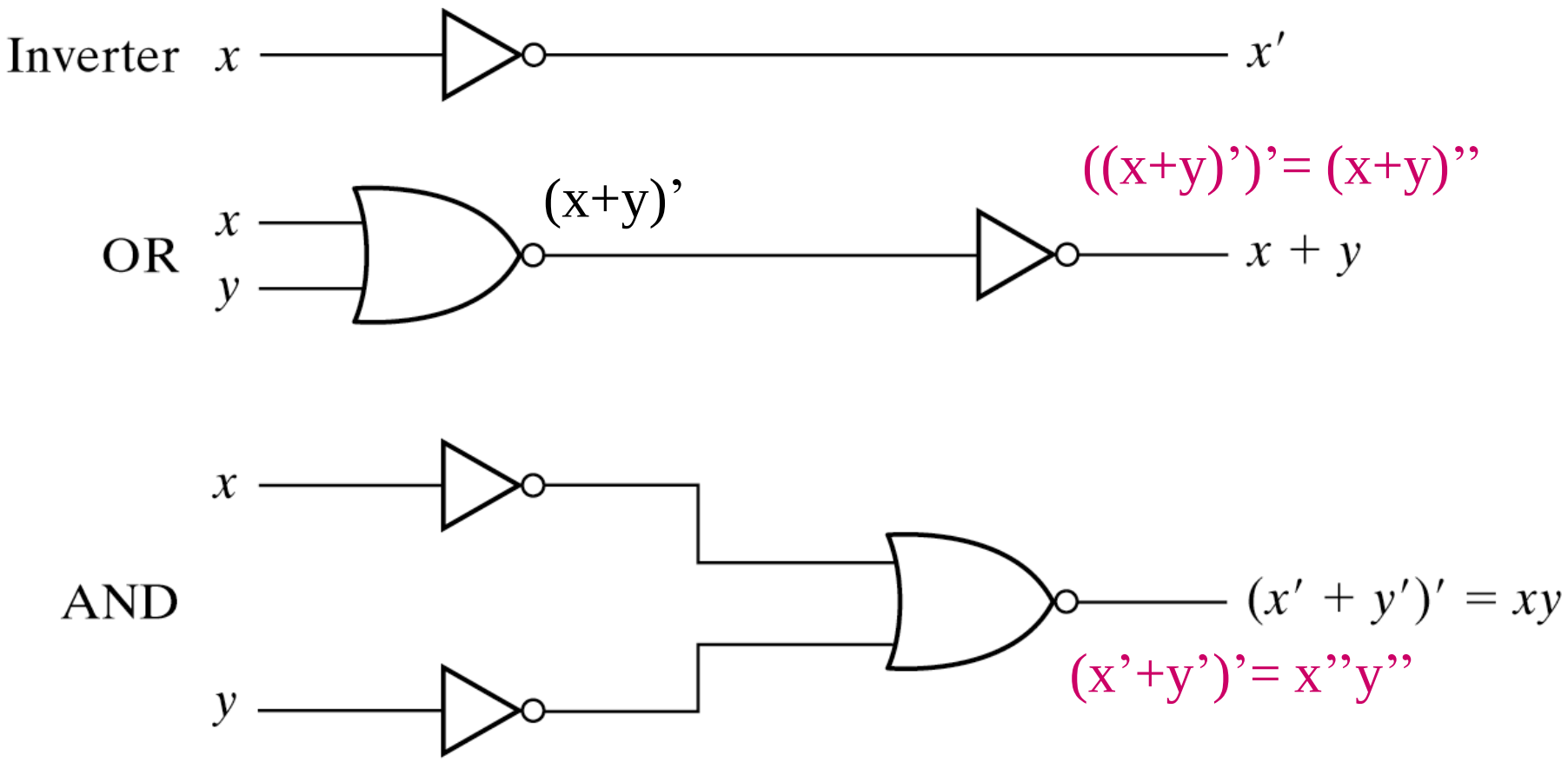
(a) OR–invert



(a) Invert–AND

Two Graphic Symbols for NOR Gate

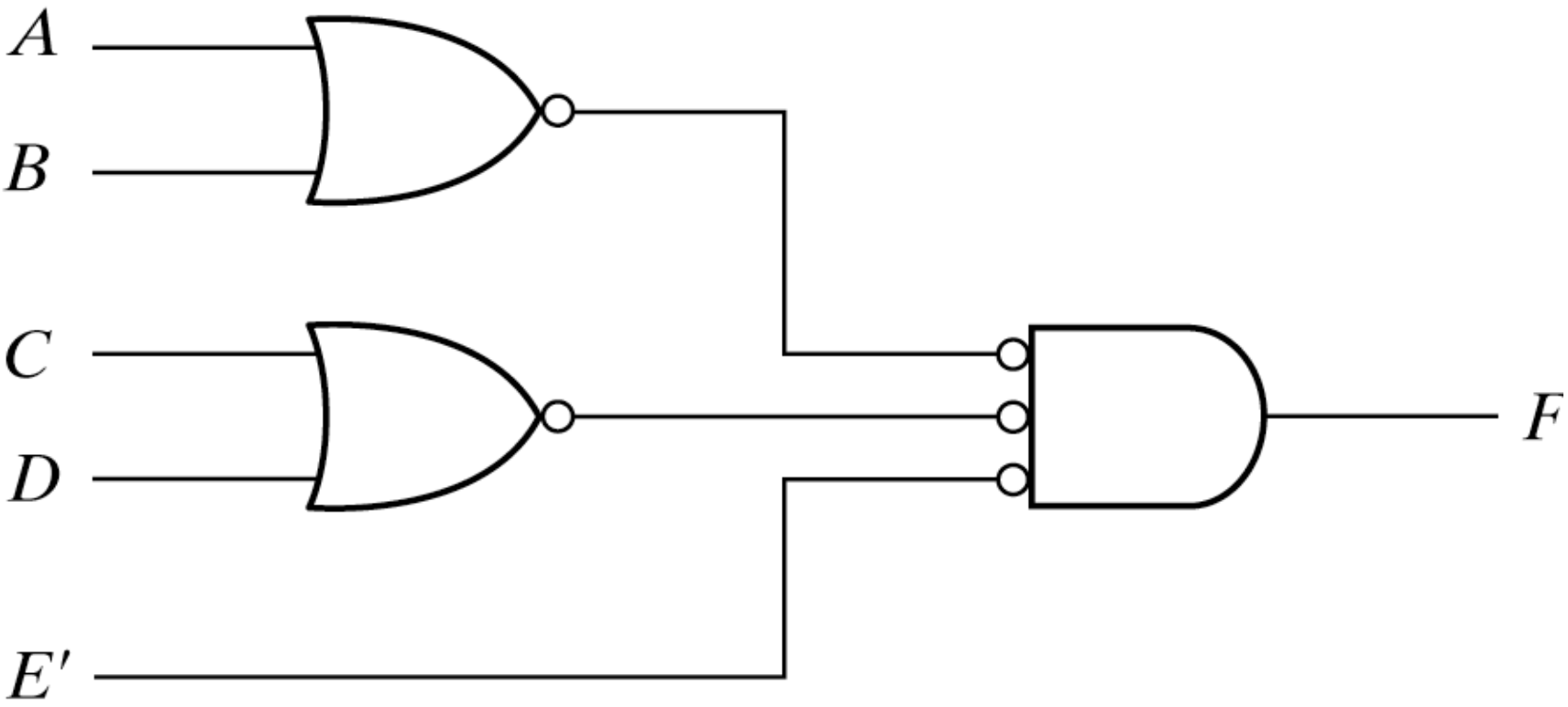
Logic Operations with NOR Gates



Logic Operations with NOR Gates

NOR gates Implementation -Examples

POS with NOR



Implementing $F = (A + B)(C + D)E$

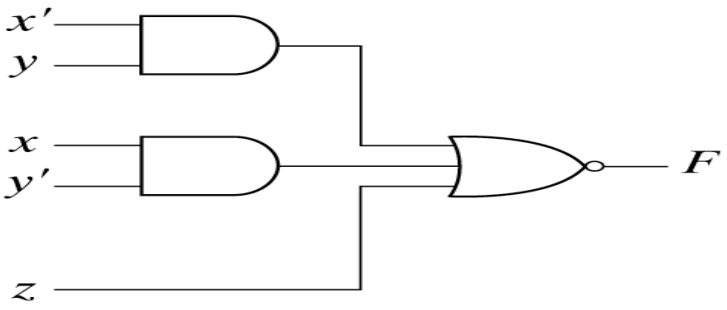
Other implementation examples

x \ yz	00	01	11	10
0	1	0	0	0
1	0	0	0	1

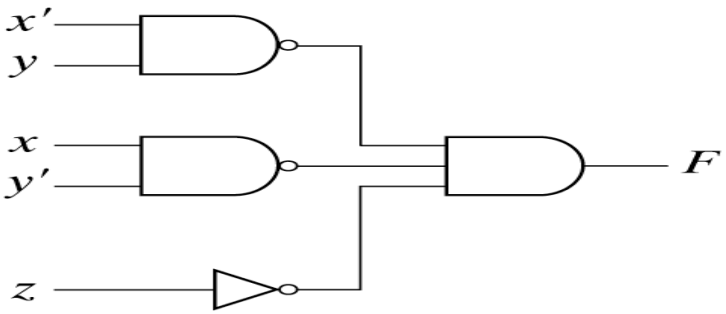
$F = x'y'z' + xyz'$

$F' = x'y + xy' + z$

(a) Map simplification in sum of products.

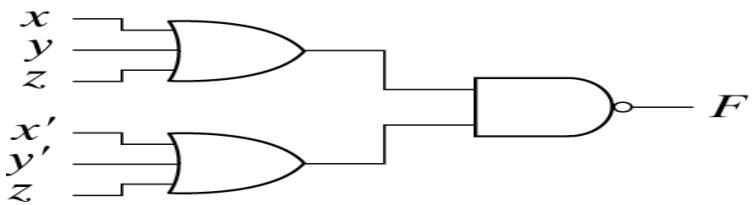


AND-NOR

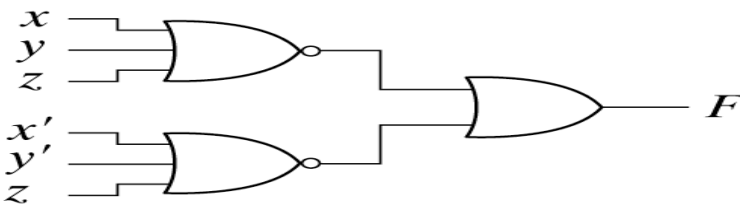


NAND-AND

(b) $F = (x'y + xy' + z)'$



OR-NAND



NOR-OR

(c) $F = [(x + y + z)(x' + y' + z)]'$

Other Two-level Implementations

Lecture 5
Karnaugh Maps

- Algebraic procedures:
  - Difficult to apply in a systematic way.
  - Difficult to tell when you have arrived at a minimum solution.
- Karnaugh map (K-map) can be used to minimize functions of up to 6 variables.
  - K-map is directly applied to two-level networks composed of AND and OR gates.
    - Sum-of-products, (SOP)
    - Product-of-sum, (POS).

Minimum SOP

- It has a minimum no. of terms.
  - That is, it has a minimum number of gates.
- It has a minimum no. of gate inputs.
  - That is, minimum no. of literals.
  - Each term in the minimum SOP is a prime implicant, i.e., it cannot be combined with others.
- It may not be unique.
  - Depend on the order in which terms are combined or eliminated.

Minimum SOP

- Example: vertical input scheme

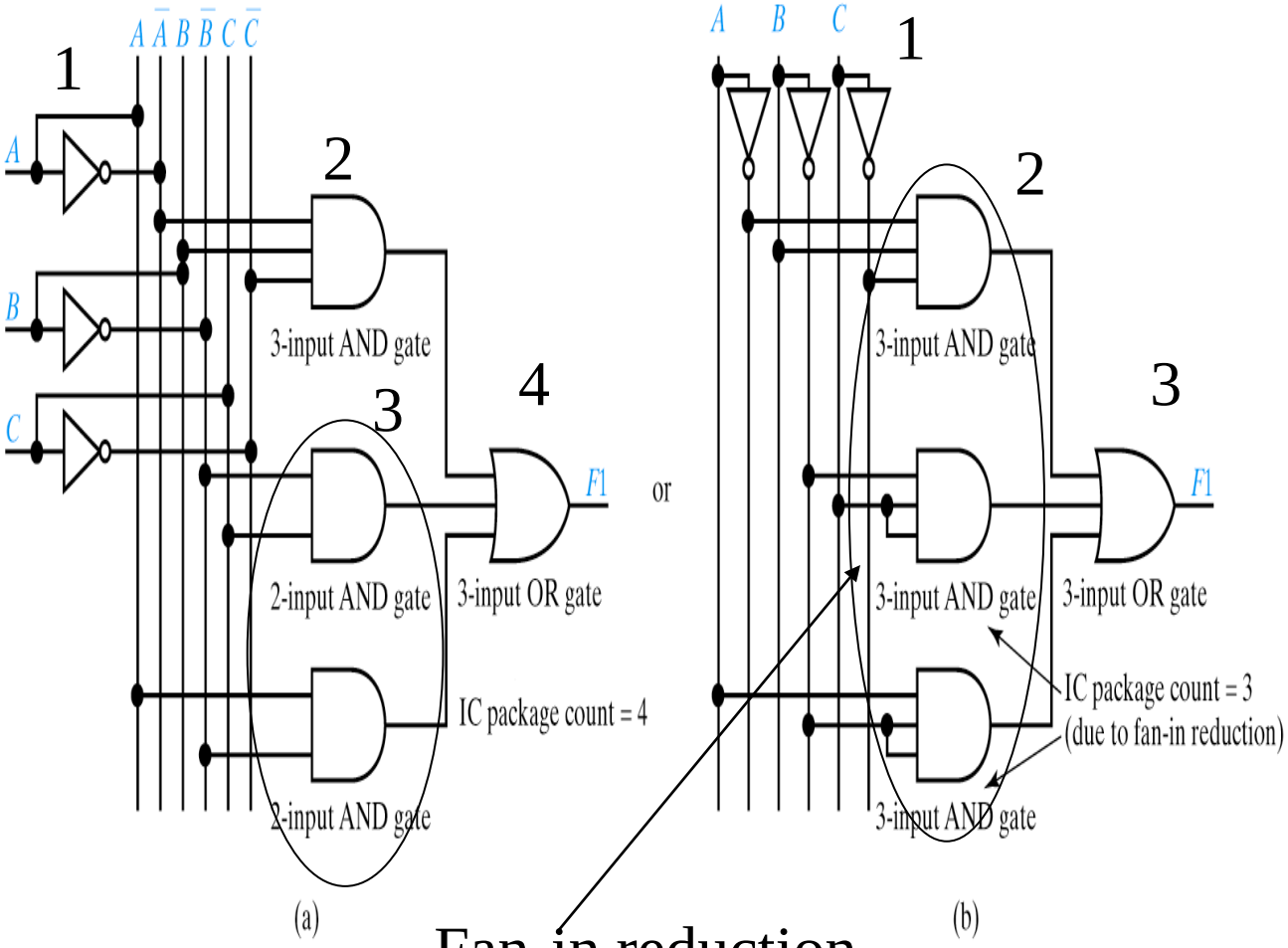


Figure 3.2.5 IC logic circuit designs for a minimum SOP form of a function using a vertical input scheme: (a) without fan-in reduction, (b) with fan-in reduction.

Minimum POS

- It has a minimum no. factors.
- It has a minimum no. of literals.
- It may not be unique.
  - Use (X+Y) (X+Y’) = X
  - Use (X +C) (X’ + D)(C+D) = (X+C)(X’+D) to eliminate term.

Minimum POS

- Example: Vertical input scheme

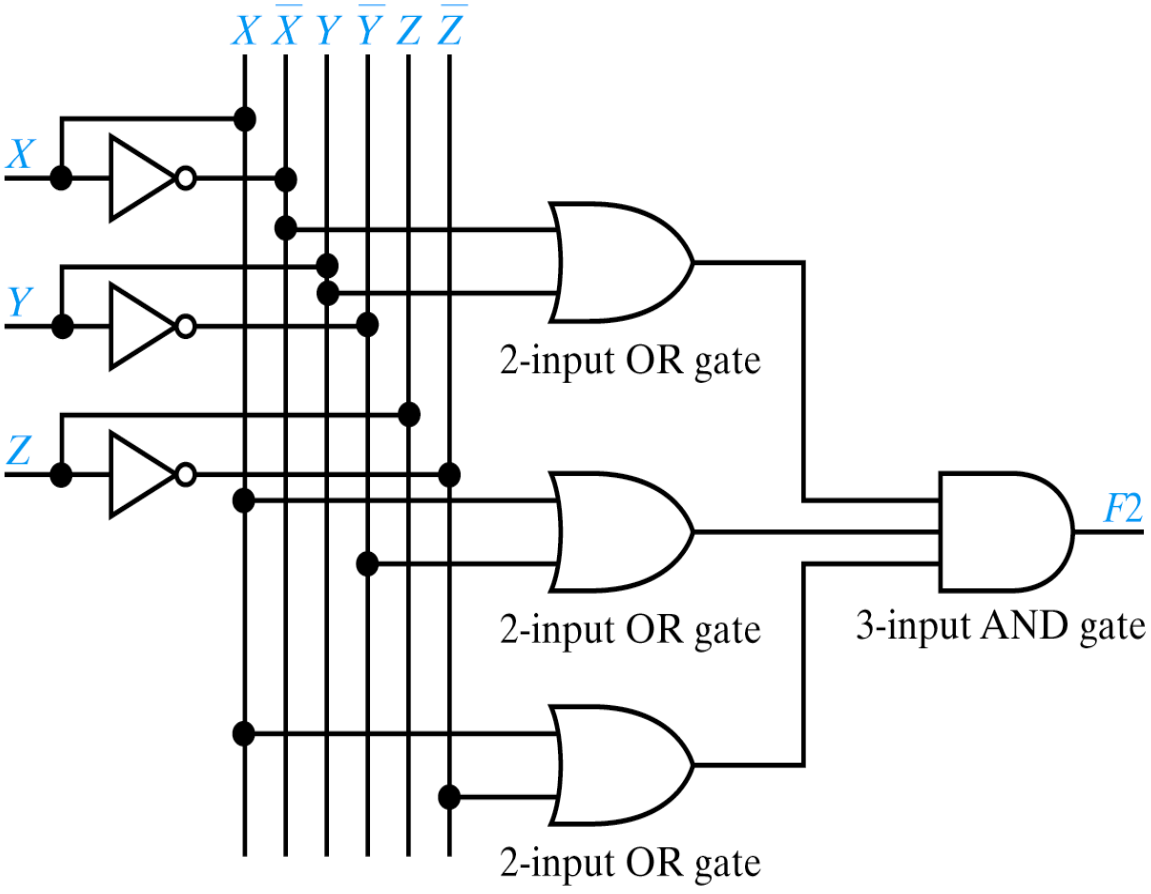
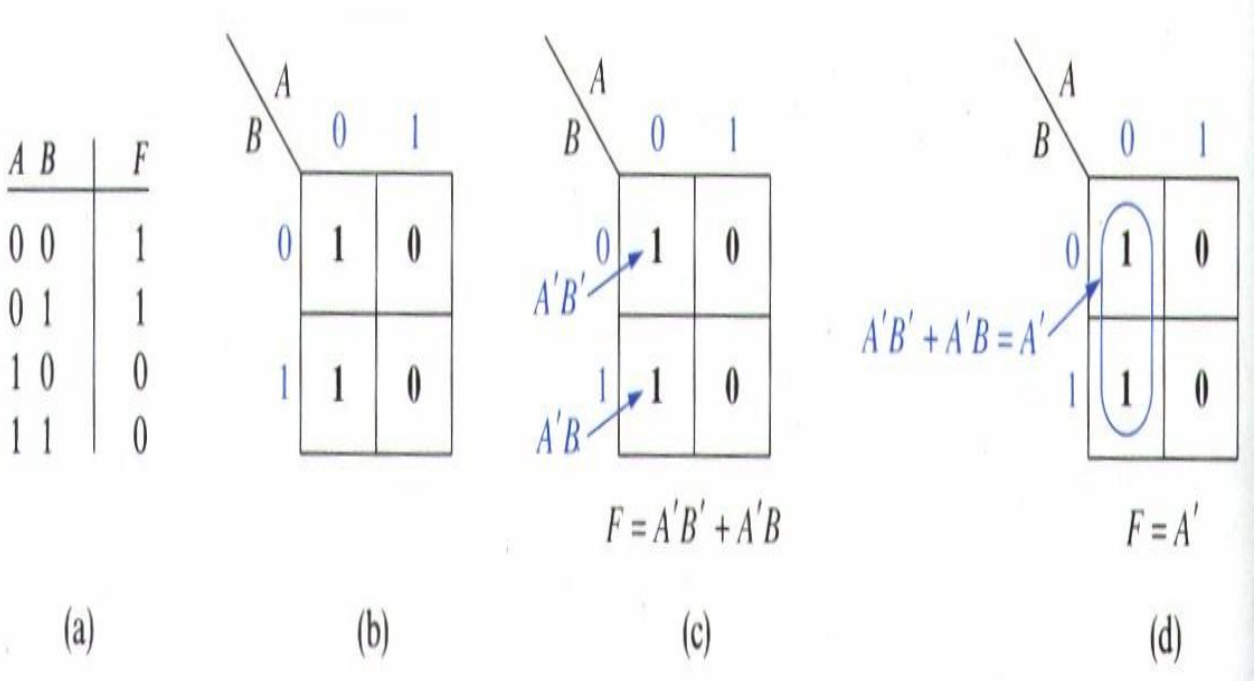


Figure 3.2.7 IC logic circuit design for a minimum POS form of a function using a vertical input scheme.

2-Variable K-map

- Place 1s and 0s from the truth table in the K-map.
- Each square of 1s = minterms.
- Minterms in adjacent squares can be combined since they differ in only one variable. Use XY' + XY = X.

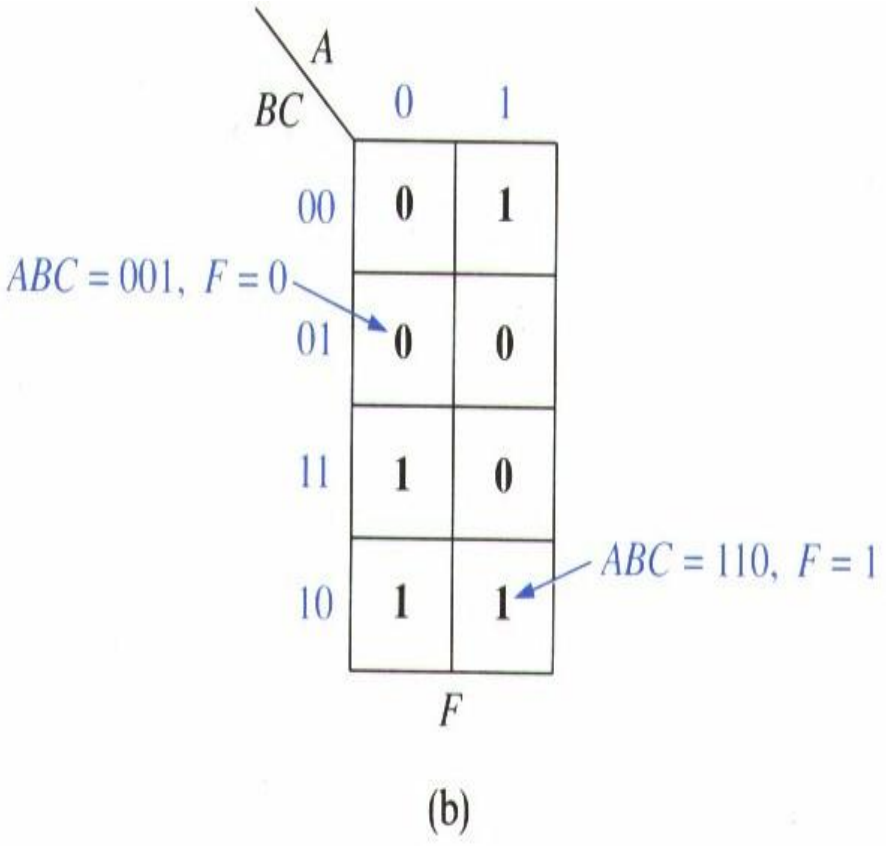


3-Variable K-map

- Note BC is listed in the order of 00, 01, 11, 10. (Gray code)
- Minterms in adjacent squares that differ in only one variable can be combined using XY' + XY = X.

A	B	C	F
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	0

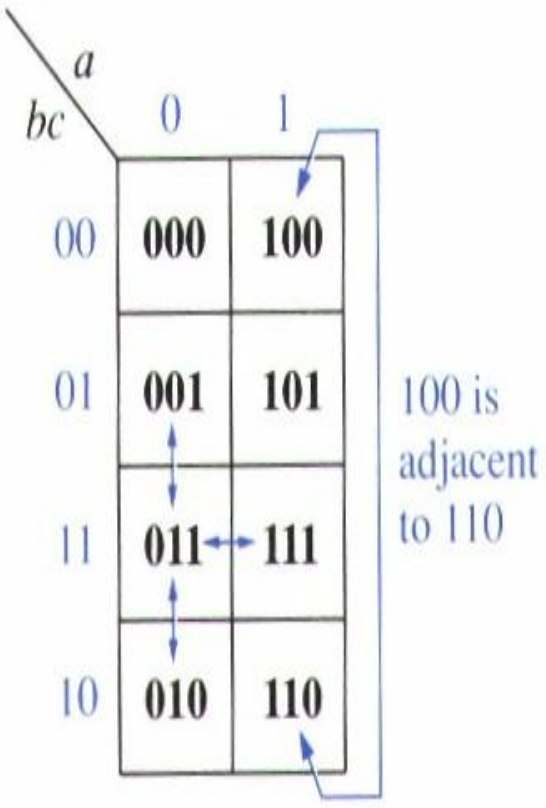
(a)



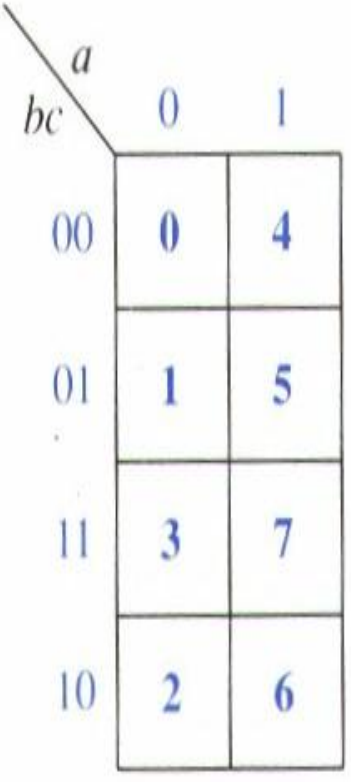
(b)

Location of Minterms

– Adjacent terms in 3-variable K map.



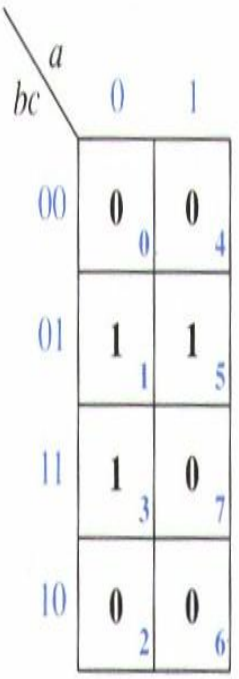
(a) Binary notation



(b) Decimal notation

K Map Example

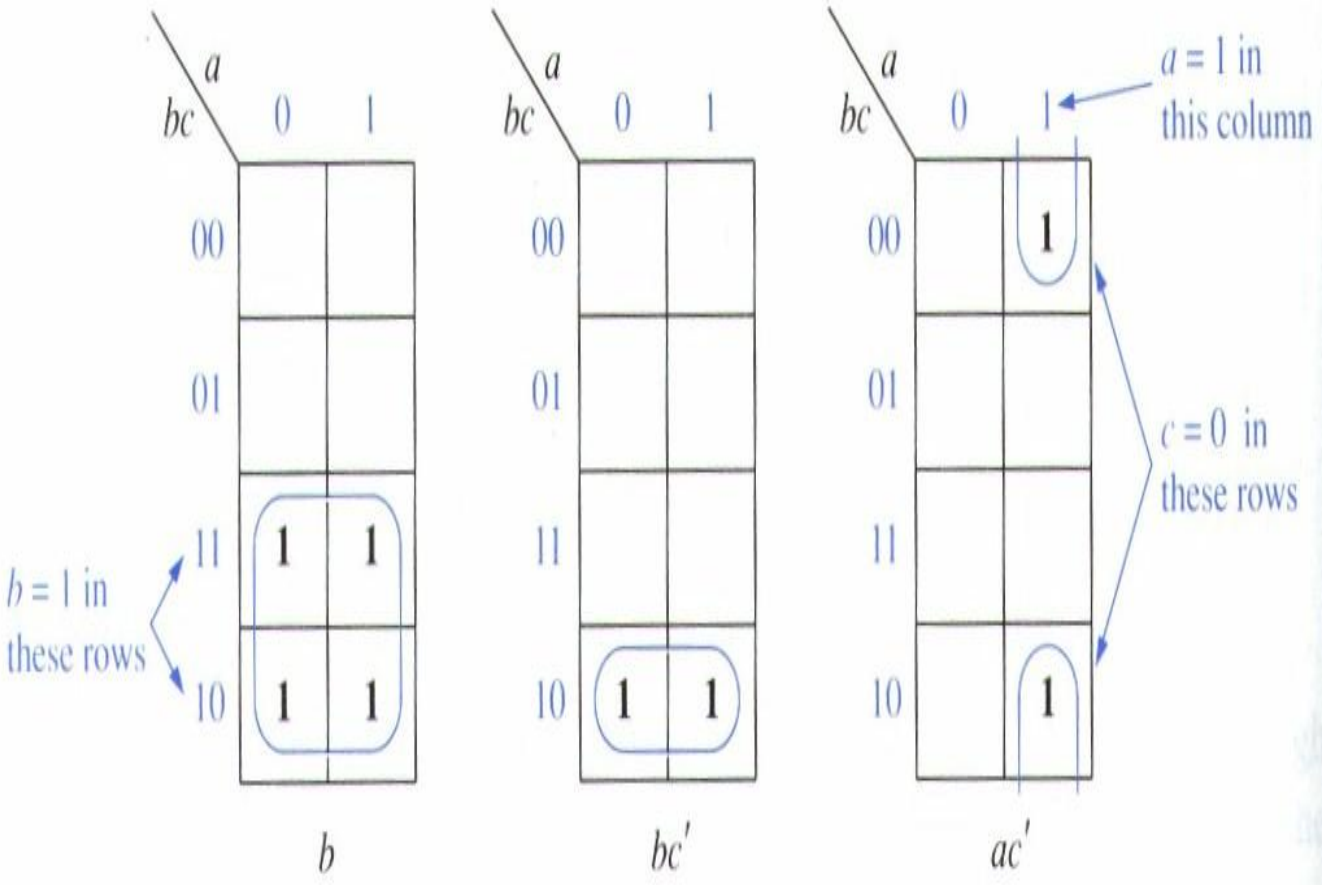
– K-map of F(a,b,c) = ∑m(1,3,5) =∏M(0,2,4,6,7)



Karnaugh Map of
$F(a, b, c) = \sum m(1, 3, 5) = \prod M(0, 2, 4, 6, 7)$

Place Product Terms on K Map

- Example
  - Place b, bc’ and ac’ in the 3-variable K map.



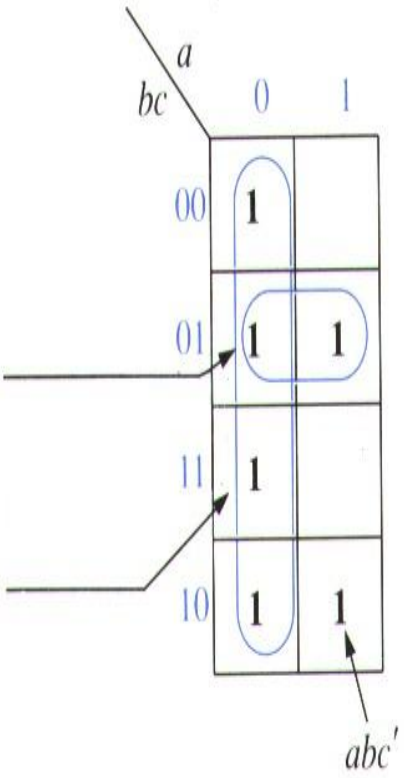
More Example

– Exercise. Plot f(a, b, c) = abc’ + b’c + a’ into the K-map.

$$f(a, b, c) = abc' + b'c + a'$$

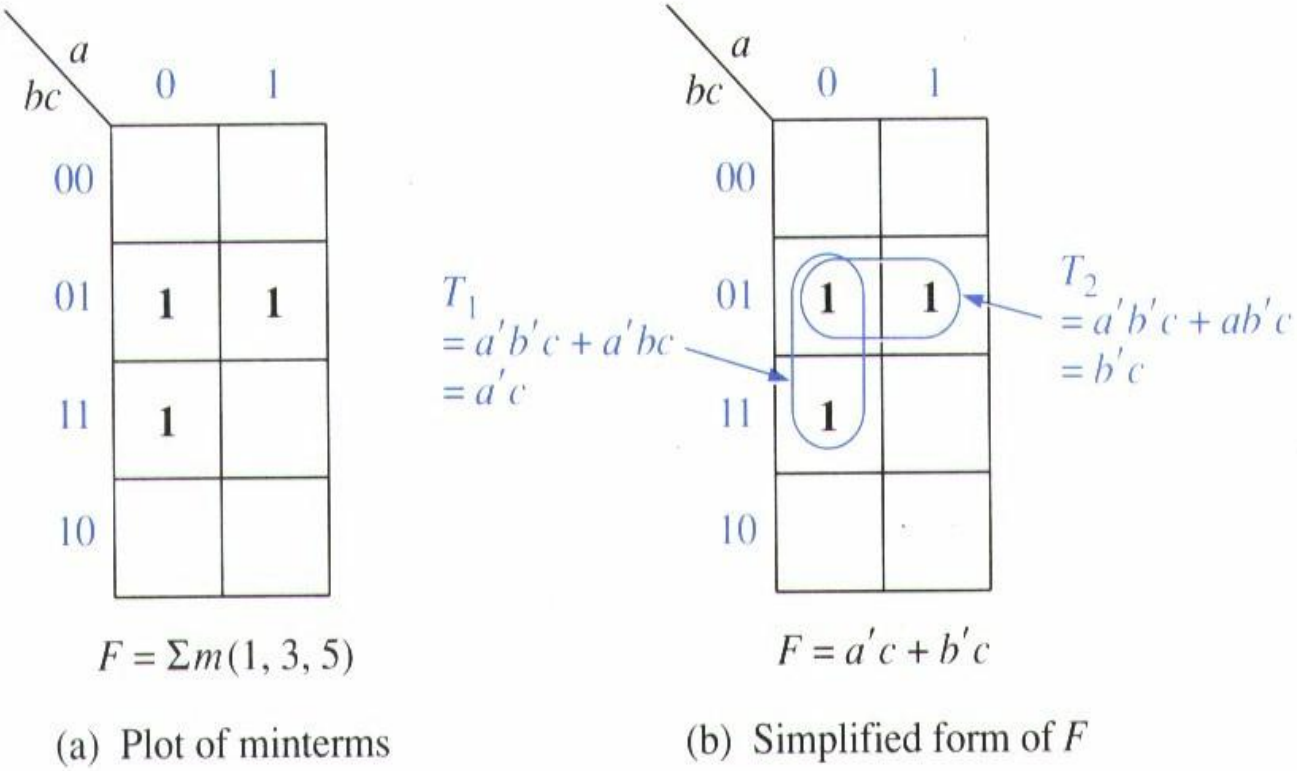
we would plot the map as follows:

1. The term abc' is 1 when $a = 1$ and $bc = 10$, so we place a 1 in the square which corresponds to the $a = 1$ column and the $bc = 10$ row of the map.
2. The term $b'c$ is 1 when $bc = 01$, so we place 1's in both squares of the $bc = 01$ row of the map.
3. The term a' is 1 when $a = 0$, so we place 1's in all the squares of the $a = 0$ column of the map. (*Note:* since there already is a 1 in the $abc = 001$ square, we do not have to place a second 1 there because $x + x = x$.)



Simplication Example

- Exercise. Simplify: F(a,b,c) = ∑m(1,3,5)
  - Procedure: place minterms into map.
  - Select adjacent 1's in group of two 1's or four 1's etc.
  - Kick off x and x'.

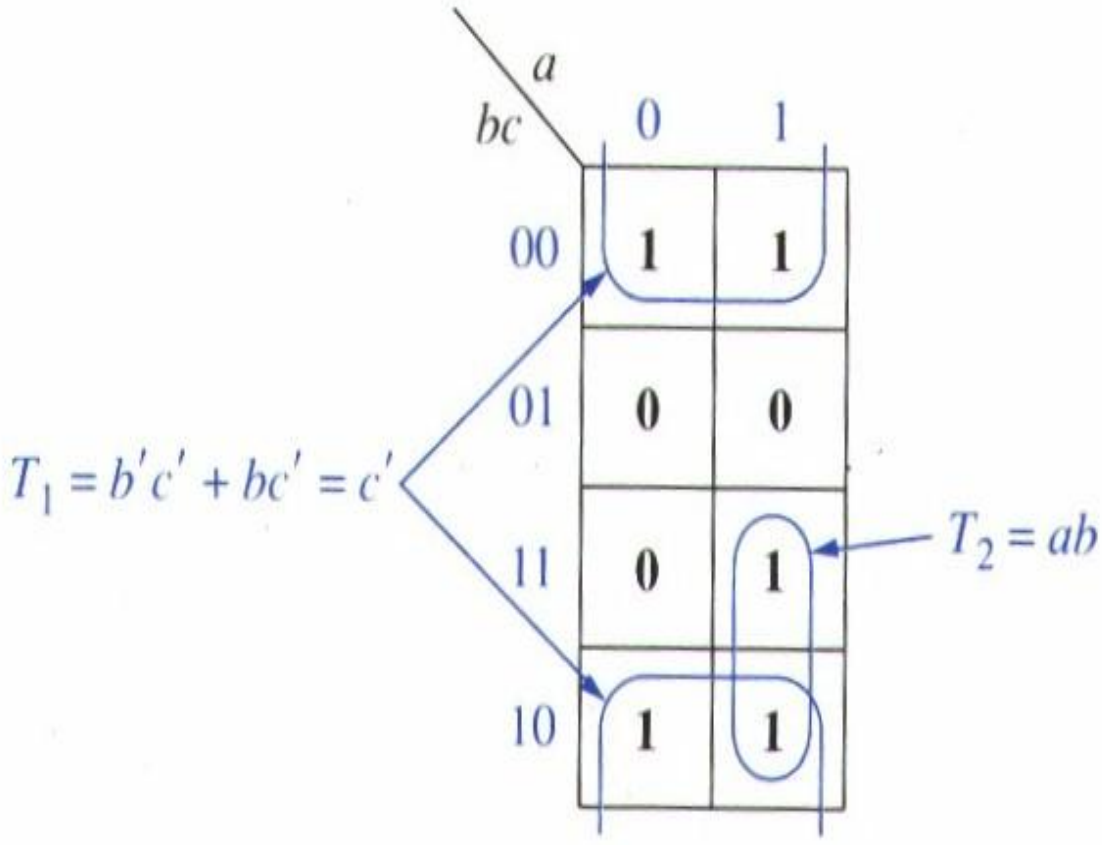


(a) Plot of minterms

(b) Simplified form of F

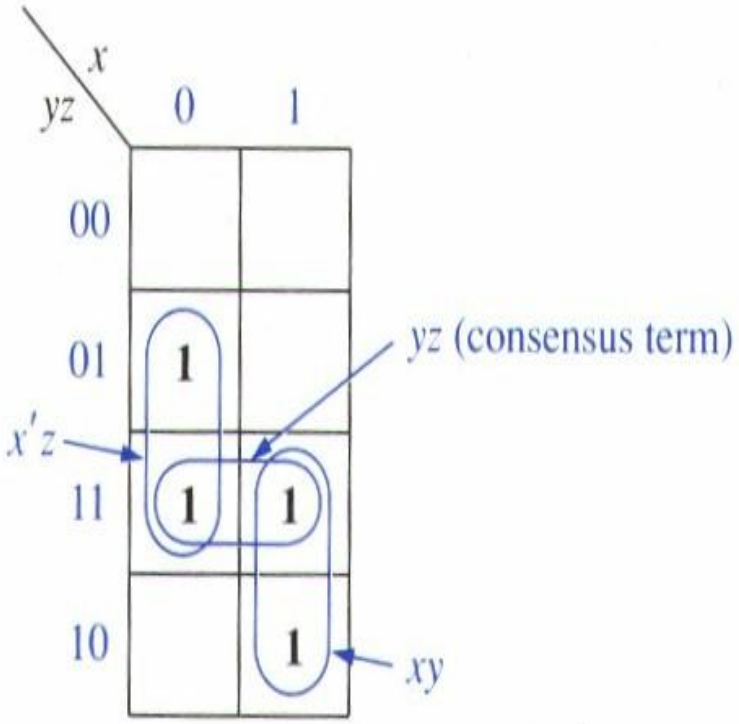
More Example

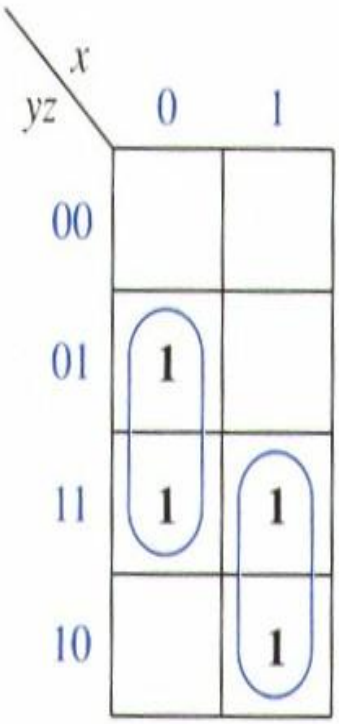
– The complement of F
  - Using four 1's to eliminate two variables.



Redundant Terms

- If a term is covered by two other terms, that term is redundant. That is, it is a consensus term.
- Example: yz is the redundant.

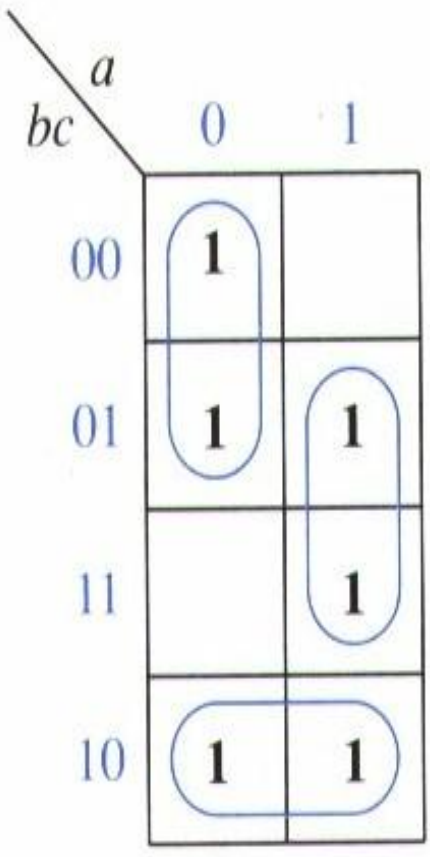




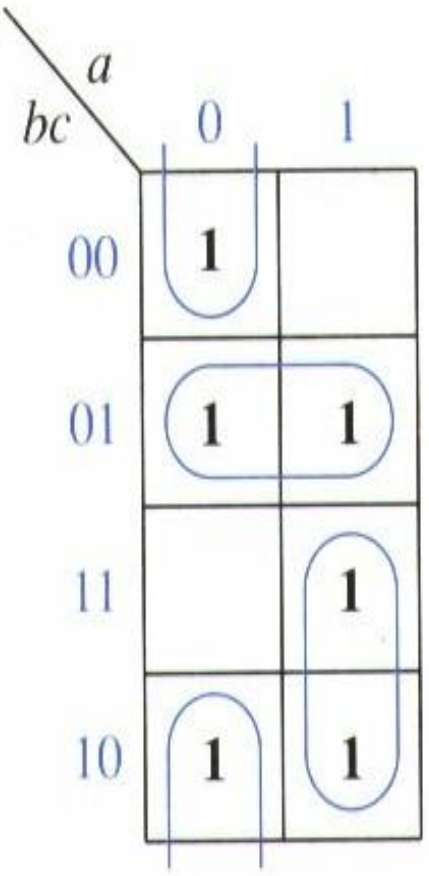
$$xy + x'z + yz = xy + x'z$$

More Than Two Minimum Solutions

- F = $\sum$ m(0,1,2,5,6,7)



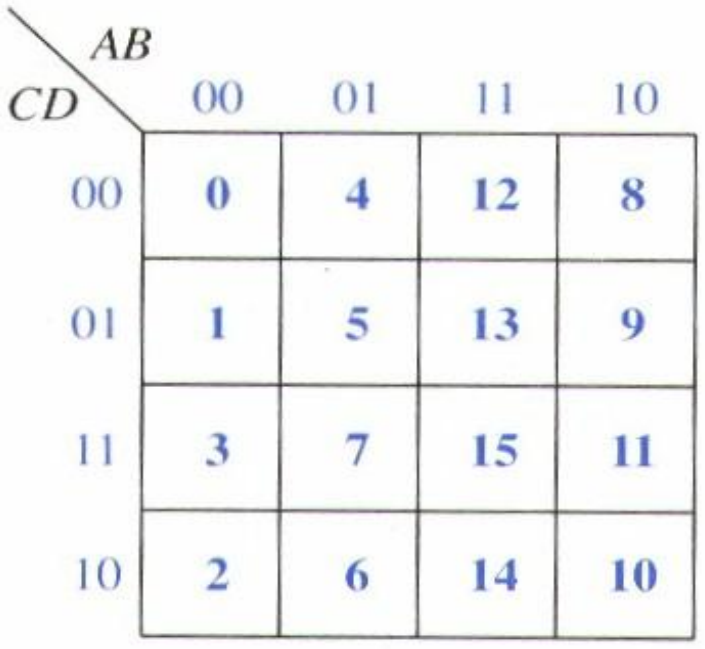
$F = a'b' + bc' + ac$



$F = a'c' + b'c + ab$

4-Variable K Map

– Each minterm is adjacent to 4 terms with which it can combine.
  - 0, 8 are adjacent squares
  - 0, 2 are adjacent squares, etc.
  - 1, 4, 13, 7 are adjacent to 5.

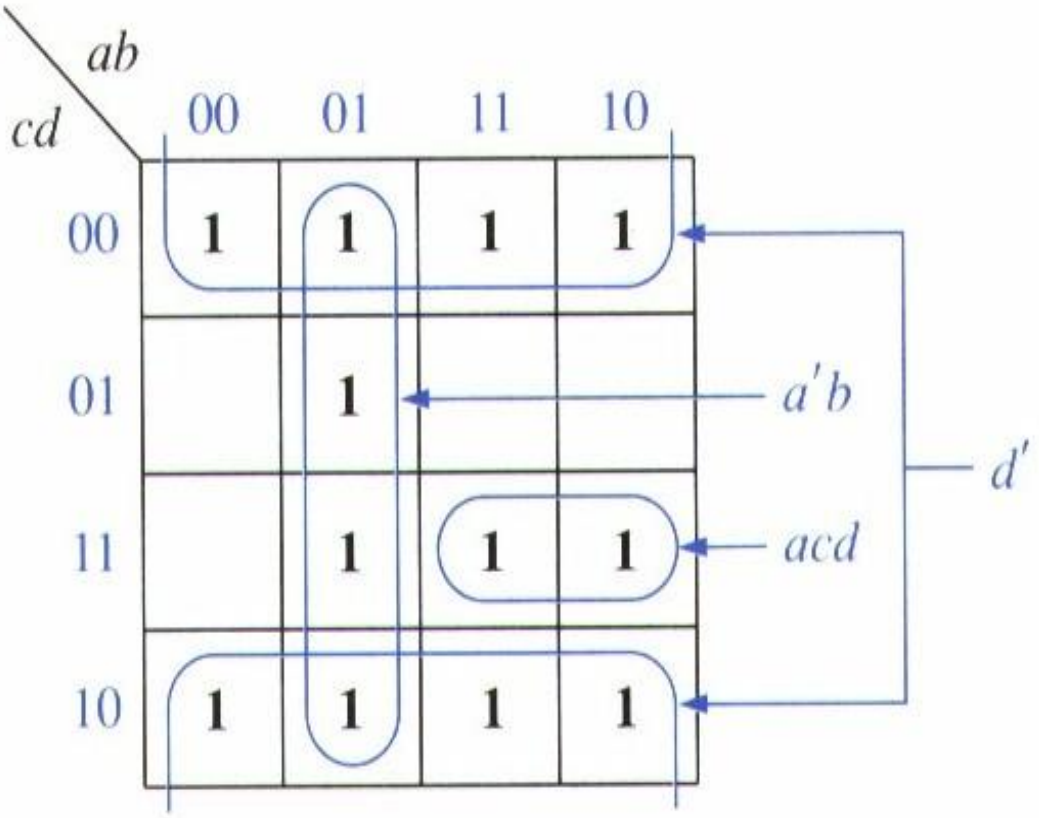


CD \ AB	00	01	11	10
00	0	4	12	8
01	1	5	13	9
11	3	7	15	11
10	2	6	14	10

Plot a 4-variable Expression

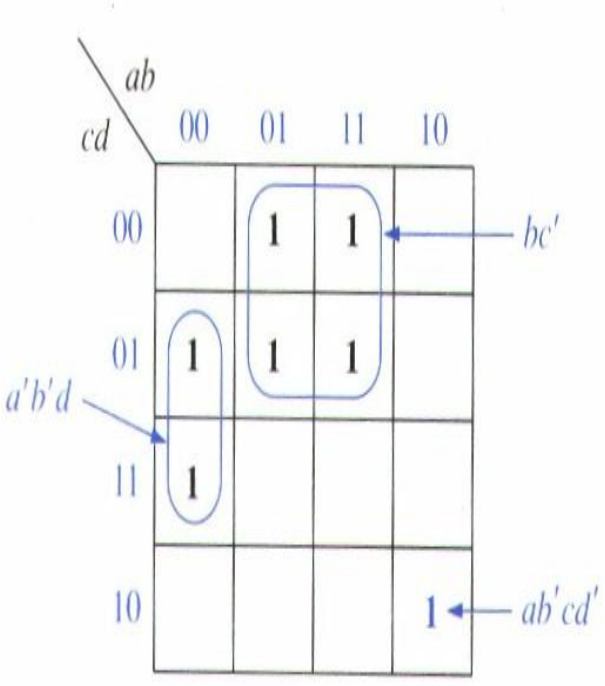
- F(a,b,c,d) = acd + a'b + d'

  acd = 1 if a=1, c=1, d=1



Simplification Example

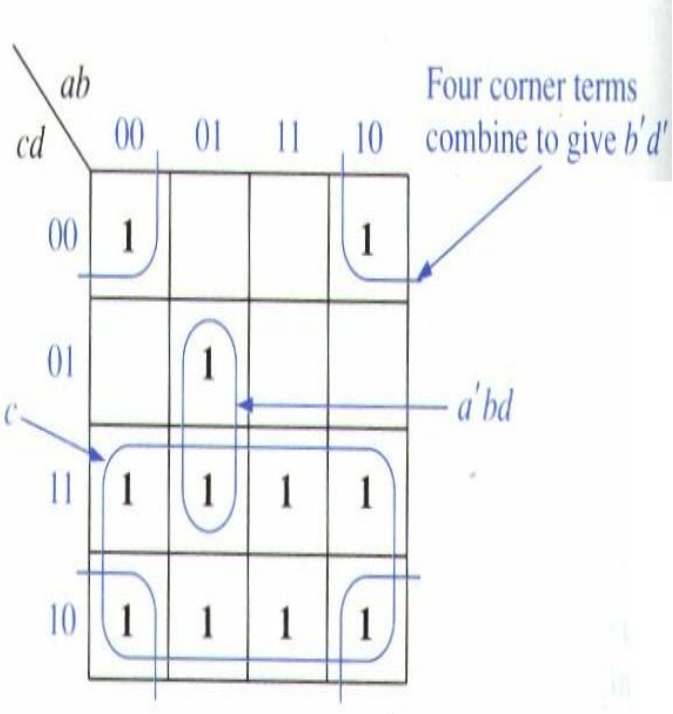
- Minterms are combined in groups of 2, 4, or 8 to eliminate 1, 2, 3 variables.
- Corner terms.



$f_1 = \Sigma m(1, 3, 4, 5, 10, 12, 13)$

$= bc' + a'b'd + ab'cd'$

(a)



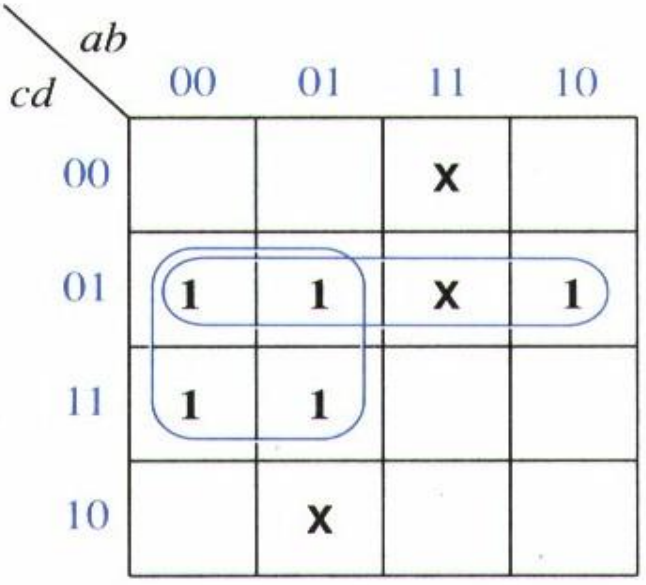
$f_2 = \Sigma m(0, 2, 3, 5, 6, 7, 8, 10, 11, 14, 15)$

$= c + b'd' + a'bd$

(b)

Simplification with Don't Care

- Don't care "x" is covered if it helps. Otherwise leave it along.



$$f = \Sigma m(1, 3, 5, 7, 9) + \Sigma d(6, 12, 13)$$
$$= a'd + c'd$$

Get a Minimum POS Using K Map

- Cover 0's to get simplified POS.
  - We want 0 in each term.

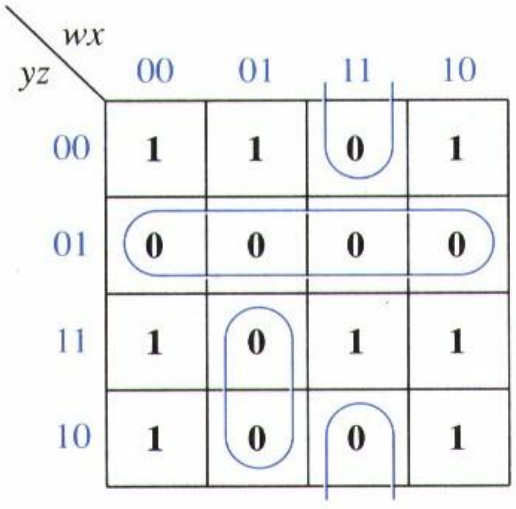
$$f = x'z' + wyz + w'y'z' + x'y$$

First, the 1's of f are plotted in Fig. 5–14. Then, from the 0's,

$$f' = y'z + wxz' + w'xy$$

and the minimum product of sums for f is

$$f = (y + z')(w' + x' + z)(w + x' + y')$$

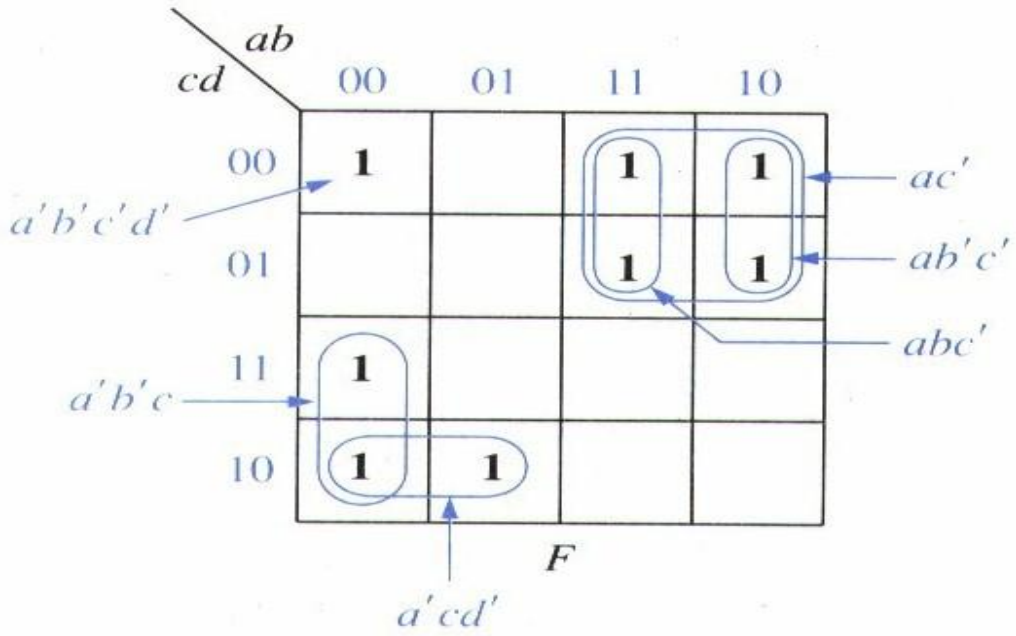


Determination of Minimum Expressions Using Essential Prime Implicants

- Definitions:
  - **Implicants**: An implicant of a function F is a single element of the on set (1) or any group of elements that can be combined together in a K-map.
  - **Prime Implicants**: An implicant that cannot be combined with another to eliminate a literal.
  - **Essential Prime Implicants**: If a particular element of the on-set is covered by a single prime implicant. That implicant is called an essential prime implicant.
- All essential primes must be part of the minimized expression.

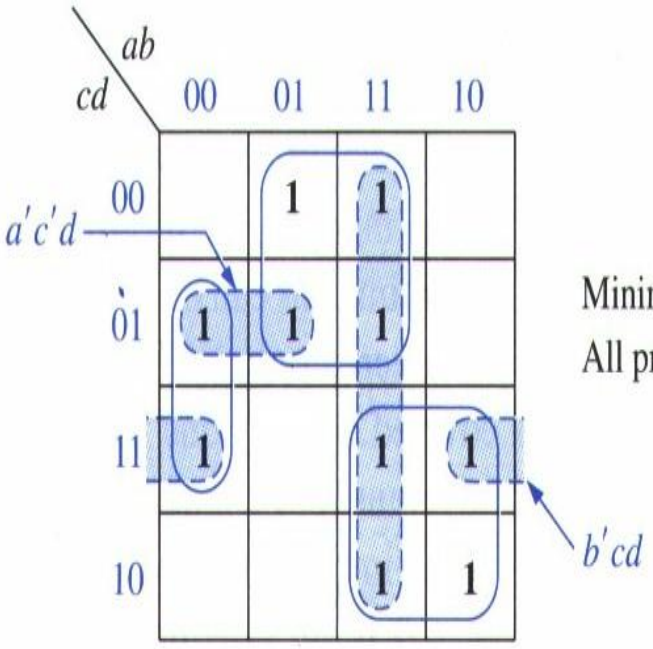
Implicant of F

- Implicant
  - Any single 1 or any group of 1's
- Prime implicant
  - An implicant that can not be combined with another term to eliminate a variable.
    - a'b'c, a'cd', ac' are prime implicant.
    - a'b'c'd' is not (combined with a'b'cd'). abc' and ab'c' are not.



Prime Implicant

- A single 1 which is not adjacent to any other 1's.
- Two adjacent 1's which are not contained in a group of four 1's. And so on.
  - Shaded loops are also prime implicants, but not part of the minimum solution.
  - a'c'd and b'cd are already covered by other group. So we do not need them.

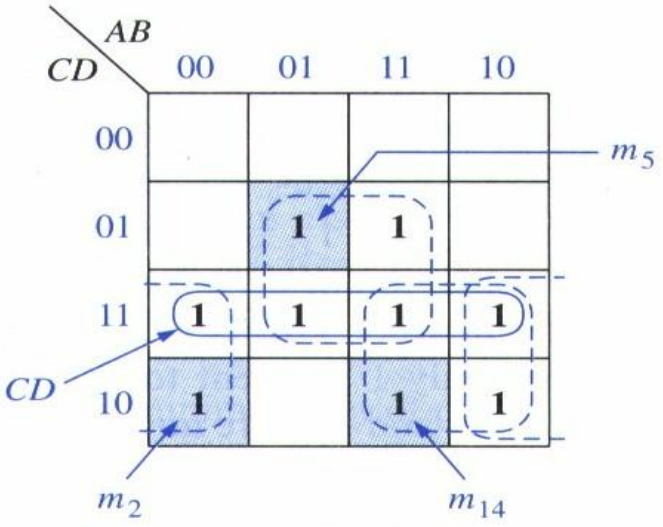


Minimum solution: $F = a'b'd + bc' + ac$

All prime implicants: $a'b'd$, bc', ac, $a'c'd$, ab, $b'cd$

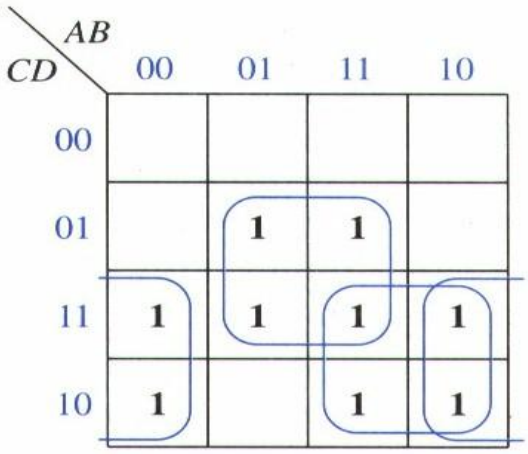
Essential Prime Implicants for Minimum SOP

- If CD is chosen first, then f has 4 terms. We don't need CD since it is covered by other group. CD is not a essential prime implicant.
- m2 is essential prime implicant since it is covered only be one prime implicant. So does m5, and m14. We need them in the answer.



$f = CD + BD + B'C + AC$

(a)

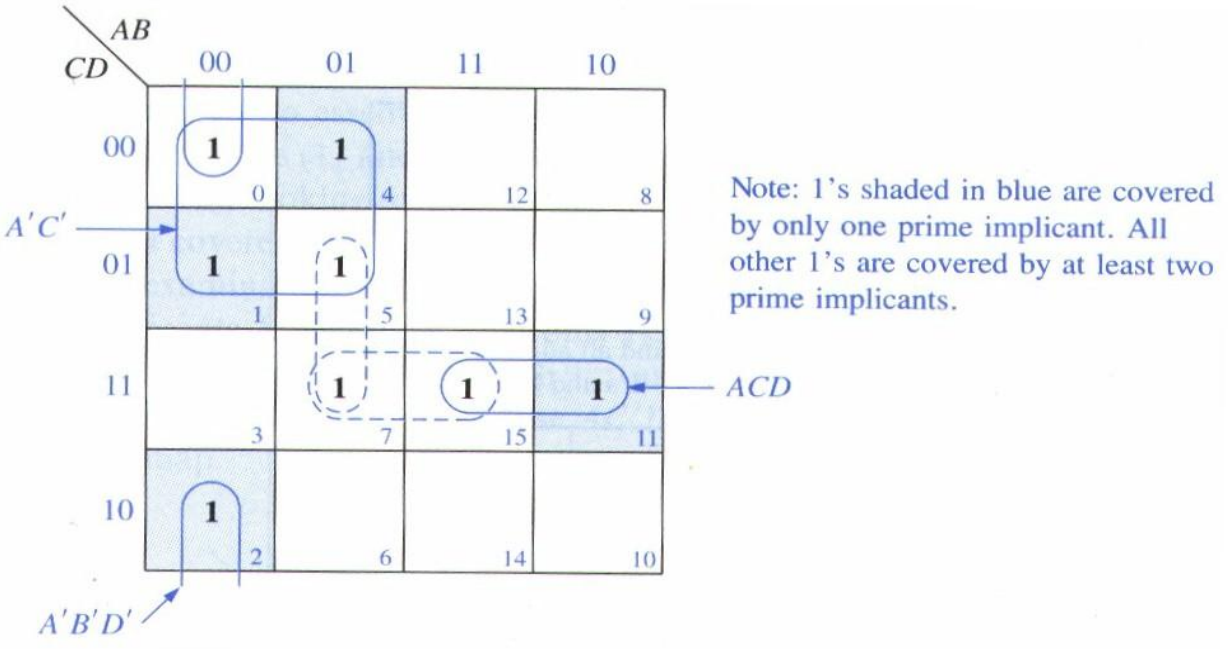


$f = BD + B'C + AC$

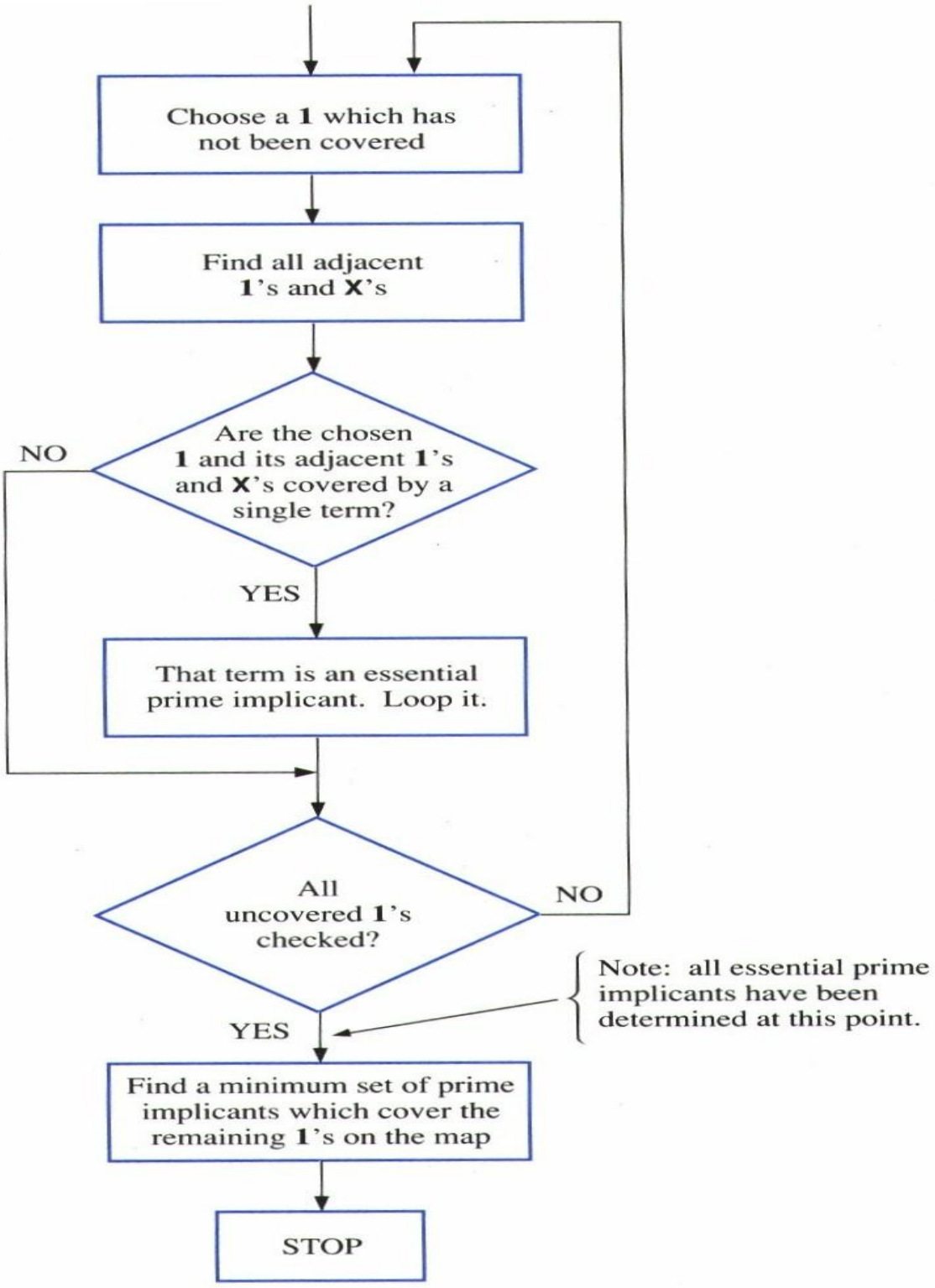
(b)

Rule of Thumb

- Look at all squares adjacent to a minterm.
  - If the given minterm and all of the 1's adjacent to it are covered by a single term, then that term is an essential prime implicant
  - But if it is covered by more than two prime implicant, we can not tell whether this term is essential or not.
  - The solution is A'C' + A'B'D' +ACD + (A'BD or BCD).

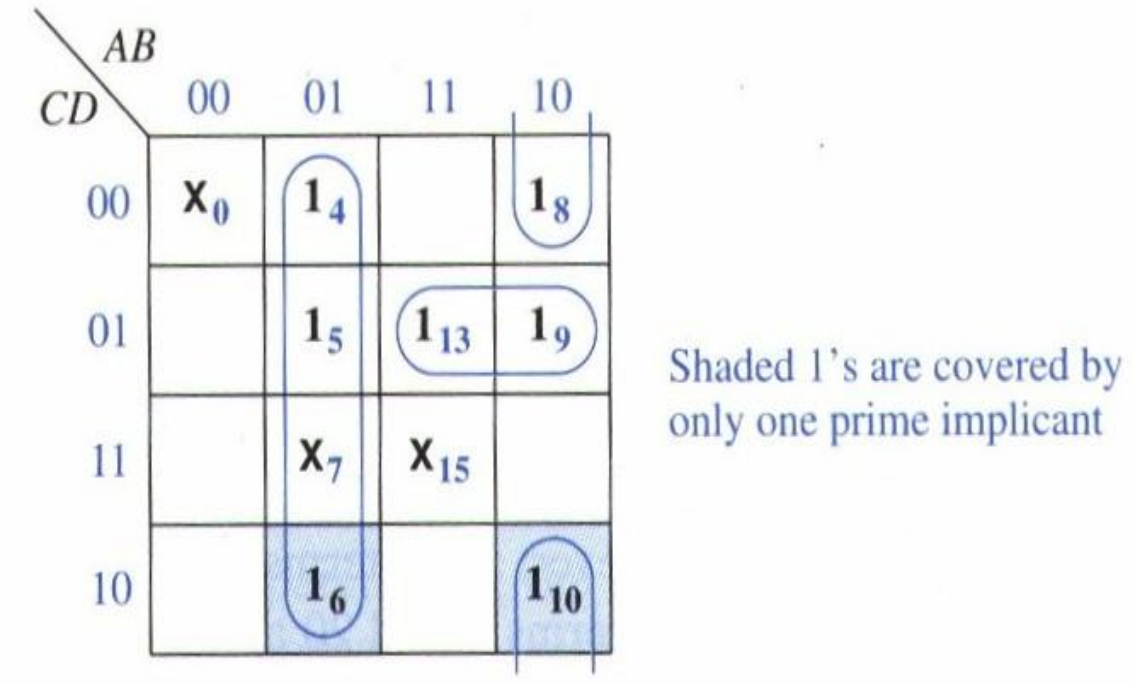


Flow Chart


Choose a 1 which has not been covered
Find all adjacent 1's and X's
Are the chosen 1 and its adjacent 1's and X's covered by a single term?
NO
YES
That term is an essential prime implicant. Loop it.
All uncovered 1's checked?
NO
YES
Note: all essential prime implicants have been determined at this point.
Find a minimum set of prime implicants which cover the remaining 1's on the map
STOP

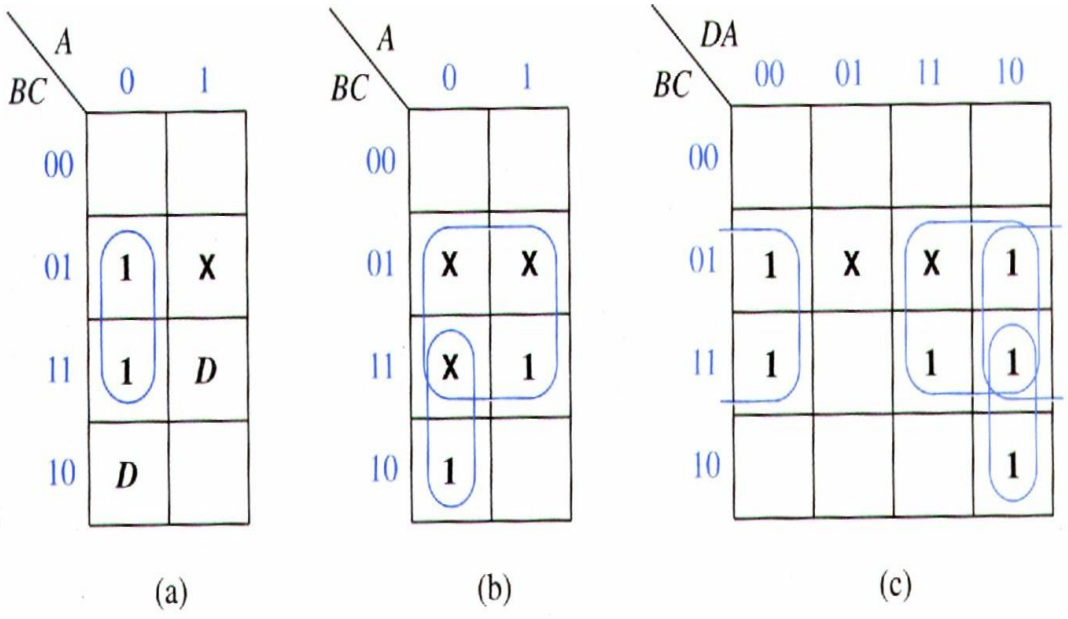
Example

- Find the 1 that is covered by only one term first (Do not share with other circle).



Map-Entered Variable

- F(A,B,C,D) = A'B'C + A'BC + A'BC'D + ABCD + (AB'C), (don't care)
  - Choose D as a map-entered variable.
  - When D = 0, F = A'C (Fig. a)
  - When D = 1, F = C + A'B (Fig. b)
    - two 1's are changed to x's since they are covered in Fig. a.
- F = A'C + D(C+A'B) = A'C + CD + A'BD



Combinational Logic

Combinational logic circuits

- → outputs logical functions of inputs
- → new outputs appear shortly after changed inputs (propagation delay)
- → no feedback loops
- → no clock

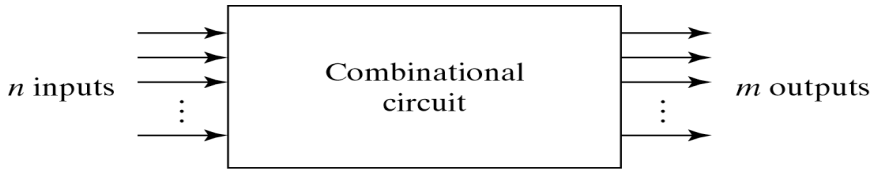


Fig. 4-1 Block Diagram of Combinational Circuit

2

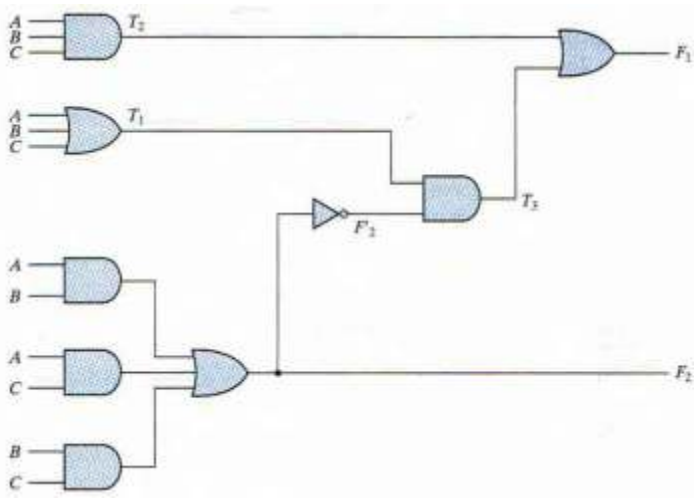
Analysis Procedure

To analyze a circuit:

1. Label all gate outputs that are a function of input variables with arbitrary symbols. Determine the Boolean functions for each gate output.
2. Label the gates that are a function of input variables and previously labeled gates with other arbitrary symbols. Find the Boolean functions for these gates.
3. Repeat the process outlined in step 2 until the outputs of the circuit are obtained.
4. By repeated substitution of previously defined functions. obtain the output Boolean functions in terms of input variables.

3

Analysis Procedure



4

Analysis Procedure

$F_2 = AB + AC + BC$

$T_1 = A + B + C$

$T_2 = ABC$



$T_3 = F_2'T_1$

$F_1 = T_3 + T_2$



$$F_1 = T_3 + T_2 = F_2'T_1 + ABC = (AB + AC + BC)'(A + B + C) + ABC$$
$$= (A' + B')(A' + C')(B' + C')(A + B + C) + ABC$$
$$= (A' + B'C')(AB' + AC' + BC' + B'C) + ABC$$
$$= A'BC' + A'B'C + AB'C' + ABC$$

5

Design Procedure

Design a circuit from a specification.

1. Determine number of required inputs and outputs.
2. Derive truth table
3. Obtain simplified Boolean functions
4. Draw logic diagram and verify correctness

6

Example: Code Converter

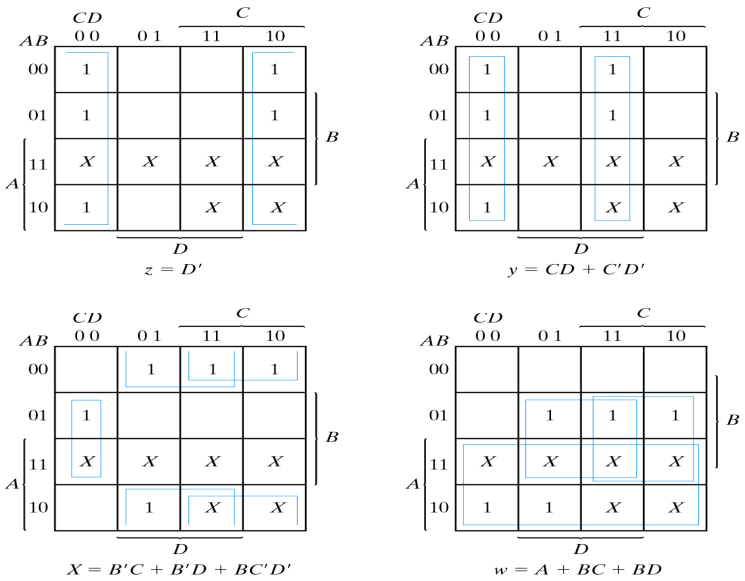
- A circuit that translates one binary code to another

- Example 3-2: BCD to Excess-3 Code Converter
  - Excess-3 code: decimal code + 3
  - BCD inputs 1010 to 1111 are don't care conditions

Decimal Digit	Input BCD				Output Excess-3			
	A	B	C	D	W	X	Y	Z
0	0	0	0	0	0	0	1	1
1	0	0	0	1	0	1	0	0
2	0	0	1	0	0	1	0	1
3	0	0	1	1	0	1	1	0
4	0	1	0	0	0	1	1	1
5	0	1	0	1	1	0	0	0
6	0	1	1	0	1	0	0	1
7	0	1	1	1	1	0	1	0
8	1	0	0	0	1	0	1	1
9	1	0	0	1	1	1	0	0

7

Simplification with K-map



8

Fig. 4-3 Maps for BCD to Excess-3 Code Converter

Further manipulation of simplified expressions

- Two-level AND-OR implementation for the circuit can be obtained directly from the Boolean expression derived from the K-MAPS.
- Further manipulation can be done on the function to allow use of common gates for multiple-output circuits.
- Thus there are several possibilities for the implementation. The following shows the implementation with 3 levels of gates.

$$W = A + BC + BD = A + B(C + D)$$

$$X = B'C + B'D + BC'D' = B'(C + D) + BC'D'$$

$$= B'(C+D) + B(C+D)'$$

$$Y = CD + C'D' = CD + (C + D)'$$

$$Z = D'$$

9

Three Level Implementation

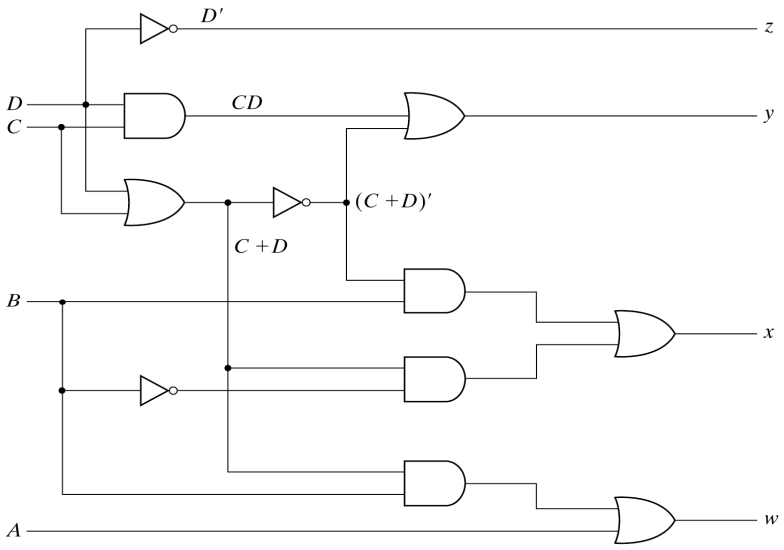


Fig. 4-4 Logic Diagram for BCD to Excess-3 Code Converter

10

Binary Adder -Subtractor

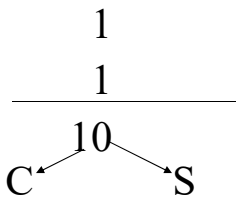
11

Half Adder

→The half-adder accepts two binary digits on its inputs and produces two binary digits on its outputs: a sum bit and a carry bit.

Truth Table

x	y	C	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

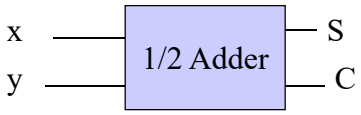


12

Half-Adder (see other implementations in Chap. 2)

Truth Table

x	y	C	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0



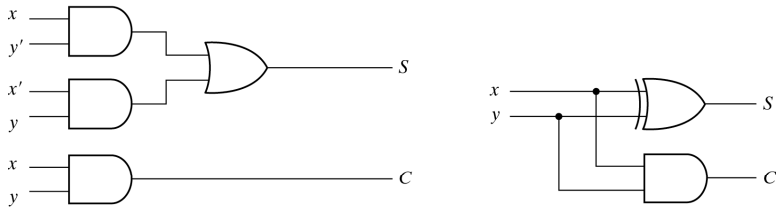
Some of products

S = x'y + xy'

C = xy

13

Half-Adder-Implementation



(a) $S = xy' + x'y$
$C = xy$

(b) $S = x \oplus y$
$C = xy$

Fig. 4-5 Implementation of Half-Adder

14

Full Adder (see other implementations in Chap. 2)

Truth Table

x	y	z	C	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

→ The Full-adder is combinational circuit



15

Full Adder (see other implementations in Chap. 2)

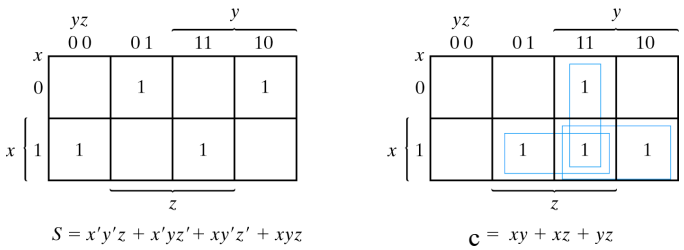


Fig. 4-6 Maps for Full Adder

S(X,Y,Z) = **Σ**m(1,2,4,7)
C (X,Y,Z) = **Σ**m(3, 5, 6, 7).

16

Full Adder (see other implementations in Chap. 2)

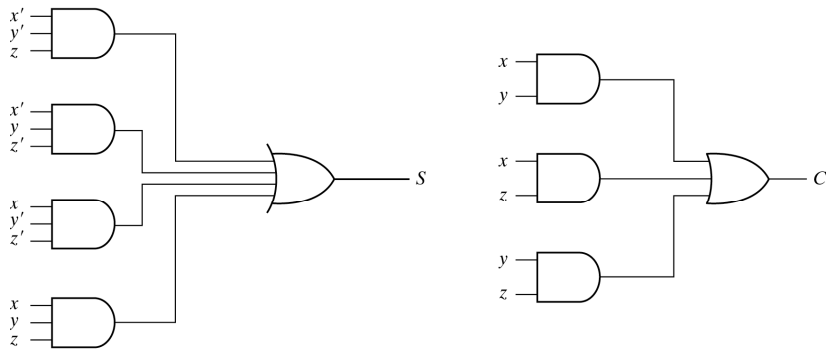


Fig. 4-7 Implementation of Full Adder in Sum of Products

17

Full Adder (same as in Chap. 2)

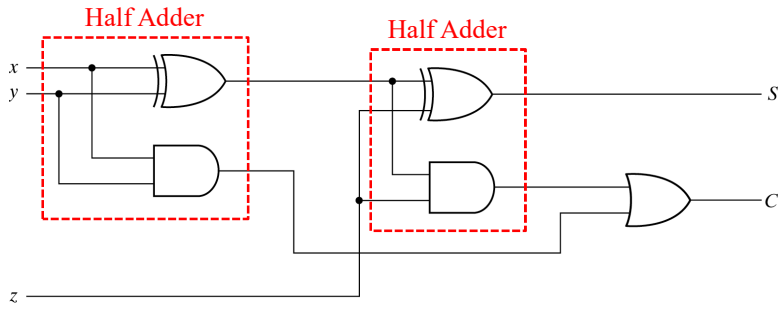
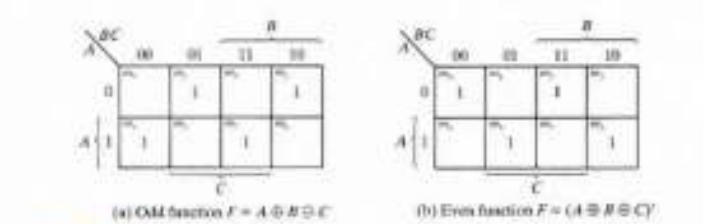


Fig. 4-8 Implementation of Full Adder with Two Half Adders and an OR Gate

18

Three -Variable Exclusive-OR



(a) Odd function $F = A \oplus B \oplus C$ (b) Even function $F = (A \oplus B \oplus C)'$

FIGURE 3.33
Map for a three-variable exclusive-OR function

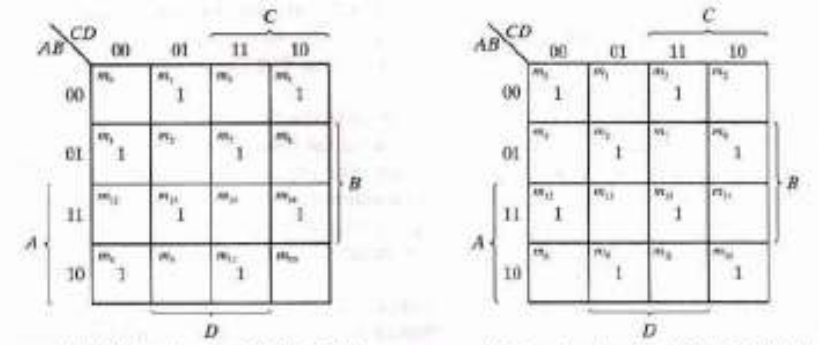


(a) 3-input odd function (b) 3-input even function

FIGURE 3.34
Logic diagram of odd and even functions

19

Four -Variable Exclusive-OR

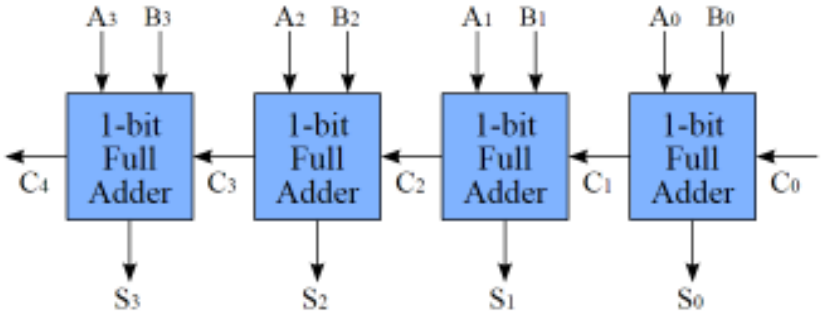


(a) Odd function $F = A \oplus B \oplus C \oplus D$ (b) Even function $F = (A \oplus B \oplus C \oplus D)'$

FIGURE 3.35
Map for a four-variable exclusive-OR function

20

Four – bit Adder



21

Adder - Subtractor

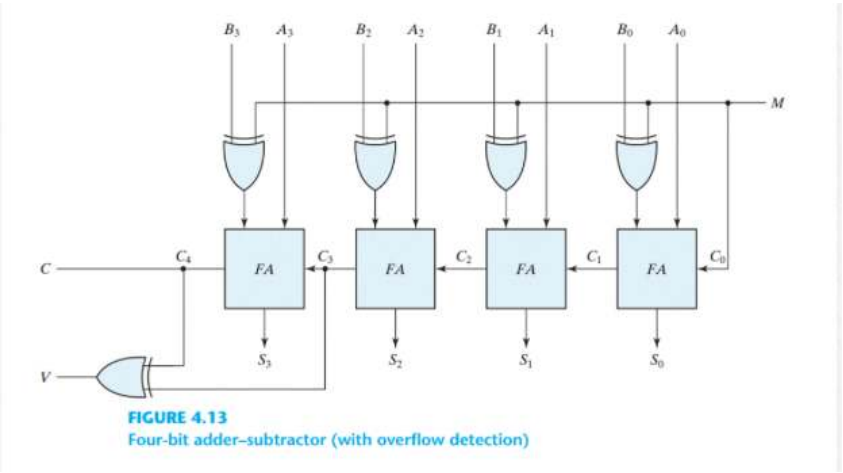
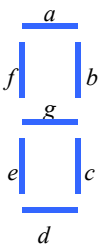


FIGURE 4.13
Four-bit adder–subtractor (with overflow detection)

22

Decoder -Example

a BCD to Seven Segment Decoder inputs data in BCD form and converts it to a seven segment output



(a) Segment designation

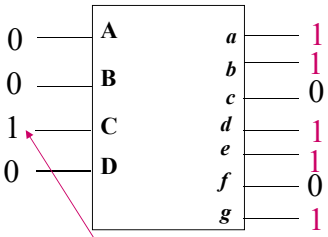


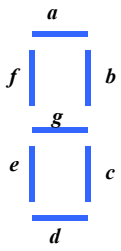
(b) Numerical designation for display

23

A- BCD to Seven Segment Decoder

A	B	C	D	a	b	c	d	e	f	g
0	0	0	0	1	1	1	1	1	1	0
0	0	0	1	0	1	1	0	0	0	0
0	0	1	0	1	1	0	1	1	0	1
0	0	1	1	1	1	1	1	0	0	1
0	1	0	0	0	1	1	0	0	1	1
0	1	0	1	1	0	1	1	0	1	1
0	1	1	0	1	0	1	1	1	1	1
0	1	1	1	1	1	1	0	0	0	0
1	0	0	0	1	1	1	1	1	1	1
1	0	0	1	1	1	1	1	0	1	1
1	0	1	0	x	x	x	x	x	x	x
1	0	1	1	x	x	x	x	x	x	x
1	1	0	0	x	x	x	x	x	x	x
1	1	0	1	x	x	x	x	x	x	x
1	1	1	0	x	x	x	x	x	x	x
1	1	1	1	x	x	x	x	x	x	x





Don't care terms

24

K-MAP

AB \ CD	00	01	11	10
00	1		1	1
01		1	1	1
11				
10	1	1		

***a* =**

AB \ CD	00	01	11	10
00	1	1	1	1
01	1		1	
11				
10	1	1		

***b* =**

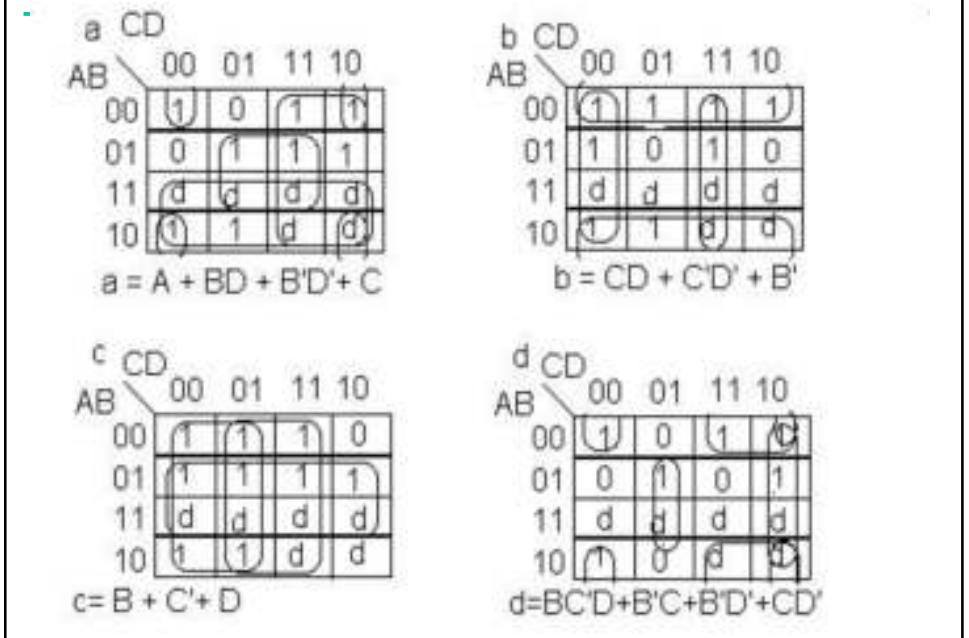
AB \ CD	00	01	11	10
00	1	1	1	
01	1	1	1	1
11				
10	1	1		

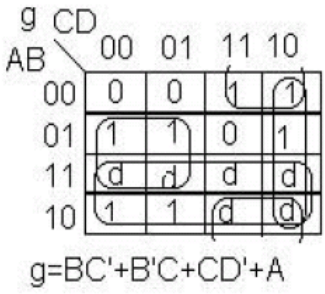
***c* =**

AB \ CD	00	01	11	10
00	1		1	1
01		1		1
11				
10	1	1		

***d* =**

25

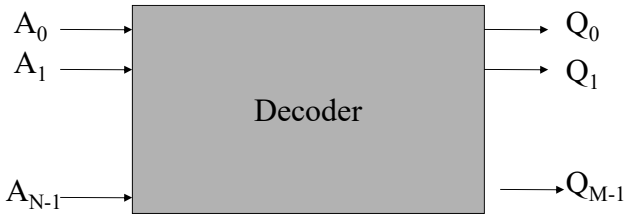




27

Decoder

- A decoder is a combinational circuit that converts binary information from n input lines to a maximum of 2^n unique output lines. → *n-to-* 2^n decoder
- If the n-bit coded information has unused combinations → less than 2^n outputs.
  → *n-to-m* decoder, $m \leq 2^n$, Example: BCD-to-7-segment decoder, where *n=4* and *m=7*



28

2-to-4 Decoder

→A 2-to-4 decoder operates according to the following truth table.

- The 2-bit input is called S1S0, and the four outputs are Q0-Q3.
- If the input is the binary number i, then output Qi is uniquely true.

S1	S0	Q0	Q1	Q2	Q3
0	0	1	0	0	0
0	1	0	1	0	0
1	0	0	0	1	0
1	1	0	0	0	1

- For example, if the input S1 S0 = 10 (decimal 2), then output Q2 is true, and Q0, Q1, Q3 are all false.
- This logic circuit "decodes" a binary number into a "one-of-four" code.

29

Building a 2-to-4 decoder?

- Same deign procedure as for the combinational logic circuit (see previous slides). From the truth table, we can derive equations for each of the four outputs (Q0-Q3), based on the two inputs (S0-S1).

S1	S0	Q0	Q1	Q2	Q3
0	0	1	0	0	0
0	1	0	1	0	0
1	0	0	0	1	0
1	1	0	0	0	1

- There is no much to be simplified. the equations are:

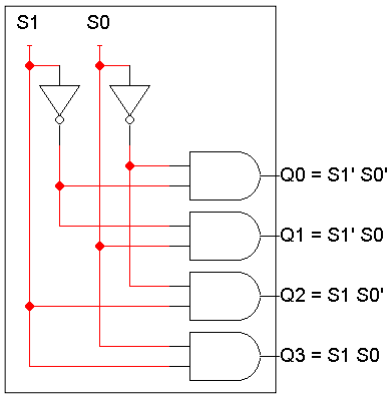
$Q0 = S1' S0'$
$Q1 = S1' S0$
$Q2 = S1 S0'$
$Q3 = S1 S0$

30

Implementation of 2-to-4 decoder

S1	S0	Q0	Q1	Q2	Q3
0	0	1	0	0	0
0	1	0	1	0	0
1	0	0	0	1	0
1	1	0	0	0	1

Q0 = *S1′ S0′*
Q1 = S1′ S0
Q2 = *S1 S0′*
Q3 = S1 S0



31

Enable inputs

- Many devices have an additional enable input, which is used to "activate" or "deactivate" the device.
- For a decoder,
  - EN=1 activates the decoder, so it behaves as specified earlier. Exactly one of the outputs will be 1.
  - EN=0 "deactivates" the decoder. By convention, that means *all* of the decoder's outputs are 0.
- We can include this additional input in the decoder's truth table:

EN	S1	S0	Q0	Q1	Q2	Q3
0	0	0	0	0	0	0
0	0	1	0	0	0	0
0	1	0	0	0	0	0
0	1	1	0	0	0	0
1	0	0	1	0	0	0
1	0	1	0	1	0	0
1	1	0	0	0	1	0
1	1	1	0	0	0	1

32

abbreviated truth tables

- In this table, note that whenever EN=0, the outputs are always 0, ***regardless*** of inputs S1 and S0.

EN	S1	S0	Q0	Q1	Q2	Q3
0	0	0	0	0	0	0
0	0	1	0	0	0	0
0	1	0	0	0	0	0
0	1	1	0	0	0	0
1	0	0	1	0	0	0
1	0	1	0	1	0	0
1	1	0	0	0	1	0
1	1	1	0	0	0	1

- We can abbreviate the table by writing x's in the input columns for S1 and S0.

EN	S1	S0	Q0	Q1	Q2	Q3
0	x	x	0	0	0	0
1	0	0	1	0	0	0
1	0	1	0	1	0	0
1	1	0	0	0	1	0
1	1	1	0	0	0	1

33

Decoder- a Minterm Generator

S1	S0	Q0	Q1	Q2	Q3
0	0	1	0	0	0
0	1	0	1	0	0
1	0	0	0	1	0
1	1	0	0	0	1

Q0= S1′ S0′
Q1 = S1′ S0
Q2= S1 S0′
Q3= S1 S0

- Decoders are sometimes called minterm generators.
  - For each of the input combinations, exactly one output is true.
  - Each output equation contains all of the input variables.
  - These properties hold for all sizes of decoders.
- Therefore we can implement arbitrary functions with decoders.
  →from a sum of minterms equation for a function, we can use a decoder (a minterm generator) to implement that function.

34

3- to- 8 line Decoder

- Three inputs, x, y, z, are decoded into eight outputs, D0 through D7
- Each output D*i* represents one of the minterms of the 3 input variables.
- D*i* = 1 when the binary number xyz = *i*
- Shorthand: D*i* = m*i*
- The output variables are *mutually exclusive*; exactly one output has the value 1 at any time, and the other seven are 0.

35

3- to- 8 line Decoder

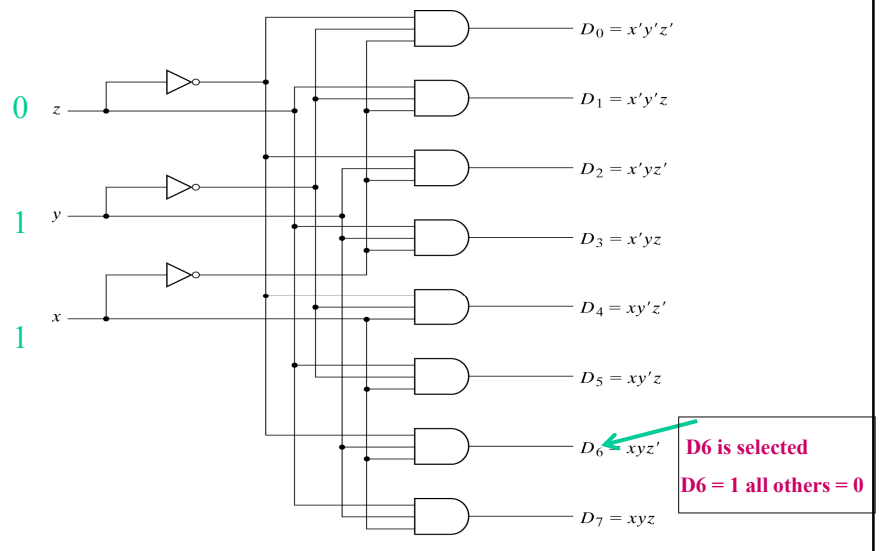


Fig. 4-18 3-to-8-Line Decoder

36

Implementing Boolean Functions Using Decoders

- Any combinational circuit can be constructed using decoders and OR gates! Why?

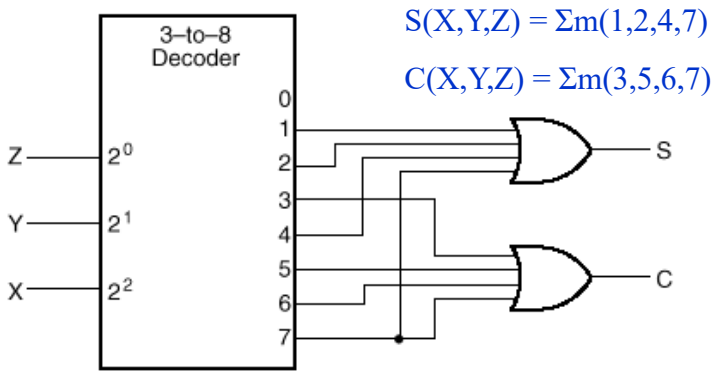
→Here is an example:
Implement a **full adder** circuit with a decoder and two OR gates.
Recall full adder equations, and let X, Y, and Z be the inputs:

$S(X,Y,Z) = \Sigma m(1,2,4,7)$
$C(X,Y,Z) = \Sigma m(3, 5, 6, 7)$.

•Since there are 3 inputs and a total of 8 minterms, we need a 3-to-8 decoder.

37

Implementing Boolean Functions Using Decoders



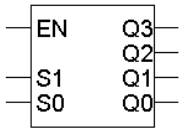
$S(X,Y,Z) = \Sigma m(1,2,4,7)$

$C(X,Y,Z) = \Sigma m(3,5,6,7)$

38

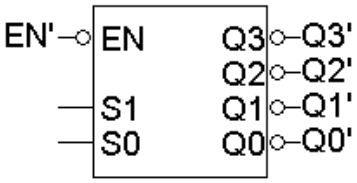
Implementing a decoder with NAND gates

- The decoders studied so far are active-high decoders (i.e. implemented with AND gates)



EN	S1	S0	Q0	Q1	Q2	Q3
0	x	x	0	0	0	0
1	0	0	1	0	0	0
1	0	1	0	1	0	0
1	1	0	0	0	1	0
1	1	1	0	0	0	1

- Active-low decoders are implemented using NAND gates (i.e. with an inverted EN input and inverted outputs).

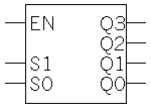


EN	S1	S0	Q0	Q1	Q2	Q3
0	0	0	0	1	1	1
0	0	1	1	0	1	1
0	1	0	1	1	0	1
0	1	1	1	1	1	0
1	x	x	1	1	1	1

39

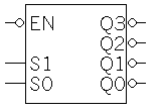
Active-High and Active-Low decoders

- Active-high decoders generate *minterms*, as we have already seen.



Q3 = S1 S0
Q2 = S1 S0'
Q1 = S1' S0
Q0 = S1' S0'

- Active-low decoders generate *Maxterms*.



Q3' = (S1 S0)' = S1' + S0'
Q2' = (S1 S0')' = S1' + S0
Q1' = (S1' S0)' = S1 + S0'
Q0' = (S1' S0')' = S1 + S0

40

Building a 3-to-8 decoder with two 2-to-4 decoders

- Another way to design a 3-to-8 decoder is to use two 2-to-4 decoders.
- from the truth table of 3-8 decoder we can notice some patterns:
  - When S2 = 0, outputs Q0-Q3 are generated as in a 2-to-4 decoder.
  - When S2 = 1, outputs Q4-Q7 are generated as in a 2-to-4 decoder.
  - → S2 can be used as an Enable input that activates/deactivates the 2-to-4 decoders.

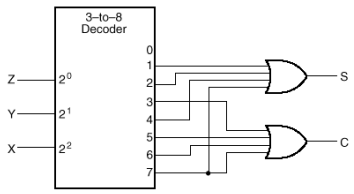
S2	S1	S0	Q0	Q1	Q2	Q3	Q4	Q5	Q6	Q7
0	0	0	1	0	0	0	0	0	0	0
0	0	1	0	1	0	0	0	0	0	0
0	1	0	0	0	1	0	0	0	0	0
0	1	1	0	0	0	1	0	0	0	0
1	0	0	0	0	0	0	1	0	0	0
1	0	1	0	0	0	0	0	1	0	0
1	1	0	0	0	0	0	0	0	1	0
1	1	1	0	0	0	0	0	0	0	1

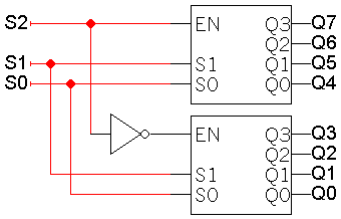
Q0 = S2' S1' S0' = m_0
Q1 = S2' S1' S0 = m_1
Q2 = S2' S1 S0' = m_2
Q3 = S2' S1 S0 = m_3
Q4 = S2 S1' S0' = m_4
Q5 = S2 S1' S0 = m_5
Q6 = S2 S1 S0' = m_6
Q7 = S2 S1 S0 = m_7

41

Building a 3-to-8 decoder with two 2-to-4 decoders

- We can use enable inputs to string decoders together. 3-to-8 decoder constructed from two 2-to-4 decoders:



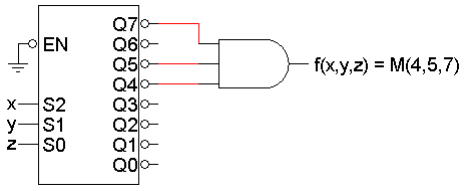


S2	S1	S0	Q0	Q1	Q2	Q3	Q4	Q5	Q6	Q7
0	0	0	1	0	0	0	0	0	0	0
0	0	1	0	1	0	0	0	0	0	0
0	1	0	0	0	1	0	0	0	0	0
0	1	1	0	0	0	1	0	0	0	0
1	0	0	0	0	0	0	1	0	0	0
1	0	1	0	0	0	0	0	1	0	0
1	1	0	0	0	0	0	0	0	1	0
1	1	1	0	0	0	0	0	0	0	1

42

Active-low decoder example

- We can use active-low decoders to implement arbitrary functions as a product of maxterms.
- For example, here is an implementation of the function f(x,y,z) = ΠM(4,5,7), using an active-low decoder.
- f(x,y,z) = ∑m(0,1,2,3,6),



- The "ground" symbol connected to EN represents logical 0, so this decoder is always enabled.
- We need an AND gate for a product of sums.

43

Decoder Expansions

- Larger decoders can be constructed using a number of smaller ones.

→ HIERARCHICAL design

- *Example:*
  A 6-to-64 decoder can be designed using four 4-to-16 and one 2-to-4 decoders. How? (tip: Use the 2-to-4 decoder to generate the enable signals to the four 4-to-16 decoders).

44

4-input tree decoder

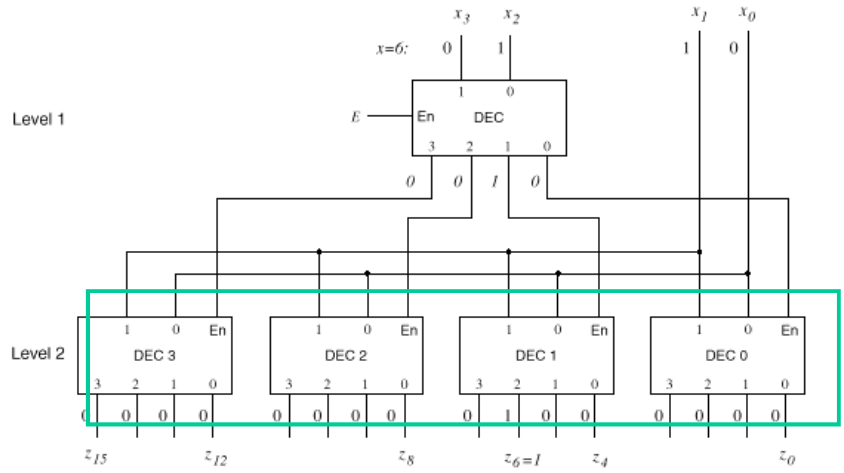


Figure 9.8. 4- nput tree decoder

Binary Encoder

•An encoder has a number of input lines, only one of which is activated at a given time.
•The opposite of the decoding process.

- takes ALL its data inputs one at a time and then converts the one whose value is equal to "1" into a single encoded output

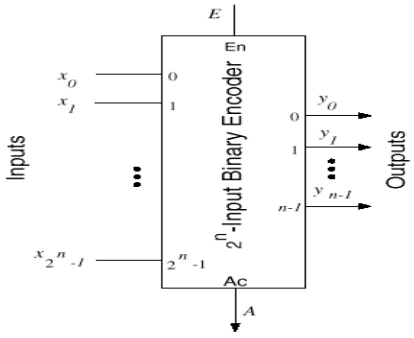


Figure 9.12. 2--input binary encoder.

Binary Encoder

Example: **8-to-3 Encoder**

x_7	x_6	x_5	x_4	x_3	x_2	x_1	x_0	y_2	y_1	y_0
0	0	0	0	0	0	0	1	0	0	0
0	0	0	0	0	0	1	0	0	0	1
0	0	0	0	0	1	0	0	0	1	0
0	0	0	0	1	0	0	0	0	1	1
0	0	0	1	0	0	0	0	1	0	0
0	0	1	0	0	0	0	0	1	0	1
0	1	0	0	0	0	0	0	1	1	0
1	0	0	0	0	0	0	0	1	1	1

47

Binary Encoder

- In the previous truth table each line selected (line 0 through 7) generates it's own binary code such as a 1 is a 001, 5 is a 101 and so on.
- Boolean functions for Outputs

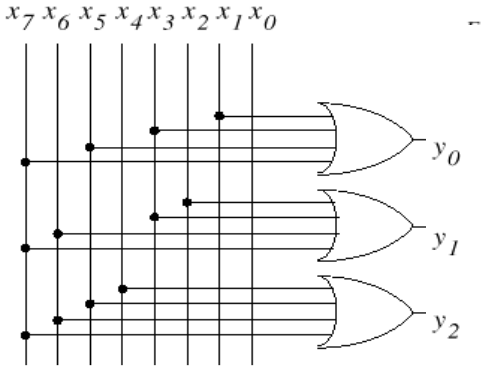
$$y2 = x4 + x5 + x6 + x7$$

$$y1 = x2 + x3 + x6 + x7$$

$$y0 = x1 + x3 + x5 + x7$$

48

Binary Encoder



49

Simple Encoder Design Issues

- There are two ambiguities associated with the design of a simple encoder:
  - Only one input can be active at any given time. If two inputs are active simultaneously, the output produces an undefined combination (for example, if x3 and x6 are 1 simultaneously, the output of the encoder will be 111. (100 and 011)
  - An output with all 0's can be generated when all the inputs are 0's,or when x0 is equal to 1.

50

Binary Encoder

Example: **8-to-3 Encoder**

x_7	x_6	x_5	x_4	x_3	x_2	x_1	x_0	y_2	y_1	y_0
0	0	0	0	0	0	0	1	0	0	0
0	0	0	0	0	0	1	0	0	0	1
0	0	0	0	0	1	0	0	0	1	0
0	0	0	0	1	0	0	0	0	1	1
0	0	0	1	0	0	0	0	1	0	0
0	0	1	0	0	0	0	0	1	0	1
0	1	0	0	0	0	0	0	1	1	0
1	0	0	0	0	0	0	0	1	1	1

51

Priority Encoders

- Solves the ambiguities multiple assigned inputs are allowed; one has priority over all others.
- Separate indication of not assigned inputs (all inputs are 0s).

52

Example: 4-to-2 Priority Encoder Truth Table

Inputs				Outputs		
D_3	D_2	D_1	D_0	A_1	A_0	V
0	0	0	0	X	X	0
0	0	0	1	0	0	1
0	0	1	X	0	1	1
0	1	X	X	1	0	1
1	X	X	X	1	1	1

53

4-to-2 Priority Encoder (cont.)

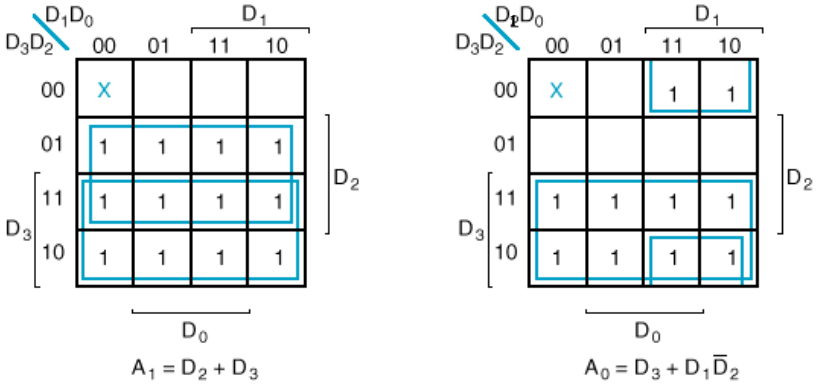
- The operation of the priority encoder is such that:

→If two or more inputs are equal to 1 at the same time, the input in the highest-numbered position will take precedence.

→A ***valid output indicator***, designated by V, is set to 1 only when one or more inputs are equal to 1. $V = D_3 + D_2 + D_1 + D_0$ by inspection.

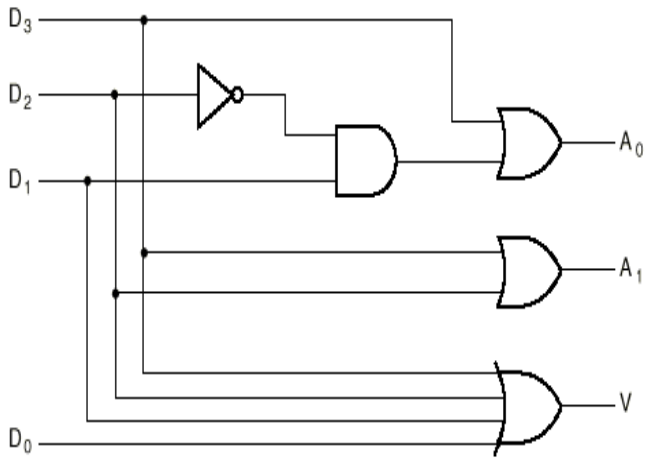
54

4-to-2 Priority Encoder (cont.)



55

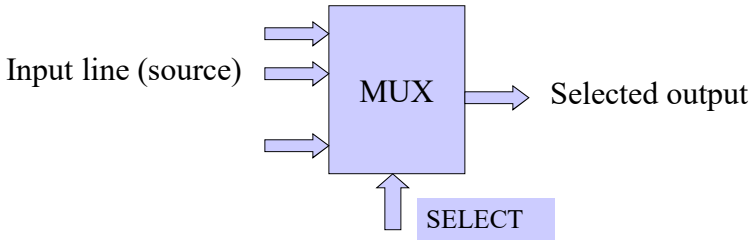
4-to-2 Priority Encoder (cont.)



56

Multiplexer (MUX): Data Selectors

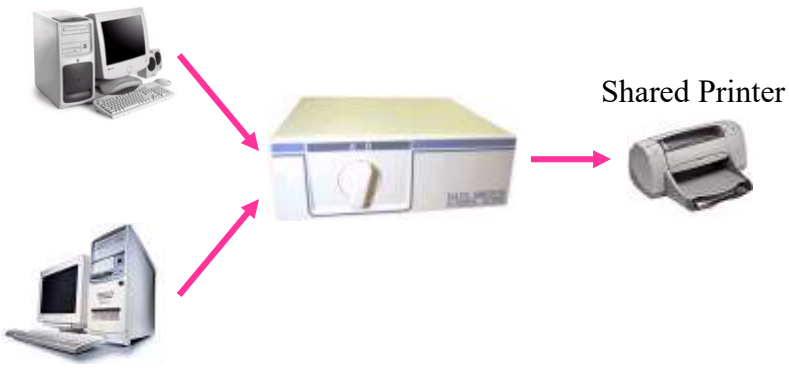
- A multiplexer selects one of several input signals and passes it on to the output.
- Routing of selected data input to the output is controlled by SELECT inputs.



57

Multiplexer

- Multiplexers, or muxes, are used to choose between resources.
- A real-life example: in the old days before networking, several computers could share one printer through the use of a switch.

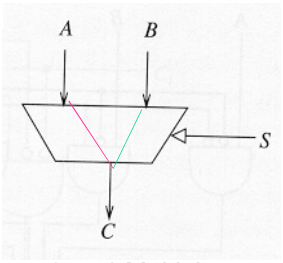


58

Multiplexer (MUX): Data Selectors [2]

- A combinational circuit with 2^n *data inputs*, 1 data output and a number of bit *control input* that select one of the data inputs

C takes the value of A or B depending on the value of S



2-to-1 Multiplexer

S	A	B	C	
0	0	0	0	
0	0	1	0	C=A;
0	1	0	1	
0	1	1	1	
1	0	0	0	
1	0	1	1	C=B;
1	1	0	0	
1	1	1	1	

59

2-to-1 Multiplexer

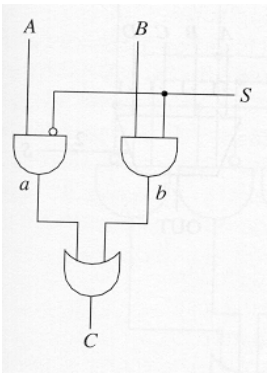
When S= 0 → C= A, when S= 1 → C = B

$C = \overline{S}.A.\overline{B} + \overline{S}.A.B + S.\overline{A}.B + S.A.B$

$= \overline{S}.A + S.B$

Inputs			Output
S	A	B	C
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

Two level implementation



60

Synchronous Sequential Logic

- A digital system is composed of

  - Registers → to store data

  - Data processing units → CPU (process or compute operations + - * % …)

- Binary information (0,1) is transferred from one set of registers into another

- Transfer can be done directly or via a processing unit to perform an operation

Registers → To store data in the memory

- A register is a group of *Binary cells*
- a register with *n* cells can store *n* bits

Example:

- Register with 8 cells can store 8 bits and can be in one of 2^8 possible states

1	0	0	0	0	0	1	1

Register A

The content is function of the interpretation given to the information stored in to it. Here it's the positive integer (+3)

Registers

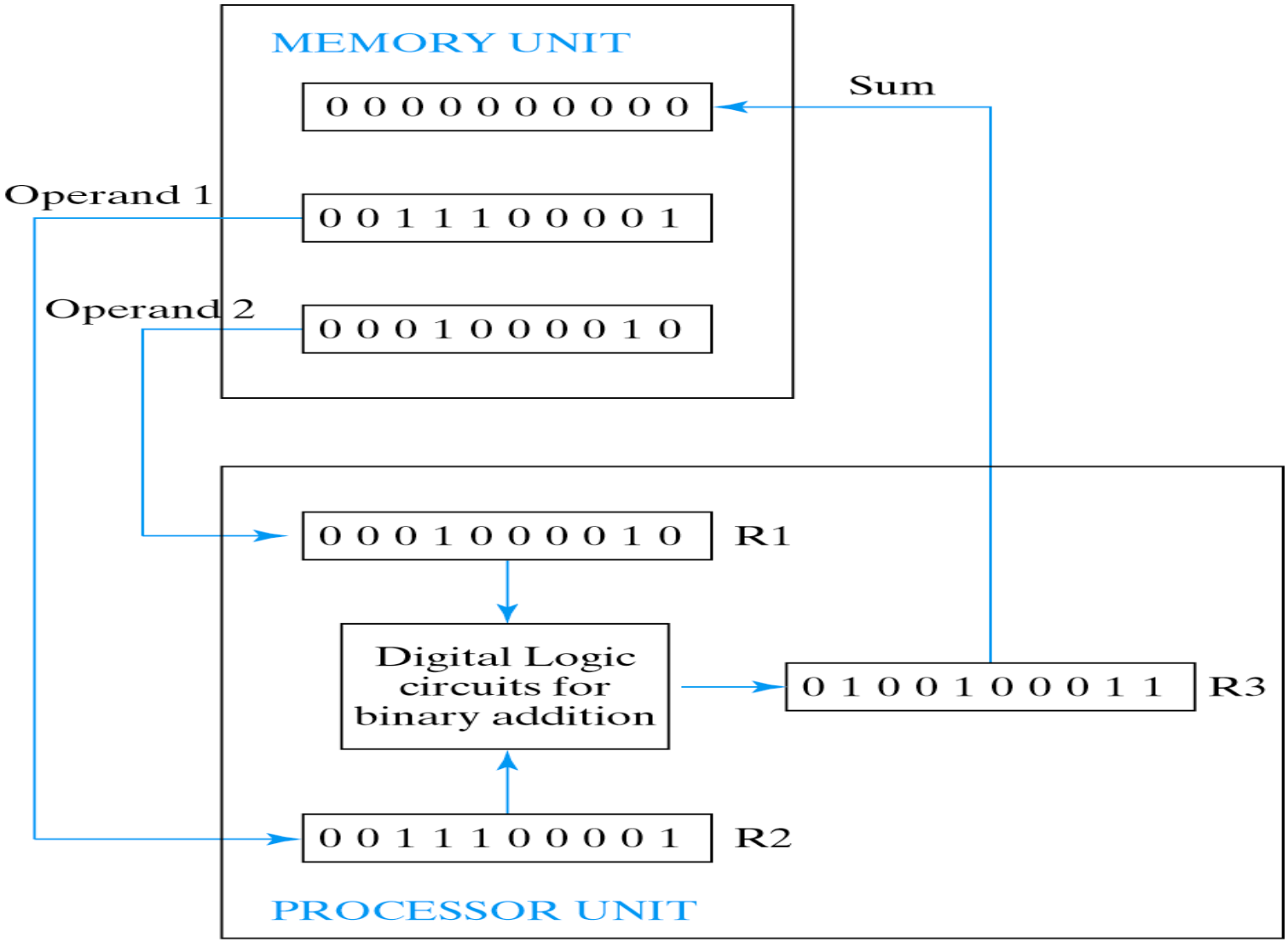


Fig. 1-2 Example of binary information processing

Sequential Circuits

→Outputs logical functions of inputs and previous history of circuit (memory)

→ After changed inputs, new outputs appear in the next clock cycle

→ Feedback loops



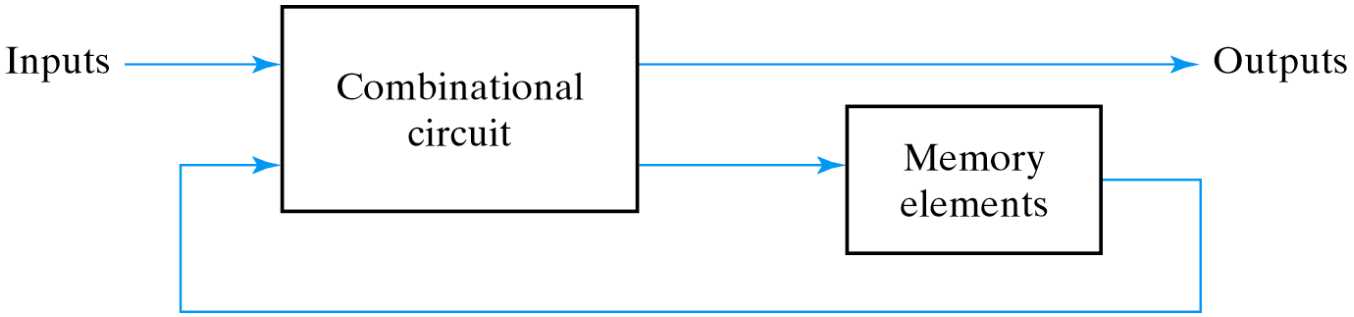


Fig. 5-1 Block Diagram of Sequential Circuit

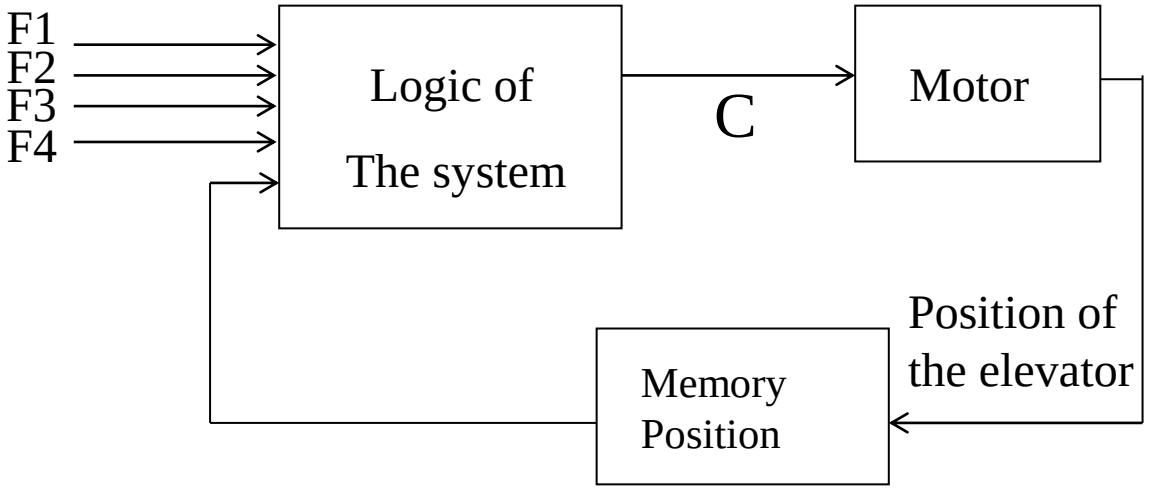
Sequential logic circuits -Example

-- Elevator Control (1/2) --

•The buttons are the input variables of the system

• The position is also an input variable which represents the current state of the elevator.

• in the figure the command control 'C' depends on the button selection AND and the position of the elevator (I.e. the previous state of the system: before the activation of the buttons)



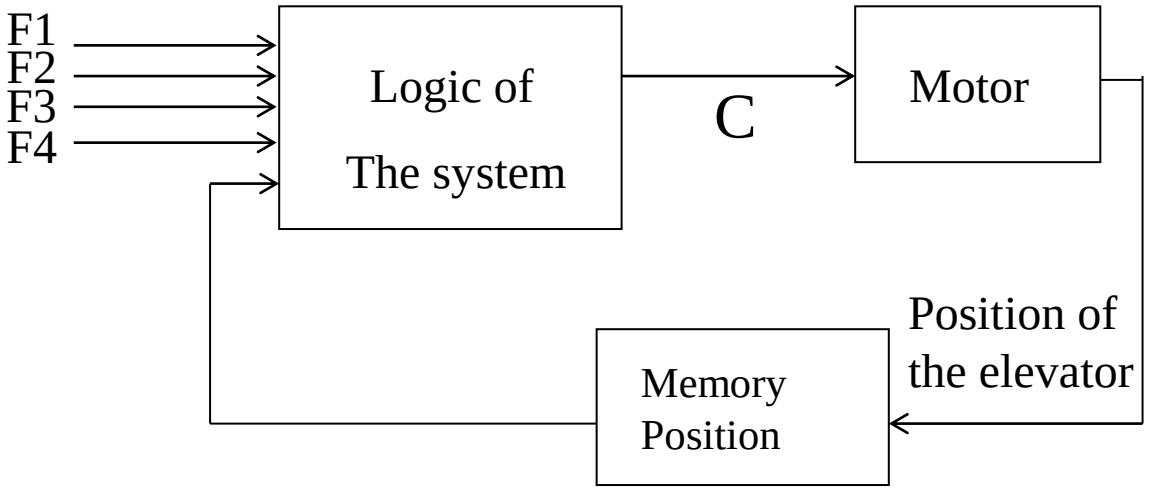
Sequential logic circuits –Example (2)

-- Elevator Control (2/2) --

This example represents a **Sequential** logic system That is the output C depends on combination of

1 – **the input variables**

2 – **AND the previous states** (internal states).



Sequential circuits: time and memory

→ ***In contrast to the combinational circuits*** *two new parameters are to be considered*

1 - ***Time***

We have to consider the **propagation delay** of the circuits

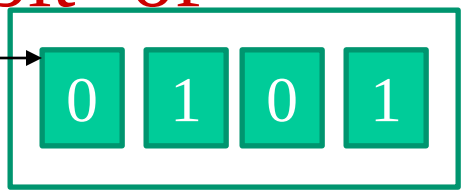
2 – ***Memory Element***

We have to keep the **previous results** and impose a predetermined order.

→ Generally speaking, the function performed by the memory element is to store information in a binary form.

Sequential logic circuits: Flip-flop

• The memory cell that stores one "bit" of information is termed **Flip-flop**.

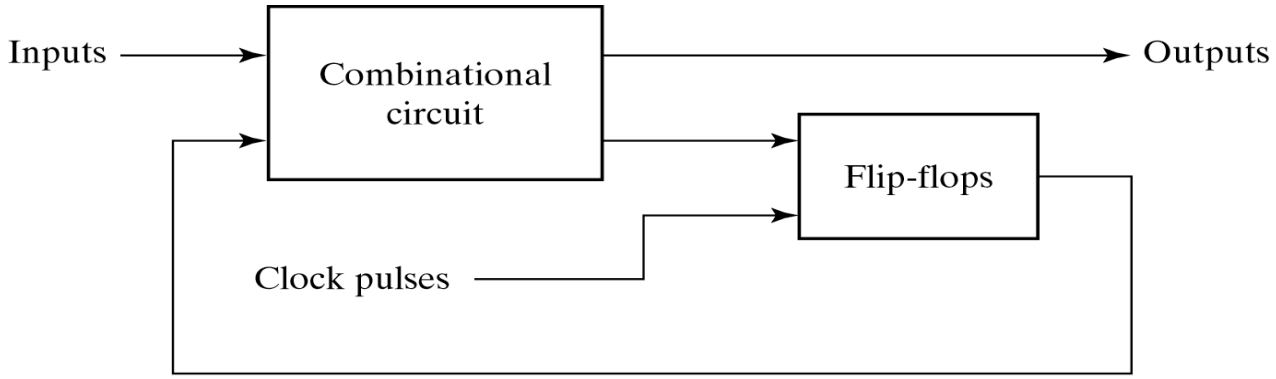


• A set of Flip-flop can be used to store several bits (one each) in a REGISTER.

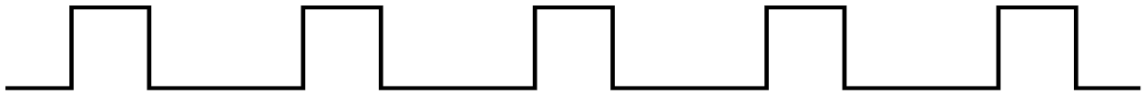
• Two different modes:

1) Asynchronous

2) Synchronous



(a) Block diagram



(b) Timing diagram of clock pulses

Fig. 5-2 Synchronous Clocked Sequential Circuit

Clock signal : definitions

• Clock is used in synchronous logic circuits to trigger the circuit (Flip-flop) allowing it to switch its state.

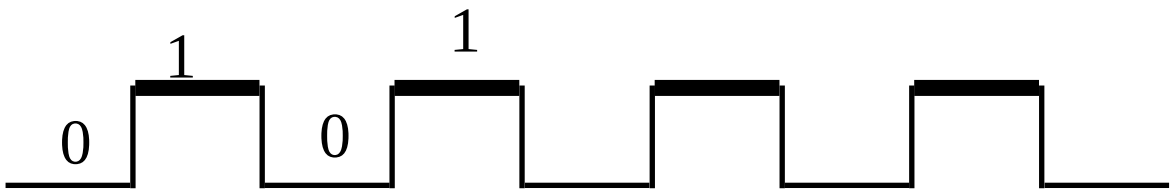
• Clock signal can trigger a flip-flop in three ways:

- During Positive level (Always 1)
- During Positive transition (or positive edge)
- During Negative transition (or negative edge)

Clock signal : definitions

1- Positive level

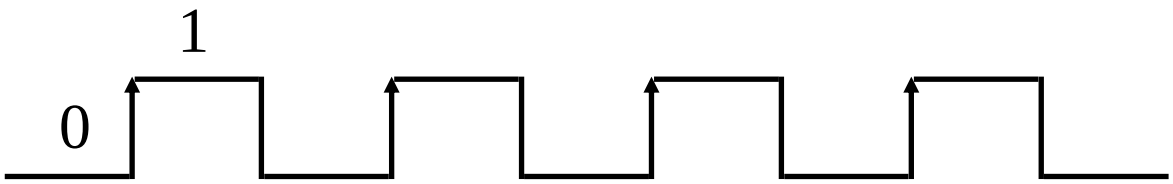
- The flip-flop is triggered while the clock pulse is at logic ‘1’
- The logic ‘0’ is **therefore** inactive state where changes to the flip-flop (state) are not allowed.

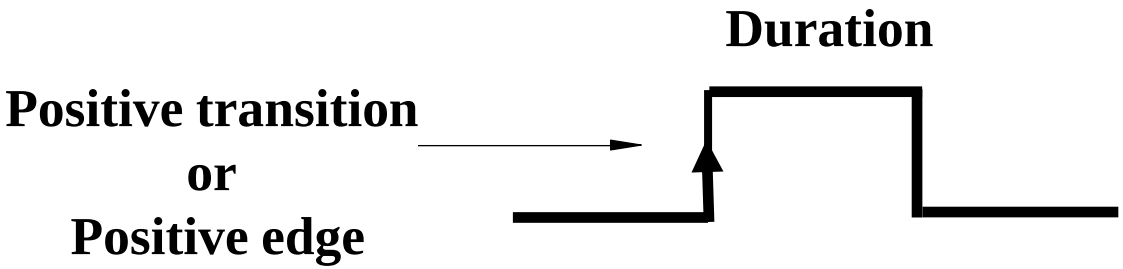


Clock signal : definitions (2)

2- Positive transition (or positive edge)

The flip flop is triggered only during the signal transition from 0 to 1 , this transition is defines as *positive edge*

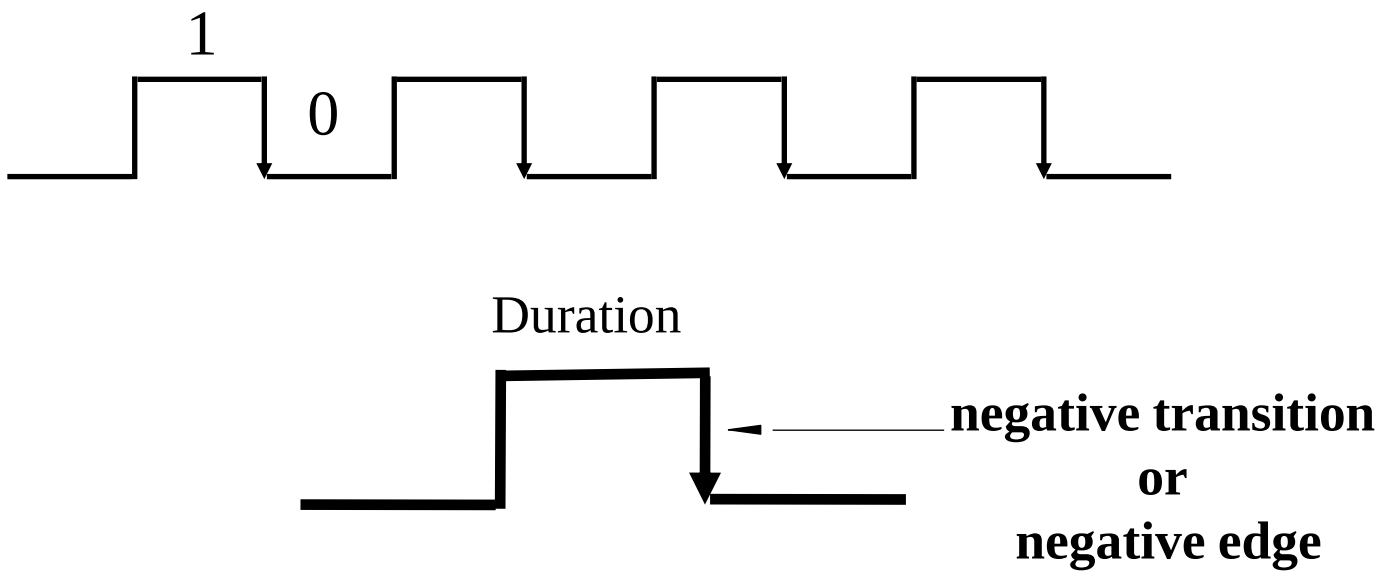




Clock signal : definitions (4)

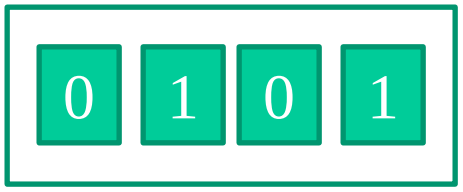
3- Negative transition (or negative edge)

The flip flop is triggered only during the signal transition from 1 to 0 , this transition is defines as *negative edge*



Memory Element (Latch)

- A flip-flop can maintain a binary state indefinitely until directed by an input signal to switch states
- The most basic types of flip-flops operate with signal levels and are referred to as Latches.
- Latches are basic circuits from which all flip-flops are constructed
- **Memory →Several Registers →Several (set) of Flip-flops -→Latches**



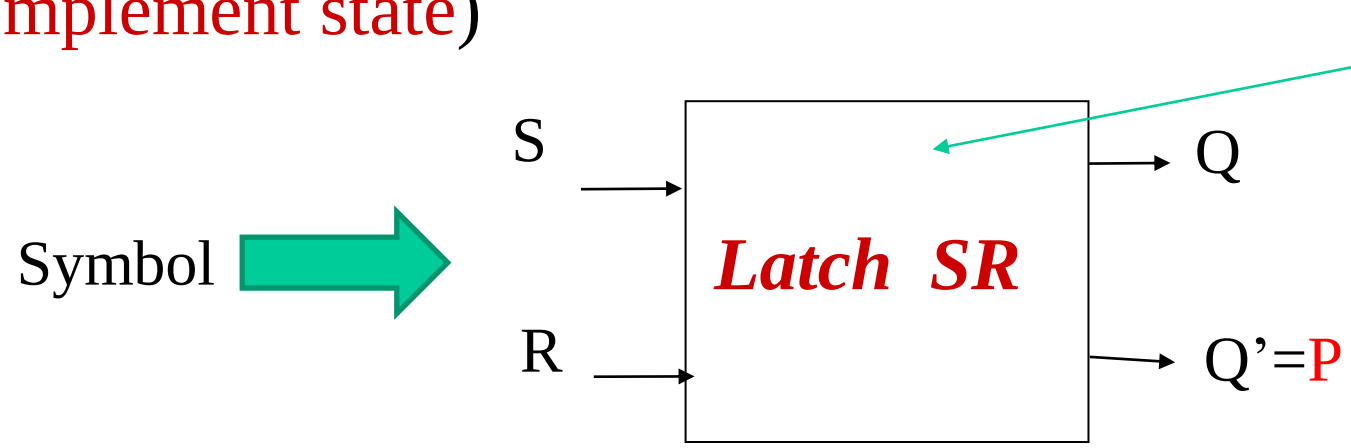
Reg.

Memory Element (SR Latch)

• Flip-flop (Latch) SR has two input variables that are defined as S and R

- S for SET → 1
- R for RESET → 0

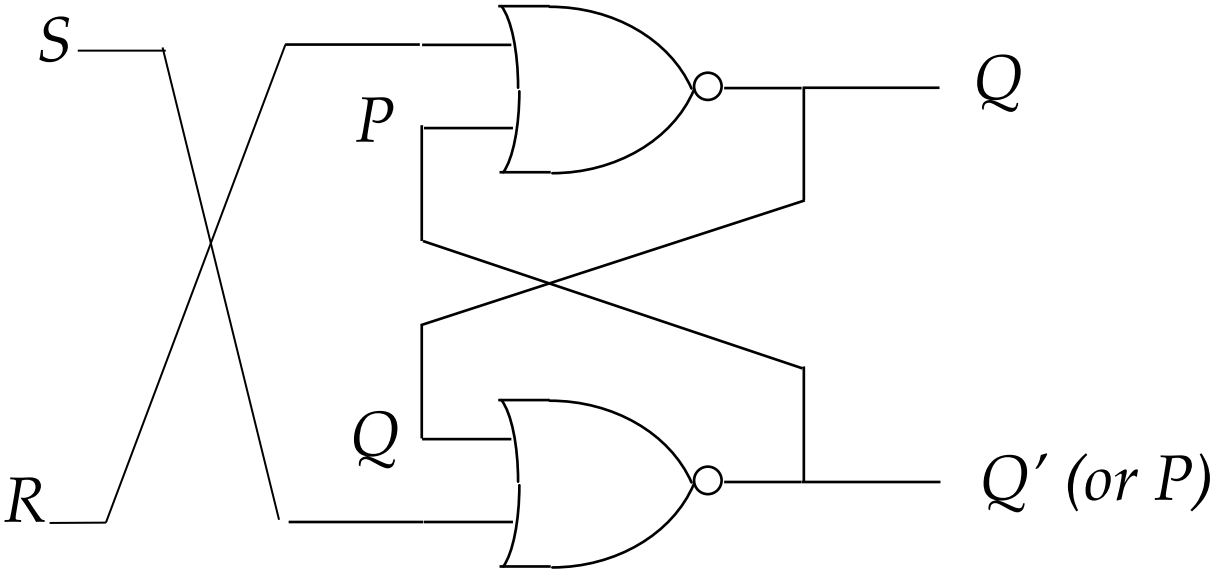
• It also has two outputs Q (normal state) and Q' (complement state)



Memory Element (SR Latch)

1- NOR Implementation

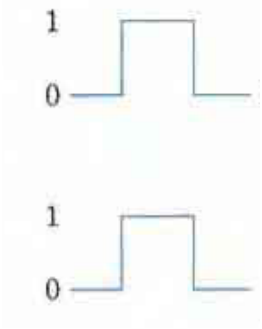
2- NAND Implementation



Q = (R+P)'= *R'P'*

P = (S+Q)'= *S'Q'*

Memory Element (SR Latch)





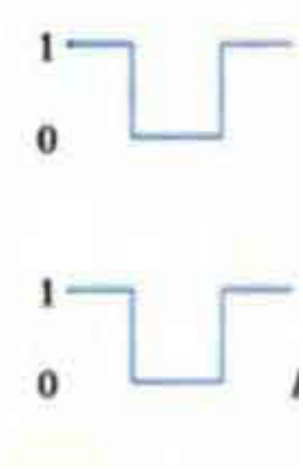
Symbol

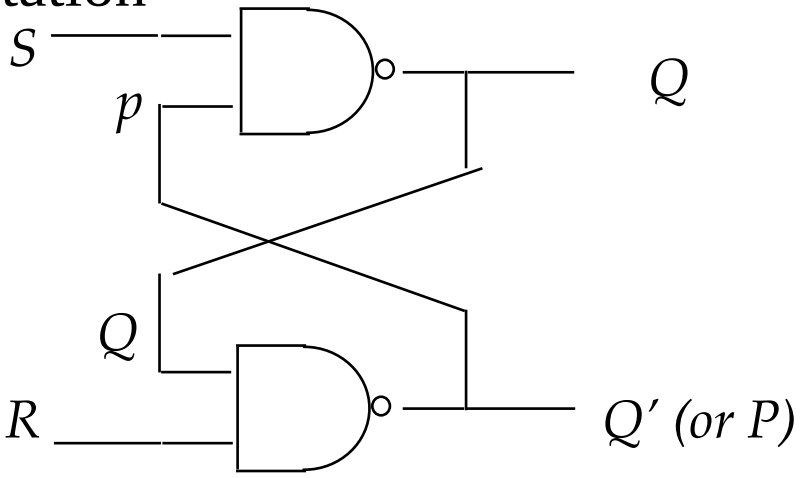
Function table for RS with NOR gates

S	R	Q	Q'	
1	0	1	0	Set to "1"
0	0	1	0	unchanged (after SR = 10)
0	**1**	0	1	Set to "0"
0	0	0	1	unchanged (after SR = 01)
1	1	?	?	Not allowed (0 0)

Memory Element (SR Latch)

2- NAND Implementation





Function table for RS with NAND gates:

S	R	Q	Q'	
1	0	0	1	set to « 0 »
1	1	0	1	unchanged (after SR = 10)
0	1	1	0	set to « 1 »
1	1	1	0	unchanged (after SR = 01)
0	0	?	?	Not allowed! (1 1)

Synchronous Sequential Circuits

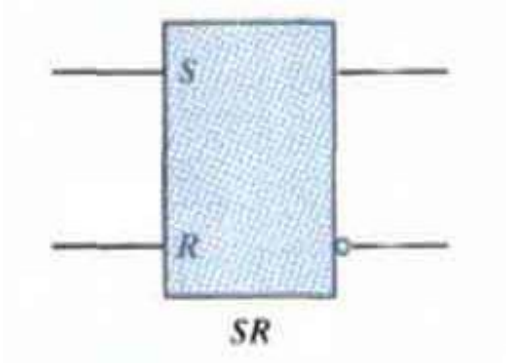
• If we add to the memory element a **synchronization signal** (i.e.clock impulse or control input) we would obtain a synchronous SR latch

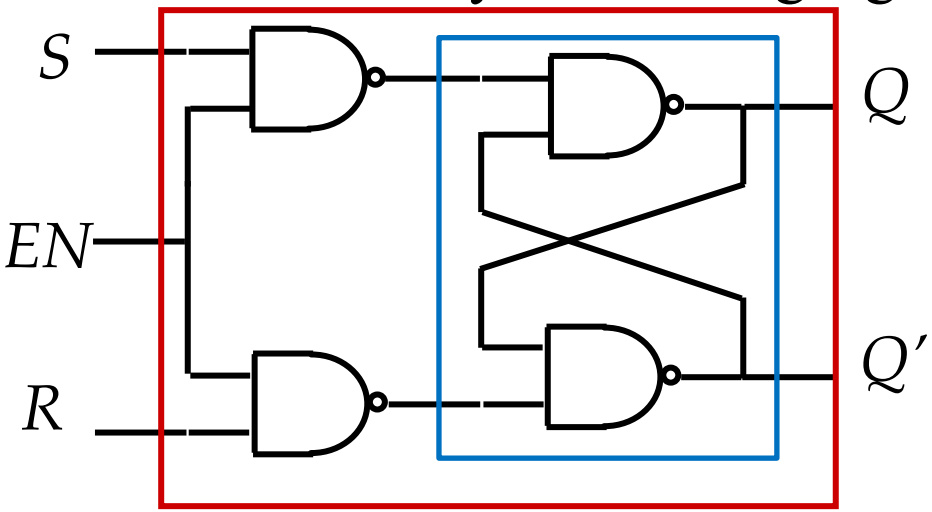
SR Latch with Control Input

- The time when a latch is allowed to change state is regulated.
- Change of state is regulated by a control signal called **ENABLE**.
- **Circuit is a NAND latch controlled by steering gates.**

Latch with Control Input: SR Latch

- Used in two principal ways.
  - as an ON/OFF signal.
  - as a synchronizing signal.





EN	S	R	Qt_{+1}	$Q't_{+1}$	Function
1	0	0	Qt	Q't	**No change**
1	0	1	0	1	**RESET**
1	1	0	1	0	**SET**
1	1	1	0	0	**Forbidden**
0	X	X	Qt	Q't	**Inhibited.**

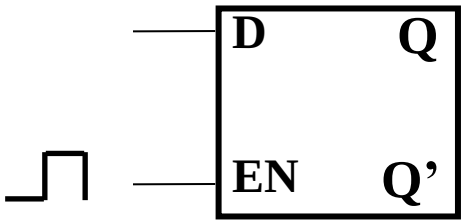
- Sometimes it's useful to avoid latch changes

  When EN = 0 → disables all latch state changes
- Control signal *enables* state change when EN = 1
- Right side of circuit same as ordinary S-R latch with NAND.

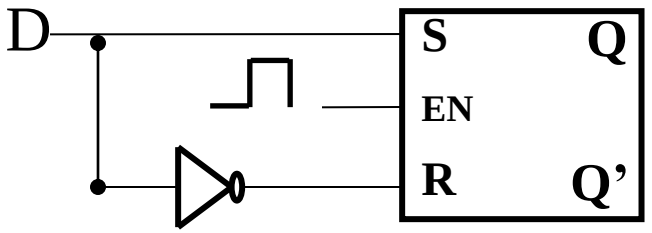
Latches with Control Input: D latch

- D for Data. Data is transferred to the Latch

Symbol



Implementation with SR latch



When EN is high, D passes from input to output (Q)

D	EN	Q	Q'
0	1	0	1
1	1	1	0
X	0	Q_n	Q_n'

Edge Sensitive Flip-Flop

- Latches respond to trigger levels on enable inputs

  Example: If EN = 1, input is reflected at output

- Flip flops store data on a positive or negative edge.

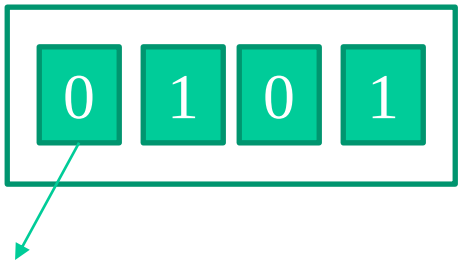
  Example: control input transitions from 0 to 1, data input available at output

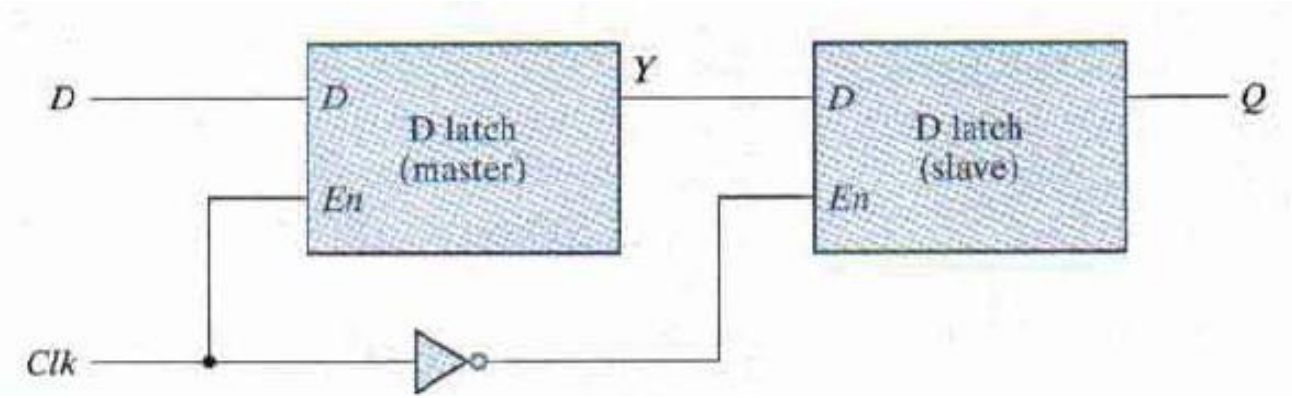
- Data remains stable in the flip flop until next positive /negative edge.

- Different types of flip flops are used for specific functions

Edge Sensitive Flip-Flop :definitions

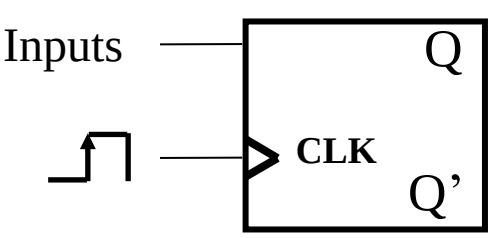
- A flip flop is made up of **two Latches** with a clock input.
- The sequential circuit output changes when its CLOCK input detects an edge. Edge-sensitive instead of level-sensitive.



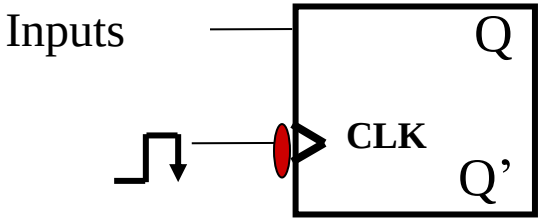


Edge Sensitive Flip-Flop :definitions

- Symbol is a triangle on the CLK (clock) input of a flip-flop.
- Flip flop can be triggered on positive or negative edge
- Bubble before *Clock (CLK)* input indicates negative edge trigger



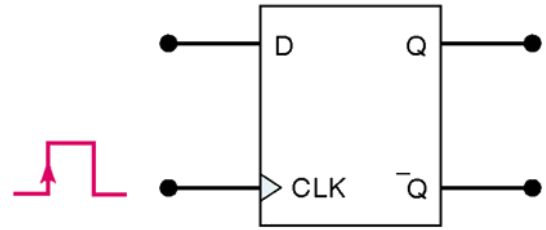
Symbol with positive edge



Symbol with negative edge

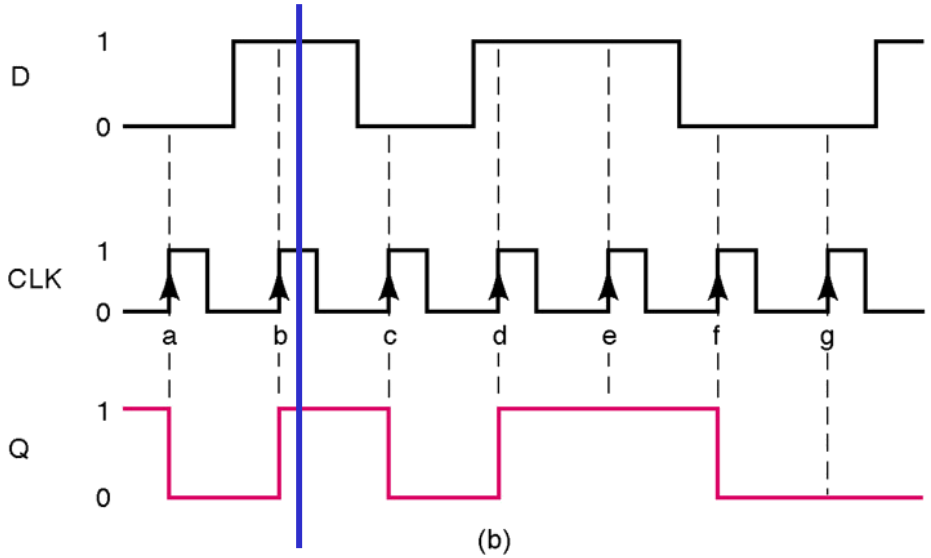
D Flip flop

- Stores a value on the positive edge of *Clock*
- Input changes at other times have no effect on output



D	CLK	Q
0	↑	0
1	↑	1

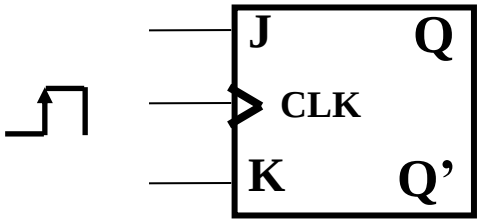
(a)



(b)

JK Flip flop

Symbol



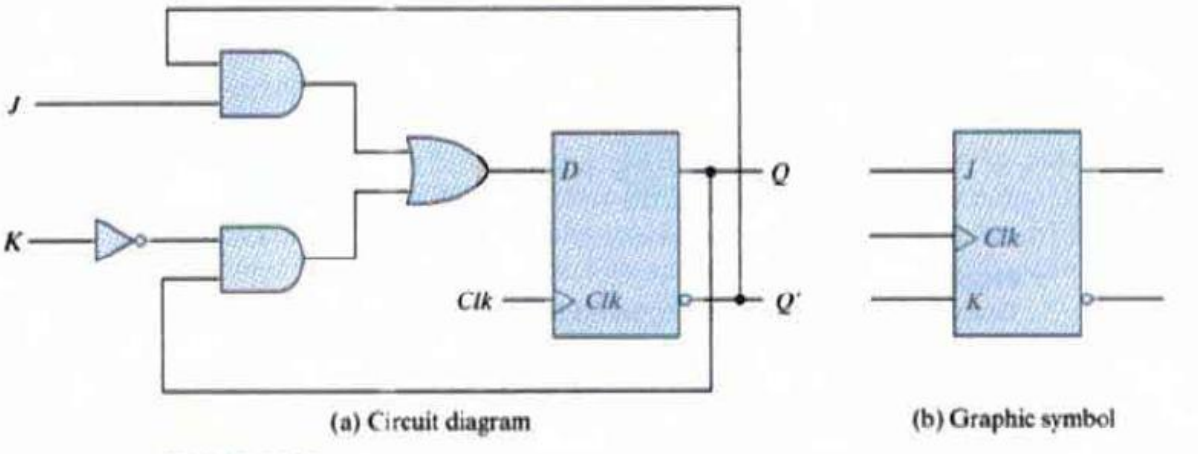
J	K	Q_{n+1}	Function
0	0	Q_n	No change
0	1	0	RESET
1	0	1	SET
1	1	Q'_n	complement (also Toggle)

Same as SR except for

K=J= 1 the JK flip flop will

Output the opposite state of Q_n.

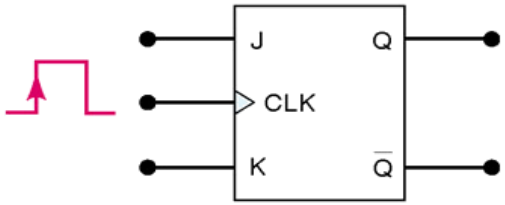
- Q_n = state *before* positive edge
- Q_{n+1} = state *after* positive edge



(a) Circuit diagram (b) Graphic symbol

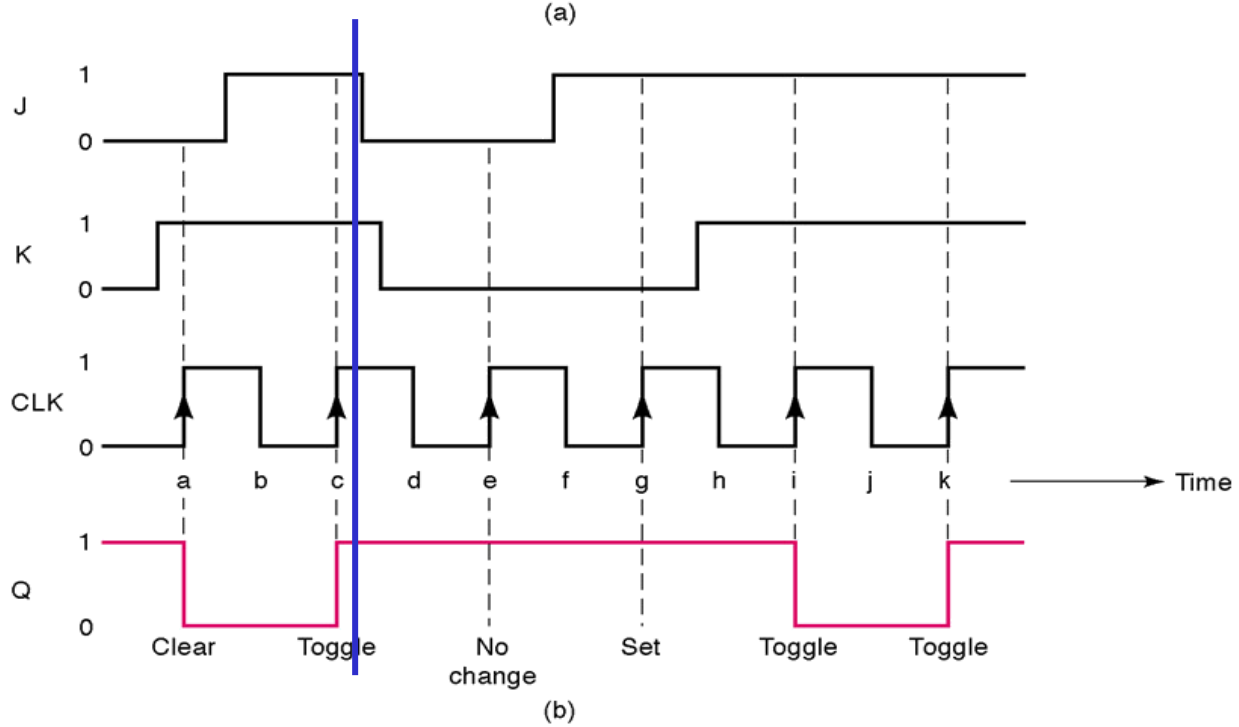
JK Flip flop

- Two data inputs, J and K
- J -> set, K -> reset, if J=K=1 then toggle (complement) output



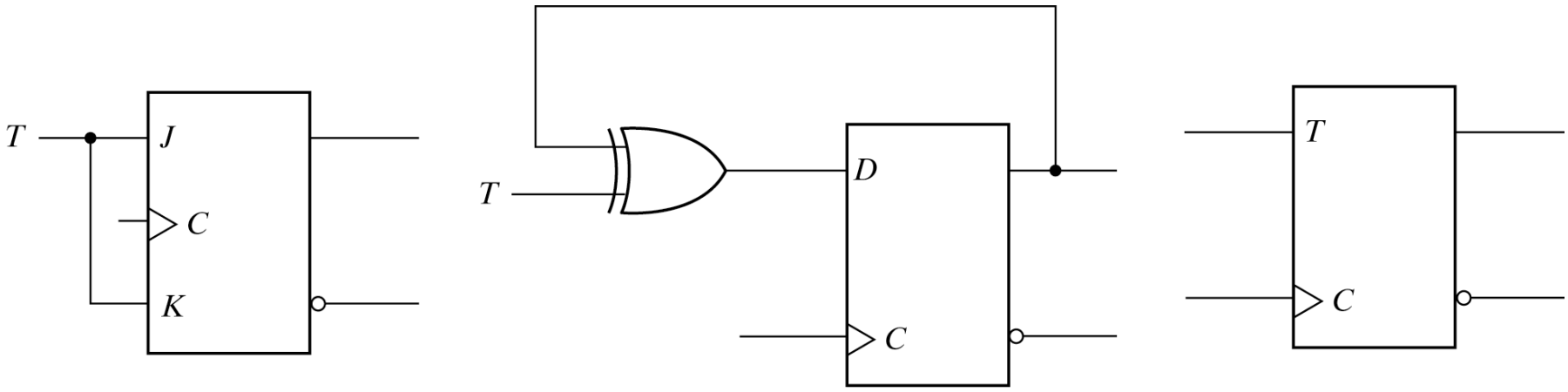
J	K	CLK	Q
0	0	↑	Q_0 (no change)
1	0	↑	1
0	1	↑	0
1	1	↑	$\overline{Q}_0$ (toggles)

(a)



(b)

T Flip-Flop



(a) From *JK* flip-flop (b) From *D* flip-flop (c) Graphic symbol

Fig. 5-13 T Flip-Flop

- Can be Created from JK or D flip flop

T	C	Q	Q'
0		Q_n	Q_n'
1		*Complement (toggle)*	

Analysis of synchronous sequential circuits

→Analysis describes what a given circuit will do under certain conditions

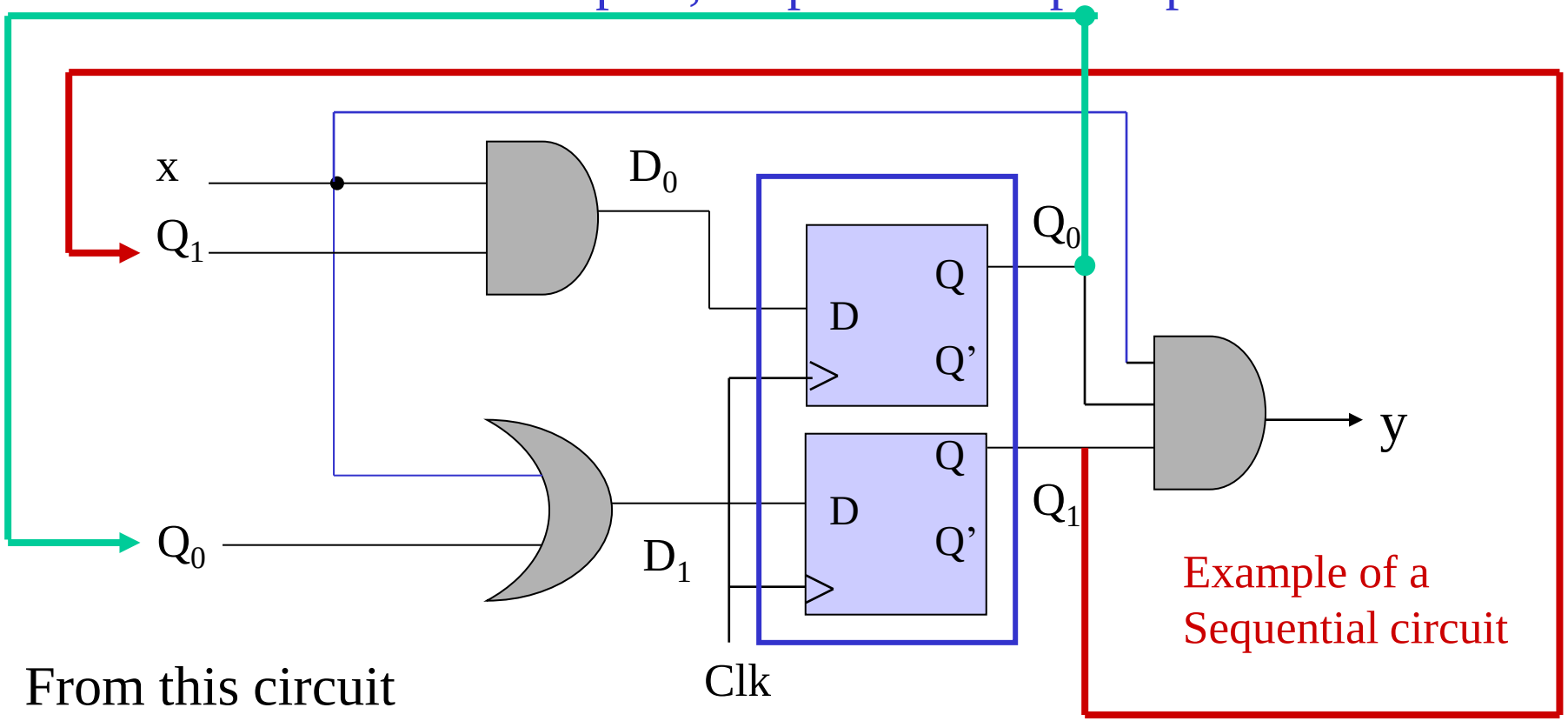
→Behavior of synchronous sequential circuit can be determined from

- **Its inputs,**

- **Its outputs and its Flip Flop state**

Flip Flop State

- Behavior of synchronous sequential circuit can be determined from inputs, outputs and Flip Flop state



From this circuit we can derive:

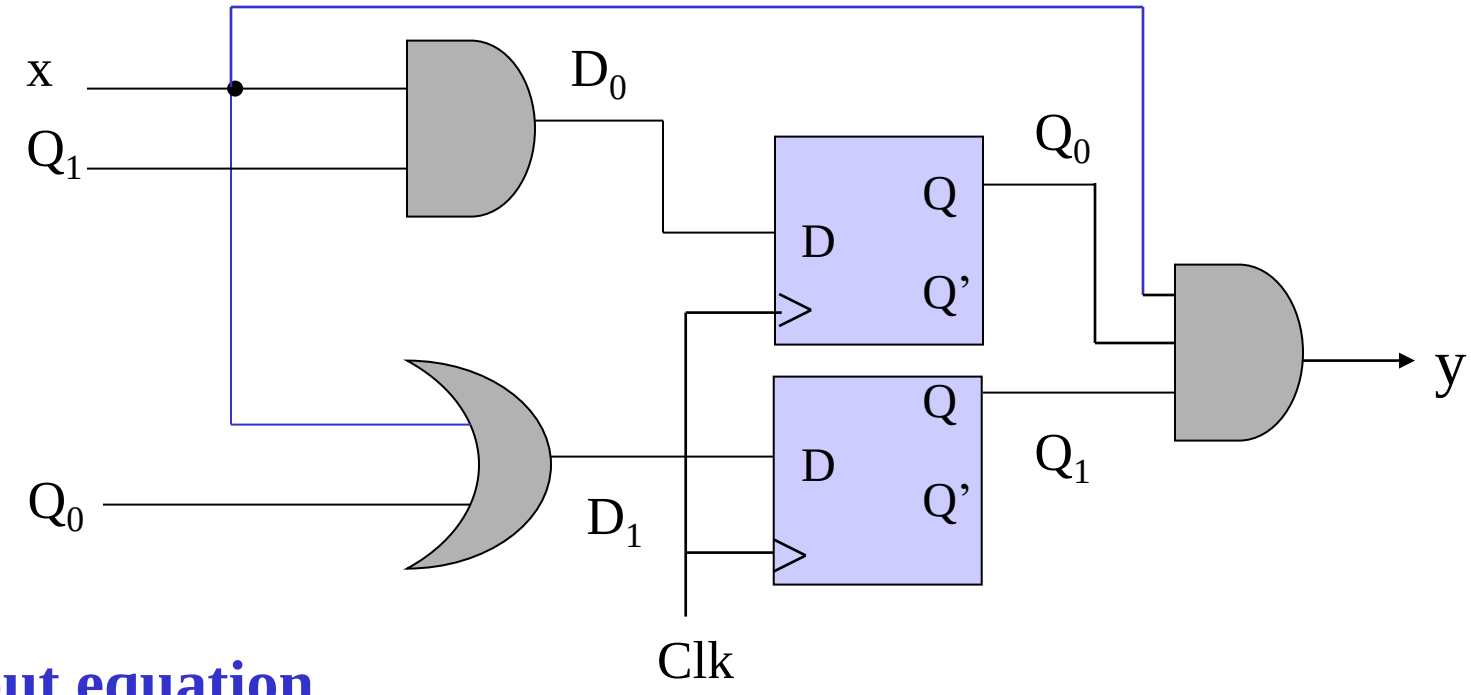
$y(t) = x(t)Q_1(t)Q_0(t)$

$Q_0(t+1) = D_0(t) = x(t)Q_1(t)$

$Q_1(t+1) = D_1(t) = x(t) + Q_0(t)$

Output and State Equations

- **Next state dependent on previous state.**



Output equation

$y(t) = x(t)Q_1(t)Q_0(t)$

State equations

$Q_0(t+1) = D_0(t) = x(t)Q_1(t)$

$Q_1(t+1) = D_1(t) = x(t) + Q_0(t)$

State Table

- Sequence of outputs, inputs, and flip flop states are listed in a table called "State Table"
- Present state indicates current value of flip flops
- Next state indicates state after next clock edge
- Output is output value on current clock edge

State Table

Present State	Next State		Output	
	x=0	x=1	x=0	x=1
0 0	00	10	0	0
0 1	10	10	0	0
1 0	00	11	0	0
1 1	10	11	0	1

$Q_1(t)\ Q_0(t)$ $Q_1(t+1)\ Q_0(t+1)$

$$y(t) = x(t)Q_1(t)Q_0(t)$$
$$Q0(t+1) = D_0(t) = x(t)Q_1(t)$$
$$Q1(t+1) = D_1(t) = x(t) + Q_0(t)$$

State Table

- All possible input combinations are listed
- All possible state combinations are listed
- Separate columns for each output value.
- Easier to use a symbol for each state.

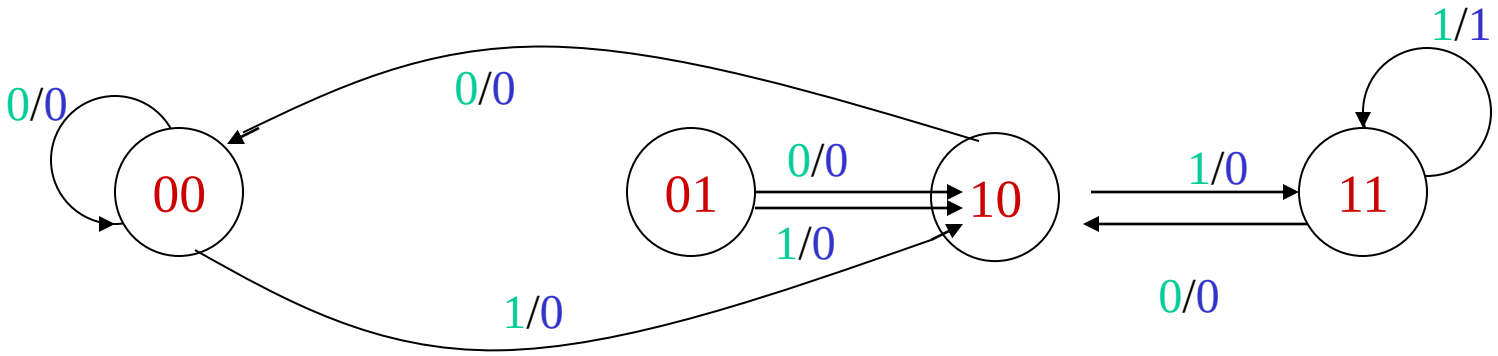
Let:
$s_0 = 00$
$s_1 = 01$
$s_2 = 10$
$s_3 = 11$

Present State	Next State		Output	
	x=0	x=1	x=0	x=1
s_0	s_0	s_2	0	0
s_1	s_2	s_2	0	0
s_2	s_0	s_3	0	0
s_3	s_2	s_3	0	1

State Diagram

- Circles indicate current state
- Arrows point to next state
- For x/y, x is input and y is output

Present State	Next State		Output	
	x=0	x=1	x=0	x=1
0 0	00	10	0	0
0 1	10	10	0	0
1 0	00	11	0	0
1 1	10	11	0	1



State Diagram

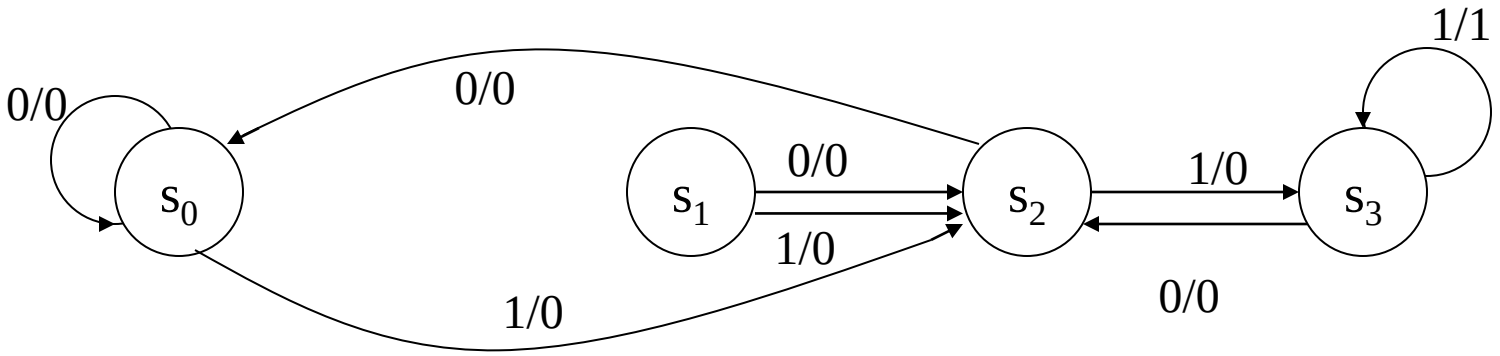
- **Each state has two arrows leaving**

  - One for x = 0 and one for x = 1

- **Unlimited arrows can enter a state**

- **Use of state names in this example**

  - Easier to identify



Analysis of synchronous sequential circuits

→Analysis describes what a given circuit will do under certain conditions

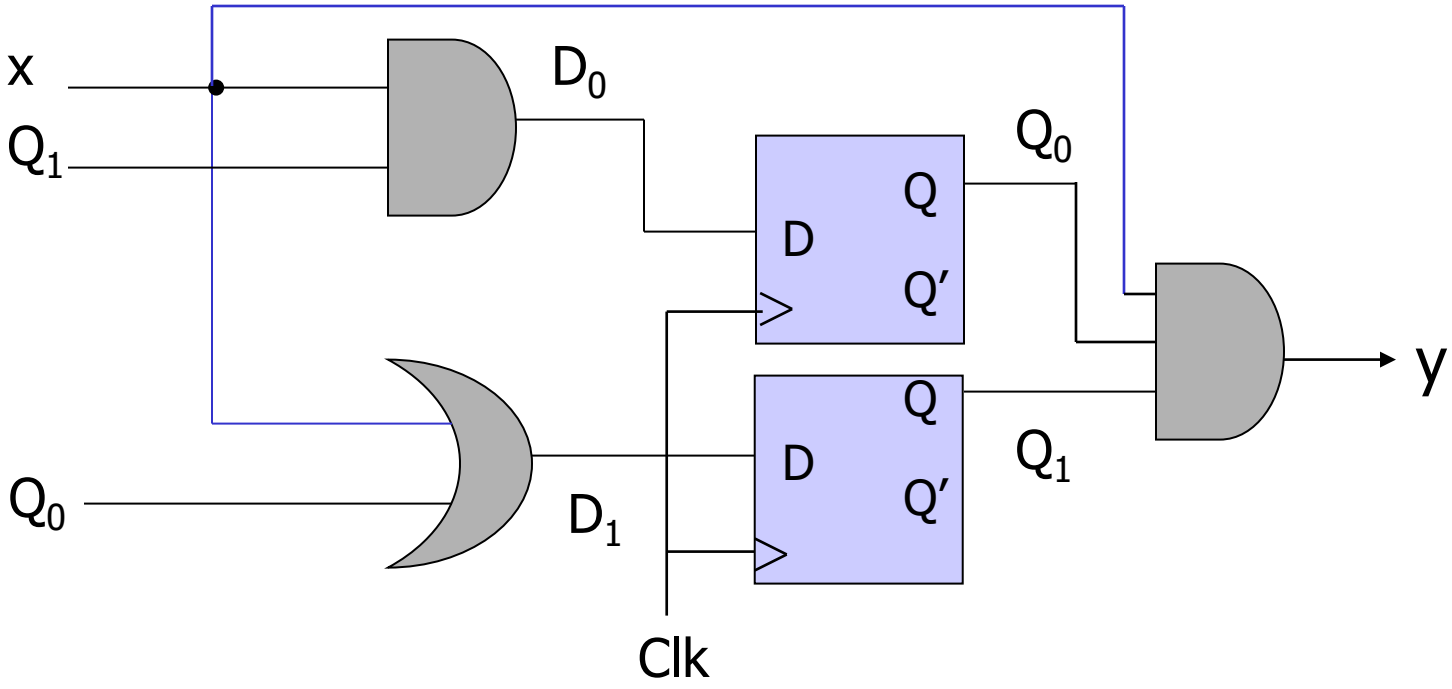
→Behavior of synchronous sequential circuit can be determined from

- Its inputs,

- Its outputs and its Flip Flop state

Flip Flop Input Equations

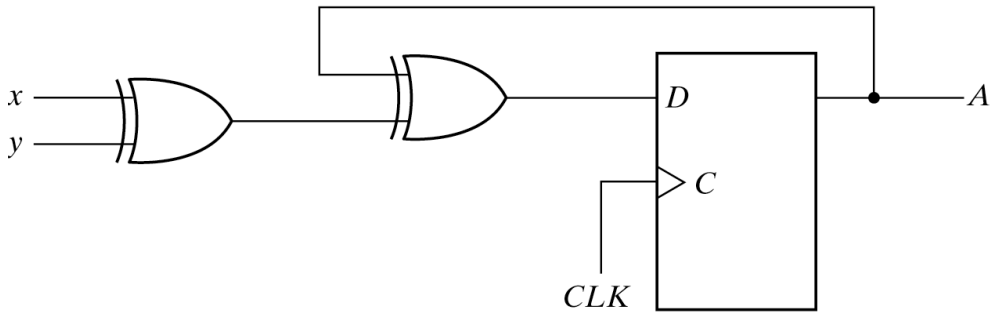
- **Boolean expressions which indicate the input to the flip flops.**



$$D_{Q0} = xQ_1$$
$$D_{Q1} = x + Q_0$$

Analysis with D Flip-Flops

- **Identify flip flop input equations** $D_A = A \oplus x \oplus y$
- **Identify output equation (no output in this example)**

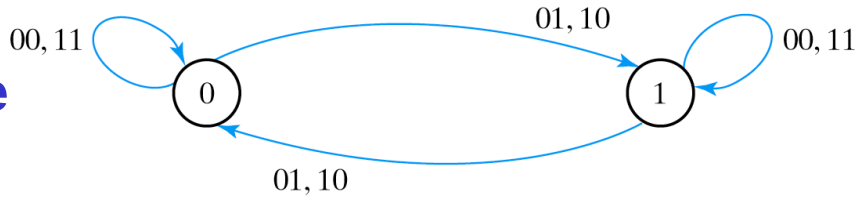


(a) Circuit diagram

Present state	Inputs		Next state
A	*x*	*y*	*A*
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1

(b) State table

Note: this example has no output



(c) State diagram

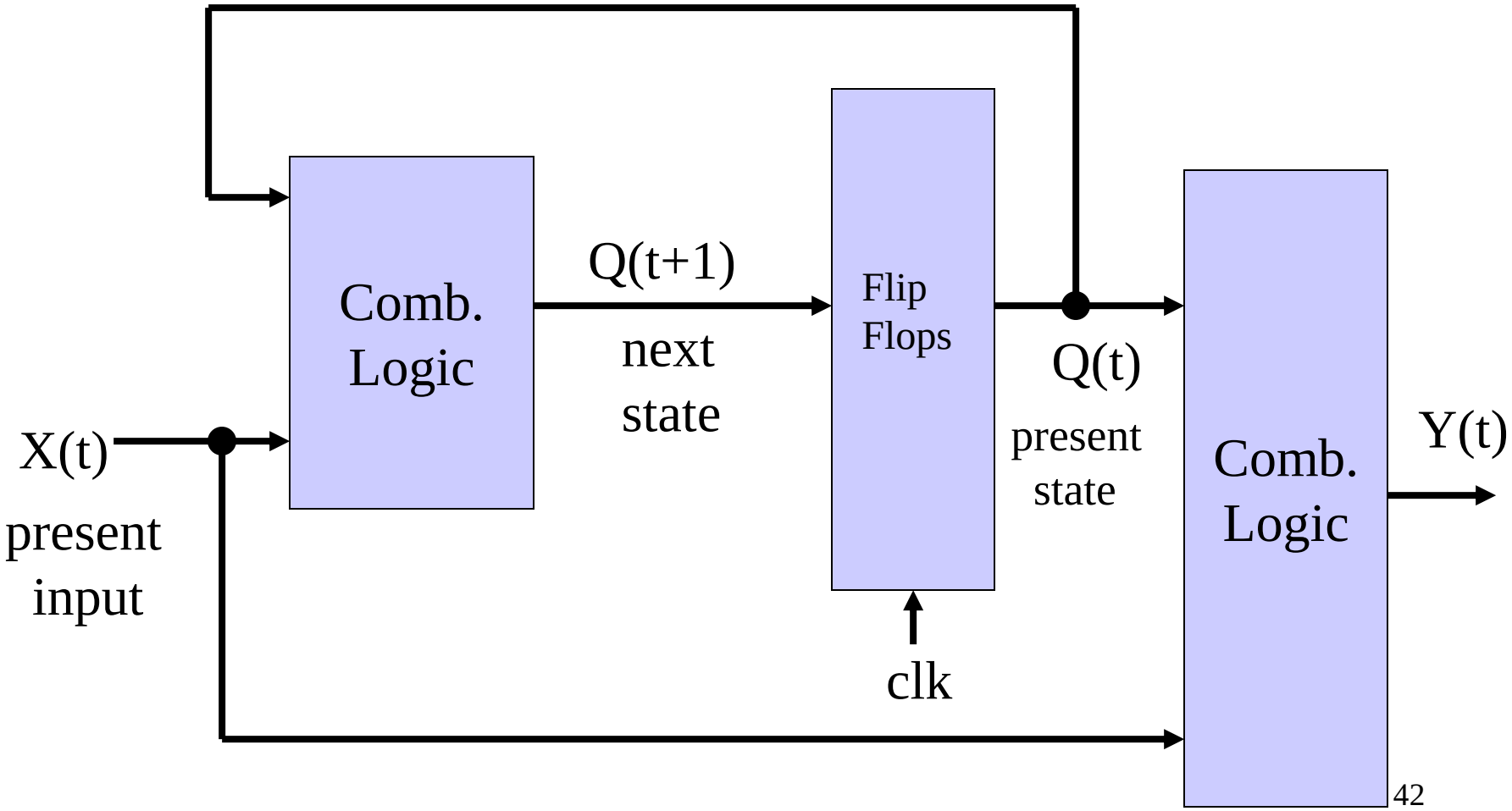
Fig. 5-17 Sequential Circuit with *D* Flip-Flop

Mealy and More Models

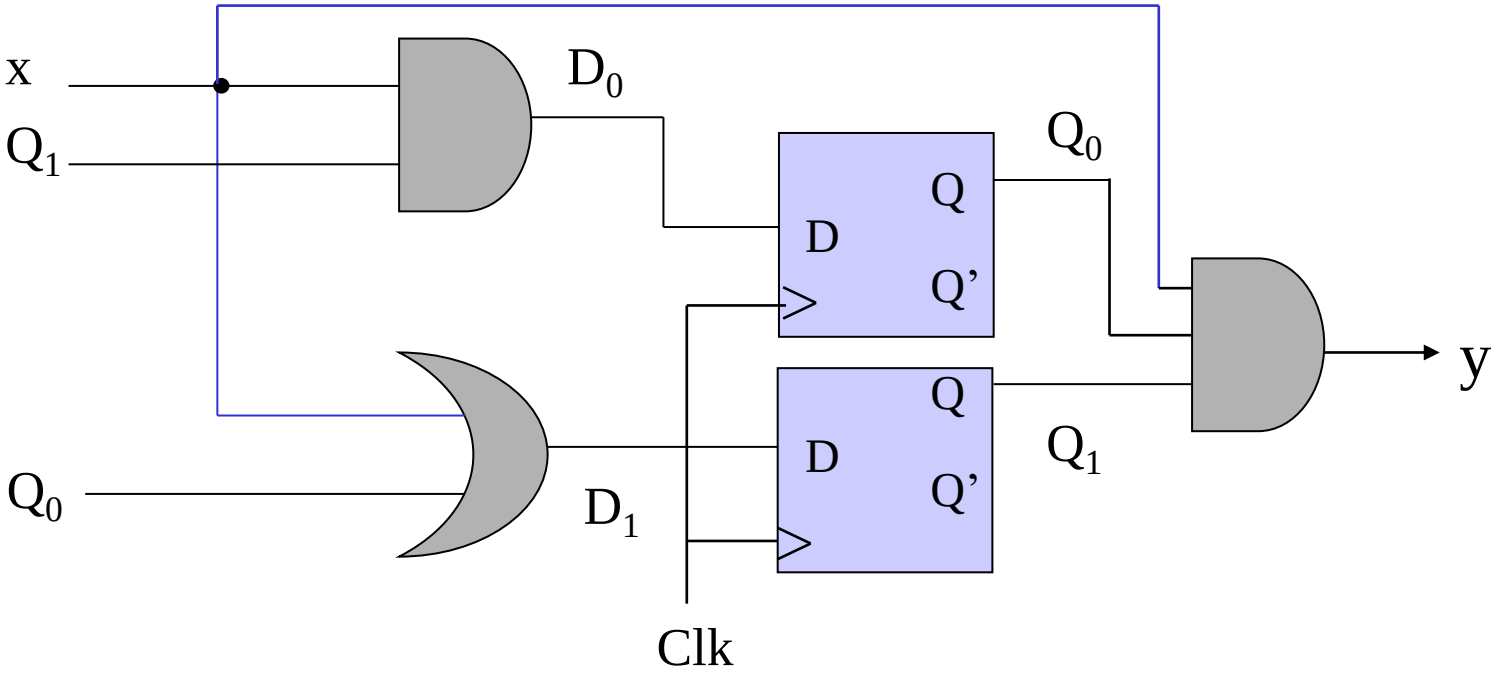
- A general model of a Sequential Circuit has
  - Inputs
  - Outputs
  - Internal States
- There are two "standard models"
  - Mealy Model, known as Mealy Finite State Machine (or Mealy machine)
  - Moore Model known as Moore Finite State Machine (or Moore Machine)

Mealy Machine

- Output based on state and present input

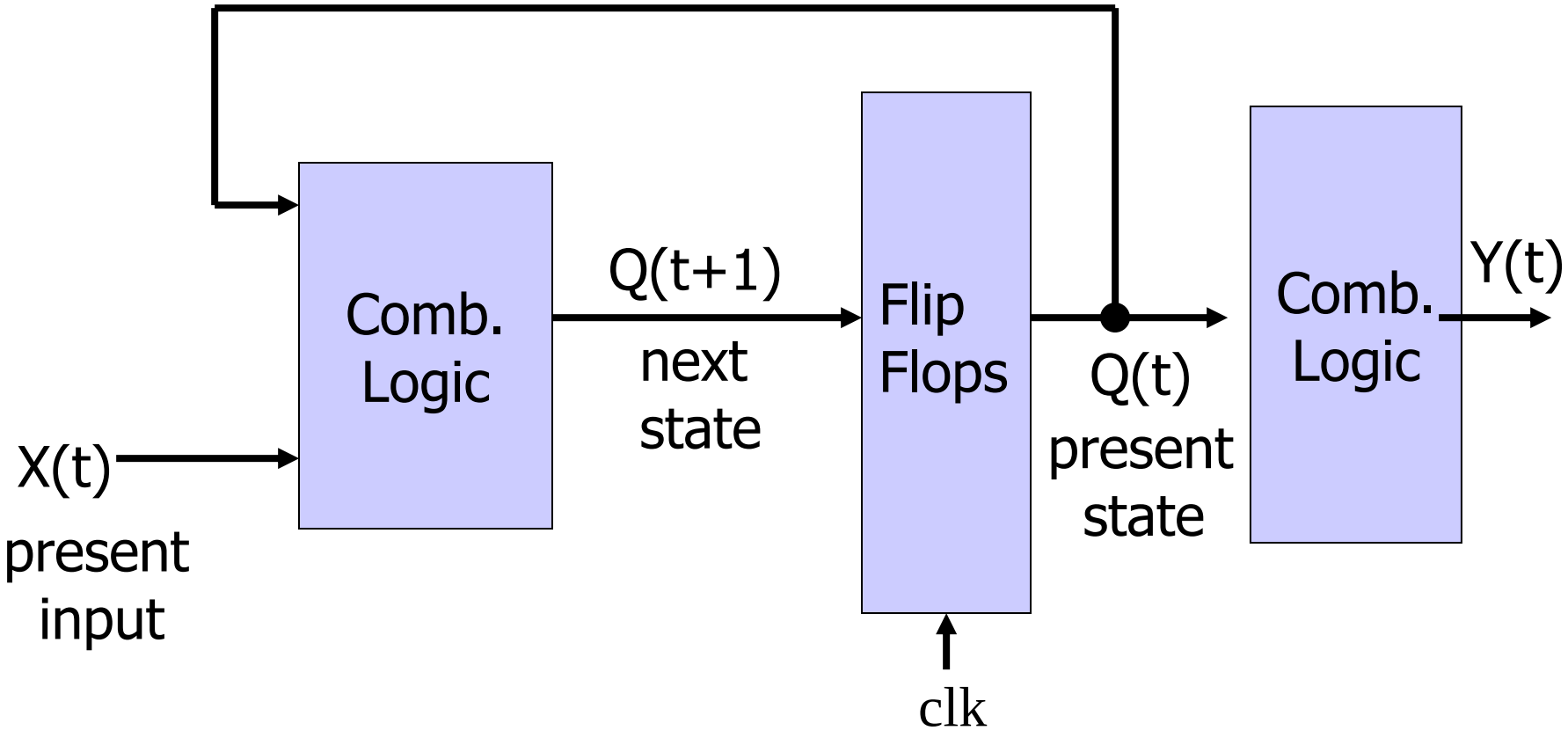


Example of Mealy Machine



Moore Machine

- Output based on state only



Example of Moore Machine

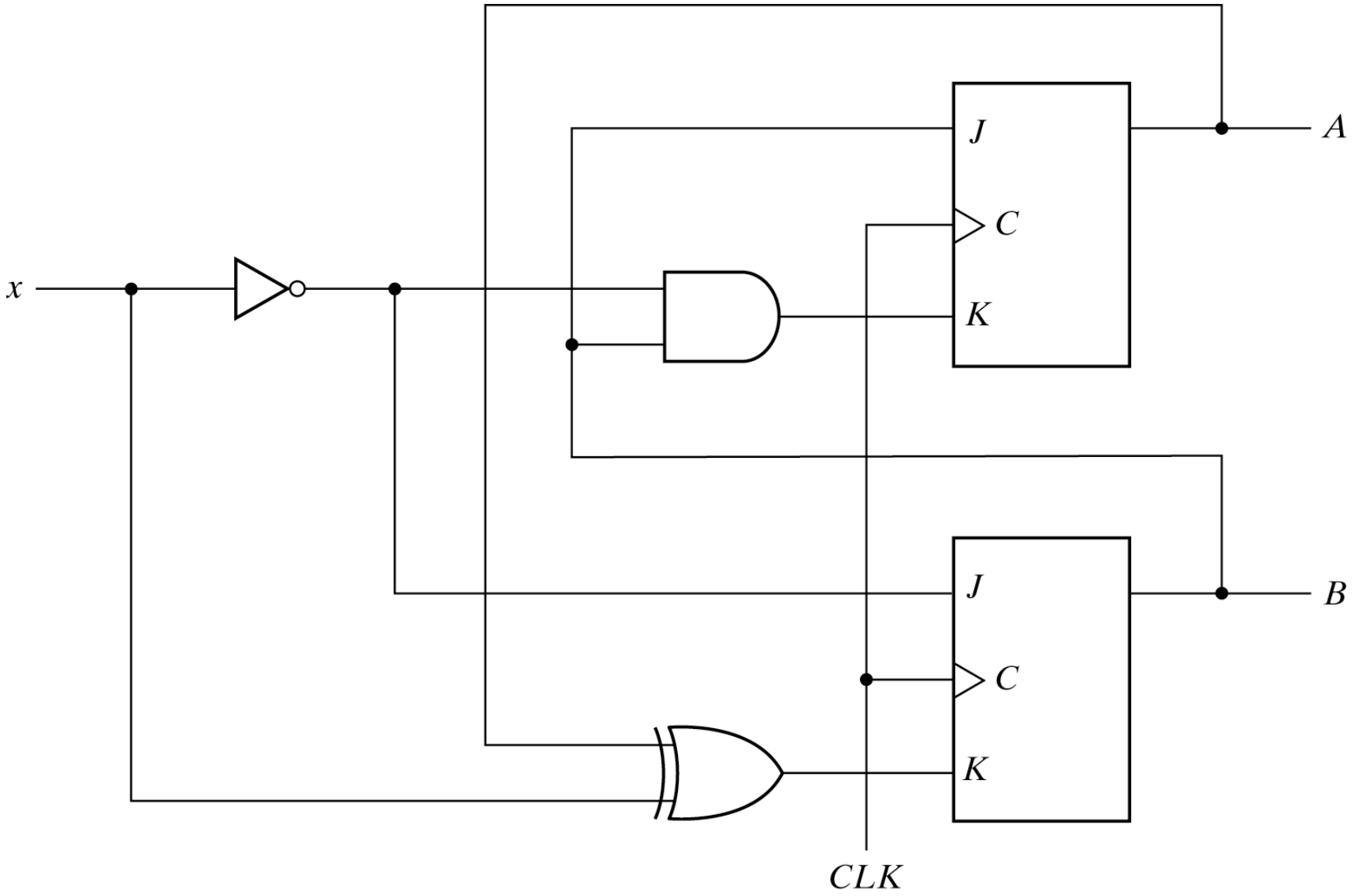
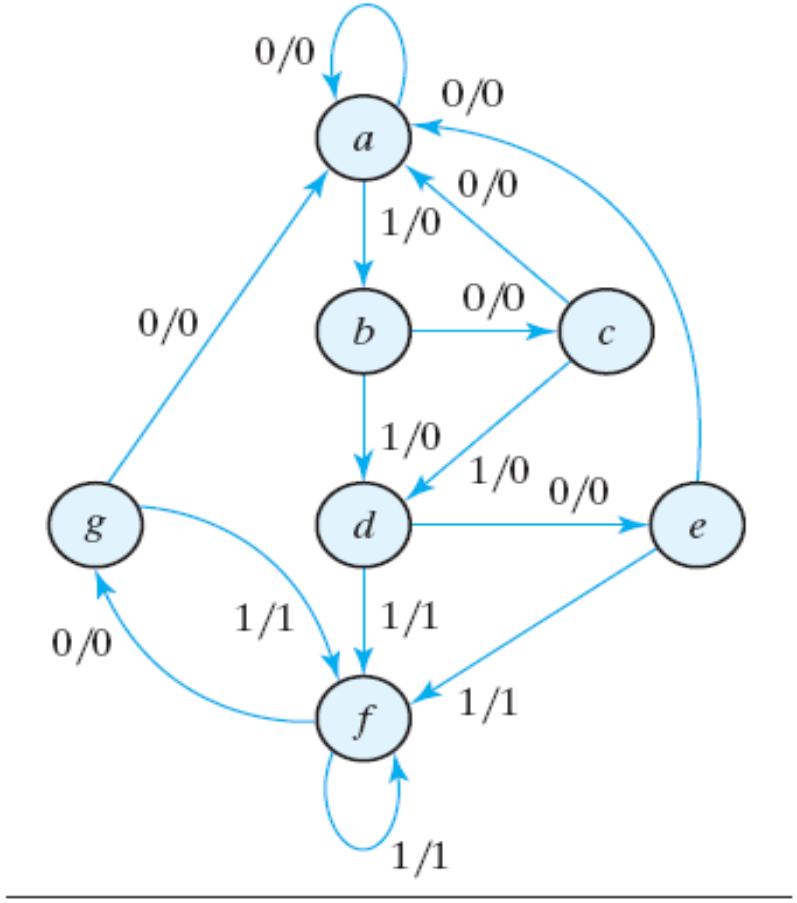


Fig. 5-18 Sequential Circuit with *JK* Flip-Flop

State Reduction (1)

- Consider a sequential circuit with the following diagram



State Reduction (2)

- Obtaining the State table from State diagram of the previous slide

State e and f are equivalent→ have the same next states and same output when x=0, x=1

State Table

Present State	Next State $x=0$	Next State $x=1$	Output $x=0$	Output $x=1$
a	a	b	0	0
b	c	d	0	0
c	a	d	0	0
d	e	f	0	1
e	a	f	0	1
f	g	f	0	1
g	a	f	0	1

Reducing the State Table

Present State	Next State $x=0$	Next State $x=1$	Output $x=0$	Output $x=1$
a	a	b	0	0
b	c	d	0	0
c	a	d	0	0
d	e	f	0	1
e	a	f	0	1
f	e	f	0	1

Remove g and replace it by e in remaining next states

State Reduction (3)

Reducing the State Table

Present State	Next State $x = 0$	Next State $x = 1$	Output $x = 0$	Output $x = 1$
a	a	b	0	0
b	c	d	0	0
c	a	d	0	0
d	e	f	0	1
e	a	f	0	1
f	e	f	0	1

f and d are equivalent

Reduced State Table

Present State	Next State $x = 0$	Next State $x = 1$	Output $x = 0$	Output $x = 1$
a	a	b	0	0
b	c	d	0	0
c	a	d	0	0
d	e	d	0	1
e	a	d	0	1

END of Chapter 5

Note: several examples are discussed in the class.
They are not included in this .PPT file

Registers and Counters

Registers

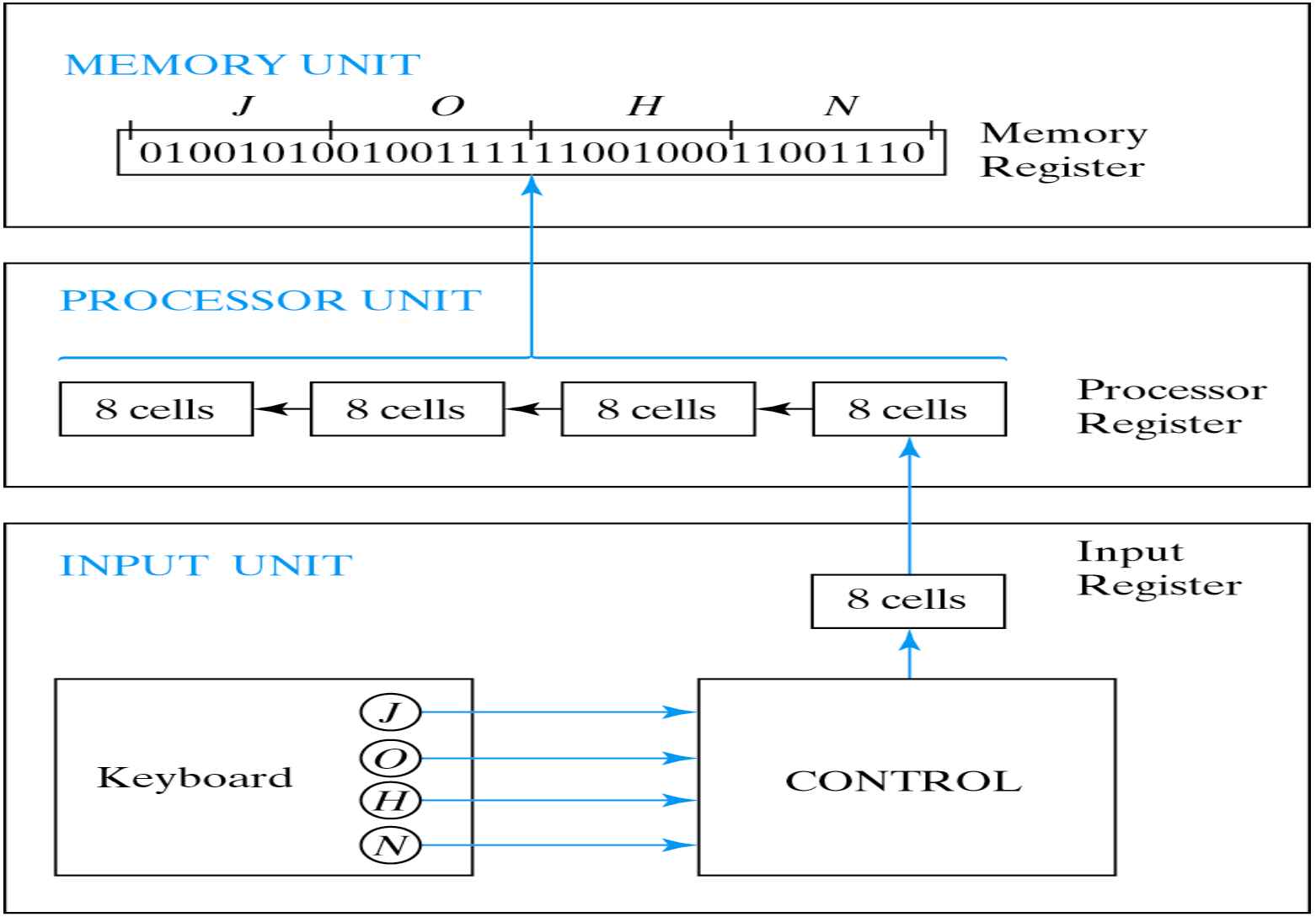


Fig. 1-1 Transfer of information with registers

Registers

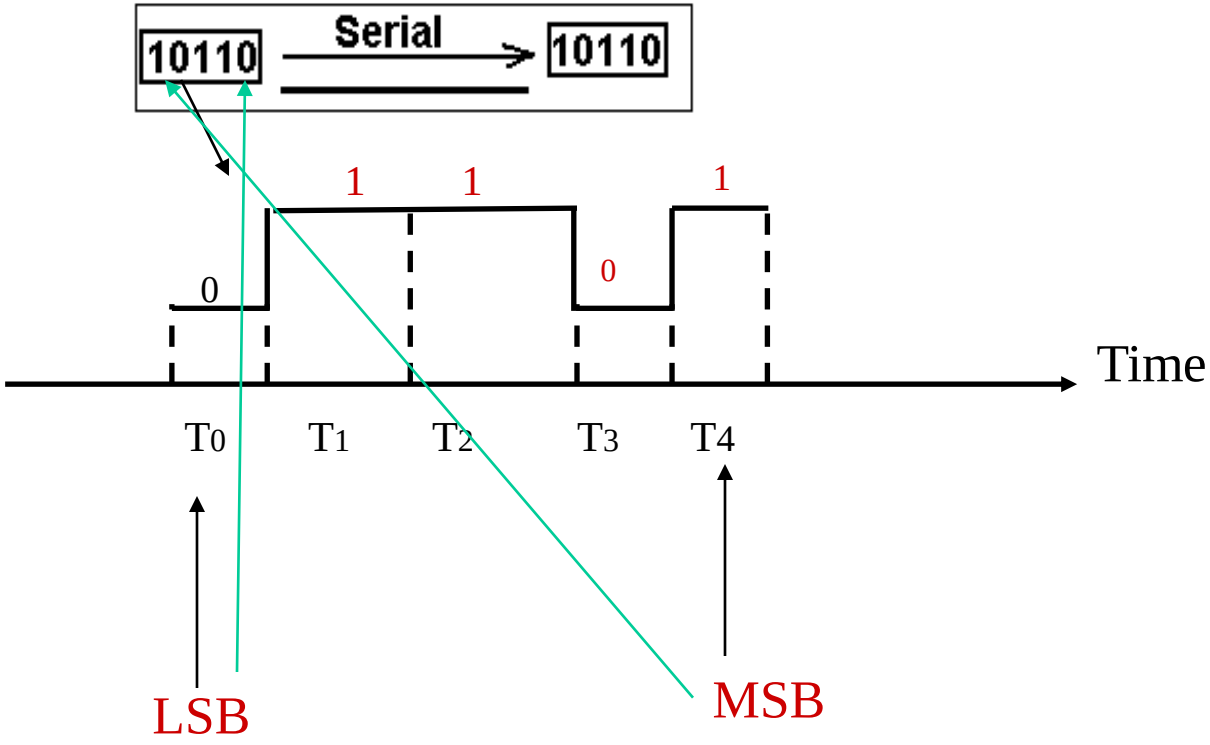
- Multiple flip flops can be combined to form a data ***register***
- **Shift registers** allow data to be transported one bit at a time
- **Registers** also allow for parallel transfer

  →Many bits transferred at the same time
- Basic components of most computers

Parallel versus Serial

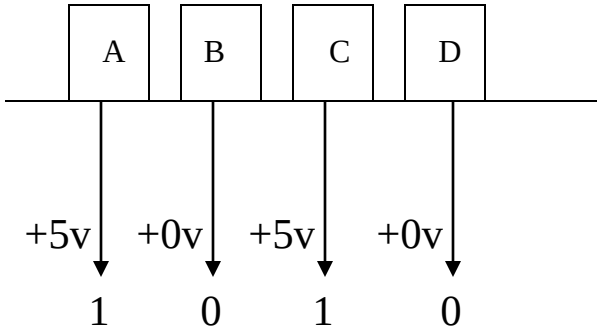
→Serial communications, transfers a binary number as a sequence of binary digits, one after another, through one data line.

- One Circuit is necessary to represent any binary number

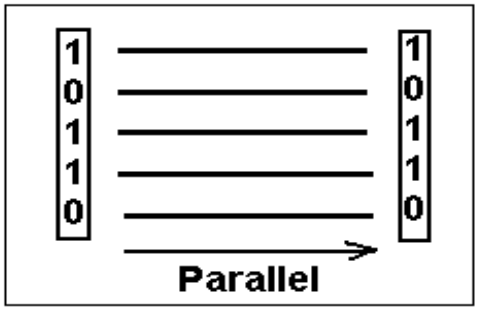


Parallel versus Serial

→ Parallel communications

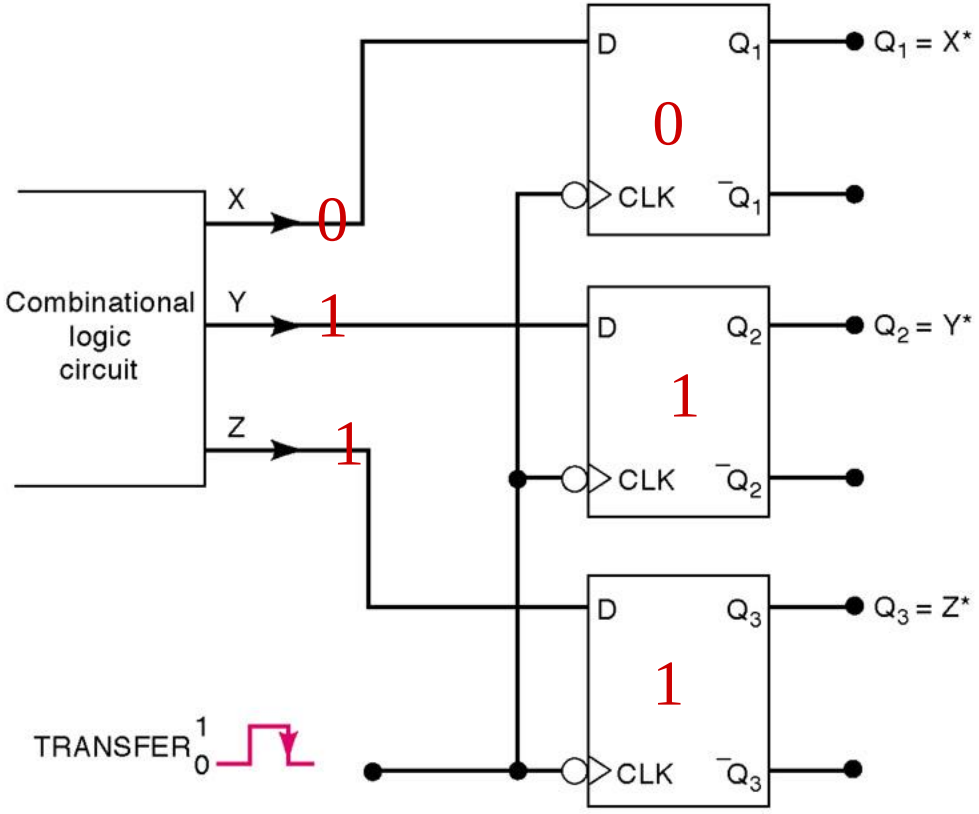


–Transfers a binary number through multiple data lines at the same time.



Parallel Data Transfer

- In this example Flip flops D store outputs from combinational logic that has 3 outputs. 3 flip flops are required
- Multiple flops can store a collection of binary data



Register with Parallel Load

- Register: Group of Flip-Flops
- Ex: D Flip-Flops
- Holds 4 bits of Data
- Loads in Parallel on Clock Transition
- Asynchronous Clear (Reset)

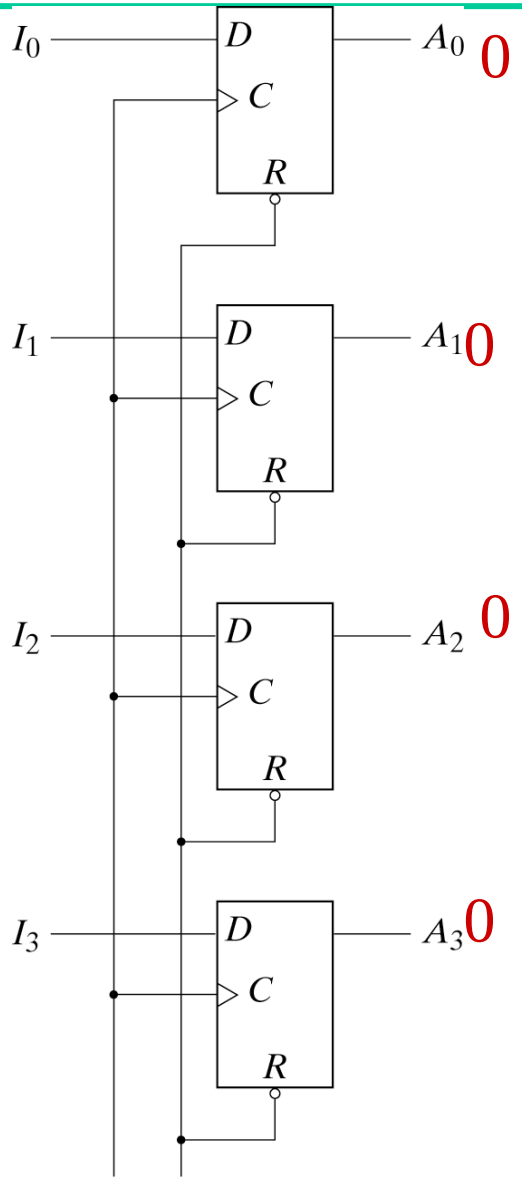


Fig. 6-1 4-Bit Register

Register with Parallel Load

- Register: Group of Flip-Flops
- Ex: D Flip-Flops
- Holds 4 bits of Data
- Loads in Parallel on Clock Transition
- Asynchronous Clear (Reset)

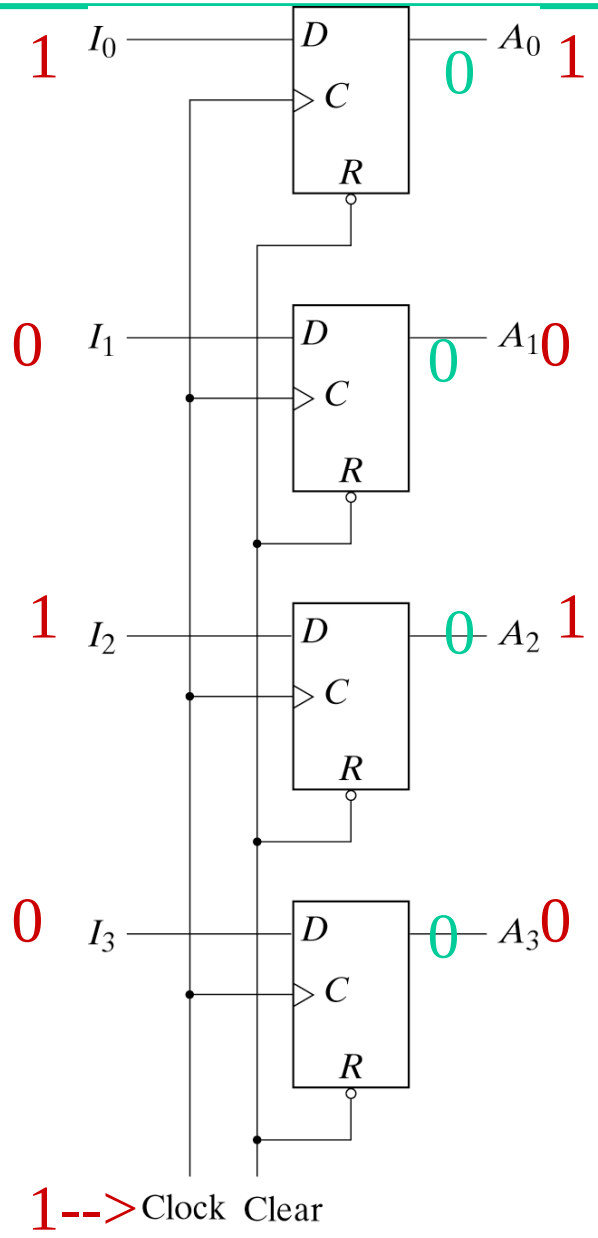
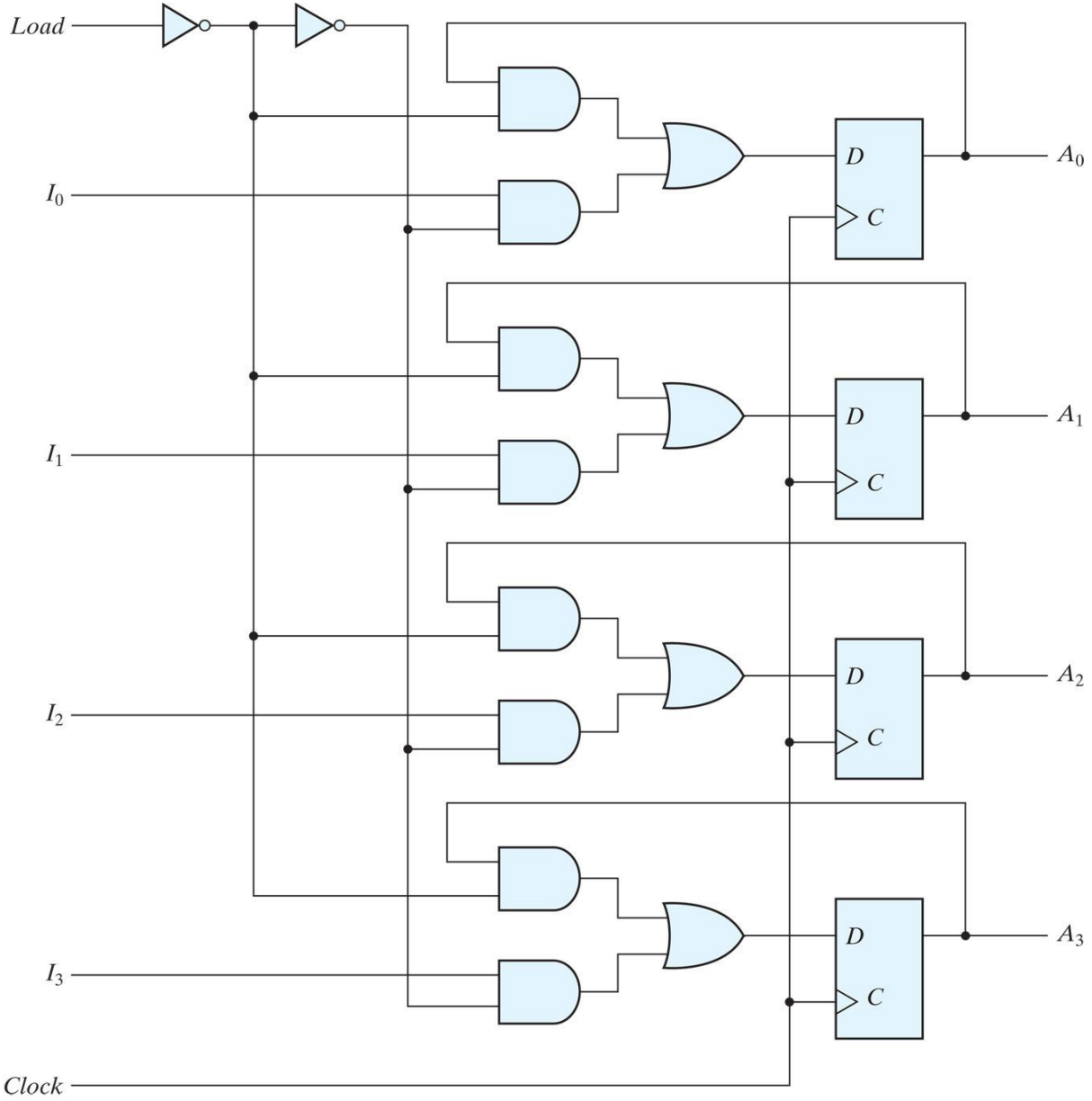


Fig. 6-1 4-Bit Register

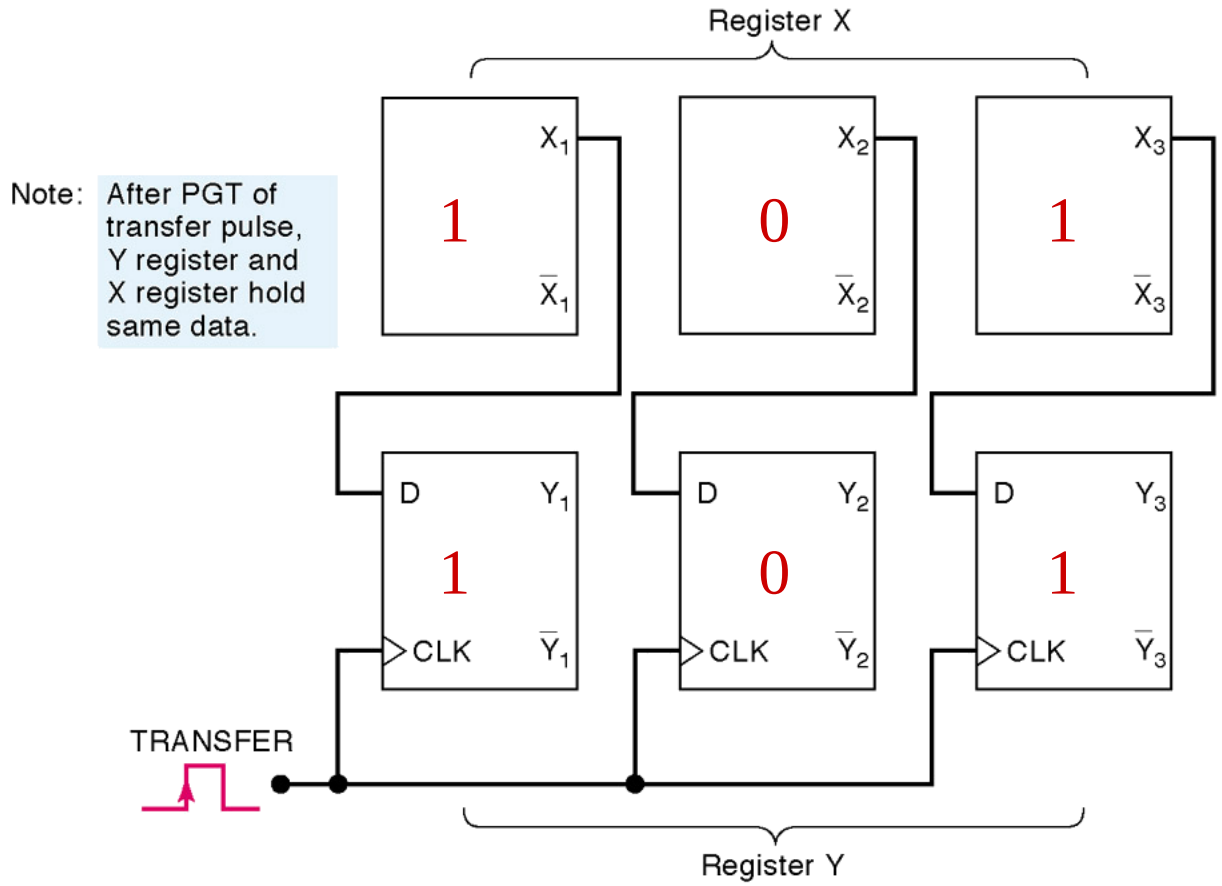
Four-bit register with parallel load



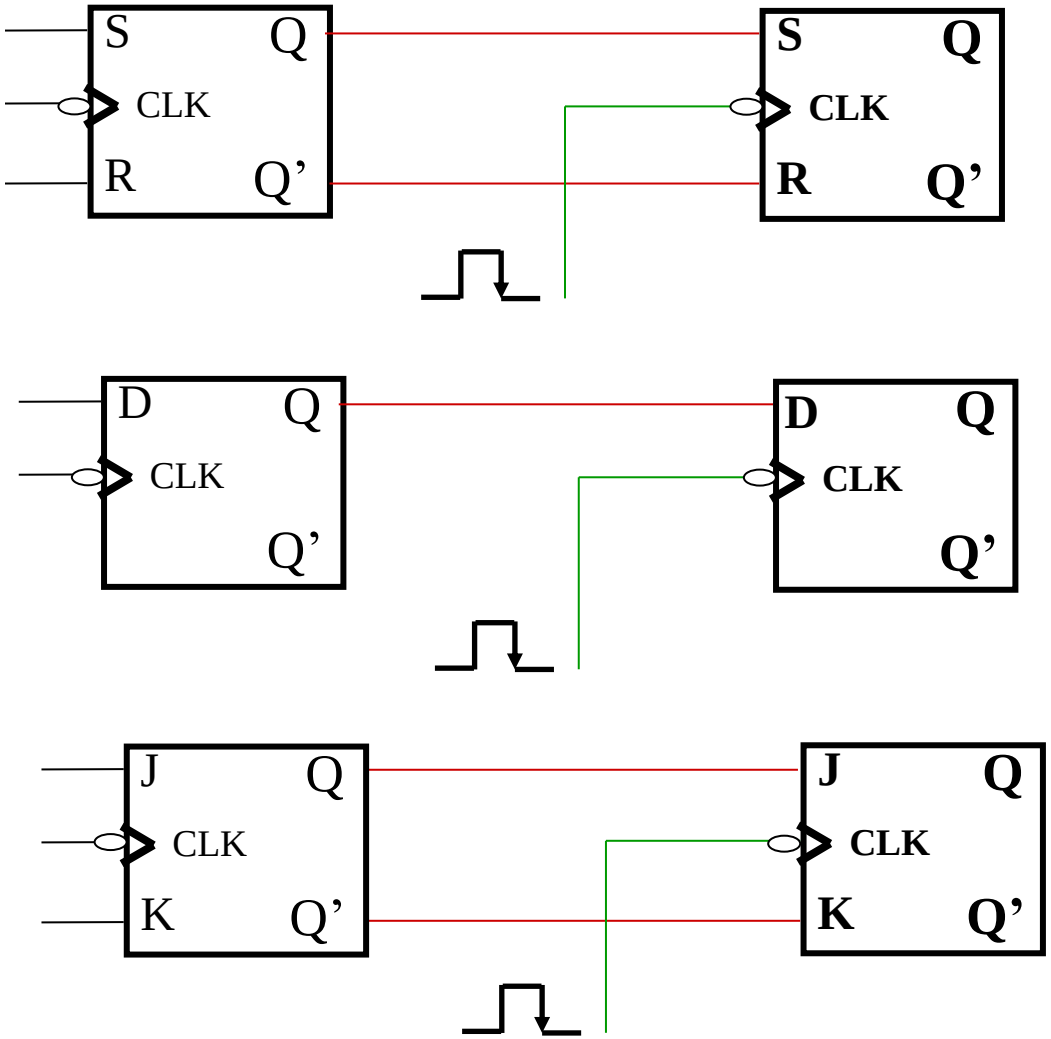
Copyright ©2013 Pearson Education, publishing as Prentice Hall

Parallel Data Transfer

- All data is transferred on one positive edge
- Data stored into register Y



Serial Transfer



Shift Registers

- Cascade chain of Flip-Flops
- Bits travel on positive edges (in this example)
- Serial in (SI) – Serial out (SO)

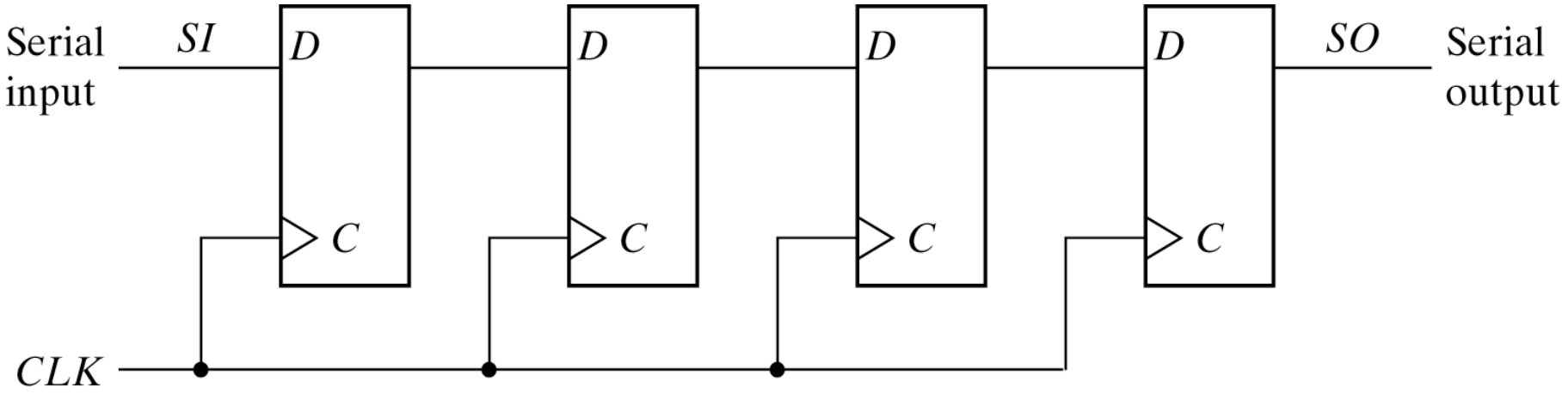
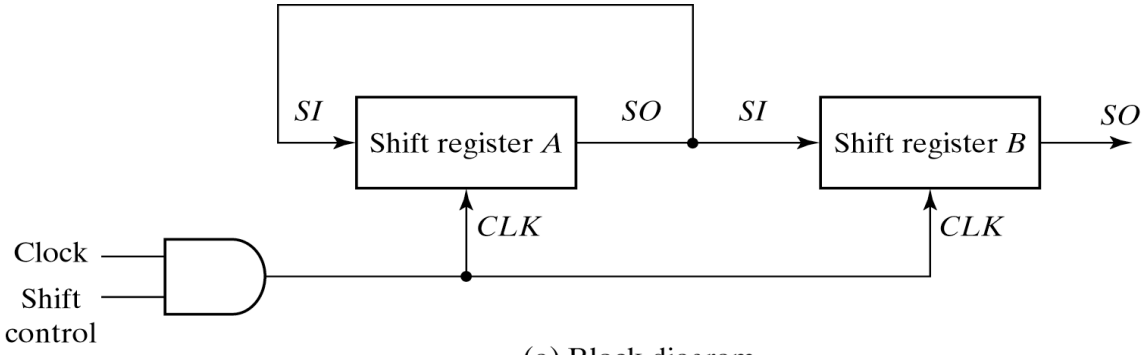


Fig. 6-3 4-Bit Shift Register

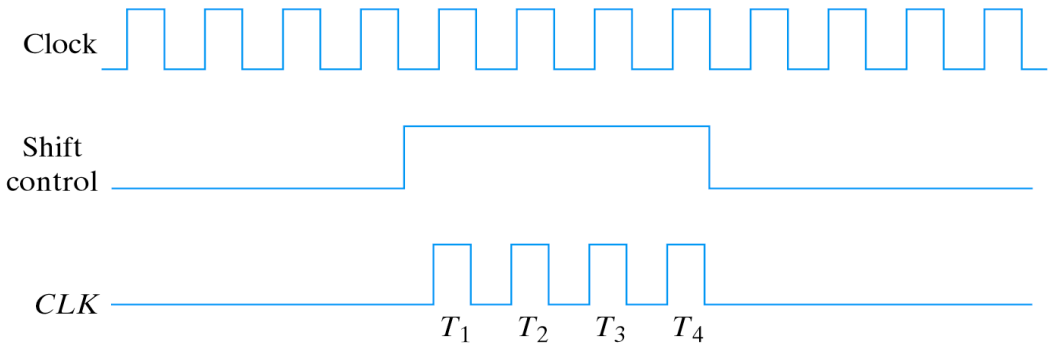
Serial Transfer

- Data is transferred one bit at a time
- Note the data loop back for register A

Time	A	B
T0	1011	0011
T1		
T2		
T3		
T4		



(a) Block diagram



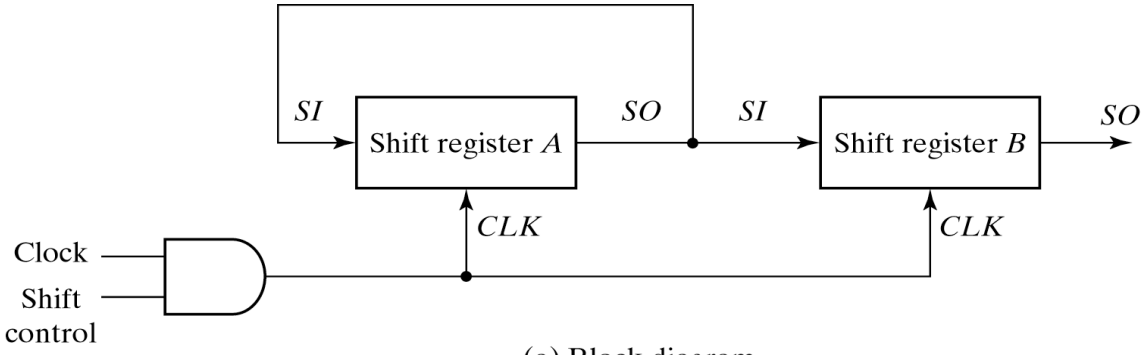
(b) Timing diagram

Fig. 6-4 Serial Transfer from Register *A* to register *B*

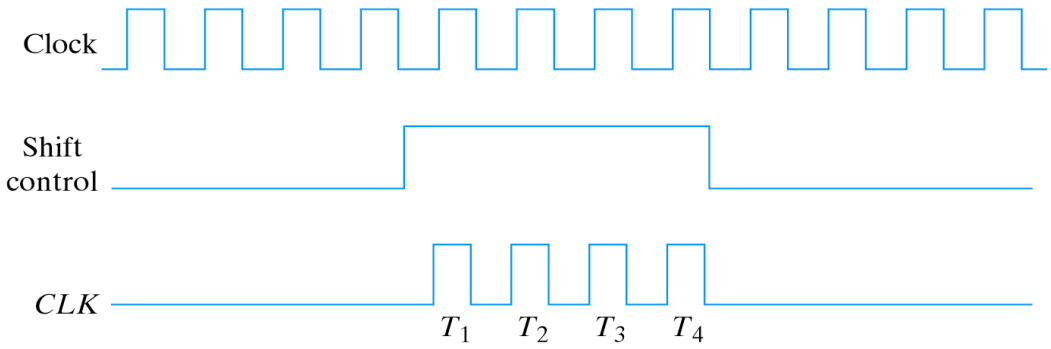
Serial Transfer

- Data is transferred one bit at a time
- Note the data loop back for register A

Time	A	B
T0	1011	0011
T1	1101	1001
T2	1110	1100
T3	0111	0110
T4	1011	1011



(a) Block diagram

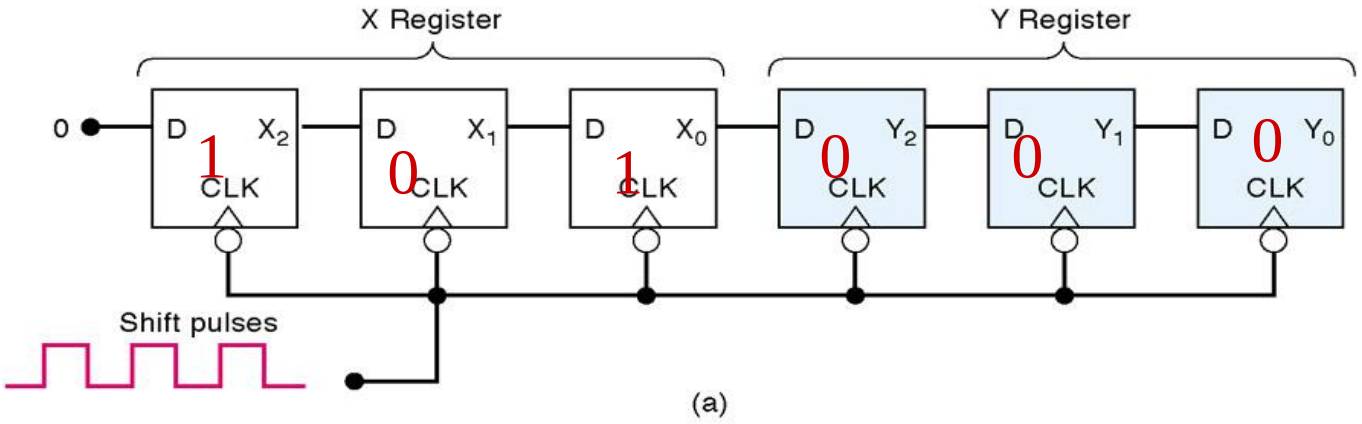


(b) Timing diagram

Fig. 6-4 Serial Transfer from Register *A* to register *B*

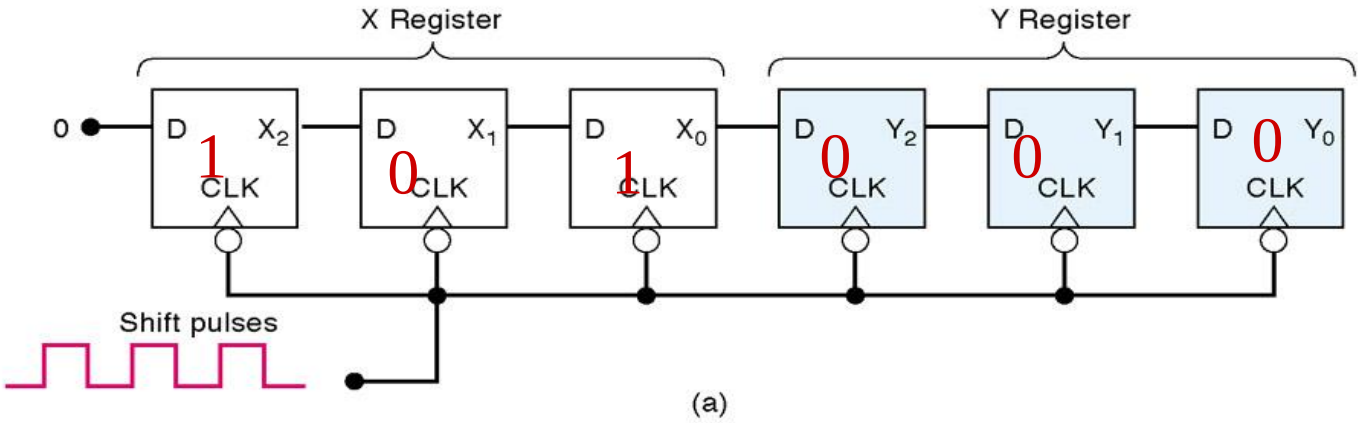
Serial transfer of Data

Transfer from register X to register Y



Serial transfer of Data

Transfer from register X to register Y

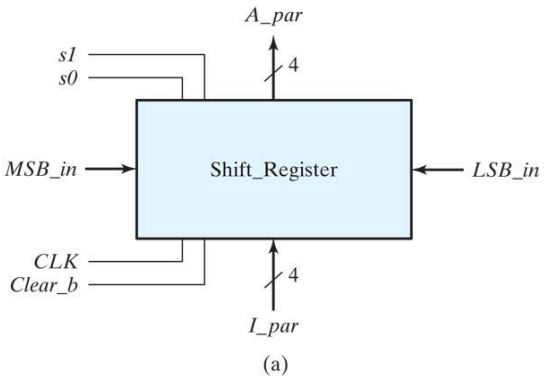


(a)

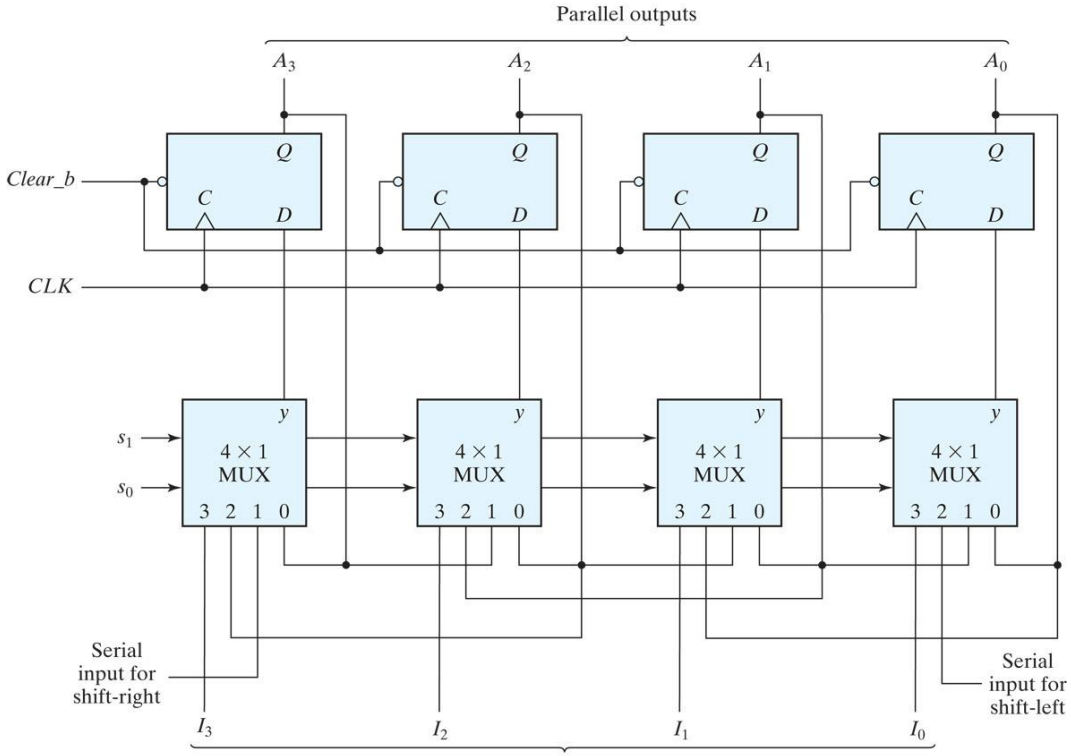
X_2	X_1	X_0	Y_2	Y_1	Y_0	
1	0	1	0	0	0	← Before pulses applied
0	1	0	1	0	0	← After first pulse
0	0	1	0	1	0	← After second pulse
0	0	0	1	0	1	← After third pulse

(b)

Four-bit universal shift register



(a)



(b)

Copyright ©2013 Pearson Education, publishing as Prentice Hall

Full Adder (see other implementations in Chap. 2)

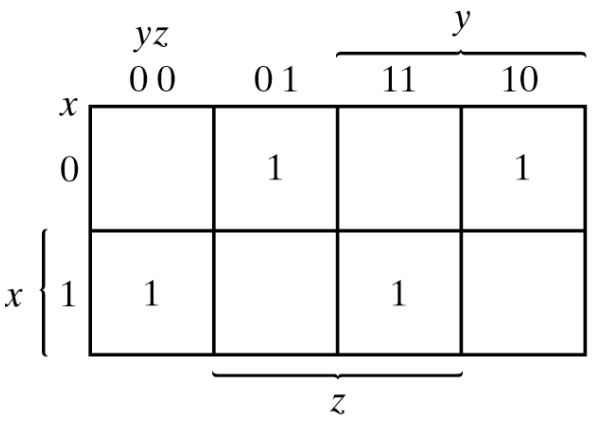
Truth Table

x	y	z	C	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

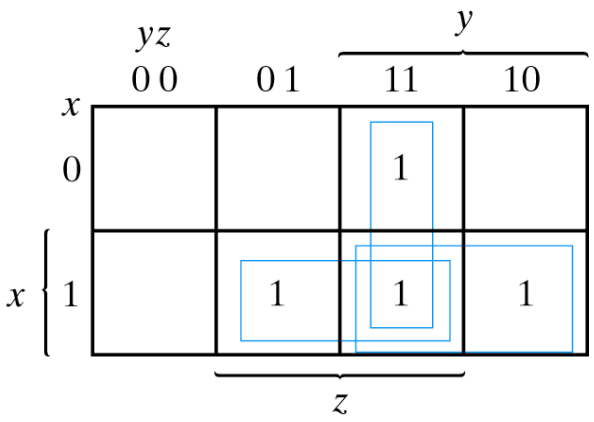
→ The Full-adder is combinational circuit



Full Adder (see other implementations in Chap. 2)



$S = x'y'z + x'yz' + xy'z' + xyz$



$C = xy + xz + yz$

Fig. 4-6 Maps for Full Adder

Full Adder (see other implementations in Chap. 2)

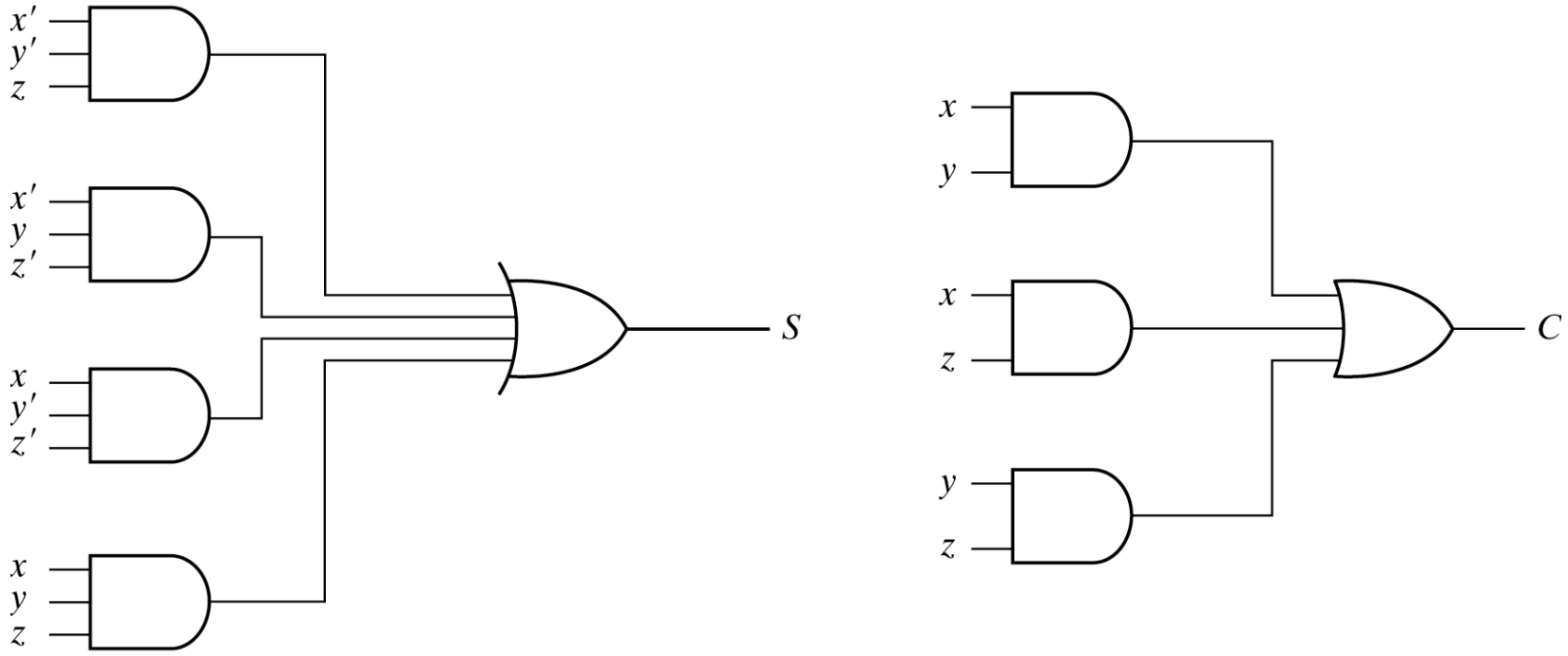


Fig. 4-7 Implementation of Full Adder in Sum of Products

Full Adder (same as in Chap. 2)

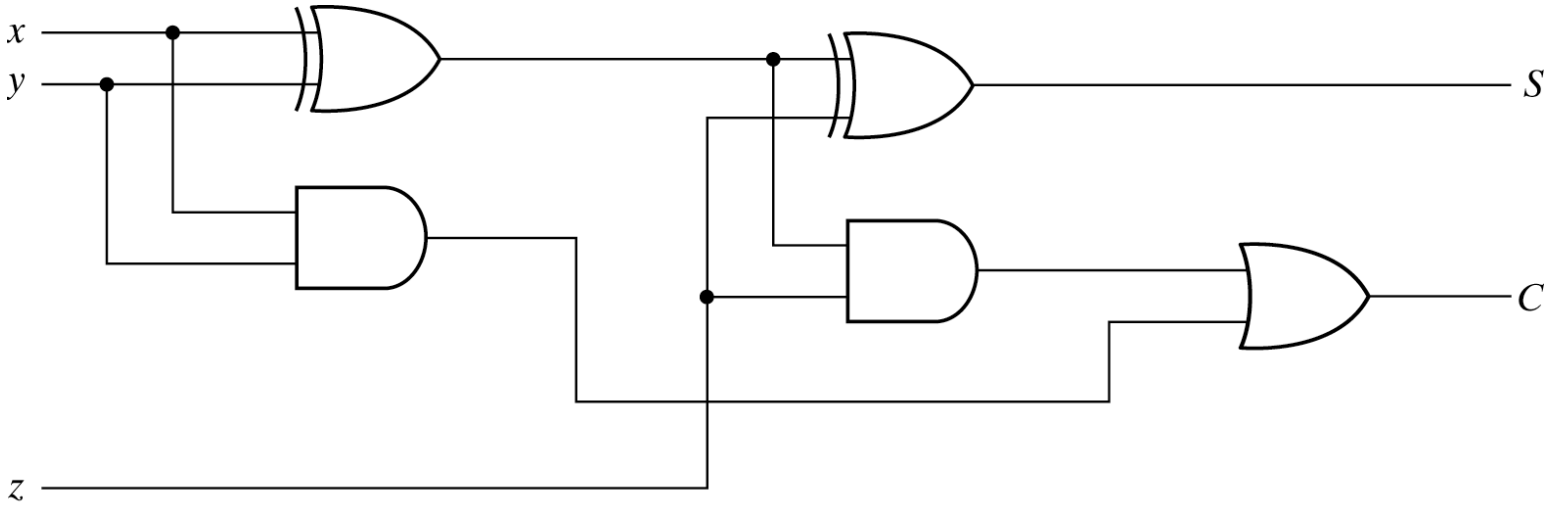


Fig. 4-8 Implementation of Full Adder with Two Half Adders and an OR Gate

Full Adder (see other implementations in Chap. 2)

Truth Table

x	y	z	C	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

→ The Full-adder is combinational circuit



Serial Addition (D Flip-Flop)

- Slower than parallel
- Low cost
- Share fast hardware on slow data

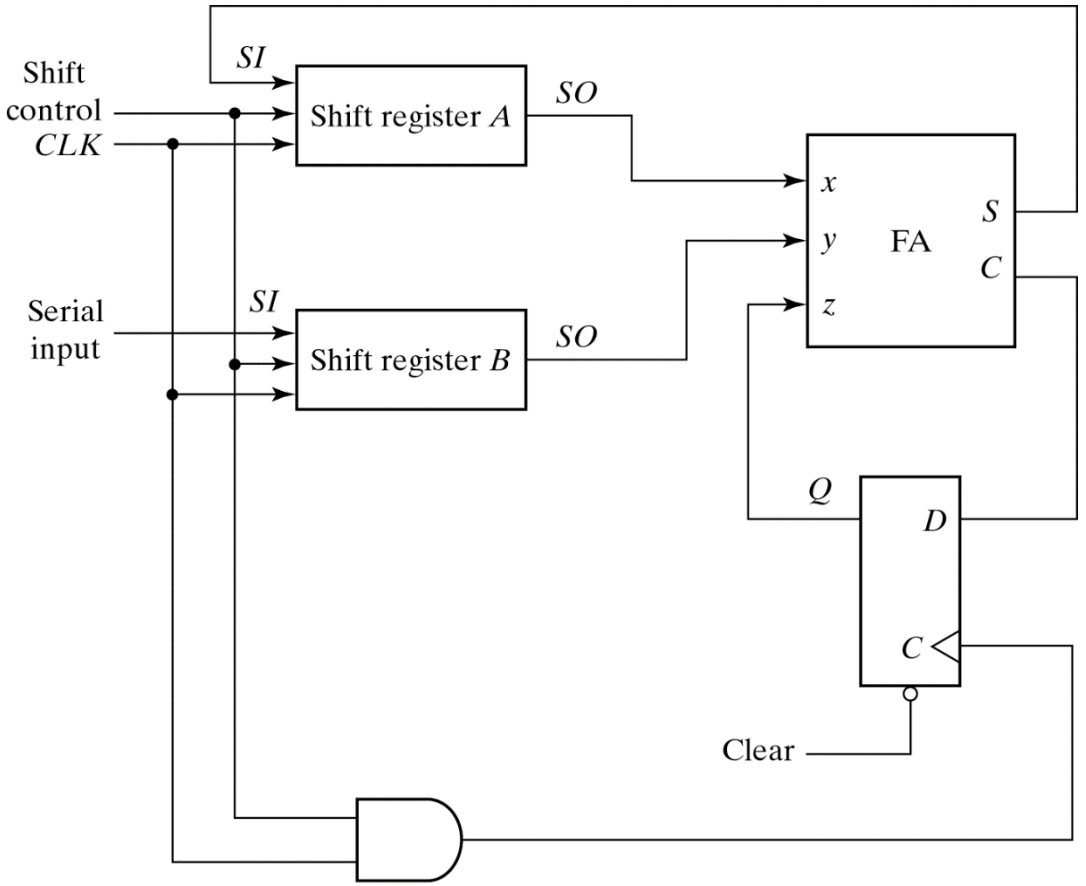


Fig. 6-5 Serial Adder

Transition Tables

J-K Flip-Flop Transition Table

J	K	Q_{n+1}	Function
0	**0**	Q_n	**No change**
0	**1**	**0**	**RESET**
1	**0**	**1**	**SET**
1	**1**	**Q'n**	**complement (also Toggle)**

- we indicate the present and next Q-output. If the flip-flop toggles or holds, we indicate that binary value.
- Also note that we need to determine both the J and K inputs.
- The "X" indicates "Don't Care" states (can be '1' or '0' input).

J-K FLIP-FLOP	
Present Next	Input J K
0 → 0	0 X
0 → 1	1 X
1 → 0	X 1
1 → 1	X 0

Designing a JK Serial Adder

- $J_Q = x \cdot y$
- $K_Q = x' \cdot y' = (x+y)'$
- $S = x \oplus y \oplus Q$

Present State	Inputs		Next State	Output	Flip_Flop Inputs	
Q	x	y	Q	S	J_Q	K_Q
0	0	0	0	0	0	x
0	0	1	0	1	0	x
0	1	0	0	1	0	x
0	1	1	1	0	1	x
1	0	0	0	1	x	1
1	0	1	1	0	x	0
1	1	0	1	0	x	0
1	1	1	1	1	x	0

New Serial Adder

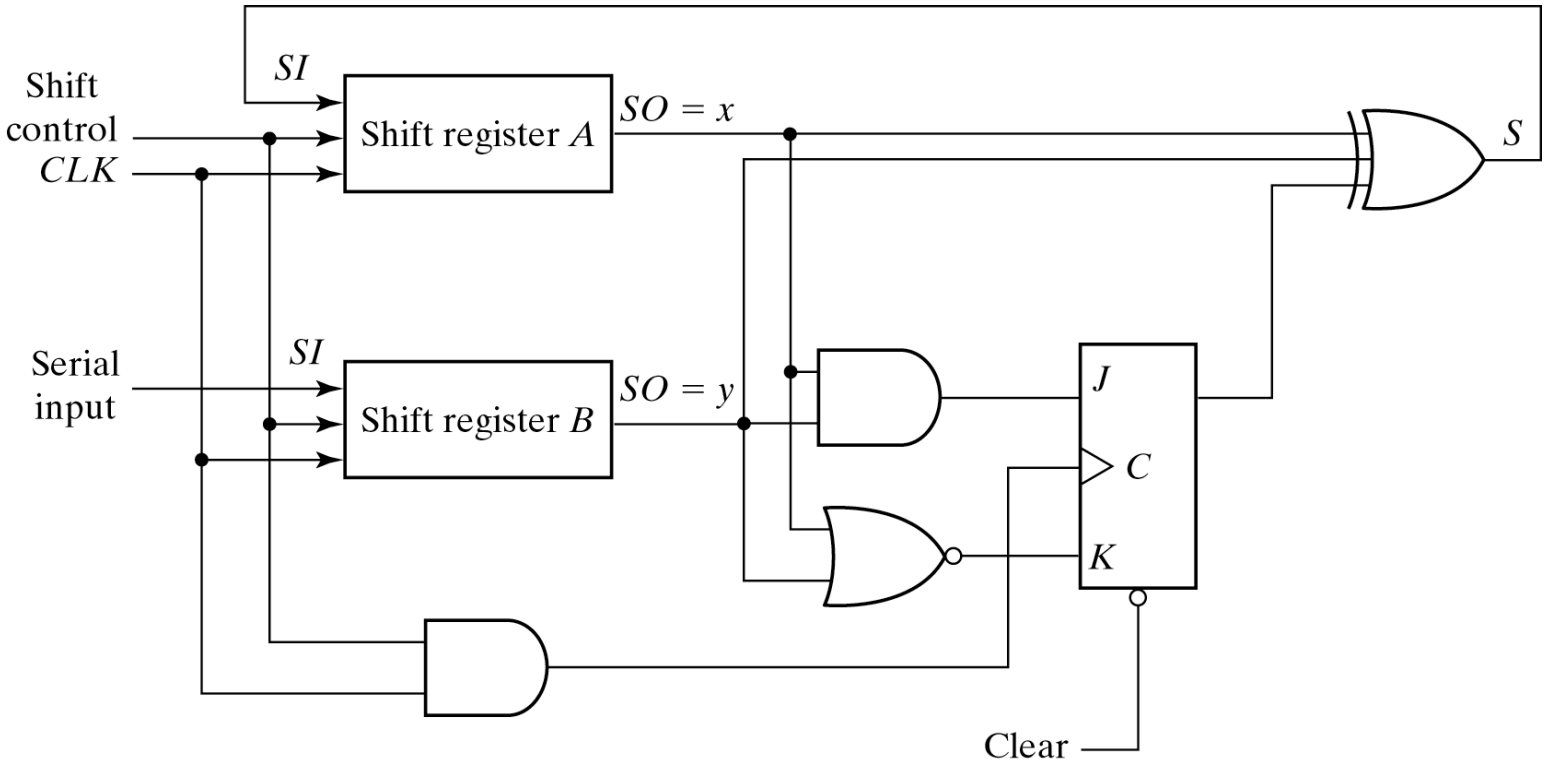


Fig. 6-6 Second form of Serial Adder

Counters

→Counter: A register that goes through a prescribed series of states

→Two main types of counters

1- asynchronous counters: also known as Ripple counters

2- Synchronous counters

→Asynchronous counters (*Ripple counters*)

– Flip flop output serves as a clock for triggering connected flip flops

→ *Synchronous counters*

– All flip flops are triggered by a clock signal at the same time

→ Synchronous counters are more widely used

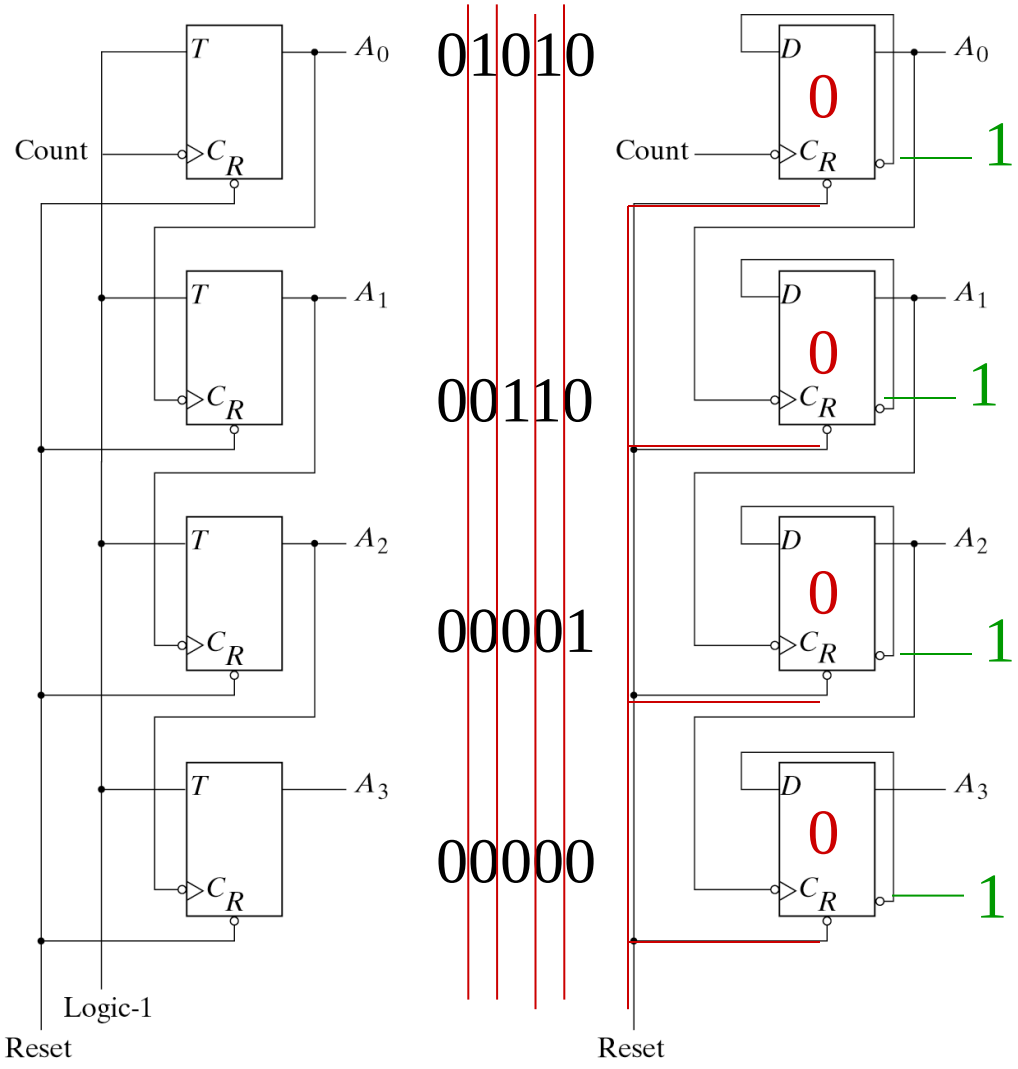
Asynchronous Counters

- Each Flip flop output controls the CLK input of the next Flip flop.
- Flip flop do not change states in exact synchronism with the applied clock pulses.
- *Ripple counter* due to the way the flip flops respond one after another in a kind of rippling effect.

A_3	A_2	A_1	A_0
0	0	0	0
0	0	0	1
0	0	1	0
0	0	1	1
0	1	0	0
0	1	0	1
0	1	1	0
0	1	1	1
1	0	0	0

Binary Ripple Counter

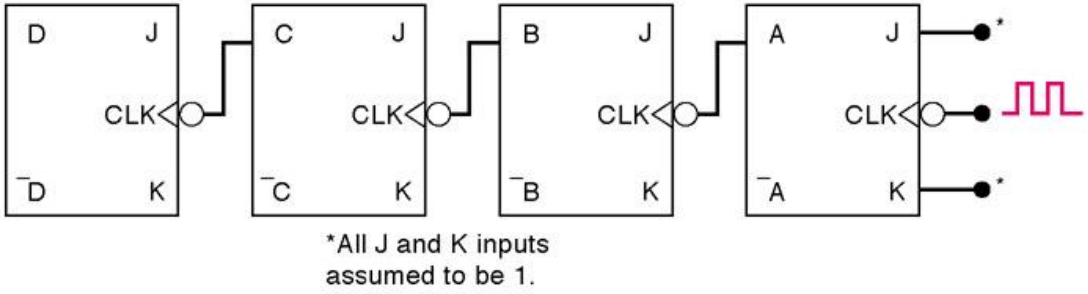
- Reset signal sets all outputs to 0
- Count signal toggles output of low-order flip flop
- Low-order flip flop provides trigger for adjacent flip flop
- Not all flops change value simultaneously
  - Lower-order flops change first



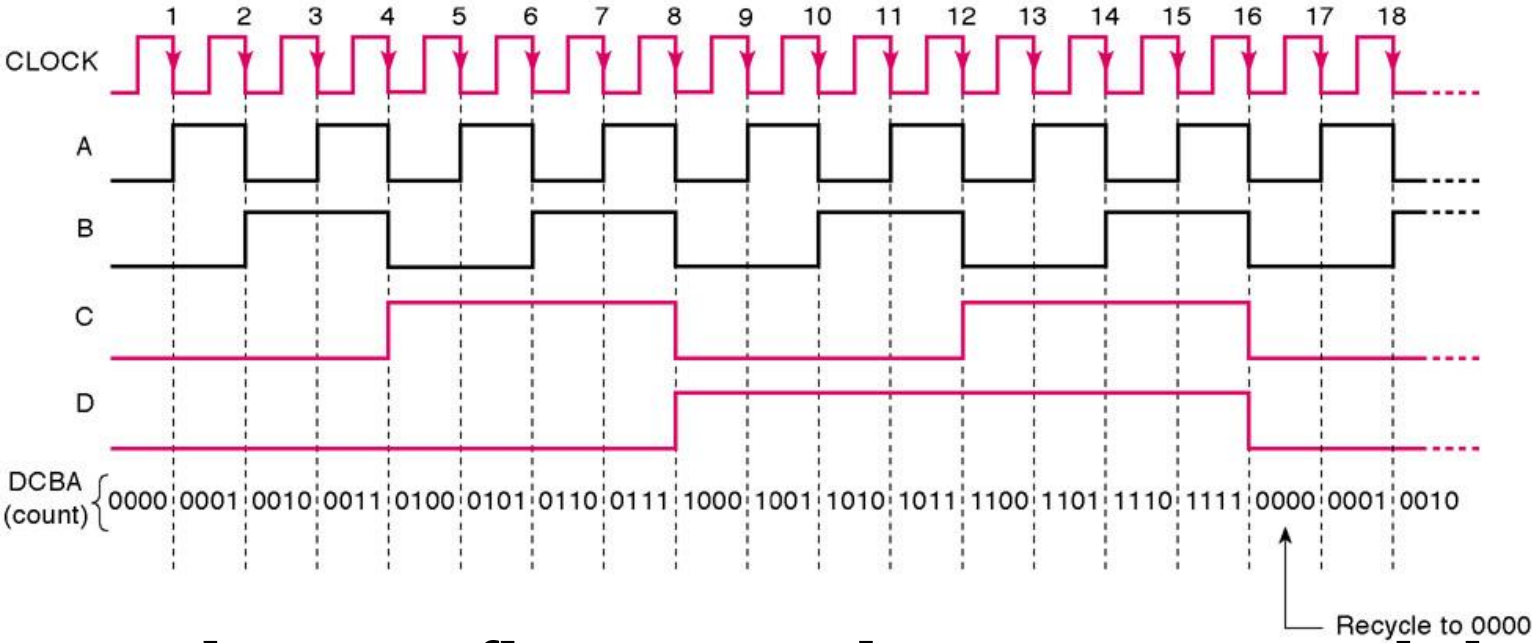
(a) With T flip-flops (b) With D flip-flops

Fig. 6-8 4-Bit Binary Ripple Counter

Another Asynchronous Ripple Counter



*All J and K inputs assumed to be 1.



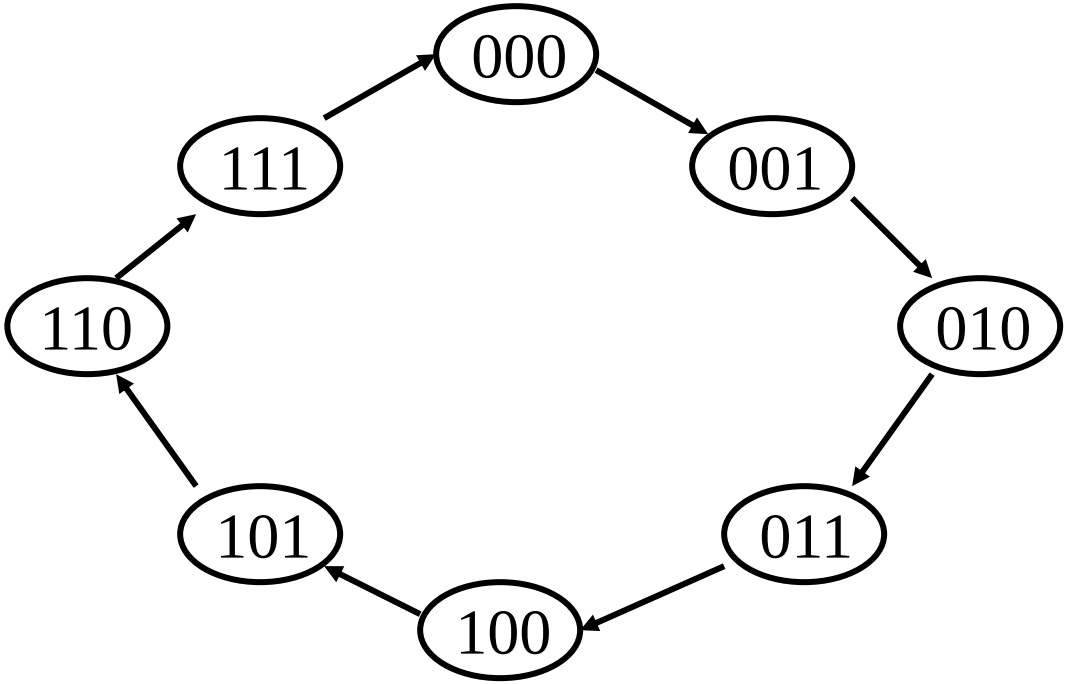
- Similar to T flop example on previous slide

Binary Counters

Counters produce a fixed sequence of binary digits, or states.

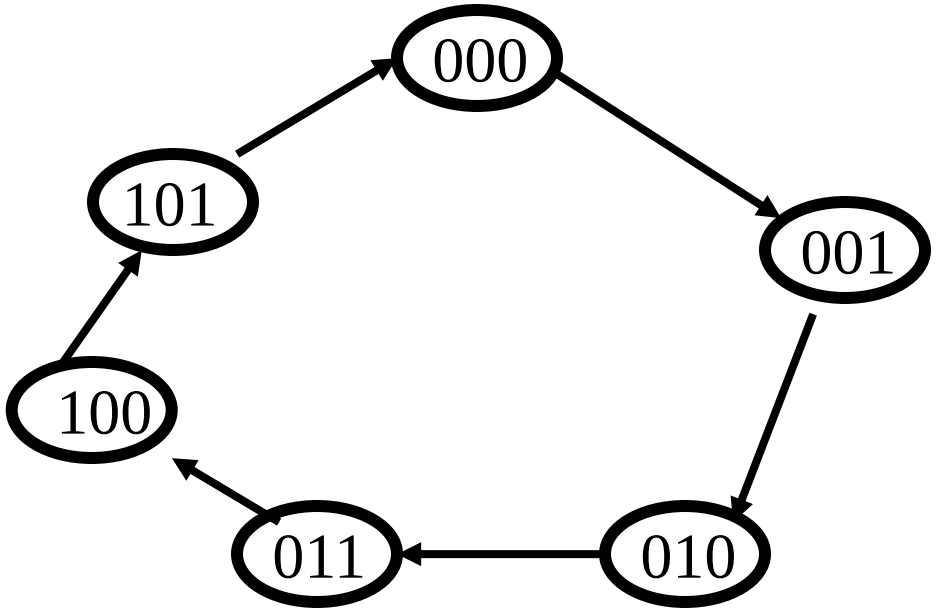
Counters are normally designed to recycle or restart the sequence.

Example:



Binary Counters: Modulus

- The number of output states is called the MODULUS, or MOD.
  - For example, a mod-6 counter has 6 unique output states → what's the max MOD ?



Counter Sequences

→*Full Sequence Count*

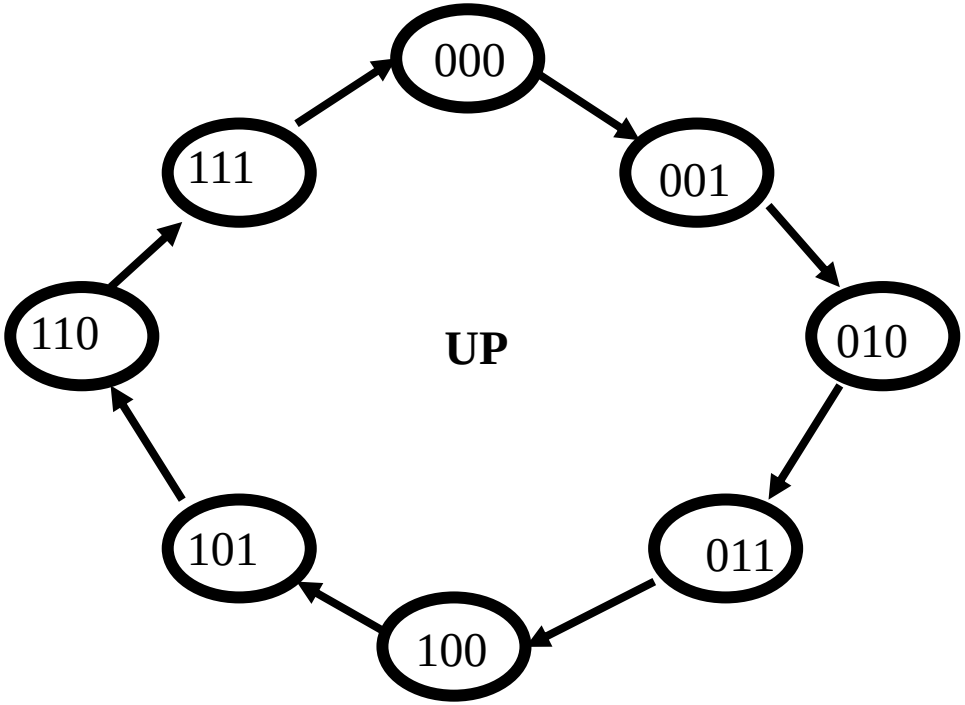
refers to a natural count that includes all possible binary numbers. It's modulus is the same as its maximum modulus.

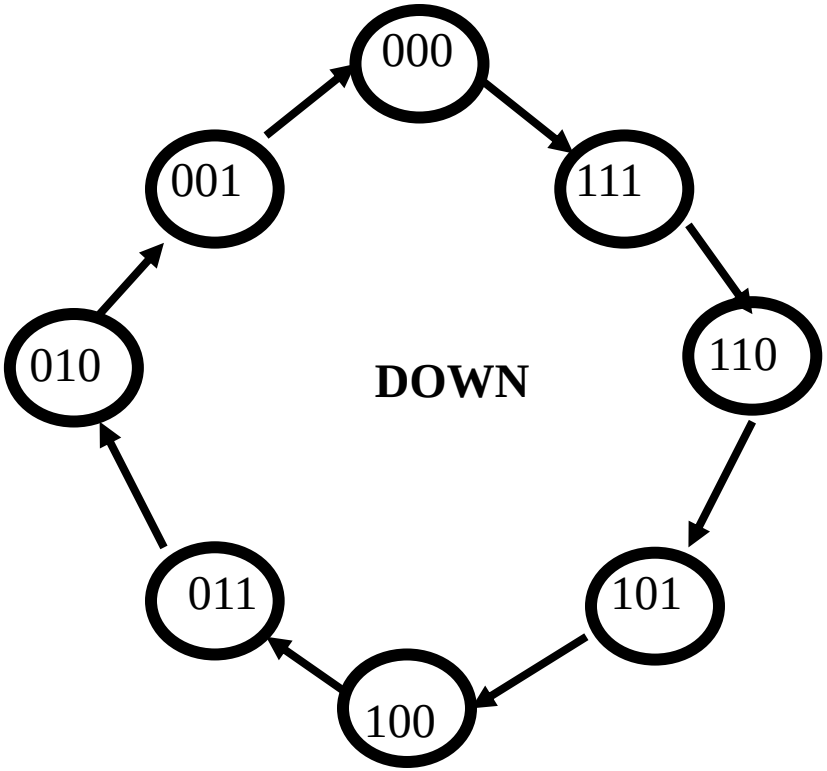
→A Truncated Sequence Count

when the modulus is less than its maximum, or where less than all possible binary numbers are used.

Binary Counters: Up/Down

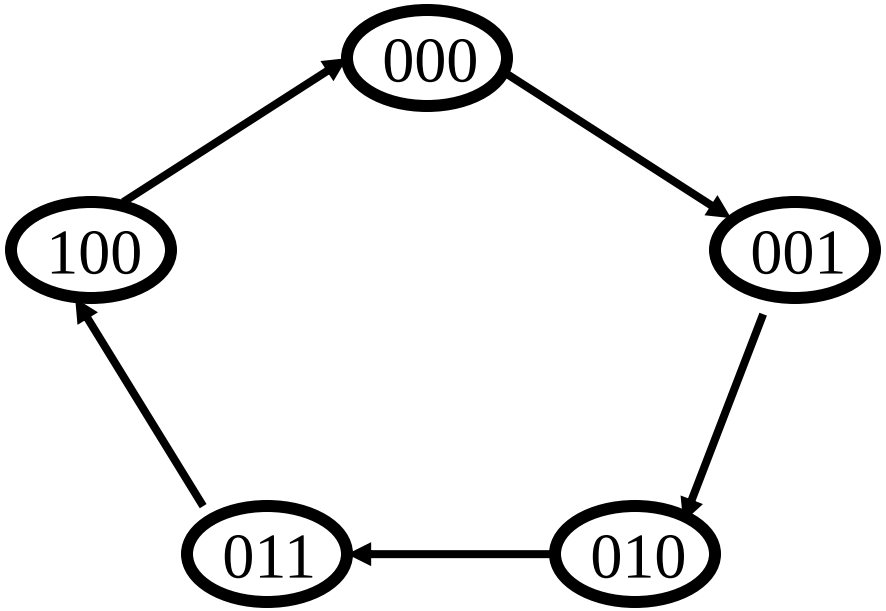
A sequential count refers to a natural numerical count.The sequence can be Up or Down:





Counter State Diagram

- Counter states are sequential, where an output state will follow another in a sequence.
- State Diagrams are used to present the sequence of these states.



State Diagram

Counter State Table

- A State Table is another means of presenting state sequences.
- As with the state diagram, the state table will help determine the next output based on the present output state.

State Table

Current			Next		
Q_c	Q_b	Q_a	Q_c	Q_b	Q_a
0	0	0	0	0	1
0	0	1	0	1	0
0	1	0	0	1	1
0	1	1	1	0	0
1	0	0	0	0	0

Design Procedure

1- Obtain from a (given) state table,

(i) the state diagram and

(ii) transition table

2- Provide the required input to each of the flip-flops by utilizing the outputs of any of the other flip-flops.

3- Use external gates, to detect specific, usable sequences of flip-flop outputs to create the necessary next input levels. (How to create inputs from outputs)

4- Use K-Maps to record and simplify the available outputs to create the inputs.

5- Implement the Circuit (i.e. Diagram)

Transition Tables

- By using K-Maps, we no longer need to utilize the toggling mode of a flip-flop (as in asynchronous). This allows us to use any edge triggered flip-flop to create the non-sequential counter.

- We need to know how the different flip-flops respond to various input states, based on their **current state.**

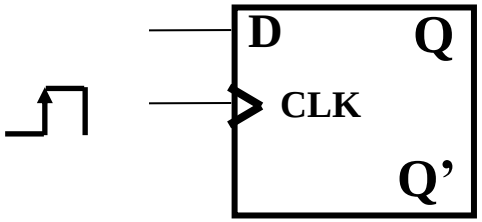
Building a Transition Table

- Building a Transition Table is done using the Function Tables of the Flip-Flops.

- Where the Function Table indicates "Q", "Q'" or "No Change", we indicate the binary value.

D Flip flop

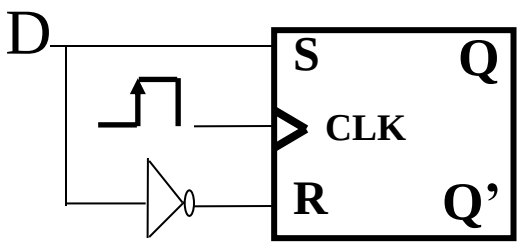
- Output changes only on the clock transition



Symbol

D	Q_{n+1}
0	0
1	1

D	CLK	Q	Q'
0	↑	0	1
1	↑	1	0
X	0	Q_0	Q_0'



D Flip flop can be implemented using SR

S = D

R =D'

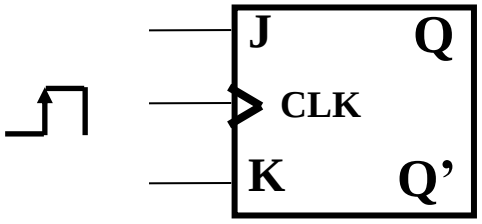
Transition Table: Example with D Flip Flop

D-FLIP-FLOP		
Present	Next	Input
0	0	0
0	1	1
1	0	0
1	1	1

- we indicate the present and the next states of the outputs.
- The input states indicates the required D input to produce the next state.

JK Flip flop

Symbol



J	K	Q_{n+1}	Function
0	0	Q_n	No change
0	1	0	RESET
1	0	1	SET
1	1	Q'_n	complement (also Toggle)

Same as SR except for

K=J= 1 the JK flip flop will

Output the opposite state of Q_n.

- Q_n = state *before* positive edge
- Q_{n+1} = state *after* positive edge

If $Q_n = 1$ then $Q_{n+1} = 0$

If $Q_n = 0$ then $Q_{n+1} = 1$

Transition Tables

J-K Flip-Flop Transition Table

- we indicate the present and next Q-output. If the flip-flop toggles or holds, we indicate that binary value.
- Also note that we need to determine both the J and K inputs.
- The “X” indicates “Don’t Care” states (can be ‘1’ or ‘0’ input).

J-K FLIP-FLOP	
Present Next	Input J K
0 → 0	0 X
0 → 1	1 X
1 → 0	X 1
1 → 1	X 0

J-K Flip Flop Transition Table

J-K FLIP-FLOP		
Present	Next	Input J K
0	0	0 X
0	1	1 X
1	0	X 1
1	1	X 0

States

Hold or Reset

Toggle or Set

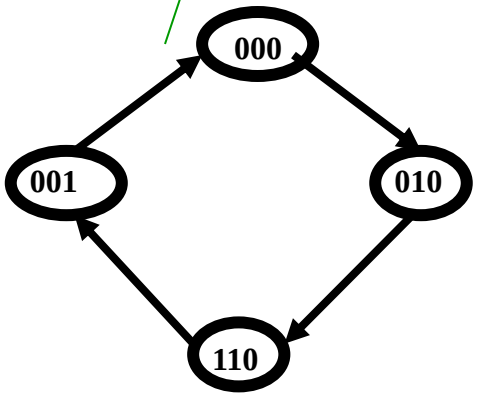
Toggle or Reset

Hold or Set

→ **each output of the J-K Flip-Flop is the result of 2 possible states.**

Building a K-Map with D Flip Flop

- To build a K-Map, we first need:
  1- State Diagram of the output sequence
  2- State Table of the output sequence
  3- Transition Table for the Flip-Flop we intend to use (here D ff)
    - Present and Next outputs
    - Necessary inputs to create Next outputs

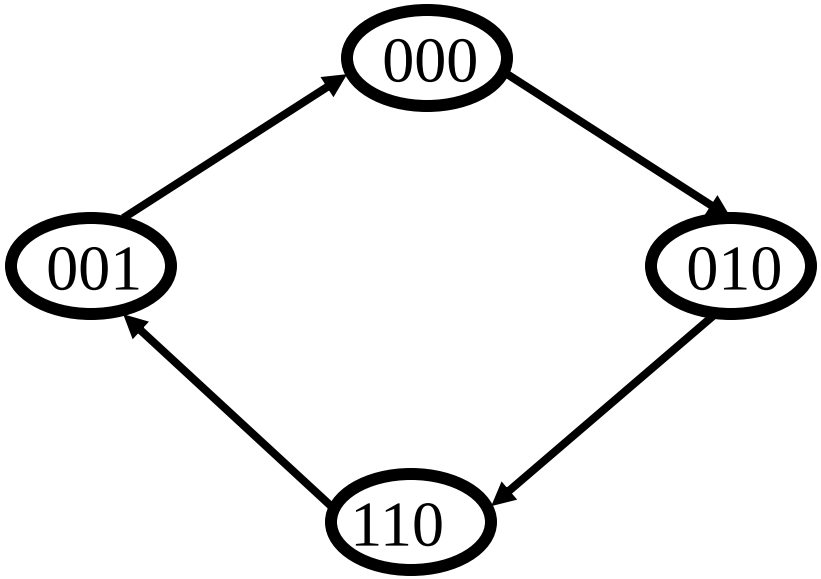


Present	Next
$Q_C Q_B Q_A$	$Q_C Q_B Q_A$
0 0 0	0 1 0
0 1 0	1 1 0
1 1 0	0 0 1
0 0 1	0 0 0

D-FLIP-FLOP		
Present	Next	Input
0	0	0
0	1	1
1	0	0
1	1	1

Building a K-Map for D Flip Flop: Step 1

- Draw the State Table and State Diagram of the output sequence.



State Diagram

Present $Q_C Q_B Q_A$	Next $Q_C Q_B Q_A$
0 0 0	0 1 0
0 1 0	1 1 0
1 1 0	0 0 1
0 0 1	0 0 0

State Table

Building the K-Map : Step 2

- Determine the type of Flip-Flop and its Transition Table. For our example, we use the D Flip-Flop:

D-FLIP-FLOP	
Present → Next	Input
0 → 0	0
0 → 1	1
1 → 0	0
1 → 1	1

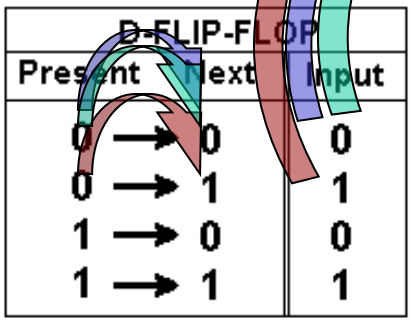
Transition Table

Building the K-Map: Step 3

- **Build the State Table with the inputs indicated:**

D-FLIP-FLOP		
Present	Next	Input
0	0	0
0	1	1
1	0	0
1	1	1

Present	Next	Input
Q_C Q_B Q_A	Q_C Q_B Q_A	D_C D_B D_A
0 0 0	0 1 0	0 1 0
0 1 0	1 1 0	1 1 0
1 1 0	0 0 1	0 0 1
0 0 1	0 0 0	0 0 0



Present			Next			Input		
Q_C	Q_B	Q_A	Q_C	Q_B	Q_A	D_C	D_B	D_A
0	0	0	0	1	0	0	1	0
0	1	0	1	1	0	1	1	0
1	1	0	0	0	1	0	0	1
0	0	1	0	0	0	0	0	0

Animated

Building the K-Map: Step 4

Build a K-Map for each output:

Present			Next			Input		
Q_C	Q_B	Q_A	Q_C	Q_B	Q_A	D_C	D_B	D_A
0	0	0	0	1	0	0	1	0
0	1	0	1	1	0	1	1	0
1	1	0	0	0	1	0	0	1
0	0	1	0	0	0	0	0	0

$Q_C Q_B$ \ Q_A	0	1
00	0	0
01	1	X
11	0	X
10	X	X

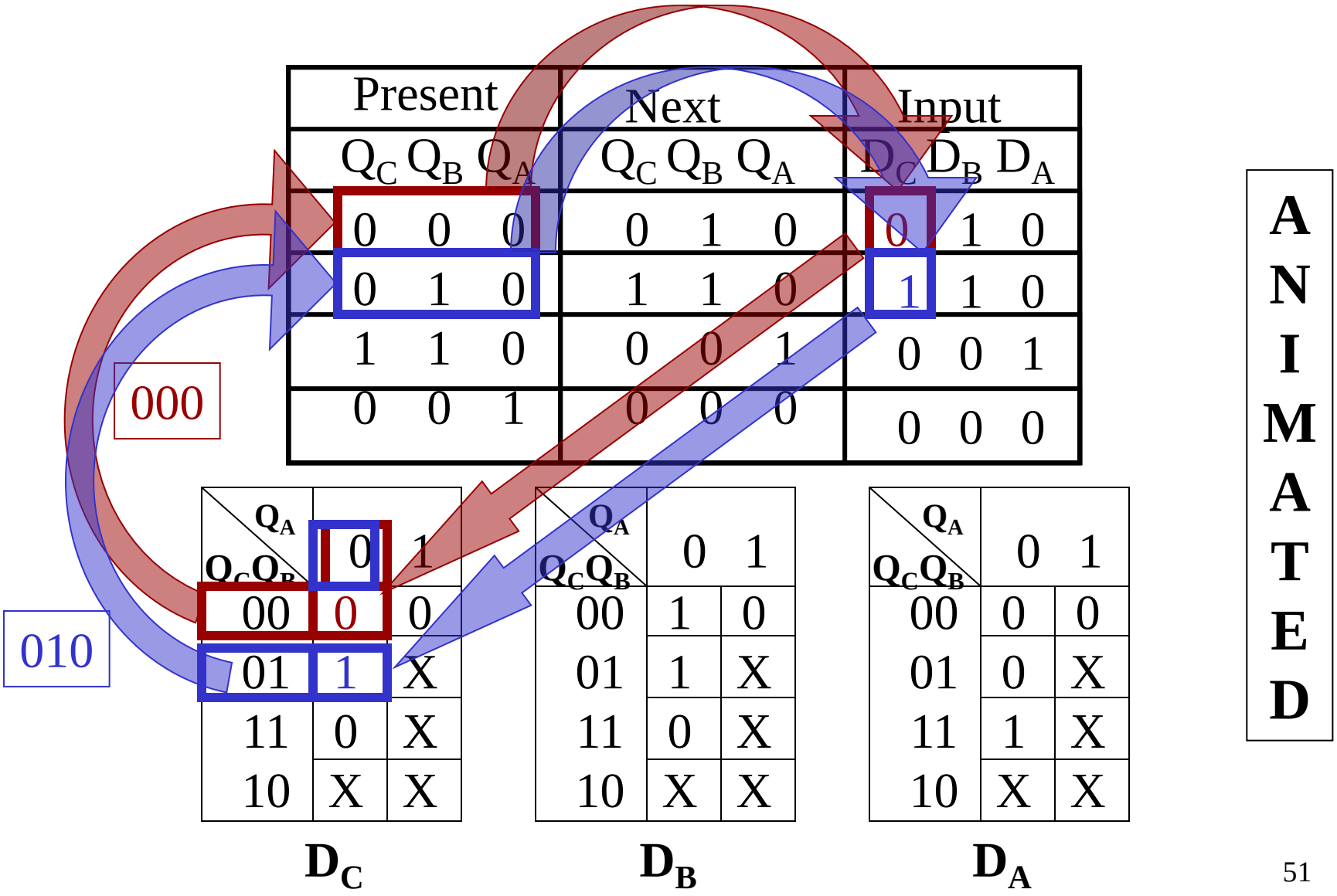
D_C

$Q_C Q_B$ \ Q_A	0	1
00	1	0
01	1	X
11	0	X
10	X	X

D_B

$Q_C Q_B$ \ Q_A	0	1
00	0	0
01	0	X
11	1	X
10	X	X

D_A


Present
Next
Input
Q_C Q_B Q_A
Q_C Q_B Q_A
D_C D_B D_A
0 0 0
0 1 0
0 1 0
1 1 0
1 1 0
1 1 0
0 0 1
0 0 1
0 0 1
0 0 1
0 0 0
0 0 0
000
010
Q_A
$Q_C Q_B$
0 1
00 0 0
01 1 X
11 0 X
10 X X
D_C
Q_A
$Q_C Q_B$
0 1
00 1 0
01 1 X
11 0 X
10 X X
D_B
Q_A
$Q_C Q_B$
0 1
00 0 0
01 0 X
11 1 X
10 X X
D_A

ANIMATED

Building the K-Map: Step 5

determine the simplified SOP for the inputs.

$Q_C Q_B$ \ Q_A	0	1
00	0	0
01	1	X
11	0	X
10	X	X

D_C

$D_C = Q'_C Q_B$

$Q_C Q_B$ \ Q_A	0	1
00	1	0
01	1	X
11	0	X
10	X	X

D_B

$D_B = Q'_C Q'_A$

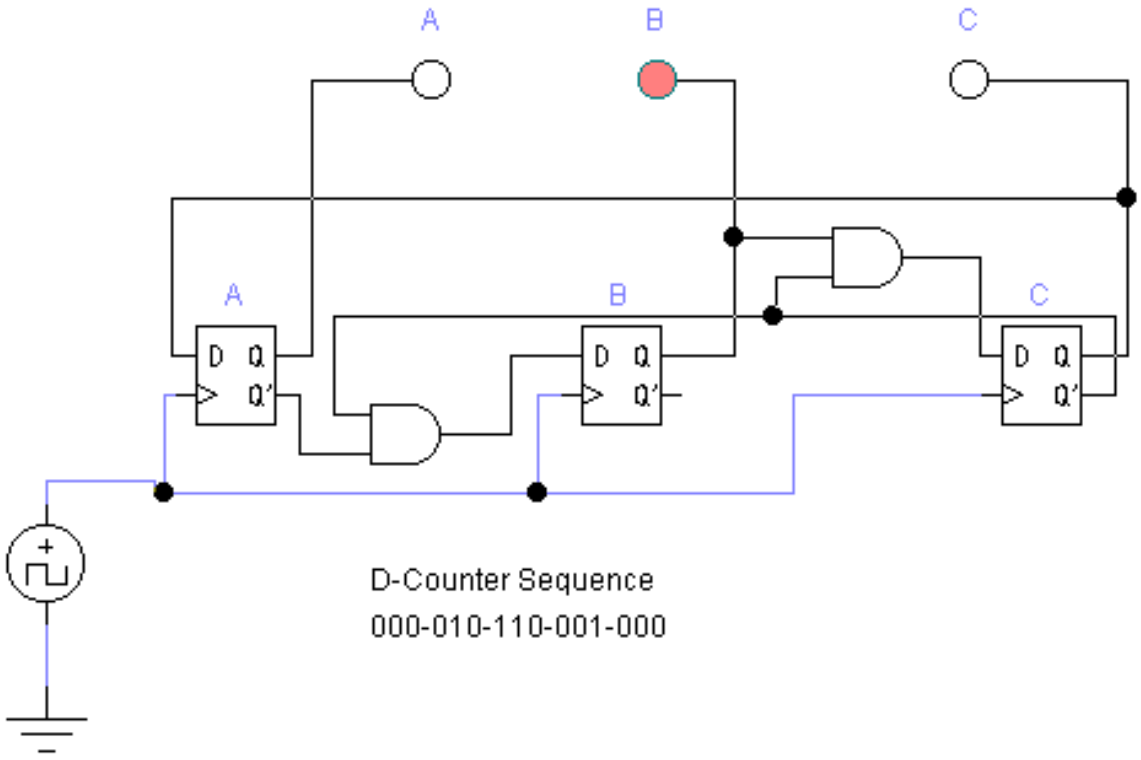
$Q_C Q_B$ \ Q_A	0	1
00	0	0
01	0	X
11	1	X
10	X	X

D_A

$D_A = Q_C$

Implementation Diagram

- Draw the Circuit Diagram
- Verify its operations.



Binary Counter Examples (will NOT be on the website)

- Examples of binary counter with JK flip Flop

1- Without missing States

2- With missing States

Binary counter with JK flip flop

Transition Table for JK FF

(a) JK flip-flop truth table

J	K	Q_n	Q_{n+1}
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	0

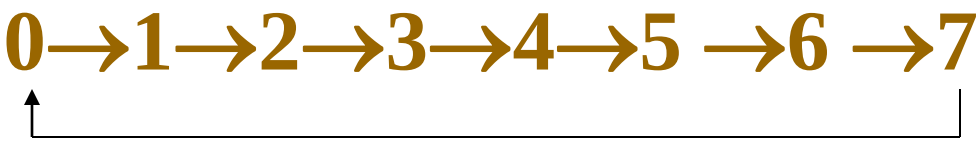
(b) Transition table for JK flip-flops

Q_n	Q_{n+1}	J	K
0	0	0	x
0	1	1	x
1	0	x	1
1	1	x	0

Binary counter with JK flip flop

- Build state diagram
- Build the state table that consists of
  - Current state output
  - Next state output
  - JK inputs for each flip-flop
- Use K-maps to simplify expressions for JK inputs
- Build the circuit for the counter

Binary counter with JK flip flop



- 3-bit binary counter
- 3 JK flip-flops are needed
- Current state and next state outputs are 3 bits each
- 3 pairs of JK inputs

Binary counter with JK flip flop

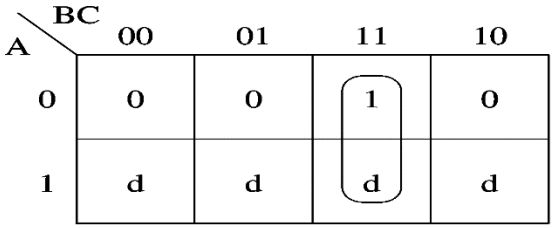
State table for the binary counter example

Present state			Next state			JK flip-flop inputs					
A	B	C	A	B	C	J_A	K_A	J_B	K_B	J_C	K_C
0	0	0	0	0	1	0	X	0	X	1	X
0	0	1	0	1	0	0	X	1	X	X	1
0	1	0	0	1	1	0	X	X	0	1	X
0	1	1	1	0	0	1	X	X	1	X	1
1	0	0	1	0	1	X	0	0	X	1	X
1	0	1	1	1	0	X	0	1	X	X	1
1	1	0	1	1	1	X	0	X	0	1	X
1	1	1	0	0	0	X	1	X	1	X	1

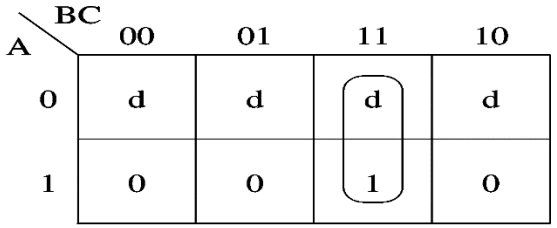
Binary counter with JK flip flop

Use K-maps to simplify expressions for JK inputs

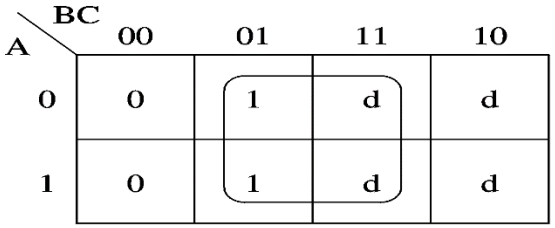
X= d



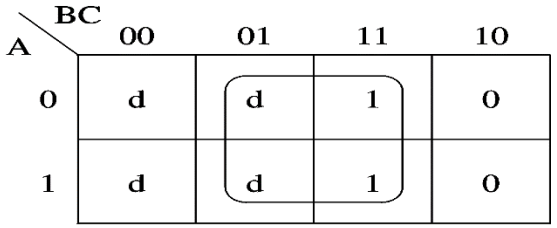
$J_A = B\,C$



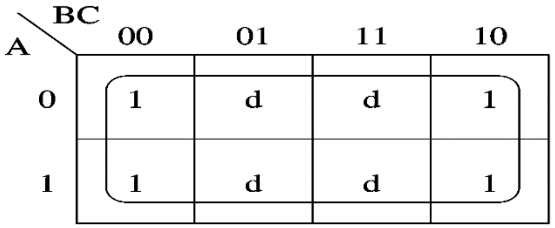
$K_A = B\,C$



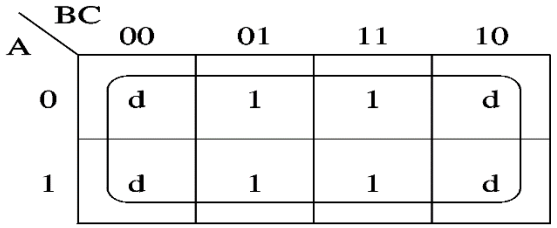
$J_B = C$



$K_B = C$



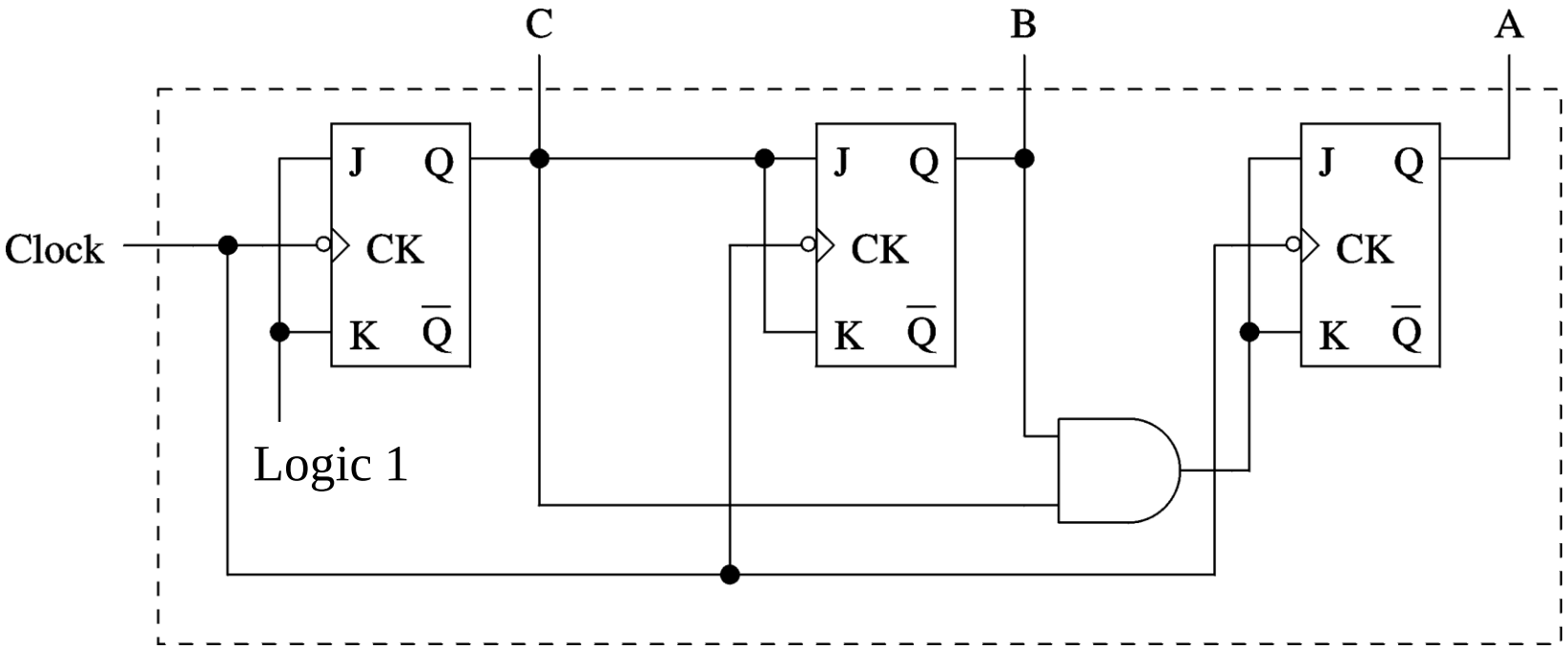
$J_C = 1$



$K_C = 1$

Binary counter with JK flip flop

- Final circuit for the binary counter

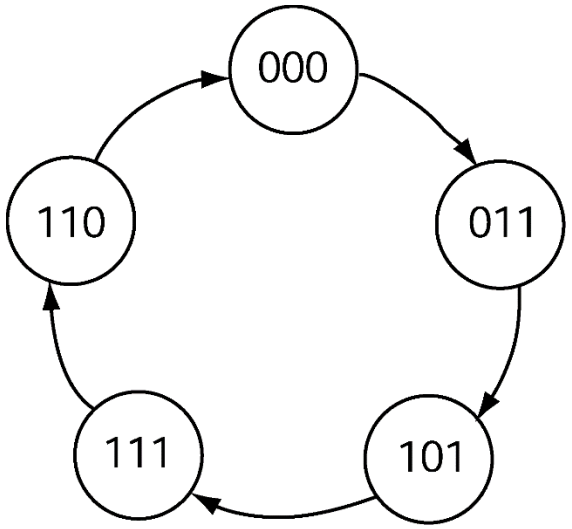


Binary counter with JK flip flop (missing states)

→ Example with missing states

0→3→5→7→6→0

- Same design process as before
- One significant change
  - Missing states
    - » 1, 2, and 4
    - » Use don't cares for these states



Binary counter with JK flip flop (missing states)

Sate table for the counter example

Present state			Next state			JK flip-flop inputs					
A	B	C	A	B	C	J_A	K_A	J_B	K_B	J_C	K_C
0	0	0	0	1	1	0	X	1	X	1	X
0	0	1	X	X	X	X	X	X	X	X	X
0	1	0	X	X	X	X	X	d	X	X	X
0	1	1	1	0	1	1	X	X	1	X	0
1	0	0	X	X	X	X	X	X	X	X	X
1	0	1	1	1	1	X	0	1	X	X	0
1	1	0	0	0	0	X	1	X	1	0	X
1	1	1	1	1	0	X	0	X	0	X	1

Binary counter with JK flip flop (missing states)

K-maps to simplify JK input expressions

X=d

A \ BC	00	01	11	10
0	0	d	1	d
1	d	d	d	d

$J_A = B$

A \ BC	00	01	11	10
0	d	d	d	d
1	d	0	0	1

$K_A = \overline{C}$

A \ BC	00	01	11	10
0	1	d	d	d
1	d	1	d	d

$J_B = 1$

A \ BC	00	01	11	10
0	d	d	1	d
1	d	d	0	1

$K_B = \overline{A} + \overline{C}$

A \ BC	00	01	11	10
0	1	d	d	d
1	d	d	d	0

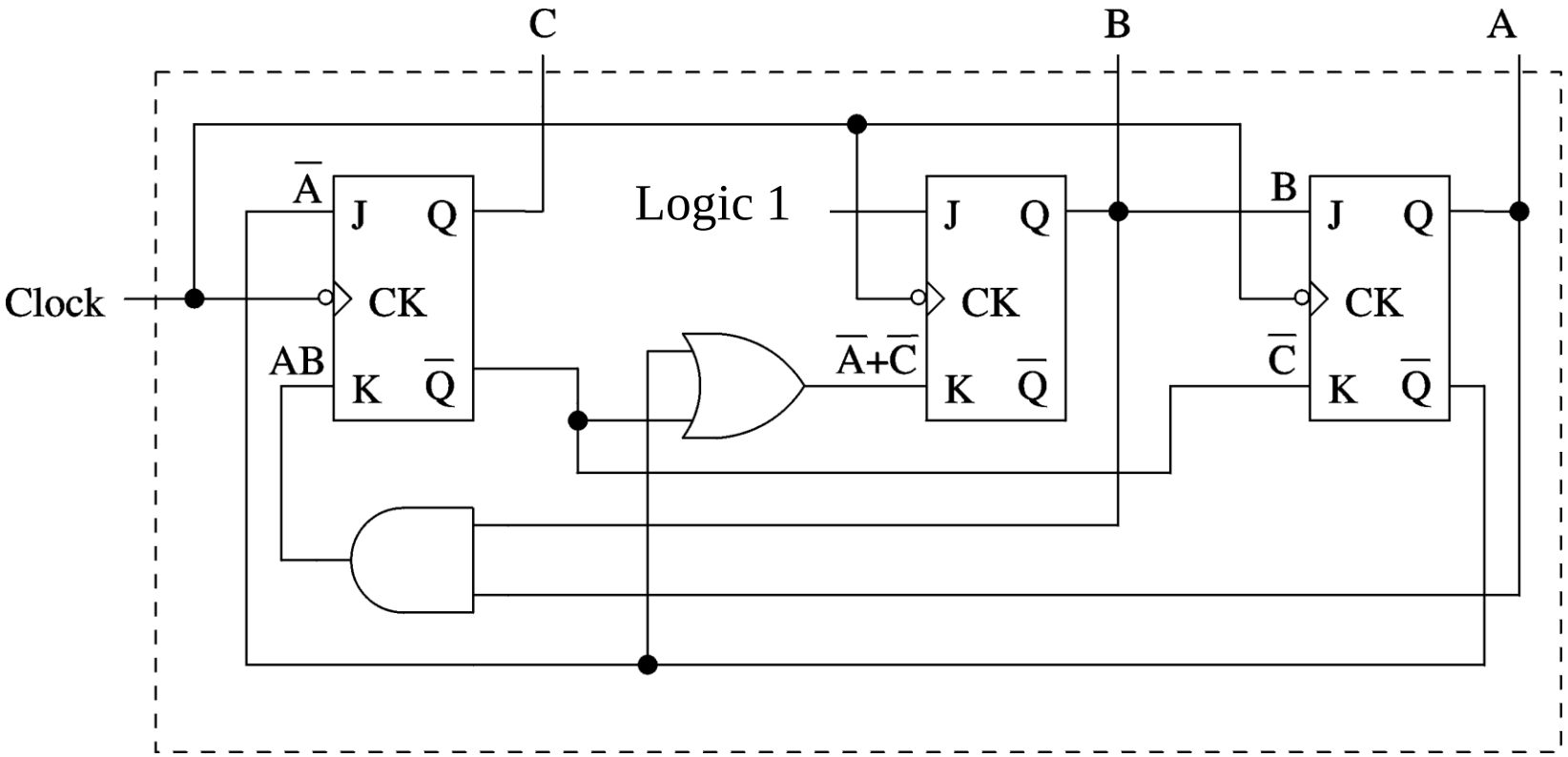
$J_C = \overline{A}$

A \ BC	00	01	11	10
0	d	d	0	d
1	d	0	1	d

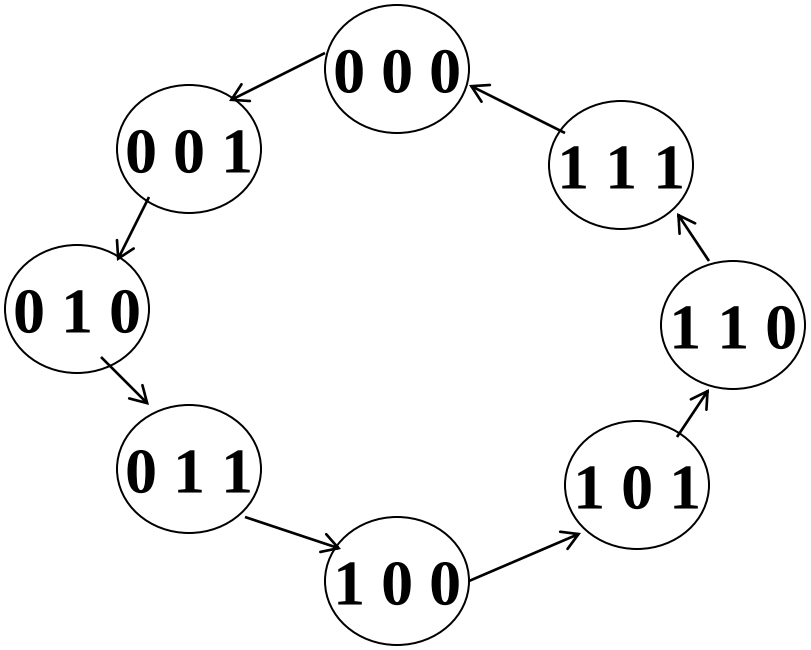
$K_C = A\,B$

Binary counter with JK flip flop (missing states)

Final circuit



Example: - Design a 3 - bit binary counter using T flip - flops

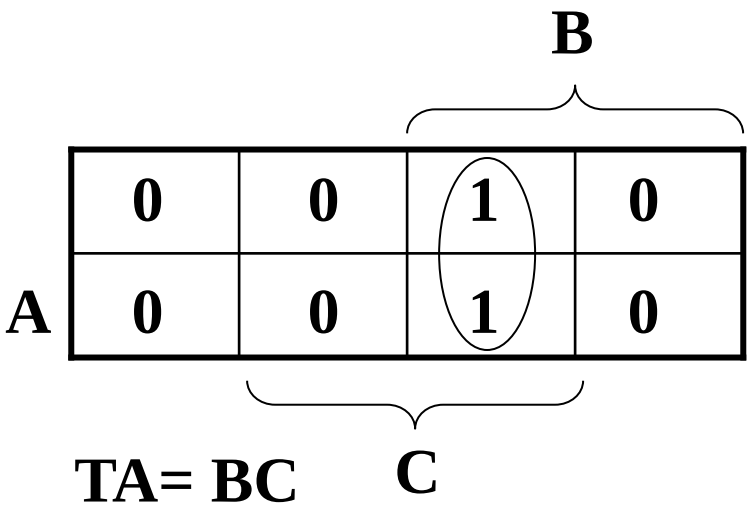


Example: - Design a 3 - bit binary counter using T flip - flops.

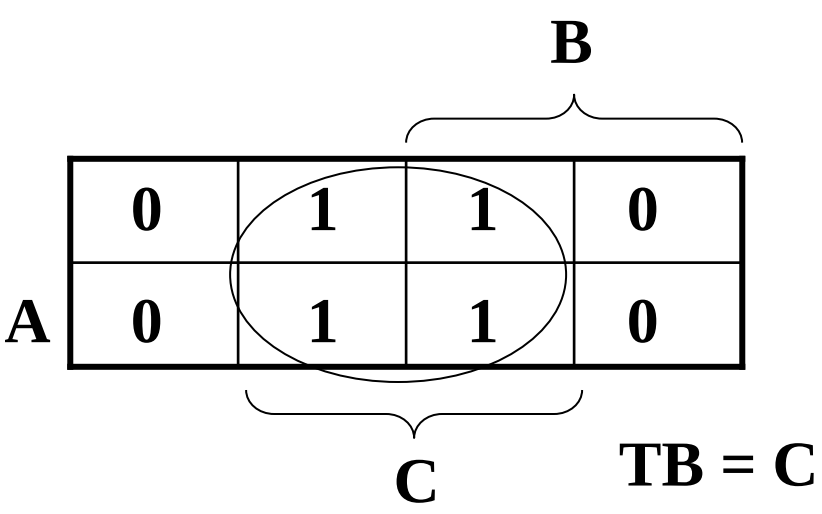
Present State A B C	Next State A B C	Flip - Flop Inputs T A	T B	T C
0 0 0	0 0 1	0	0	1
0 0 1	0 1 0	0	1	1
0 1 0	0 1 1	0	0	1
0 1 1	1 0 0	1	1	1
1 0 0	1 0 1	0	0	1
1 0 1	1 1 0	0	1	1
1 1 0	1 1 1	0	0	1
1 1 1	0 0 0	1	1	1

Example: - Design a 3 - bit binary counter using T flip - flops.

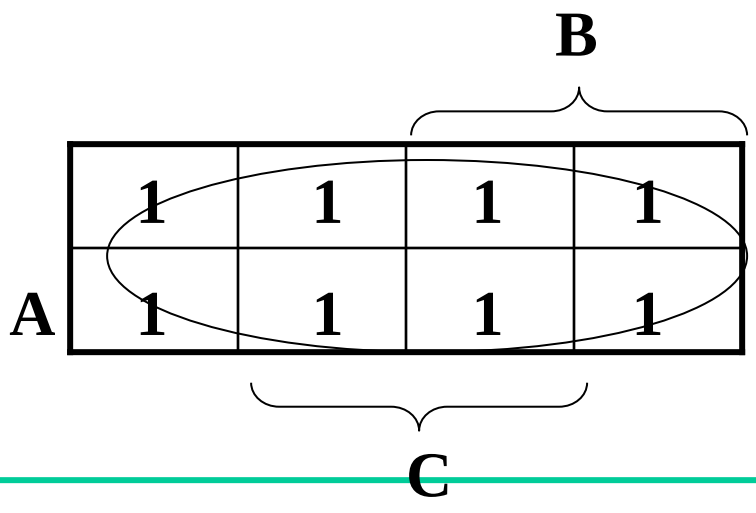
- Kmap



TA= BC

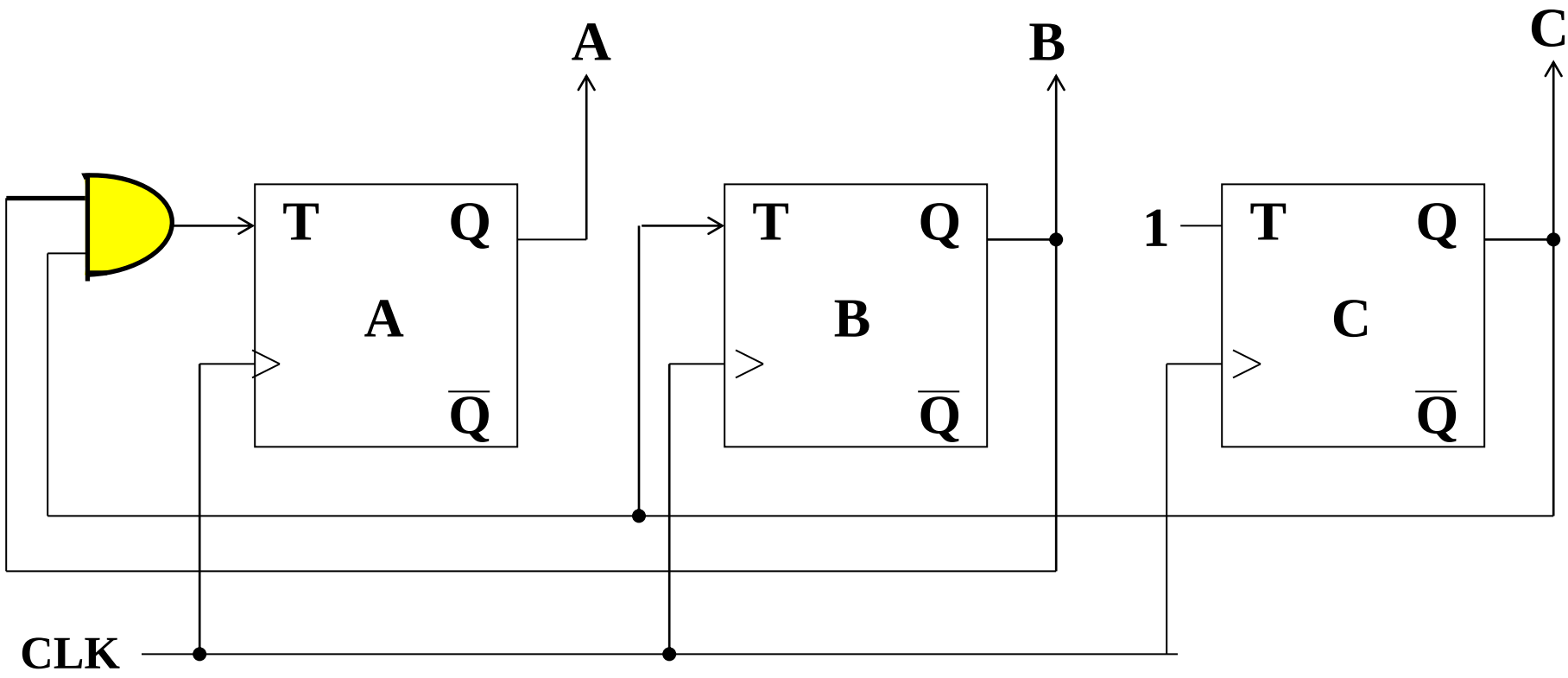


TB = C



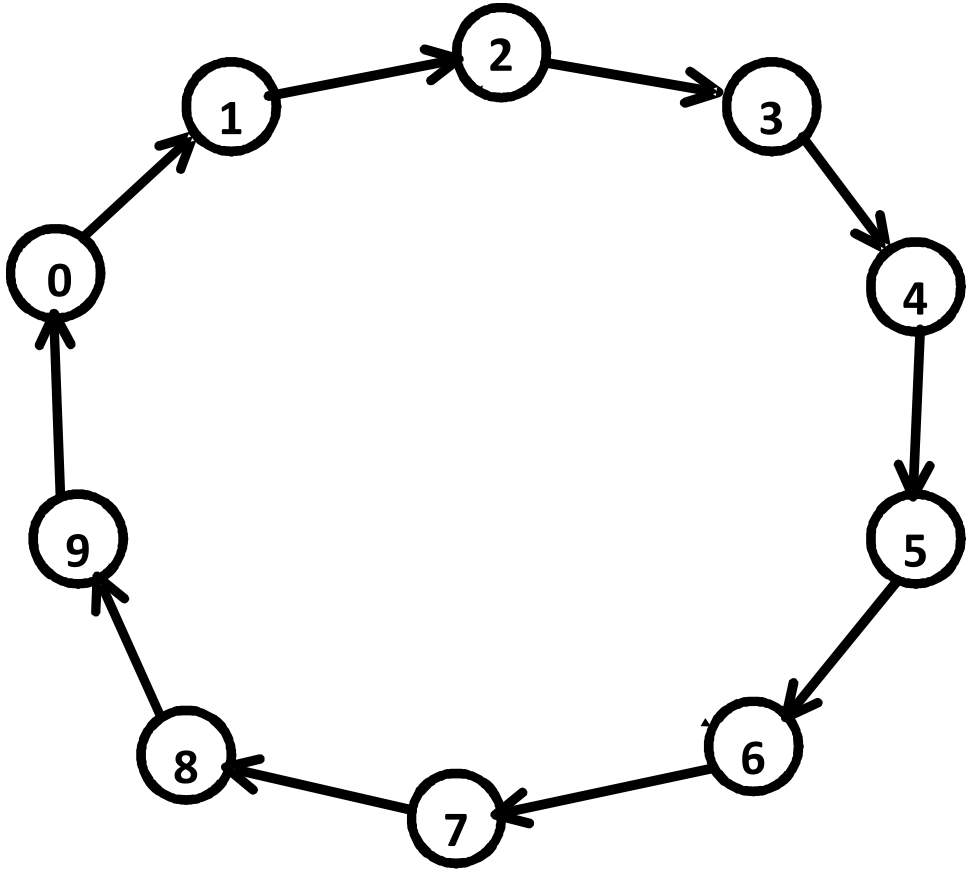
TC = 1

Example: - Design a 3 - bit binary counter using T flip - flops.



Design: Synchronous BCD

- Design a BCD counter using T Flip Flop



Design: Synchronous BCD

- State table

Current State Q_8 Q_4 Q_2 Q_1	Next State Q_8 Q_4 Q_2 Q_1	T-FF inputs T_8 T_4 T_2 T_1
0 0 0 0	0 0 0 1	
0 0 0 1	0 0 1 0	
0 0 1 0	0 0 1 1	
0 0 1 1	0 1 0 0	
0 1 0 0	0 1 0 1	
0 1 0 1	0 1 1 0	
0 1 1 0	0 1 1 1	
0 1 1 1	1 0 0 0	
1 0 0 0	1 0 0 1	
1 0 0 1	0 0 0 0	

Design: Synchronous BCD

- **We can use the sequential logic model to design a synchronous BCD counter with T flip-flops. Below is the State Table.**

Current State Q_8 Q_4 Q_2 Q_1	Next State Q_8 Q_4 Q_2 Q_1	T-FF inputs T_8 T_4 T_2 T_1
0 0 0 0	0 0 0 1	0 0 0 1
0 0 0 1	0 0 1 0	0 0 1 1
0 0 1 0	0 0 1 1	0 0 0 1
0 0 1 1	0 1 0 0	0 1 1 1
0 1 0 0	0 1 0 1	0 0 0 1
0 1 0 1	0 1 1 0	0 0 1 1
0 1 1 0	0 1 1 1	0 0 0 1
0 1 1 1	1 0 0 0	1 1 1 1
1 0 0 0	1 0 0 1	0 0 0 1
1 0 0 1	0 0 0 0	1 0 0 1

- **Don't care states have been left out here.**

Synchronous BCD (Continued)

- **Use K-Maps to minimize the next state function:**

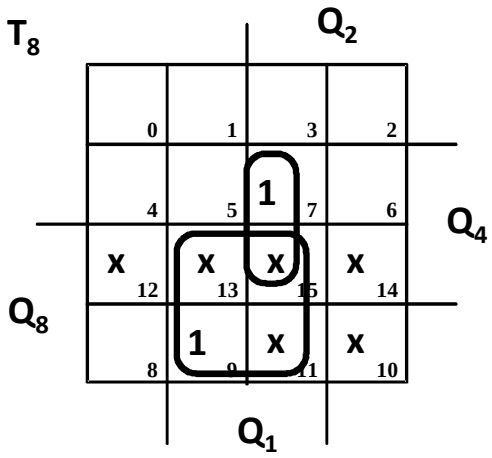
$T_8 = Q_8 \bullet Q_1 + Q_4 \bullet Q_2 \bullet Q_1$

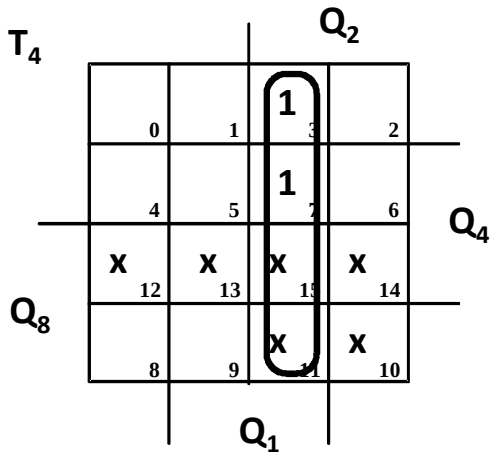
$T_4 = Q_2 \bullet Q_1$

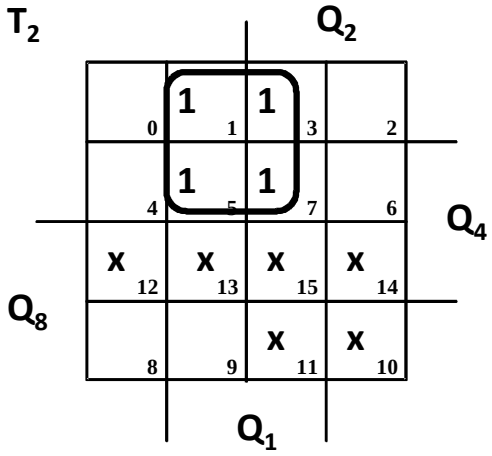
$T_2 = Q_8' \bullet Q_1$

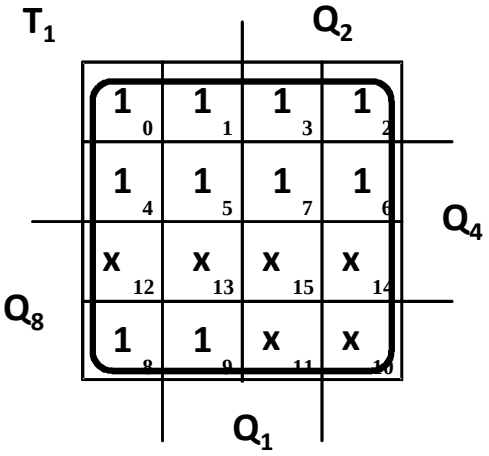
$T_1 = \text{"1"}$

Note: <u>Don't Cares</u> are included here.



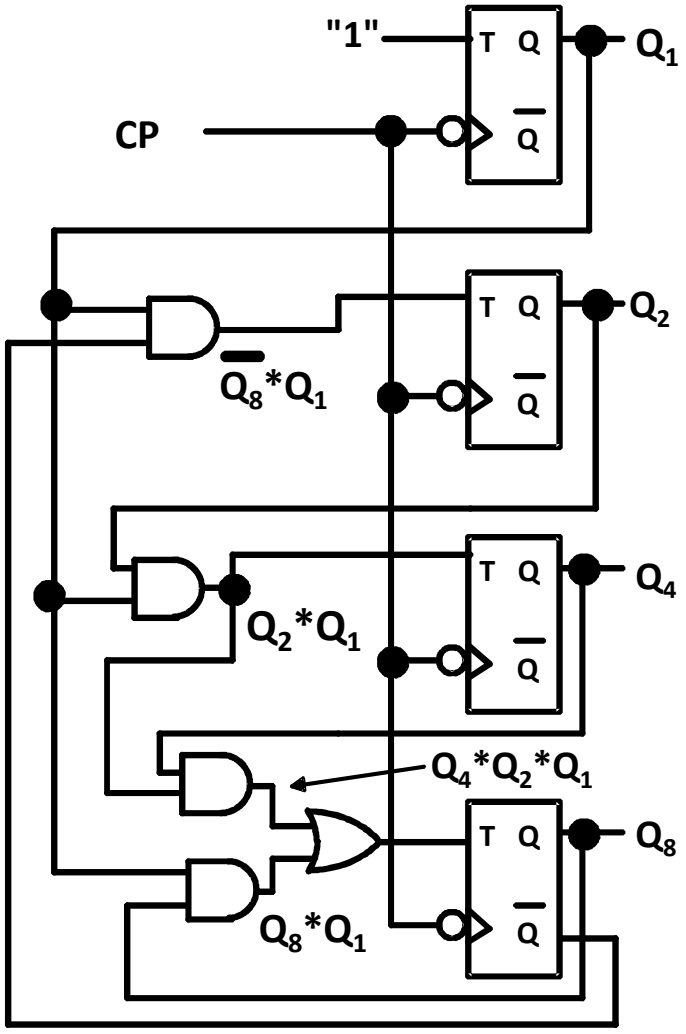






Synchronous BCD (Continued)

- **The minimized circuit:**



Designing a Synchronous BCD Counter

Present State				Next State				Output	Next State			
Q_8	Q_4	Q_2	Q_1	Q_8	Q_4	Q_2	Q_1	y	TQ_8	TQ_4	TQ_2	TQ_1
0	0	0	0	0	0	0	1	0	0	0	0	1
0	0	0	1	0	0	1	0	0	0	0	1	1
0	0	1	0	0	0	1	1	0	0	0	0	1
0	0	1	1	0	1	0	0	0	0	1	1	1
0	1	0	0	0	1	0	1	0	0	0	0	1
0	1	1	0	0	1	1	0	0	0	0	1	1
0	1	0	1	0	1	1	1	0	0	0	0	1
0	1	1	1	1	0	0	0	0	1	1.	1	1
1	0	0	0	1	0.	0	1	0	0	0	0	1
1	0.	0	1	0	0	0	0	1	1	0	0	1

$TQ_1 = 1$, $TQ_2 = Q'_8Q_1$, $TQ_4 = Q_2Q_1$, $TQ_8 = Q_8Q_1 + Q_4Q_2Q_1$ $y = Q_8Q_1$

,

Asynchrone, BCD Counter

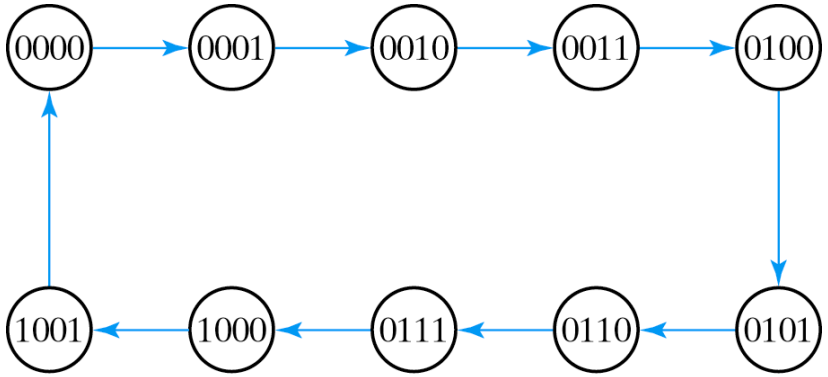
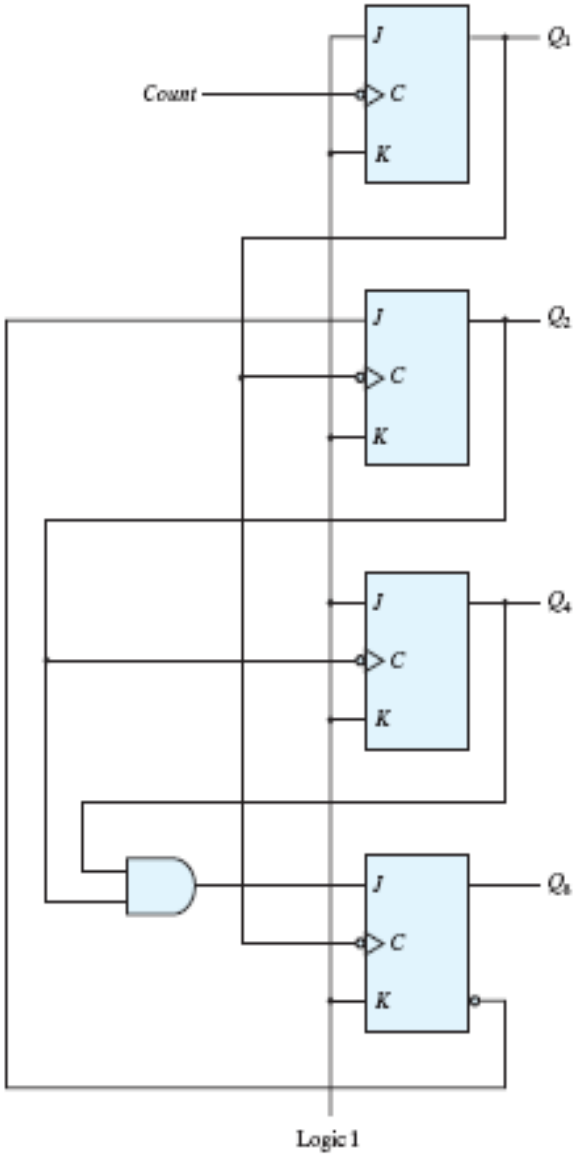


Fig. 6-9 State Diagram of a Decimal BCD-Counter



Decimal (BCD) Counter

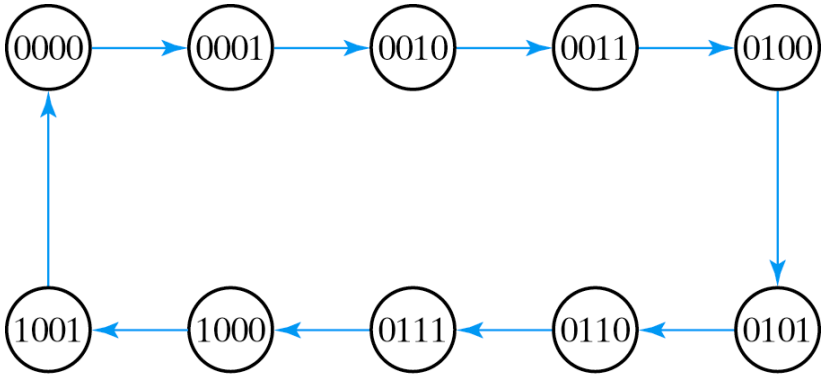


Fig. 6-9 State Diagram of a Decimal BCD-Counter

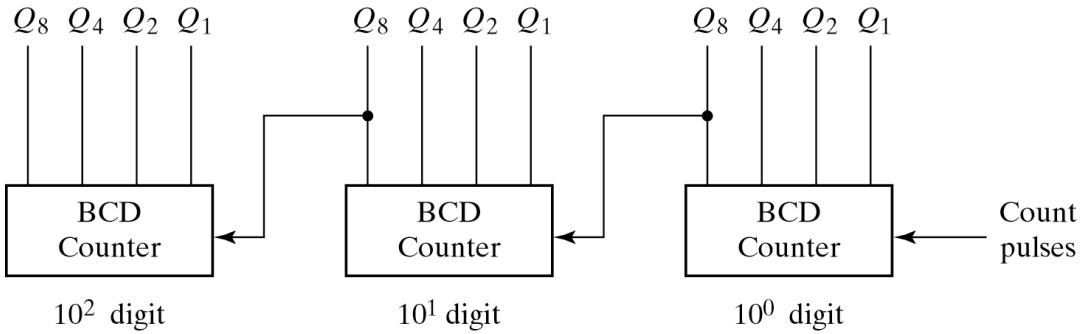
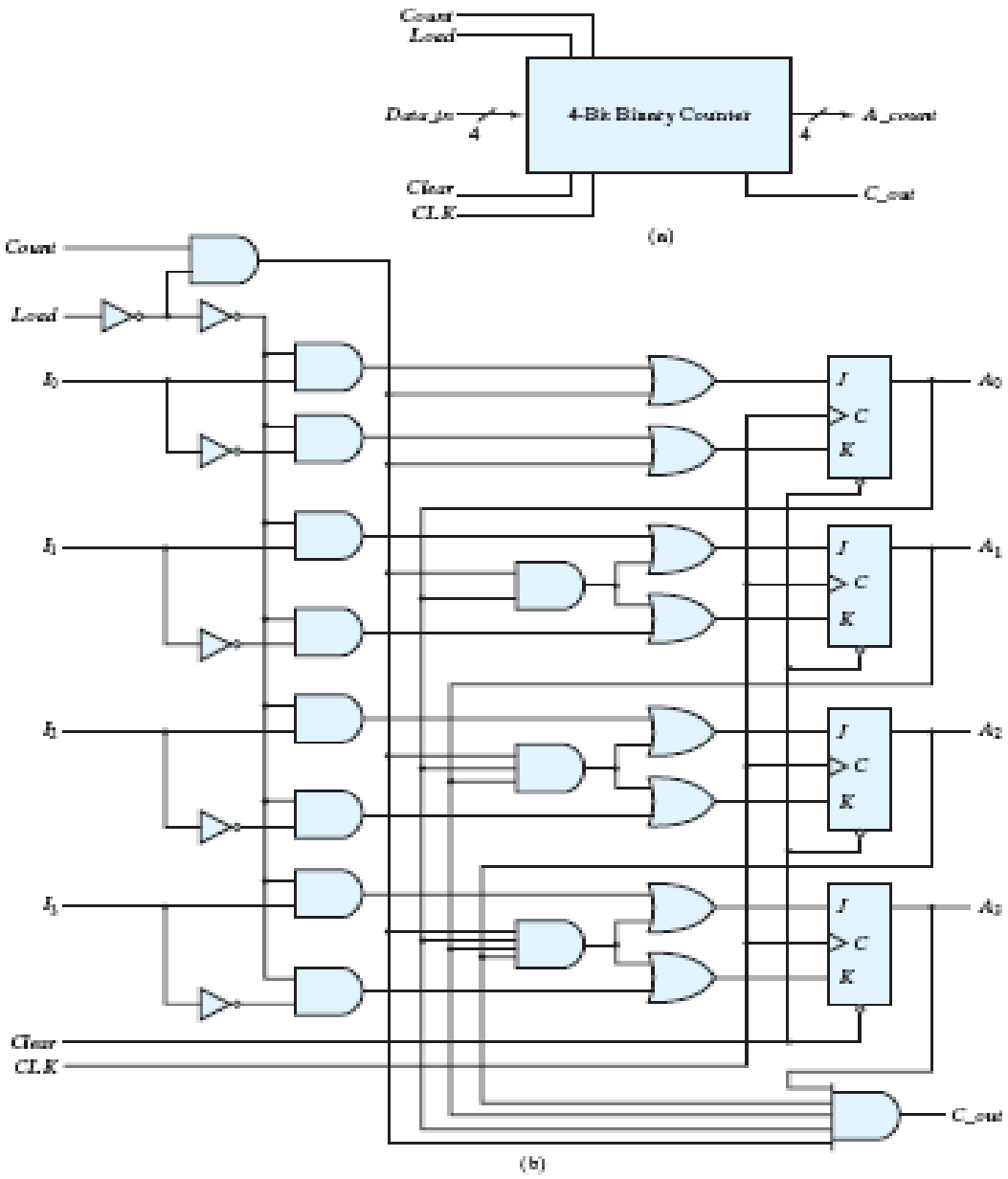


Fig. 6-11 Block Diagram of a Three-Decade Decimal BCD Counter

Counter with Parallel load



More BCD Counters (with Parallel Load)

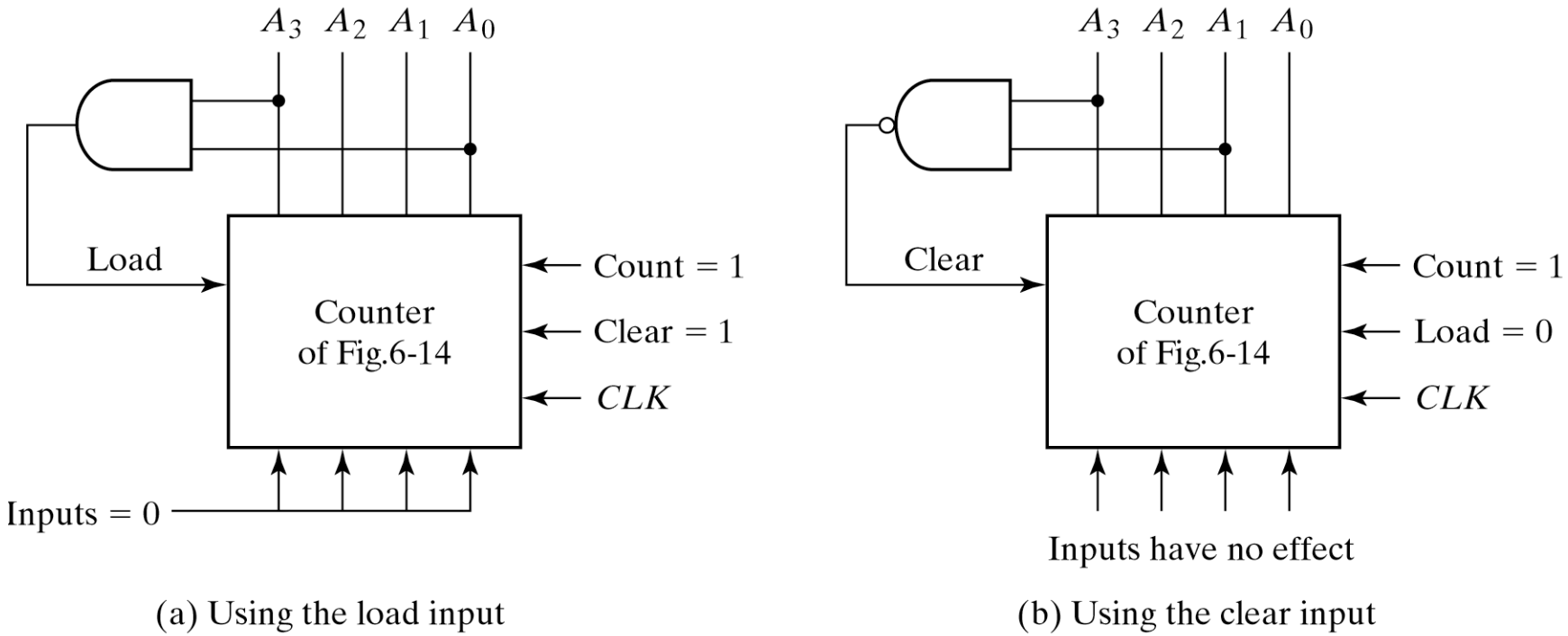


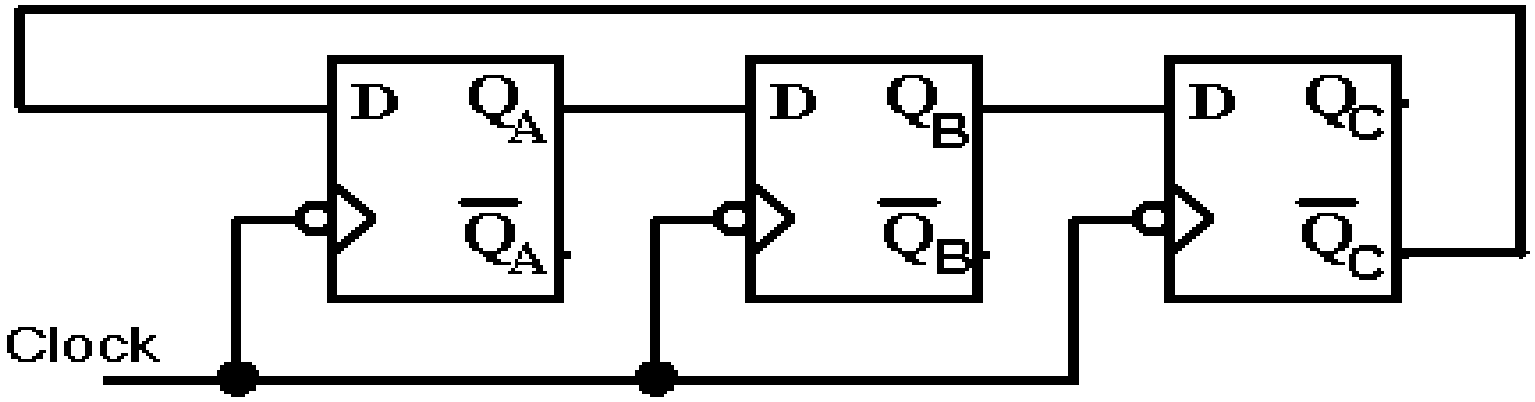
Fig. 6-15 Two ways to Achieve a BCD Counter Using a Counter with Parallel Load

Johnson Counter

- A Johnson Counter re-circulates the last flip-flop Q' (inverted) output back to the input of the first Flip-Flop.

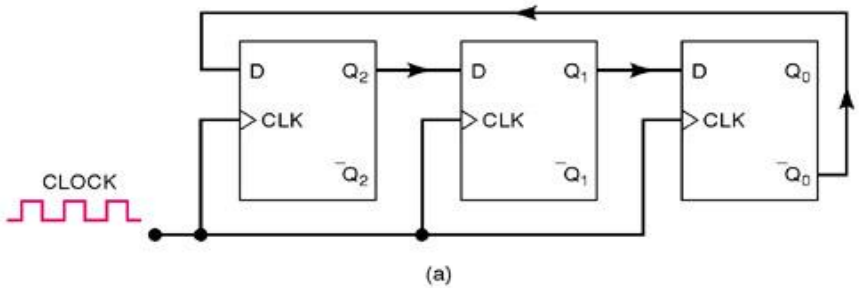
000, 100, 110,111,011,001, 000, 100, 110, ….

Initial state

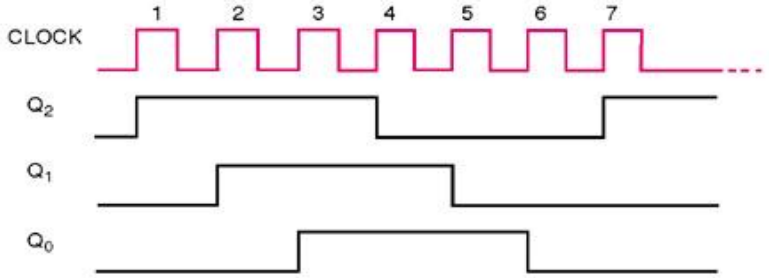


Johnson Counter

Johnson Counter



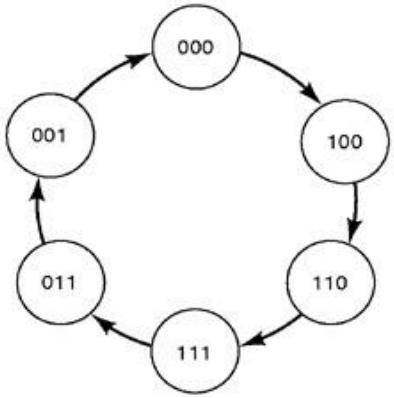
(a)



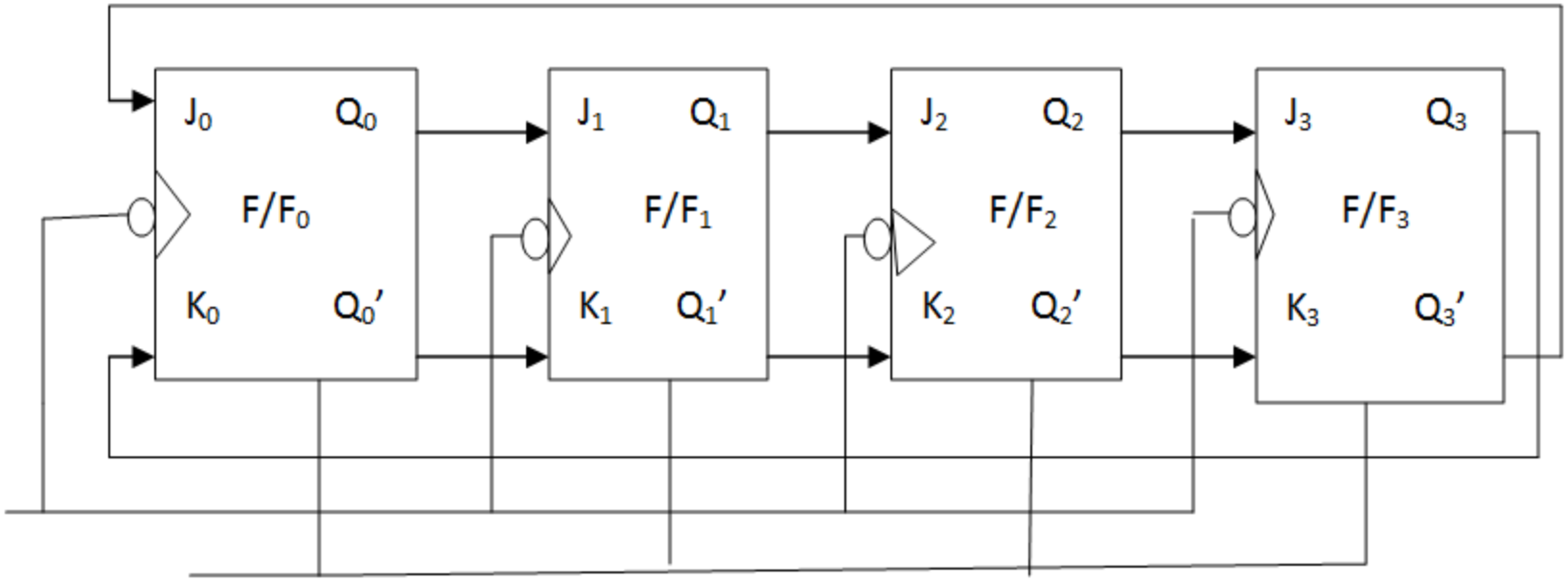
(b)

Q_2	Q_1	Q_0	CLOCK pulse
0	0	0	0
1	0	0	1
1	1	0	2
1	1	1	3
0	1	1	4
0	0	1	5
0	0	0	6
1	0	0	7
1	1	0	8
.	.	.	.
.	.	.	.
.	.	.	.

(c)



(d)

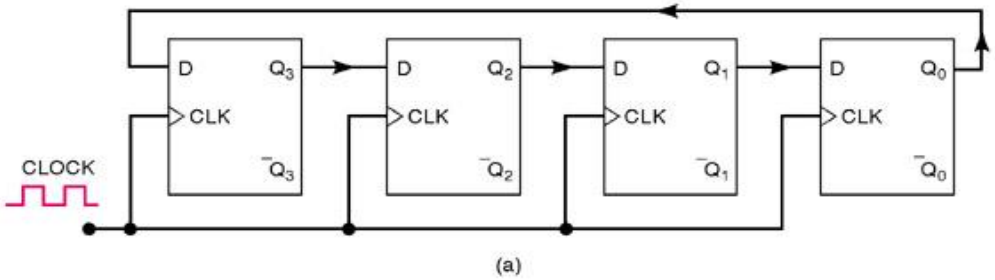

J0
Q0
F/F0
K0
Q0'
J1
Q1
F/F1
K1
Q1'
J2
Q2
F/F2
K2
Q2'
J3
Q3
F/F3
K3
Q3'

Ring Counter

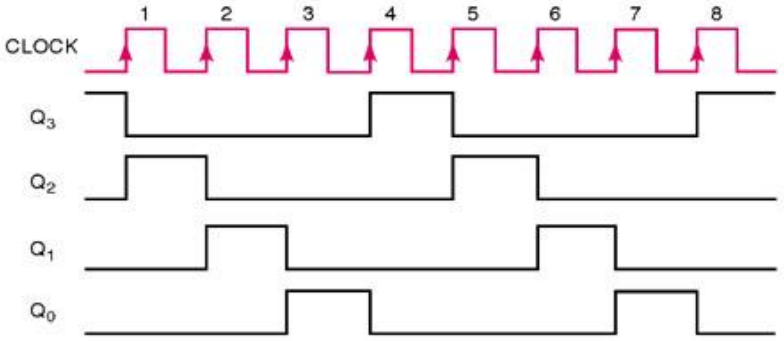
- A ring counter takes the serial output of the last Flip-Flop of a shift register and provides it to the serial input of the first Flip-Flop.

- This is also known as a re-circulating shift register.

Ring Counter



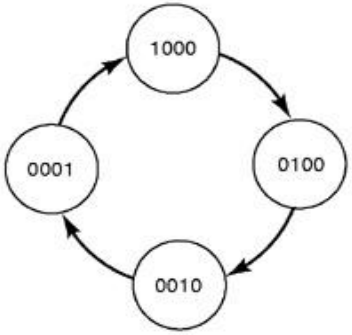
(a)



(b)

Q_3	Q_2	Q_1	Q_0	CLOCK pulse
1	0	0	0	0
0	1	0	0	1
0	0	1	0	2
0	0	0	1	3
1	0	0	0	4
0	1	0	0	5
0	0	1	0	6
0	0	0	1	7
.	.	.	.	.
.	.	.	.	.

(c)



(d)

Sequential Logic Circuits: Examples

Note:

In addition to the examples presented here other examples were discussed in the class:

-Asynchronous counters (4 examples are discussed in the class)

-Synchronous counters (3 examples are discussed in the class)

- Steps for building synchronous counters (comprehensive examples with animation is used here to show the steps with the D flip Flops)

* Convert from standard to canonical form

ex. Express the Boolean function $F = A + B'C$ as a sum of minterms

① $F = A + B'C$

$= A(B + B') + B'C$

$= AB + AB' + B'C$

$= AB(C + C') + AB' + B'C$

$= ABC + ABC' + AB' + B'C$

$= ABC + ABC' + AB'(C + C') + B'C$

$= ABC + ABC' + AB'C + AB'C' + B'C$

$= ABC + ABC' + AB'C + AB'C' + B'C(A + A')$

$= ABC + ABC' + AB'C + AB'C' + AB'C + A'B'C$

111 110 101 100 101 001

$= m_7 + m_6 + m_5 + m_4 + m_5 + m_1$

$F = \Sigma(1, 4, 5, 6, 7)$

② From truth table

$F = A + B'C$

ABC	
0 00	0
0 01	1 m_1
0 10	0
0 11	0
1 00	1 m_4
1 01	1 m_5
1 10	1 m_6
1 11	1 m_7

$F(A, B, C) = \Sigma(1, 4, 5, 6, 7)$

$= \Pi(0, 2, 3)$

Even Parity Generator

Let us assume that a 3-bit message is to be transmitted with an even parity bit. Let the three inputs A, B and C are applied to the circuits and output bit is the parity bit P. The total number of 1s must be even, to generate the even parity bit P. The figure below shows the truth table of even parity generator in which 1 is placed as parity bit in order to make all 1s as even when the number of 1s in the truth table is odd.

3-bit message			Even parity bit generator (P)
A	B	C	Y
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1

The K-map simplification for 3-bit message even parity generator is

K-Map Simplification

A \ BC	00	01	11	10
0	0	1	0	1
1	1	0	1	0

From the above truth table, the simplified expression of the parity bit can be written as

$$P = \overline{A}\ \overline{B}\ C + \overline{A}\ B\ \overline{C} + A\ \overline{B}\ \overline{C} + A\ B\ C$$

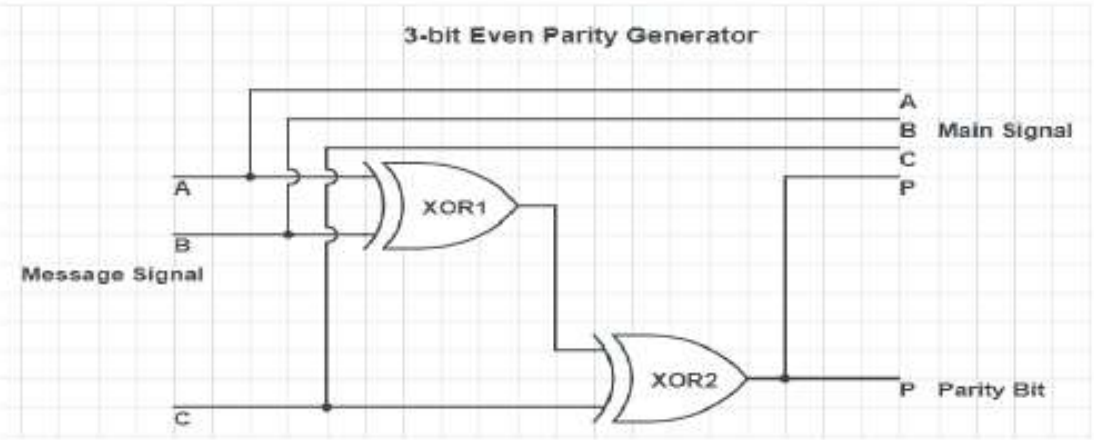
$$= \overline{A}\ (\overline{B}\ C + B\ \overline{C}) + A\ (\overline{B}\ \overline{C} + B\ C)$$

$$= \overline{A}\ (B \oplus C) + A\ (\overline{B \oplus C})$$

$$P = A \oplus B \oplus C$$

The above expression can be implemented by using two Ex-OR gates. The logic diagram of even parity generator with two Ex – OR gates is shown below. The three bit message along with the parity generated by this circuit which is transmitted to the receiving end where parity

checker circuit checks whether any error is present or not. To generate the even parity bit for a 4-bit data, three Ex-OR gates are required to add the 4-bits and their sum will be the parity bit.



Odd Parity Generator

Let us consider that the 3-bit data is to be transmitted with an odd parity bit. The three inputs are A, B and C and P is the output parity bit. The total number of bits must be odd in order to generate the odd parity bit. In the given truth table below, 1 is placed in the parity bit in order to make the total number of bits odd when the total number of 1s in the truth table is even.

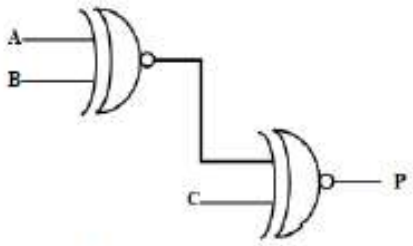
3-bit message			Odd parity bit generator (P)
A	B	C	Y
0	0	0	1
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	0

The truth table of the odd parity generator can be simplified by using K-map as

A \ BC	00	01	11	10
0	1	0	1	0
1	0	1	0	1

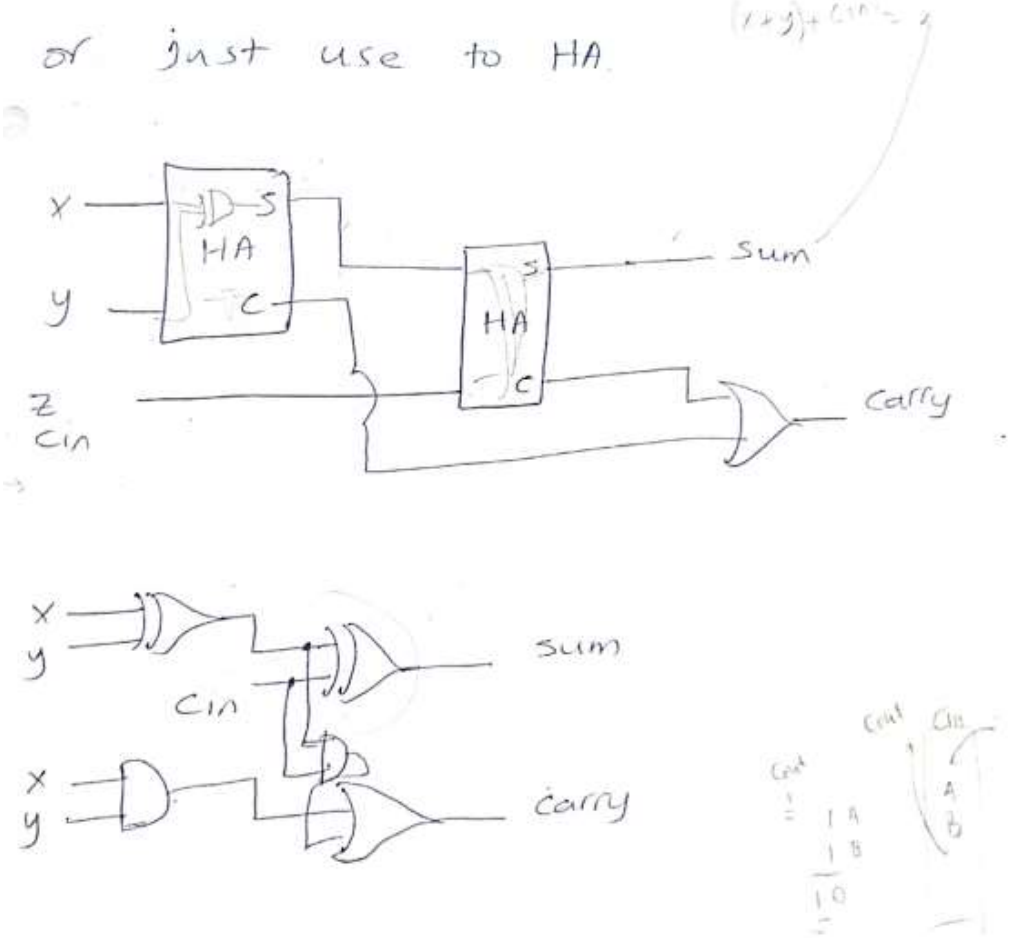
The output parity bit expression for this generator circuit is obtained as

$$P = (A \oplus B \oplus C)'$$



Odd Parity Generator for three bit Data Word

The Full Adder also can be implemented as:



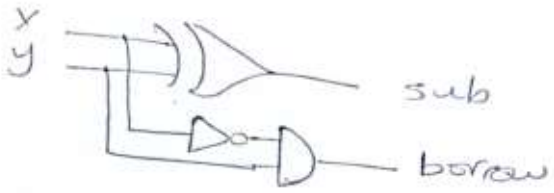
Subtractor:

Subtractor:-

x	y	sub	borrow
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

$sub = x \oplus y$

$borrow = x'y$



2

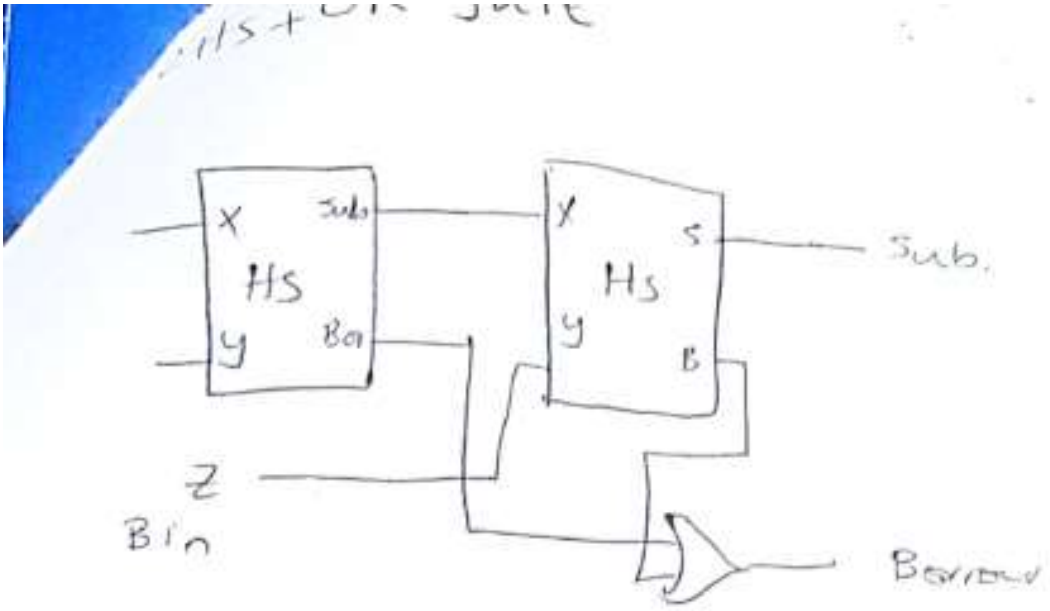
* Full subtractor:-

x	y	Bin	sub	Bout
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	1	0	0	0
1	1	1	1	1

$sub = x \oplus y + Bin$

$Bout = xy' + x'Bin + y\,Bin$

$= xy' + B_{in}(x+y)'$


X
Sub
HS
y
Bo
x
S
HS
y
B
Sub.
Z
Bin

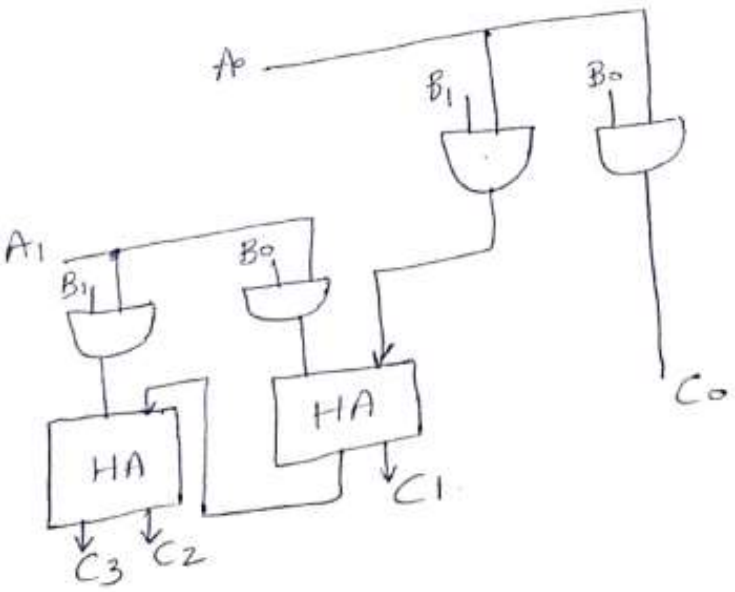
Multiplier:-

Design a Multiplier for two 2-bit numbers A & B.

$$C = A \times B$$

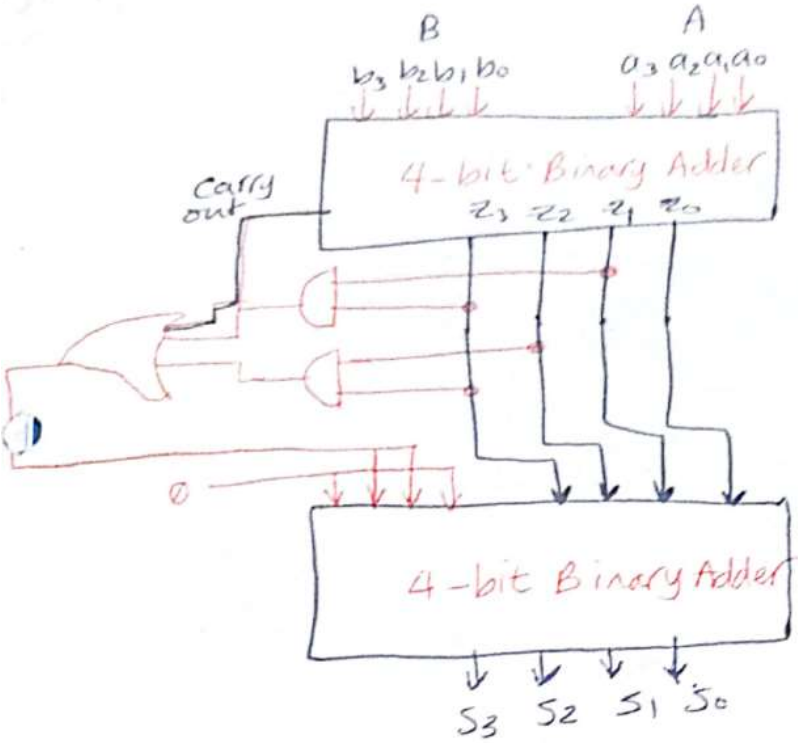
$$A = A_1 A_0$$
$$B = B_1 B_0$$

$$\begin{array}{r} B_1 \quad B_0 \\ A_1 \quad A_0 \quad \times \\ \hline A_0B_1 \quad A_0B_0 \\ A_1B_1 \quad A_1B_0 \quad + \\ \hline C_3 \quad C_2 \quad C_1 \quad C_0 \end{array}$$



BCD Adder:-

- A BCD adder adds two BCD digits & produces a sum digit in BCD



1 0 0 0
1 0 0 1
1 0 1 0
1 0 1 1
1 1 0 0
1 1 0 1
1 1 1 0
1 1 1 1

The Truth Table for 2-bit Comparator:

Table 1. Truth Table of 2-Bit Magnitude Comparator

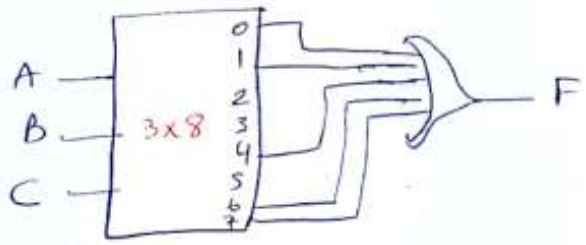
INPUT				OUTPUT		
A1	A0	B1	B0	A>B	A=B	A<B
0	0	0	0	0	1	0
0	0	0	1	0	0	1
0	0	1	0	0	0	1
0	0	1	1	0	0	1
0	1	0	0	1	0	0
0	1	0	1	0	1	0
0	1	1	0	0	0	1
0	1	1	1	0	0	1
1	0	0	0	1	0	0
1	0	0	1	1	0	0
1	0	1	0	0	1	0
1	0	1	1	0	0	1
1	1	0	0	1	0	0
1	1	0	1	1	0	0
1	1	1	0	1	0	0
1	1	1	1	0	1	0

Implement the Boolean function

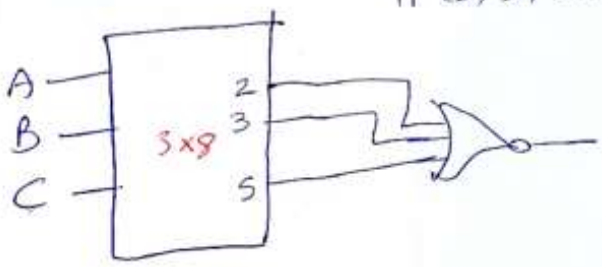
$F(A,B,C) = \Sigma(0,1,4,6,7)$

using 3x8 Decoder

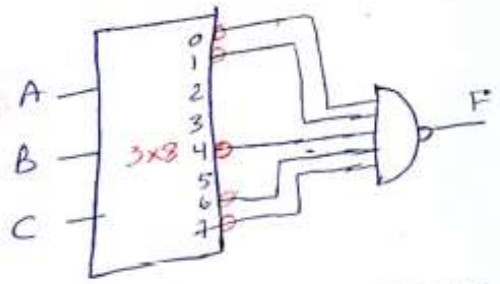
Active High



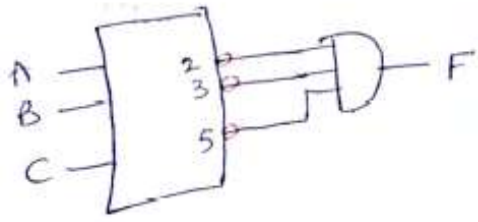
$\Pi(2,3,5)$



Active Low



$\Pi(2,3,5)$



a MUX with (n-1) select inputs

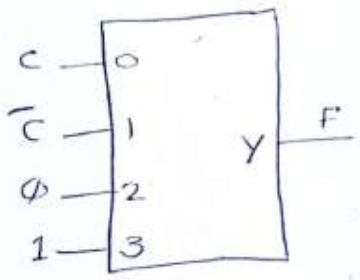
plement

$F(A,B,C) = \Sigma(1,2,4,7)$

using a 4x1 MUX.

A	B	C	F	
0	0	0	0	C
0	0	1	1	
0	1	0	1	$\bar{C}$
0	1	1	0	
1	0	0	0	0
1	0	1	0	
1	1	0	1	1
1	1	1	1	

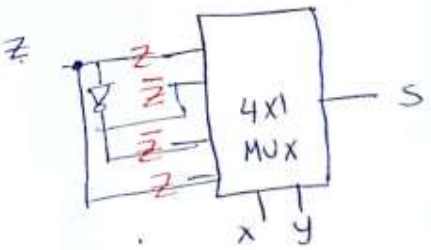
select

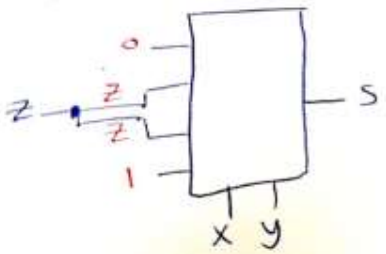


Ex. Design a full Adder using 4x1 MUXs

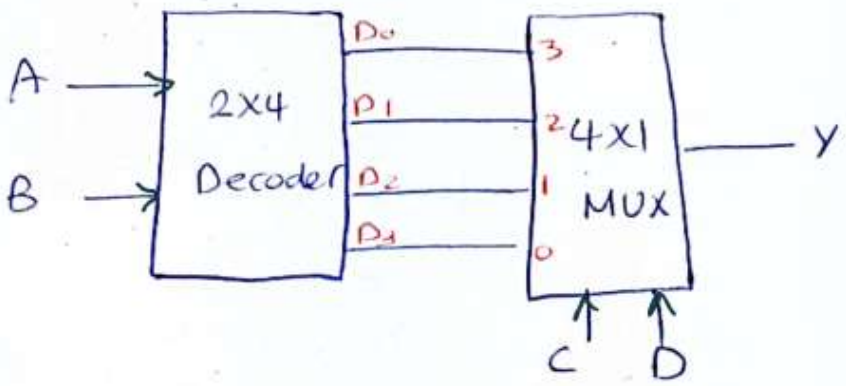
X	y	z	S	C	
0	0	0	0	0	0
0	0	1	z	1	
0	1	0	$\bar{z}$	0	z
0	1	1	0	1	
1	0	0	$\bar{z}$	0	z
1	0	1	0	1	
1	1	0	z	1	1
1	1	1	z	1	

select





1 Fill the following table



A	B	C	D	Y
0	0	0	0	0
0	0	0	1	0
0	0	1	0	0
0	0	1	1	1
0	1	0	0	0
0	1	0	1	0
0	1	1	0	1
0	1	1	1	0
1	0	0	0	0
1	0	0	1	1
1	0	1	0	0
1	0	1	1	0
1	1	0	0	1
1	1	0	1	0
1	1	1	0	0
1	1	1	1	0